# МАТЕРИАЛЫ ДЛЯ ГОСУДАРСТВЕННОЙ РЕГИСТРАЦИИ ПРОГРАММЫ ДЛЯ ЭВМ
# АРМ
# МОДУЛИ «СПРАВОЧНИКИ» И «ЭКОЛОГИЯ»
# ОПИСАНИЕ ФУНКЦИОНАЛЬНЫХ ВОЗМОЖНОСТЕЙ, ФРАГМЕНТЫ ИСХОДНОГО ТЕКСТА И ЭКРАННЫЕ ФОРМЫ

Адреса модулей: http://msk-arm-gs01.ntcees.ru/refdata/; http://msk-arm-gs01.ntcees.ru/ecology/

Дата подготовки материалов: 29 октября 2026 года

Состав: главная страница, 25 страниц списков, 25 форм добавления, служебные операции и взаимодействия с другими модулями.. Дополнено описанием модуля «Экология»: отчетные данные ПГ и ЗВ, анализ, расчеты действующих и новых ТЭС, сводные расчеты, агрегированные суммы, тексты формул и рассмотрение топливных пакетов. Разделы 1–6 относятся к «Справочникам», разделы 7–17 к «Экологии».

Документ подготовлен как отдельный файл, предусмотренный пунктом «Исходный текст программы (листинг)» шаблона «Сведения для регистрации программы». Приведены фрагменты, а не полный исходный текст программы.

Основание описания: текущие исходные файлы рабочего проекта и экранные формы действующего сайта. Номер релиза серверного приложения отдельно не устанавливался: совпадение серверной сборки со всеми локальными исходными файлами не утверждается.

## СОДЕРЖАНИЕ

1. Сведения по форме предоставленного шаблона ..... 5
2. Назначение и архитектура модуля ..... 8
2.1. Слои реализации ..... 8
2.2. Версии БД и межверсионная идентичность ..... 9
2.3. Общие правила обработки и ограничения ..... 10
3. Главная страница и периоды планирования ..... 11
3.1. Модели и маршрут главной страницы ..... 12
3.2. Изменение текущего года ..... 13
3.3. Служебные операции периодов и взаимодействия ..... 15
4. Страницы справочников и формы добавления ..... 16
4.1. Годы ..... 16
4.2. Федеральные округа ..... 21
4.3. Субъекты РФ ..... 26
4.4. Зарубежные страны, имеющие общие границы с РФ ..... 32
4.5. Объединенные энергосистемы ..... 36
4.6. Региональные энергосистемы ..... 41
4.7. Энергорайоны ..... 46
4.8. Энергоузлы ..... 51
4.9. Типы энергосистем ..... 55
4.10. Синхронные зоны ..... 59
4.11. Энергозоны ..... 64
4.12. Генерирующие компании ..... 68
4.13. Типы топлива ..... 72
4.14. Виды топлива ..... 76
4.15. Типы электростанций ..... 83
4.16. Типы состояния оборудования ..... 87
4.17. Типы агрегатов ..... 91
4.18. Типы ТЭС ..... 95
4.19. Типы агрегатов ТЭС ..... 99
4.20. Типы агрегатов ПГУ ..... 103
4.21. Типы групп оборудования ..... 107
4.22. Типы технологий ..... 113
4.23. Доступность технологии ..... 117
4.24. Типы затрат на производство ЭЭ и ТЭ ..... 121
4.25. Виды экономической деятельности (ВЭД) ..... 125
5. Взаимодействие с другими модулями ..... 130
5.1. Пример сквозного взаимодействия: субъект РФ → энергозона → потребление ..... 131
5.2. Пример сквозного взаимодействия: тип группы → технология → оборудование ..... 131
5.3. Граница модуля: управление версиями БД ..... 132
6. Полнота материалов и перечень страниц ..... 133
6.1. Сведения об источниках и проверке ..... 134
7. Модуль «Экология»: назначение и архитектура ..... 136
7.1. Страницы, вкладки и границы описания ..... 136
7.2. Программные слои и модели данных ..... 136
7.3. Права, версия БД и журнал ..... 138

7.4. Условия получения экранных форм ... 139
8. Главная страница и отчетные данные ГИС ТЭК ... 141
8.1. Главная страница модуля ... 141
8.2. Парниковые газы: отчетные данные ГИС ТЭС ... 142
8.3. Загрязняющие вещества: отчетные данные ГИС ТЭС ... 146
9. Анализ отчетных данных и сопоставление с модулем «Топливо» ... 150
9.1. Парниковые газы: анализ с котельными ... 150
9.2. Парниковые газы: анализ без котельных ... 152
9.3. Загрязняющие вещества: анализ с котельными ... 155
9.4. Загрязняющие вещества: анализ без котельных ... 157
10. Расчет по действующим ТЭС ... 161
10.1. Данные по модулю Топливо ... 161
10.2. Характеристики топлив и продуктов сгорания ... 163
10.3. Коэффициенты очистки и подавления ... 166
10.4. Зола: выбросы твердых веществ ... 168
10.5. Сера: выбросы диоксида серы ... 170
10.6. Азот: выбросы оксидов азота ... 172
10.7. CO: выбросы монооксида углерода ... 174
10.8. ПГ: прямой расчет CO2 ... 176
10.9. ПГ через ТДж: расчет CO2 по энергии топлива ... 178
10.10. Объемы и удельные выбросы действующих ТЭС ... 180
11. Расчет по новым ТЭС ... 183
11.1. Принятые топливные показатели новых ТЭС ... 183
11.2. Нормативы удельных выбросов новых ТЭС ... 185
11.3. Объемы выбросов новых ТЭС ... 188
12. Сводные данные расчетов ... 190
12.1. Общие топливные показатели действующих и новых ТЭС ... 190
12.2. Объемы выбросов по России ... 191
12.3. Удельные выбросы по России ... 193
12.4. Динамика изменения выбросов ... 195
13. Агрегированные суммы отчетно-аналитического контура ... 198
13.1. ПГ: агрегированные суммы с котельными ... 198
13.2. ПГ: агрегированные суммы без котельных ... 200
13.3. ЗВ: агрегированные суммы с котельными ... 203
13.4. ЗВ: агрегированные суммы без котельных ... 205
14. Тексты подсказок формул ... 209
14.1. Редактор текстов подсказок формул ... 209
15. Рассмотрение данных, переданных из модуля «Топливо» ... 212
15.1. Рассмотрение пакета данных из модуля «Топливо» ... 212
16. Сквозная логика и взаимодействие с другими модулями ... 214
16.1. Отчет, выверенные параметры и агрегаты ... 214
16.2. Передача топлива и расчет выбросов ТЭС ... 215
16.3. Формулы, единицы измерения и отсутствующие данные ... 217
16.4. Обмен Excel и интерактивные действия ... 218
17. Полнота описания модуля «Экология» и источники ... 221
17.1. Перечень экранных представлений ... 221
17.2. Скрытые совместимые вкладки сводного расчета ... 222
17.3. Маршруты страниц и служебные операции ... 224

17.4. Проверка и ограничения..... 225

**1. Сведения по форме предоставленного шаблона**

| № | Сведения по шаблону | Содержание для модуля «Справочники» |
|---|---|---|
| 1 | Название программы (полное и сокращенное) | Программа: АРМ (обозначение, предоставленное заявителем). Полное регистрационное наименование подлежит уточнению. Описываемая часть: модуль «Справочники» (refdata). |
| 2 | Назначение, область применения, функциональные возможности (в реферативной форме); ~ 10 строк. | Модуль «Справочники» программы АРМ предназначен для централизованного ведения нормативно-справочной информации, используемой при планировании развития электроэнергетики. Обеспечивает ведение административных территорий, энергосистем и энергозон, компаний, топлива, типов станций, агрегатов, групп оборудования, технологий, видов затрат и ВЭД. Реализует поиск, фильтрацию, сортировку, постраничный просмотр, табличное редактирование, добавление и удаление записей. Для предусмотренных маршрутов поддерживаются импорт и экспорт Excel. Управляет текущим годом, признаками годов и периодами планирования. Работает в контексте выбранной версии БД; для поддерживаемых справочников предусмотрено изменение во всех версиях по ref_uuid. Предоставляет единые классификаторы модулям генерации, топлива, экономики, электропотребления, потребности в мощности, энергетических балансов, экологии и рациональной структуры. Доступ осуществляется через веб-интерфейс после аутентификации. |
| 3 | Тип ЭВМ | Клиент-серверная веб-система. Сервер и клиентские персональные компьютеры; работа пользователя через веб-браузер. Точная аппаратная конфигурация заявляемого экземпляра подлежит уточнению. |
| 4 | Язык программирования | Python; JavaScript; шаблоны HTML/Jinja2 и CSS. Доступ к реляционным данным через SQLAlchemy (СУБД PostgreSQL). |
| 5 | Тип и версия используемой ОС | В документации проекта описано серверное развертывание на Astra Linux 1.8.4. Текущий рабочий каталог находится в среде Windows. ОС и версия заявляемого экземпляра подтверждаются заявителем. |
| 6 | Объем программы для ЭВМ в машиночитаемой форме в единицах, кратных числу байт | Объем всей регистрируемой программы и состав поставки указываются заявителем. Размер выборки исходных файлов этого описания не является объемом программы. |
| 7 | Дата окончательного завершения программы | Дата окончательного завершения программы указывается заявителем. 29.10.2026 – дата |

| № | Сведения по шаблону | Содержание для модуля «Справочники» |
|---|---|---|
| | (число, месяц, год) | подготовки описания, не подтверждение завершения разработки. |
| 8 | ФИО каждого автора; дата рождения; адрес местожительства (включая индекс) | Заполняется заявителем: сведения не предоставлены. |
| 9 | В каком порядке указывать ФИО авторов? | Заполняется заявителем: сведения не предоставлены. |
| 10 | Краткое описание авторского вклада в данную программу (Написание исходного текста программного модуля …; или Написание исходного текста подпрограммы …; или Разработка программ …; или Написание исходного текста программы – 00%) | Авторский вклад и доли определяются заявителем. Документ описывает реализацию модуля, но не устанавливает авторство конкретных лиц. |
| 11 | Исходный текст программы (листинг) (можно не подряд, а фрагментами). Допускается включать в состав этих мат-лов подготовительные материалы, полученные в ходе ее разработки, а также порождаемые ею аудиовизуальные отображения в любой визуально воспринимаемой форме. Экранные формы включаются в состав документов заявки в том случае, если они охраноспособны по нормам авторского права, т.е. являются результатом творческой деятельности автора. | Фрагменты исходного текста приведены в разделах 2–6, с указанием файлов и строк; экранные формы включены по месту описания. Представлены фрагменты, не полный текст. |
| 12 | Контактное лицо из числа авторов, тел. | Заполняется заявителем: сведения не предоставлены. |

Сведения об авторах, их персональных данных, контактах, окончательном названии, объеме и дате завершения не определяются по имени пользователя в интерфейсе или по дате изменения файлов. Вопрос о включении экранных форм в состав охраняемых материалов решается заявителем с учетом прав на используемые элементы интерфейса.

## 2. Назначение и архитектура модуля

Модуль «Справочники» программы АРМ предназначен для централизованного ведения нормативно-справочной информации, используемой при планировании развития электроэнергетики. Обеспечивает ведение административных территорий, энергосистем и энергозон, компаний, топлива, типов станций, агрегатов, групп оборудования, технологий, видов затрат и ВЭД. Реализует поиск, фильтрацию, сортировку, постраничный просмотр, табличное редактирование, добавление и удаление записей. Для предусмотренных маршрутов поддерживаются импорт и экспорт Excel. Управляет текущим годом, признаками годов и периодами планирования. Работает в контексте выбранной версии БД; для поддерживаемых справочников предусмотрено изменение во всех версиях по ref_uuid. Предоставляет единые классификаторы модулям генерации, топлива, экономики, электропотребления, потребности в мощности, энергетических балансов, экологии и рациональной структуры. Доступ осуществляется через веб-интерфейс после аутентификации.

### 2.1. Слои реализации

| Слой | Реализация и роль |
|---|---|
| Модели | SQLAlchemy-модели app/refdata/models: таблицы классификаторов, внешние ключи, ограничения уникальности, привязка к версии БД и устойчивый ref_uuid. |
| Маршруты | Flask Blueprint refdata_bp; обработка GET/POST, получение параметров формы, вызов сервисов, flash-сообщения, перенаправление или render_template. |
| Сервисы | app/refdata/services: выборка, фильтры, валидация, изменение данных, импорт/экспорт и синхронизация между версиями. |
| Формы | WTForms-классы app/refdata/forms: поля, подписи и валидаторы форм фильтрации и добавления. |
| Представление | Jinja2/HTML app/templates/refdata, общий base.html, клиентские скрипты для таблиц, поиска и иерархии топлива. |
| Общие подсистемы | Версии БД, аутентификация и права, журналирование, кэш вариантов выбора, транзакции и разрешение внешних ключей. |

Основной поток просмотра: браузер → GET-маршрут → сервис выборки с контекстом версии → SQLAlchemy/БД → пагинация и варианты выбора → шаблон Jinja2 → HTML. Поток изменения: POST → чтение и проверка значений → сервис изменения → проверка зависимостей и сохранение → сообщение → перенаправление на список.

Регистрация Blueprint и маршрутов. Источник: app/refdata/routes/__init__.py, строки 1–18.

```
  1  from flask import Blueprint
  2
  3  refdata_bp = Blueprint('refdata_bp', __name__)
  4
  5  from .refdata_routes import *
  6  from .territories.federal_district_routes import *
  7  from .territories.regional_district_routes import *
  8  from .territories.foreign_border_country_routes import *
  9  from .energy_systems.union_energy_system_routes import *
 10  from .energy_systems.regional_energy_system_routes import *
 11  from .energy_systems.energy_area_routes import *
 12  from .energy_systems.energy_zone_routes import *
 13  from .energy_systems.energy_system_type_routes import *
 14  from .energy_systems.synchronous_area_routes import *
 15  from .energy_systems.energy_unit_routes import *
 16  from .gen_companies.gen_company_routes import *
 17  from .fuels.fuel_routes import *
 18  from .fuels.fuel_type_routes import *
```

## 2.2. Версии БД и межверсионная идентичность

Записи классификаторов имеют database_version_id. Первичный ключ id идентифицирует строку. У справочников с RefdataUuidMixin поле ref_uuid позволяет сопоставить логически одну и ту же запись в разных версиях. Числовой id связанной записи нельзя безусловно копировать в другую версию: для межверсионной операции используется разрешение внешнего ключа по ref_uuid.

Разрешение связей между версиями. Источник: app/refdata/services/refdata_all_versions_common.py, строки 43–64.

```
 43  def fk_id_for_version(model_cls, source_pk_id: int | None, target_version_id: int | None) -> int |
None:
 44      """
 45      Находит id строки с тем же ref_uuid в указанной версии БД,
 46      по исходному pk записи из текущей (или любой) версии.
 47
 48      При активном fk_id_resolution_cache() один раз подгружает все версии
 49      по ref_uuid и дальше отвечает из памяти.
 50      """
 51      if source_pk_id is None:
 52          return None
 53      if target_version_id is None:
 54          raise ValueError(
 55              f"Нельзя сопоставить {model_cls.__name__} с версией БД id=None: "
 56              "записи без версии не участвуют в синхронизации «во всех версиях»."
 57          )
 58
 59      source_pk = int(source_pk_id)
 60      target_vid = int(target_version_id)
 61      cache = _fk_id_cache.get()
 62      cache_key = (model_cls.__name__, source_pk, target_vid)
 63      if cache is not None and cache_key in cache:
 64          return cache[cache_key]
```

Операции с признаком all_versions=1 вызываются только там, где они реализованы в маршруте и сервисе. Для справочника годов такой режим не предусмотрен. Проверки has_admin и is_admin различаются: конкретный маршрут может требовать общие административные полномочия либо именно роль admin. Документ не приравнивает эти признаки.

Серверная проверка межверсионной операции. Источник: app/refdata/routes/refdata_all_versions_guard.py, строки 11–23.

```
  11  def block_all_versions_without_admin(current_user) -> bool:
  12      """
  13      True – запрос следует заблокировать (нужен redirect и flash).
  14      """
  15      if not all_versions_requested():
  16          return False
  17      if getattr(current_user, "has_admin", False):
  18          return False
  19      flash(
  20          "Операции во всех версиях базы данных доступны только пользователям с ролью
администратора.",
  21          "danger",
  22      )
  23      return True
```

## 2.3. Общие правила обработки и ограничения

Большинство списков совмещает просмотр и изменение: GET получает фильтры, сортировку и номер страницы, POST передает массивы идентификаторов и значений строк. Удаление обычно задается флажками и обрабатывается при отправке формы. Фактический результат зависит от проверок сервиса, ограничений БД и наличия зависимых записей; возможность безусловного удаления не заявляется.

Наличие login_required означает требование входа в систему, но не доказывает отдельного административного ограничения для каждой операции. Для годов предусмотрен has_admin_required; изменение текущего года требует roles_required(["admin"]). Видимость кнопки в шаблоне и серверная проверка прав описываются раздельно.

Сохранение выполняется транзакционными средствами проекта. В сервисах применяются commit/rollback и общий _commit_with_retry; отдельные операции используют дополнительную обработку конфликтов и ограничений целостности. Изменения и ошибки регистрируются через log_to_db. Журналирование не является отдельной пользовательской страницей справочника.

## 3. Главная страница и периоды планирования

Адрес: GET /refdata/. Страница предоставляет навигацию по группам справочников и показывает текущий год, период СиПР и горизонт Генеральной схемы. Текущий год выбирается из справочных годов выбранной версии. При отсутствии выбранной версии маршрут передает нулевые значения и пустой список вариантов выбора.

Для текущего года 2025 на действующем сайте показаны СиПР 2026–2031 и Генеральная схема 2026–2042. Это значения конкретного состояния данных, а не постоянные параметры программы.




Рисунок 1. Главная страница модуля «Справочники». Действующий сайт, 29.10.2026. Показан фрагмент экранной формы.

### 3.1. Модели и маршрут главной страницы

Используются Year (календарный год и его признак), YearFeature (факт, текущий/оценка, план), YearService (границы СиПР и даты), DatabaseVersion (контекст выбранной версии).

Модель параметров периода. Источник: app/refdata/models/years/year_service_model.py, строки 13–34.

```
 13  class YearService(db.Model, AuditMixin, VersionedModelMixin):
 14      __tablename__ = 'gs_sys_year_service'
 15      __table_args__ = {
 16          "schema": SCHEMA_REFDATA,
 17          "extend_existing": True
 18      }
 19
 20      id = db.Column(db.Integer, primary_key=True, autoincrement=True)
 21
 22      # Годы СиПР
 23      year_sipr_start = db.Column(db.Integer, nullable=True)  # Год начала СиПР
 24      year_sipr_end = db.Column(db.Integer, nullable=True)    # Год конца СиПР
 25
 26      # Даты СиПР
 27      date_sipr_start = db.Column(db.Date, nullable=True)  # Дата начала СиПР
 28      date_sipr_end = db.Column(db.Date, nullable=True)    # Дата конца СиПР
 29
 30      # Аудит
 31      created_at = db.Column(db.DateTime(timezone=True), server_default=func.now(), nullable=False)
 32      updated_at = db.Column(db.DateTime(timezone=True), server_default=func.now(),
onupdate=func.now(), nullable=False)
 33
 34      # FK -> DatabaseVersion
```

Маршрут главной страницы. Источник: app/refdata/routes/refdata_routes.py, строки 19–44.

```
 19  @refdata_bp.route("/")
 20  @login_required
 21  def refdata():
 22      """Страница справочников, в т.ч. блок управления годами/СиПР."""
 23      version_id = get_current_version()
 24
 25      if version_id:
 26          year_info = get_current_year_info(version_id)
 27      else:
 28          # Если версия не выбрана – по ТЗ показываем нули по годам
 29          year_info = {
 30              "current_year": 0,
 31              "sipr_start": 0,
 32              "sipr_end": 0,
 33              "date_sipr_start": None,
 34              "date_sipr_end": None,
 35          }
 36
 37      year_select_years = get_reference_years() if version_id else []
 38
 39      return render_template(
 40          "refdata/refdata.html",
 41          year_info=year_info,
 42          year_select_years=year_select_years,
 43          general_scheme_end_year=get_general_scheme_end_year(year_info["current_year"]),
 44      )
```

### 3.2. Изменение текущего года

POST /refdata/update_current_year проверяет роль admin, выбранную версию и наличие года в get_reference_years(). Сервис update_current_year проставляет признаки: годы меньше выбранного – «факт», равный – «текущий (оценка)», большие – «план». Границы СиПР сохраняются как current_year + 1 и current_year + 6. После сохранения очищаются связанные кэши годов.

Проверки маршрута изменения года. Источник: app/refdata/routes/refdata_routes.py, строки 47–69.

```
@refdata_bp.route("/update_current_year", methods=["POST"])
@login_required
@roles_required(["admin"])
def update_current_year_refdata():
    """Обновление текущего года из формы на странице справочников."""
    version_id = get_current_version()
    if not version_id:
        flash("Текущая версия БД не установлена", "danger")
        return redirect(url_for("refdata_bp.refdata"))

    try:
        year = int(request.form.get("current_year", 0))
    except (TypeError, ValueError):
        flash("Некорректное значение года", "danger")
        return redirect(url_for("refdata_bp.refdata"))

    if year not in get_reference_years():
        flash("Некорректное значение года", "danger")
        return redirect(url_for("refdata_bp.refdata"))

    success, message = update_current_year(version_id, year, current_user)
    flash(message, "success" if success else "danger")

```

Изменение признаков годов и периода СиПР. Источник: app/refdata/services/year_management_services.py, строки 156–188.

```

        # Правило: меньше текущего -> факт, текущий -> текущий (оценка), больше -> план
        years = (
            db.session.query(Year)
            .filter(Year.database_version_id == version_id)
            .all()
        )
        for y in years:
            if y.number < new_year:
                y.id_year_feature = fact_feature.id
            elif y.number > new_year:
                y.id_year_feature = plan_feature.id
            else:
                y.id_year_feature = current_year_feature.id

        # Автоматически обновляем SIPR годы в таблице YearService
        year_service = db.session.query(YearService).filter_by(
            database_version_id=version_id
        ).first()

        if not year_service:
            # Создаем новую запись YearService
            year_service = YearService(
                database_version_id=version_id,
                year_sipr_start=new_year + 1,
                year_sipr_end=new_year + 6
            )
            db.session.add(year_service)
        else:
            # Обновляем существующую запись
            year_service.year_sipr_start = new_year + 1
            year_service.year_sipr_end = new_year + 6

```

Горизонт Генеральной схемы определяется по шестилетнему циклу разработки, начиная с 2024 года, на 18 лет. Для current_year < 2024 или отсутствующего года конец не определяется; для 2024–2029 возвращается 2042, для 2030–2035 – 2048. Начальная граница в шаблоне выводится из sipr_start.

Правило горизонта Генеральной схемы. Источник: app/refdata/services/year_management_services.py, строки 19–24.

```
def get_general_scheme_end_year(current_year: int | None) -> int | None:
    """Разработка с 2024 года каждые 6 лет на 18 лет; актуализация не сдвигает конец."""
    if current_year is None or current_year < 2024:
        return None
    cycle = (current_year - 2024) // 6
    return 2024 + cycle * 6 + 18
```

HTML выбора текущего года. Источник: app/templates/refdata/refdata.html, строки 17–35.

```
  17                            <form method="post" action="{{
url_for('refdata_bp.update_current_year_refdata') }}" class="d-flex align-items-center gap-2 mb-0 flex-
fill">
  18                                <span class="fw-bold text-nowrap">Текущий год:</span>
  19                                <div class="input-group flex-grow-1">
  20                                    <select class="form-select" id="current-year-select"
name="current_year"
  21                                            {% if not current_user.has_admin %}disabled{% endif %}>
  22                                        <option value="0" {% if year_info.current_year == 0
%}selected{% endif %}>
  23                                            не установлен
  24                                        </option>
  25                                        {% for y in year_select_years %}
  26                                            <option value="{{ y }}" {% if year_info.current_year == y
%}selected{% endif %}>
  27                                                {{ y }}
  28                                            </option>
  29                                        {% endfor %}
  30                                    </select>
  31                                    {% if current_user.has_admin %}
  32                                        <button type="submit" class="btn btn-primary">
  33                                            <i class="bi bi-check-lg"></i>
  34                                        </button>
  35                                    {% endif %}
```

HTML периода Генеральной схемы. Источник: app/templates/refdata/refdata.html, строки 46–55.

```
  46                            <div class="d-flex align-items-center gap-2 flex-fill">
  47                                <span class="fw-bold text-nowrap">Генеральная схема:</span>
  48                                <div class="input-group flex-grow-1">
  49                                    <div class="form-control bg-light fw-bold fs-5 text-center">
  50                                        {% if general_scheme_end_year is not none %}
  51                                            {{ year_info.sipr_start }} – {{
general_scheme_end_year }}
  52                                        {% else %}
  53                                            не установлен
  54                                        {% endif %}
  55                                    </div>
```

### 3.3. Служебные операции периодов и взаимодействия

| Метод и адрес | Назначение |
|---|---|
| POST /refdata/update_sipr_dates | Сохранение дат СиПР через HTML-маршрут; отдельной формы на текущей главной странице нет. |
| GET /refdata/api/current_year_info | JSON-информация о годах текущей версии; без выбранной версии возвращается набор нулевых значений. |
| POST /refdata/api/update_current_year | Изменение текущего года через JSON. Маршрут проверяет диапазон 2000–2100, в отличие от HTML-маршрута с проверкой get_reference_years(). |
| POST /refdata/api/update_sipr_years | Изменение начала и конца СиПР через JSON, с проверками маршрута и сервиса. |
| POST /refdata/api/update_sipr_dates | Изменение дат СиПР через JSON. |

Данные о периодах передаются общим сервисам годов и далее используются в годовых фильтрах и расчетных представлениях. Смена текущего года изменяет классификацию временной оси; она не означает автоматического выполнения всех расчетов АРМ. Сервисные адреса возвращают JSON либо перенаправление и не являются дополнительными самостоятельными HTML-страницами.

## 4. Страницы справочников и формы добавления

Ниже описаны все 25 справочников, доступных из навигации модуля, включая административный справочник годов. Для каждого отдельно рассматриваются список и форма добавления. Технические адреса приведены с префиксом /refdata. Номера строк соответствуют файлам рабочей копии на дату подготовки. Листинги фрагментарные: промежуточные части функций не выдаются за полный исполняемый модуль. В русскоязычном тексте листингов нормализована типографика.

### 4.1. Годы

#### 4.1.1. Страница списка

Адрес: GET, POST /refdata/years. Обработчик: years_list().

Задает календарные годы и их признаки. Таблица позволяет просматривать, искать и редактировать годы; отдельная форма создает новый год. Признак года выбирается из YearFeature. Календарный номер года не равен внутреннему первичному ключу id.

Поля и параметры HTML-формы: per_page, page, sort_by, sort_dir, year_filter, year_ids[], year_numbers[], year_features[], year_delete[].

Обработка: маршрут получает параметры отображения, при POST читает изменяемые строки и флажки удаления, вызывает сервисы и возвращает пользователя к списку с параметрами просмотра. Для обычного просмотра строится выборка и передается в шаблон вместе с пагинацией и вариантами выбора связанных справочников.

Доступ к списку, добавлению и экспорту годов ограничен has_admin_required. Межверсионных кнопок у этого справочника нет.

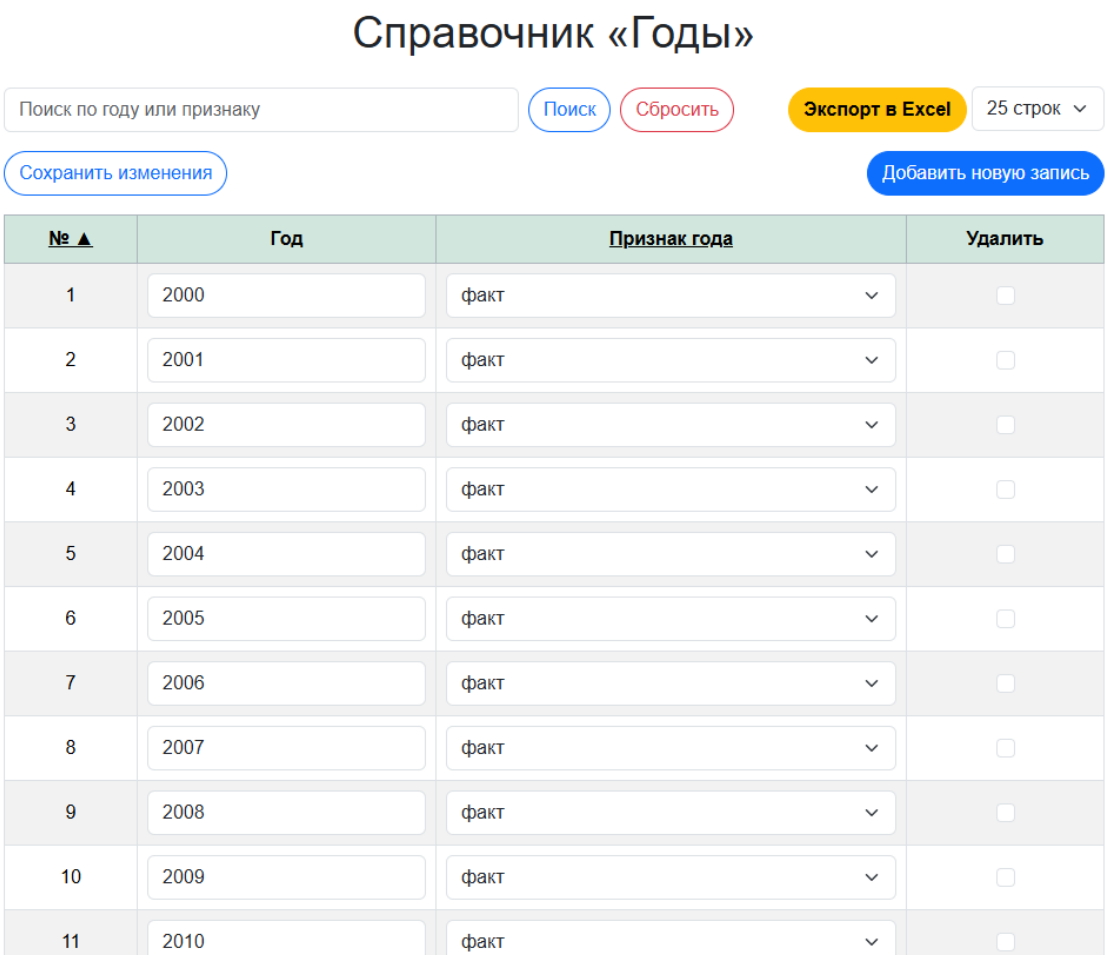



Рисунок 2. Годы. Действующий сайт, 29.10.2026. Показан фрагмент экранной формы.

Модель: Year. Поля SQLAlchemy: id, number, id_year_feature, year_feature, database_version_id, created_at, updated_at, machine_powers, machine_fuels, machine_tes_types, machine_names, pgu_machine_names, fd_economic_activity_consumption_parameters, rf_economic_activity_consumption_parameters, fd_economic_activity_accum_fixed_capital_parameters, rf_economic_activity_accum_fixed_capital_parameters, fd_economic_activity_product_output_parameters, rf_economic_activity_product_output_parameters, fd_electrical_intensity_year_parameters, rf_electrical_intensity_year_parameters, fd_population_parameters, fd_accum_monetary_income_parameters, fd_population_consumption_year_parameters, station_energy_generations, espp_energy_generations, regional_energy_system_energy_generations.

Модель данных – фрагмент объявления. Источник: app/refdata/models/years/year_model.py, строки 12–35.

```
class Year(db.Model, AuditMixin, VersionedModelMixin):
    __tablename__ = 'gs_sys_years'
    __table_args__ = (
        db.UniqueConstraint(
            "number",
            "database_version_id",
            name="uq_gs_years_number_version",
        ),
        {
            "schema": SCHEMA_REFDATA,
            "extend_existing": True,
        },
    )

    id = db.Column(db.Integer, primary_key=True, autoincrement=True)
    number = db.Column(db.Integer, nullable=False, index=True)

    # FK -> YearFeature
    id_year_feature = db.Column(
        db.Integer,
        db.ForeignKey(f'{SCHEMA_REFDATA}.gs_sys_year_features.id', ondelete='RESTRICT'),
        nullable=True,
        index=True
    )
```

Связь модели с другой сущностью. Источник: app/refdata/models/years/year_model.py, строки 36–36.

```
    year_feature = db.relationship('YearFeature', back_populates='years')
```

Маршрут списка – вход и параметры. Источник: app/refdata/routes/years/year_routes.py, строки 38–53.

```
@refdata_bp.route("/years", methods=["GET", "POST"])
@login_required
@has_admin_required
def years_list():
    """Страница справочника «Годы» (таблица + редактирование)."""

    user = session.get("username", "Неизвестный пользователь")
    log_to_db(user, "Открыта страница: годы", entity_type="year")

    # Для dropdown (строго по активной версии)
    version_id = get_current_version()

    # Форма
    form = YearFilterForm()

    # Параметры отображения
```

Маршрут списка – передача изменений сервису. Источник: app/refdata/routes/years/year_routes.py, строки 135–140.

```
                raise ValueError(f"Обнаружены дублирующиеся ID годов: {duplicates}")

            update_year_service(payload, user)
            flash("Изменения успешно сохранены.", "success")
        except ValueError as e:
            flash(str(e), "danger")
```

Сервис списка – выборка и пагинация. Источник: app/refdata/services/years/year_services.py, строки 64–72.

```
@no_autoflush
def get_year_list(page, per_page, year_filter=None, sort_by="number", sort_dir="asc"):
    """Получает список годов с пагинацией, фильтрацией и сортировкой."""
    query = year_query(
        year_filter=year_filter,
        sort_by=sort_by,
        sort_dir=sort_dir,
    )
    return query.paginate(page=page, per_page=per_page, error_out=False)
```

Сервис изменения – сохранение и журналирование. Источник: app/refdata/services/years/year_services.py, строки 149–158.

```
 149
 150          db.session.flush()
 151
 152      try:
 153          _commit_with_retry()
 154          return updated_ids
 155      except IntegrityError as e:
 156          db.session.rollback()
 157          raise ValueError(
 158              "Ошибка сохранения данных. Возможно, нарушены уникальные ограничения или внешние
ключи."
```

HTML/Jinja2 списка – элементы формы. Источник: app/templates/refdata/years/year/year.html, строки 15–28.

```
  15      {% endif %}
  16
  17      <form method="GET" action="{{ url_for('refdata_bp.years_list') }}" class="mb-3"
id="filterForm">
  18          <div class="row g-2 align-items-center">
  19              <div class="col-12 col-lg-8">
  20                  <div class="d-flex align-items-center gap-2 flex-wrap flex-lg-nowrap">
  21                      <div class="flex-grow-1" style="min-width: 220px;">
  22                          {{ form.year_filter(class="form-control", value=year_filter,
placeholder="Поиск по году или признаку") }}
  23                      </div>
  24                      <button type="submit" class="btn btn-outline-primary"
onclick="this.form.page.value=1">
  25                          Поиск
  26                      </button>
  27                      <a href="{{ url_for('refdata_bp.years_list') }}" class="btn btn-outline-danger"
data-no-preloader>
  28                          Сбросить
```

Взаимодействие с другими модулями. Годы используются как общая временная ось мощности, топлива, экономики, электропотребления и балансов. Ссылки id_year ведут на Year.id; в отдельных экономических моделях применяется сочетание year_number и database_version_id. Изменение текущего года выполняется отдельно на главной странице и не тождественно добавлению записи Year.

Подтверждающие ссылки на потребителей модели:
app/ecology/models/new_tes_norm_model.py:45 – "Year",
app/economics/models/federal_district_accum_fixed_capital_parameter_model.py:52 – "Year",
app/electrical_intensity/models/federal_district_electrical_intensity_year_parameter_model.py:54 – "Year",
app/energy_balance/models/espp_energy_generation_model.py:6 – - year_number – календарный год (ссылка на Year.number вместе с database_version_id);
app/fuel/models/fue_distribution_parameter_model.py:42 – year = db.relationship("Year", foreign_keys=[id_year])

### 4.1.2. Форма добавления

Адрес: GET, POST /refdata/add_year. Обработчик: add_year().

Назначение: создание записи справочника «Годы». Вводимые поля формы: number, year_feature. Список полей формы может быть уже модели: дополнительные реквизиты изменяются на странице списка или заполняются сервисом.

GET создает форму и подготавливает варианты выбора. POST с валидной формой формирует данные и передает их сервису добавления. При успехе выполняется переход к списку; сообщения о результате выводятся пользователю. Кнопка отмены возвращает к списку без создания записи. Создаваемая сущность – Year, модель и ее связи приведены выше в подразделе 4.1.1.

Поля WTForms и валидаторы: csrf_token = HiddenField(); number = IntegerField('Год', validators=[DataRequired(message='Поле «Год» обязательно.'),

NumberRange(min=1900, max=2200, message='Год должен быть в диапазоне 1900–2200.')]); year_feature = SelectField('Признак года', choices=[], coerce=int, validators=[DataRequired(message='Выберите признак года.')]).

Отдельной кнопки добавления во всех версиях в этой форме нет.

Модель добавляемого объекта Year – основной реквизит. Источник: app/refdata/models/years/year_model.py, строки 27–27.

```
27      number = db.Column(db.Integer, nullable=False, index=True)
```

Маршрут добавления – вход в форму. Источник: app/refdata/routes/years/year_routes.py, строки 220–232.

```
220 @refdata_bp.route("/add_year", methods=["GET", "POST"])
221 @login_required
222 @has_admin_required
223 def add_year():
224     """Страница добавления записи в справочник «Годы»."""
225
226     user = session.get("username", "Неизвестный пользователь")
227     log_to_db(user, "Открыта страница добавления года", entity_type="year")
228
229     version_id = get_current_version()
230     if not version_id:
231         flash("Текущая версия БД не установлена", "danger")
232         return redirect(url_for("refdata_bp.years_list"))
```

Маршрут добавления – вызов сервиса. Источник: app/refdata/routes/years/year_routes.py, строки 251–260.

```
251                 {
252                     "number": form.number.data,
253                     "year_feature_id": form.year_feature.data,
254                 }
255             ]
256             add_year_service(payload, user)
257             flash("Новая запись успешно добавлена.", "success")
258
259             total_records = year_query(year_filter=year_filter).count()
260             last_page = (total_records + per_page - 1) // per_page
```

Сервис добавления – создание сущности. Источник: app/refdata/services/years/year_services.py, строки 201–210.

```
201             if exists:
202                 raise ValueError(f"Год «{number}» уже существует в текущей версии БД.")
203
204             obj = Year(number=number, id_year_feature=feature_id)
205             set_db_version_on_create(obj)
206
207             db.session.add(obj)
208             db.session.flush()
209             created_ids.append(obj.id)
210
```

HTML/Jinja2 формы добавления. Источник: app/templates/refdata/years/year/year_add.html, строки 19–33.

```
19          <div class="mb-3">
20              <label for="number" class="form-label">Год</label>
21              {{ form.number(class="form-control", id="number") }}
22              {% if form.number.errors %}
23                  <div class="text-danger">{{ form.number.errors|join(', ') }}</div>
24              {% endif %}
25          </div>
26
27          <div class="mb-3">
28              <label for="year_feature" class="form-label">Признак года</label>
29              {{ form.year_feature(class="form-select", id="year_feature") }}
30              {% if form.year_feature.errors %}
31                  <div class="text-danger">{{ form.year_feature.errors|join(', ') }}</div>
32              {% endif %}
33          </div>
```

Взаимодействие при добавлении. Годы используются как общая временная ось мощности, топлива, экономики, электропотребления и балансов. Ссылки id_year ведут на Year.id; в отдельных экономических моделях применяется

сочетание year_number и database_version_id. Изменение текущего года выполняется отдельно на главной странице и не тождественно добавлению записи Year. После создания запись становится доступна для выбора в зависимых формах при их следующем построении с учетом версии и используемых механизмов кэширования.

4.1.3. Экспорт, импорт и дополнительные операции

| Адрес и метод | Обработчик | Результат |
|---|---|---|
| GET /refdata/export_years | export_years | Файл Excel для скачивания |

Отдельный маршрут импорта в этом справочнике не зарегистрирован. Поддержка импорта не приписывается всем справочникам по аналогии.

## 4.2. Федеральные округа

4.2.1. Страница списка

Адрес: GET, POST /refdata/federal_district. Обработчик: federal_district_list().

Ведет перечень федеральных округов и варианты их наименований. Предусмотрены поиск, изменение строк таблицы, добавление записей, импорт и экспорт Excel. Округа являются верхним уровнем административной группировки субъектов РФ.

Поля и параметры HTML-формы: per_page, page, sort_by, sort_dir, federal_district_filter, all_versions, federal_district_ids[], display_orders[], federal_district_names[], federal_district_full_names[], federal_district_abr_names[], federal_district_delete[], js/refdata/textarea_autoresize.js.

Обработка: маршрут получает параметры отображения, при POST читает изменяемые строки и флажки удаления, вызывает сервисы и возвращает пользователя к списку с параметрами просмотра. Для обычного просмотра строится выборка и передается в шаблон вместе с пагинацией и вариантами выбора связанных справочников.

Модель: FederalDistrict. Поля SQLAlchemy: id, display_order, name, name_full, name_abr, created_at, updated_at, database_version_id, regional_districts, demand_parameters.

Модель данных – фрагмент объявления. Источник: app/refdata/models/territories/federal_district_model.py, строки 13–35.

```
class FederalDistrict(db.Model, AuditMixin, VersionedModelMixin, RefdataUuidMixin):
    __tablename__ = 'gs_sys_federal_districts'
    __table_args__ = {"schema": SCHEMA_REFDATA}

    id = db.Column(db.Integer, primary_key=True, autoincrement=True)

    # Порядок отображения
    display_order = db.Column(db.Integer, nullable=True)

    name = db.Column(db.String(80), nullable=False, index=True)
    name_full = db.Column(db.String(80), nullable=True)
    name_abr = db.Column(db.String(80), nullable=True)

    created_at = db.Column(db.DateTime(timezone=True), server_default=func.now(), nullable=False)
    updated_at = db.Column(db.DateTime(timezone=True), server_default=func.now(),
onupdate=func.now(), nullable=False)

    # Поле для связи с версией БД
    database_version_id = db.Column(
        db.Integer,
        db.ForeignKey(f"{SCHEMA_REFDATA}.gs_database_versions.id", ondelete="SET NULL"),
        nullable=True,
        index=True
    )
```

Связь модели с другой сущностью. Источник: app/refdata/models/territories/federal_district_model.py, строки 44–44.

```
    regional_districts = db.relationship('RegionalDistrict', back_populates='federal_district')
```

Маршрут списка – вход и параметры. Источник: app/refdata/routes/territories/federal_district_routes.py, строки 36–51.

```
@refdata_bp.route("/federal_district", methods=["GET", "POST"])
@login_required
def federal_district_list():
    """Маршрут для отображения списка федеральных округов."""

    user = session.get('username', 'Неизвестный пользователь')
    log_to_db(user, "Открыта страница федеральных округов", entity_type="federal_district")

    # Создание формы
    form = FederalDistrictFilterForm()

    # Получение параметров запроса
    page = request.args.get("page", 1, type=int)
    per_page = request.args.get("per_page", 25, type=int)
    federal_district_filter = request.args.get("federal_district_filter", "").strip()
    sort_by = request.args.get("sort_by", "display_order")
```

Маршрут списка – передача изменений сервису. Источник: app/refdata/routes/territories/federal_district_routes.py, строки 146–151.

```
                            sort_dir=sort_dir,
                    ))
                update_federal_district_all_versions_service(federal_district_data, user)
                flash("Изменения применены во всех версиях БД (по ref_uuid).", "success")
            else:
                update_federal_district_service(federal_district_data, user)
```

Сервис списка – выборка и пагинация. Источник: app/refdata/services/territories/federal_district_services.py, строки 153–171.

```
153  @no_autoflush
154  def get_federal_district_list(
155      page,
156      per_page,
157      federal_district_filter=None,
158      sort_by="display_order",
159      sort_dir="asc"
160  ):
161      """ Получает список федеральных округов с пагинацией, фильтрацией и сортировкой. """
162
163      # Базовый запрос
164      query = federal_district_query(
165          federal_district_filter=federal_district_filter,
166          sort_by=sort_by,
167          sort_dir=sort_dir,
168      )
169
170      # Пагинация
171      return query.paginate(page=page, per_page=per_page, error_out=False)
```

Сервис изменения – сохранение и журналирование. Источник: app/refdata/services/territories/federal_district_services.py, строки 263–272.

```
263
264      try:
265          # Сохранение изменений в базе данных
266          # Фиксация транзакции (устойчивый коммит)
267          _commit_with_retry()
268
269          if updated_ids:
270              log_to_db(user, "Сохранены изменения по федеральным округам", f"Измененных записей:
{len(updated_ids)} (id: {updated_ids})", entity_type="federal_district")
271          else:
272              log_to_db(user, "Изменений по федеральным округам не обнаружено", "",
entity_type="federal_district")
```

HTML/Jinja2 списка – элементы формы. Источник: app/templates/refdata/territories/federal_district/federal_district.html, строки 236–249.

```
236                                  <!-- Строки таблицы. Столбец "Наименование ФО" -->
237                                  <td>
238                                      <input type="text" name="federal_district_names[]" class="form-
control" value="{{ item.name }}">
239                                  </td>
240
241                                  <!-- Строки таблицы. Столбец "Наименование полное ФО" -->
242                                  <td>
243                                      <input type="text" name="federal_district_full_names[]"
class="form-control" value="{{ item.name_full }}">
244                                  </td>
245
246                                  <!-- Строки таблицы. Столбец "Наименование сокращенное ФО" -->
247                                  <td>
248                                      <input type="text" name="federal_district_abr_names[]"
class="form-control" value="{{ item.name_abr }}">
249                                  </td>
```

Взаимодействие с другими модулями. RegionalDistrict.id_federal_district связывает субъект РФ с округом. Территориальная группировка используется в экономике, электропотреблении, электроемкости и потребности в мощности. Изменение наименования справочника отражается при последующем построении списков и отчетов, но само по себе не является запуском расчета.

Подтверждающие ссылки на потребителей модели:
app/economics/models/federal_district_accum_fixed_capital_parameter_model.py:35 – "FederalDistrict",
app/electrical_intensity/models/federal_district_electrical_intensity_coefficient_model.py:24 – "FederalDistrict",
app/energy_consumption/models/territories/federal_district_energy_consumption_parameter_model.py:26 – "FederalDistrict",
app/power_demand/models/territories/federal_district_demand_parameter_model.py:2 – """Параметры нагрузки (demand) для федерального округа (FederalDistrict)."""

app/fuel/services/fuel_imports/import_fuel_refdata_services.py:46 – from app.refdata.models.territories.federal_district_model import FederalDistrict

### 4.2.2. Форма добавления

Адрес: GET, POST /refdata/add_federal_district. Обработчик: add_federal_district().

Назначение: создание записи справочника «Федеральные округа». Вводимые поля формы: name, name_full, name_abr. Список полей формы может быть уже модели: дополнительные реквизиты изменяются на странице списка или заполняются сервисом.

GET создает форму и подготавливает варианты выбора. POST с валидной формой формирует данные и передает их сервису добавления. При успехе выполняется переход к списку; сообщения о результате выводятся пользователю. Кнопка отмены возвращает к списку без создания записи. Создаваемая сущность – FederalDistrict, модель и ее связи приведены выше в подразделе 4.2.1.

Поля WTForms и валидаторы: csrf_token = HiddenField(); name = StringField('Короткое наименование ФО', validators=[DataRequired(message='Поле «Короткое наименование ФО» обязательно.'), Length(min=3, max=255, message='Длина от 3 до 255 символов.')]); name_full = StringField('Полное наименование ФО', validators=[DataRequired(message='Поле «Полное наименование ФО» обязательно.'), Length(min=2, max=255, message='Длина от 2 до 255 символов.')]); name_abr = StringField('Сокращенное наименование ФО', validators=[DataRequired(message='Поле «Сокращенное наименование ФО» обязательно.'), Length(min=2, max=255, message='Длина от 2 до 255 символов.')]).

Шаблон содержит кнопку добавления во всех версиях для административного пользователя. Серверная ветвь all_versions=1 вызывает специализированную проверку доступа и межверсионный сервис; при обычном добавлении используется выбранная версия.

Модель добавляемого объекта FederalDistrict – основной реквизит. Источник: app/refdata/models/territories/federal_district_model.py, строки 22–22.

```
22      name = db.Column(db.String(80), nullable=False, index=True)
```

Маршрут добавления – вход в форму. Источник: app/refdata/routes/territories/federal_district_routes.py, строки 190–202.

```
190  @refdata_bp.route("/add_federal_district", methods=["GET", "POST"])
191  @login_required
192  def add_federal_district():
193      """ Маршрут для добавления нового федерального округа. """
194
195      user = session.get('username', 'Неизвестный пользователь')
196      log_to_db(user, "Открыта страница добавления федерального округа",
entity_type="federal_district")
197
198      # Создание формы
199      form = AddFederalDistrictForm()
200
201      # Сохранение текущих фильтров и параметров отображения
202      page                         = request.args.get("page", 1, type=int)
```

Маршрут добавления – вызов сервиса. Источник: app/refdata/routes/territories/federal_district_routes.py, строки 235–245.

```
                                            sort_dir=sort_dir,
                        ))
                    add_federal_district_all_versions_service(payload, user)
                    flash("Новая запись добавлена во всех версиях БД (общий ref_uuid).", "success")
                else:
                    add_federal_district_service(payload, user)
                    flash("Новая запись успешно добавлена.", "success")

                # Перенаправление на список с сохранением параметров и переходом к новой записи
                total_records = federal_district_query(
                                    federal_district_filter).count()
```

Сервис добавления – создание сущности. Источник: app/refdata/services/territories/federal_district_services.py, строки 329–350.

```
                    raise ValueError(f"Запись с сокращенным наименованием «{name_abr}» уже
существует.")

                # Создаем новую запись
                obj = FederalDistrict(
                    name=name,
                    name_full=name_full or None,
                    name_abr=name_abr,
                )
                set_db_version_on_create(obj)
                db.session.add(obj)
                db.session.flush()  # получить id без полного коммита

                log_to_db(
                    user,
                    "Создан федеральный округ",
                    (
                        f"Наименование: {name};"
                        f"Полное наименование: {_dash(name_full)};"
                        f"Сокращенное наименование: {_dash(name_abr)}",
                    ),
                    entity_type="federal_district",
                    entity_id=obj.id)
```

HTML/Jinja2 формы добавления. Источник: app/templates/refdata/territories/federal_district/federal_district_add.html, строки 22–36.

```
        <div class="mb-3">
            <label for="name" class="form-label">Наименование ФО</label>
            {{ form.name(class="form-control", id="name") }}

            <label for="name" class="form-label">Полное наименование ФО</label>
            {{ form.name_full(class="form-control", id="name_full") }}

            <label for="name" class="form-label">Сокращенное наименование ФО</label>
            {{ form.name_abr(class="form-control", id="name_abr") }}
            {% if form.name.errors %}
                <div class="text-danger">
                    {{ form.name.errors|join(', ') }}
                </div>
            {% endif %}
        </div>
```

Взаимодействие при добавлении. RegionalDistrict.id_federal_district связывает субъект РФ с округом. Территориальная группировка используется в экономике, электропотреблении, электроемкости и потребности в мощности. Изменение наименования справочника отражается при последующем построении списков и отчетов, но само по себе не является запуском расчета. После создания запись становится доступна для выбора в зависимых формах при их следующем построении с учетом версии и используемых механизмов кэширования.

### 4.2.3. Экспорт, импорт и дополнительные операции

| Адрес и метод | Обработчик | Результат |
|---|---|---|

| Адрес и метод | Обработчик | Результат |
|---|---|---|
| POST /refdata/import_federal_district | import_federal_district | Обработка загруженного файла и сообщение о результате |
| GET /refdata/export_federal_district | export_federal_district | Файл Excel для скачивания |

Отдельный маршрут импорта в этом справочнике предусмотрен; наличие маршрута не означает наличие видимой кнопки загрузки в текущем шаблоне.

### 4.3. Субъекты РФ

4.3.1. Страница списка

Адрес: GET, POST /refdata/regional_district. Обработчик: regional_district_list().

Хранит порядковый номер, краткое и полное наименования субъекта РФ, формы наименования в родительном и дательном падежах, связи с федеральным округом, энергозоной и синхронной зоной. В списке доступны текстовый поиск и фильтры по связанным классификаторам; региональные энергосистемы связаны с субъектами отношением многие-ко-многим.

Поля и параметры HTML-формы: per_page, page, sort_by, sort_dir, federal_district_filter, energy_zone_filter, synchronous_area_filter, regional_district_filter, all_versions, regional_district_ids[], region_ids[], regional_district_names[], regional_district_full_names[], regional_district_rp_names[], regional_district_dp_names[], federal_districts[], energy_zones[], synchronous_areas[], regional_district_delete[], js/refdata/textarea_autoresize.js.

Обработка: маршрут получает параметры отображения, при POST читает изменяемые строки и флажки удаления, вызывает сервисы и возвращает пользователя к списку с параметрами просмотра. Для обычного просмотра строится выборка и передается в шаблон вместе с пагинацией и вариантами выбора связанных справочников.

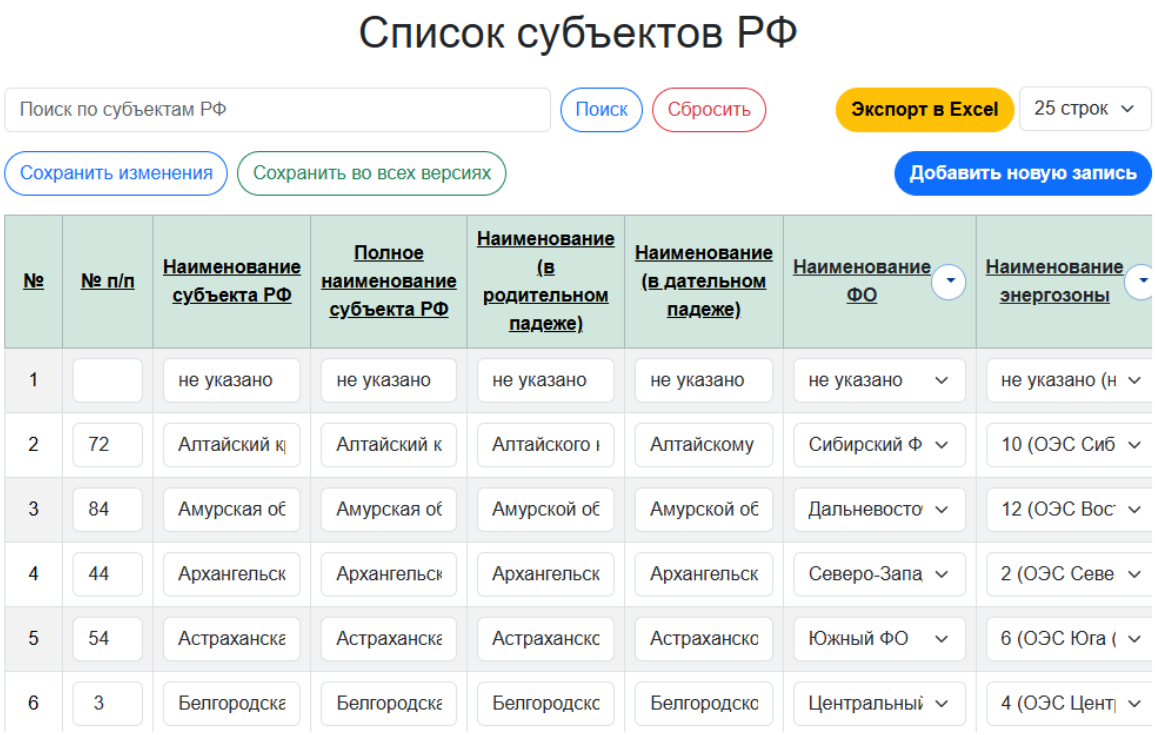



Рисунок 3. Субъекты РФ. Действующий сайт, 29.10.2026. Показан фрагмент экранной формы.

Модель: RegionalDistrict. Поля SQLAlchemy: id, region_number, region_id, name, name_full, name_rp, name_dp, id_federal_district, federal_district, regional_energy_systems, stations, energy_areas, energy_units, id_energy_zone, energy_zone, id_synchronous_area, synchronous_area, created_at, updated_at, database_version_id, demand_parameters.

Модель данных – фрагмент объявления. Источник: app/refdata/models/territories/regional_district_model.py, строки 14–32.

```
14  class RegionalDistrict(db.Model, AuditMixin, VersionedModelMixin, RefdataUuidMixin):
15      __tablename__ = 'gs_sys_regional_districts'
16      __table_args__ = {"schema": SCHEMA_REFDATA}
17
18      # Идентификатор субъекта РФ
19      id = db.Column(db.Integer, primary_key=True, autoincrement=True)
20
21      # Номер региона РФ (может быть пустым)
22      region_number = db.Column(db.String(3), nullable=True, index=True)
23
24      # Номер порядковый (может быть пустым)
25      region_id = db.Column(db.String(3), nullable=True, index=True)
26
27      # Наименование сокращенное
28      name = db.Column(db.String(80), nullable=False, index=True)
29
30      # Полное наименование
31      name_full = db.Column(db.String(255), nullable=False)
32
```

Связь модели с другой сущностью. Источник: app/refdata/models/territories/regional_district_model.py, строки 46–46.

```
46      federal_district = db.relationship('FederalDistrict', back_populates='regional_districts')
```

Маршрут списка – вход и параметры. Источник: app/refdata/routes/territories/regional_district_routes.py, строки 42–57.

```
 42  @refdata_bp.route("/regional_district", methods=["GET", "POST"])
 43  @login_required
 44  def regional_district_list():
 45      """Маршрут для отображения списка субъектов РФ."""
 46
 47      user = session.get('username', 'Неизвестный пользователь')
 48      log_to_db(user, "Открыта страница субъектов РФ", entity_type="regional_district")
 49
 50      # Создание формы
 51      form = RegionalDistrictFilterForm()
 52
 53      # Получение параметров запроса
 54      page                        = request.args.get("page", 1, type=int)
 55      per_page                    = request.args.get("per_page", 25, type=int)
 56      sort_by                     = request.args.get("sort_by", "id")
 57      sort_dir                    = request.args.get("sort_dir", "asc")
```

Маршрут списка – передача изменений сервису. Источник: app/refdata/routes/territories/regional_district_routes.py, строки 175–180.

```
175                             synchronous_area_filter=synchronous_area_filter,
176                         ))
177                 update_regional_district_all_versions_service(regional_district_data, user)
178                 flash("Изменения применены во всех версиях БД (по ref_uuid).", "success")
179             else:
180                 update_regional_district_service(regional_district_data, user)
```

Сервис списка – выборка и пагинация. Источник: app/refdata/services/territories/regional_district_services.py, строки 193–214.

```
193  @no_autoflush
194  def get_regional_district_list(
195      page,
196      per_page,
197      regional_district_filter=None,
198      federal_district_filter=None,
199      energy_zone_filter=None,
200      synchronous_area_filter=None,
201      sort_by="id",
202      sort_dir="asc",
203  ):
204      """ Получает список субъектов РФ с пагинацией, фильтрацией и сортировкой. """
205
206      # Базовый запрос
207      query = regional_district_query(
208          regional_district_filter=regional_district_filter,
209          federal_district_filter=federal_district_filter,
210          energy_zone_filter=energy_zone_filter,
211          synchronous_area_filter=synchronous_area_filter,
212          sort_by=sort_by,
213          sort_dir=sort_dir,
214      )
```

Сервис изменения – сохранение и журналирование. Источник: app/refdata/services/territories/regional_district_services.py, строки 386–395.

```
386
387      try:
388          # Сохранение изменений в базе данных
389          # Фиксация транзакции (устойчивый коммит)
390          _commit_with_retry()
391
392          if updated_ids:
393              log_to_db(user, "Сохранены изменения по субъектам РФ",
394                        f"Измененных записей: {len(updated_ids)} (id: {updated_ids})",
entity_type="regional_district")
395          else:
```

HTML/Jinja2 списка – элементы формы. Источник:
app/templates/refdata/territories/regional_district/regional_district.html, строки 493–506.

```
 493                                    <!-- Строки таблицы. Столбец "Субъект РФ" -->
 494                                    <td>
 495                                        <input type="text" name="regional_district_names[]"
class="form-control" value="{{ item.name }}">
 496                                    </td>
 497
 498                                    <!-- Строки таблицы. Столбец "Субъект РФ (полное наименование)" -->
 499                                    <td>
 500                                        <input type="text" name="regional_district_full_names[]"
class="form-control" value="{{ item.name_full }}">
 501                                    </td>
 502
 503                                    <!-- Строки таблицы. Столбец "Наименование (в родительном падеже)"
-->
 504                                    <td>
 505                                        <input type="text" name="regional_district_rp_names[]"
class="form-control" value="{{ item.name_rp }}">
 506                                    </td>
```

Взаимодействие с другими модулями. Субъекты РФ используются станциями, энергорайонами, энергоузлами, показателями потребности в мощности и территориальными расчетами. Сервис _sync_energy_zone_aggregates_for_ids обновляет территориальные агрегаты энергозон через общий сервис синхронизации при изменении соответствующих связей. При смене внешнего ключа меняется принадлежность объекта к расчетной группировке.

Подтверждающие ссылки на потребителей модели:
app/energy_balance/models/regional_energy_system_transfer_model.py:5 – Источник перетока – RegionalEnergySystem или RegionalDistrict (субъект РФ), ровно одна связь.
app/energy_consumption/models/territories/regional_district_energy_consumption_parameter_model.py:26 – "RegionalDistrict",
app/fuel/models/fue_equipment_group_fuel_param_model.py:23 – from app.refdata.models.territories.regional_district_model import RegionalDistrict
app/generation/models/station/station_model.py:85 – regional_district = db.relationship('RegionalDistrict', back_populates='stations')
app/power_demand/models/territories/regional_district_demand_parameter_model.py:2 – """Параметры нагрузки (demand) для субъекта РФ (RegionalDistrict)."""

### 4.3.2. Форма добавления

Адрес: GET, POST /refdata/add_regional_district. Обработчик: add_regional_district().

Назначение: создание записи справочника «Субъекты РФ». Вводимые поля формы: name, name_full, name_rp, name_dp, federal_district. Список полей формы может быть уже модели: дополнительные реквизиты изменяются на странице списка или заполняются сервисом.

GET создает форму и подготавливает варианты выбора. POST с валидной формой формирует данные и передает их сервису добавления. При успехе выполняется переход к списку; сообщения о результате выводятся пользователю. Кнопка отмены возвращает к списку без создания записи. Создаваемая сущность – RegionalDistrict, модель и ее связи приведены выше в подразделе 4.3.1.

Поля WTForms и валидаторы: csrf_token = HiddenField(); name = StringField('Наименование субъекта РФ', validators=[DataRequired(message='Поле «Наименование субъекта РФ» обязательно.'), Length(min=3, max=255, message='Длина от 3 до 255 символов.')]); name_full = StringField('Полное наименование субъекта РФ', validators=[DataRequired(message='Поле «Полное наименование субъекта РФ» обязательно.'), Length(min=3, max=255, message='Длина от 3 до 255 символов.')]); name_rp = StringField('Наименование (в

родительном падеже)', validators=[DataRequired(message='Поле «Наименование (в родительном падеже)» обязательно.'), Length(min=3, max=255, message='Длина от 3 до 255 символов.')]); name_dp = StringField('Наименование (в дательном падеже)', validators=[DataRequired(message='Поле «Наименование (в дательном падеже)» обязательно.'), Length(min=3, max=255, message='Длина от 3 до 255 символов.')]); federal_district = SelectField('Федеральный округ', choices=[], coerce=int, validators=[DataRequired(message='Выберите федеральный округ.')]); energy_zone = SelectField('Энергозона', choices=[], coerce=int, validators=[Optional()]); synchronous_area = SelectField('Синхронная зона', choices=[], coerce=int, validators=[Optional()]).

Шаблон содержит кнопку добавления во всех версиях для административного пользователя. Серверная ветвь all_versions=1 вызывает специализированную проверку доступа и межверсионный сервис; при обычном добавлении используется выбранная версия.

Модель добавляемого объекта RegionalDistrict – основной реквизит. Источник: app/refdata/models/territories/regional_district_model.py, строки 28–28.

```
28      name = db.Column(db.String(80), nullable=False, index=True)
```

Маршрут добавления – вход в форму. Источник: app/refdata/routes/territories/regional_district_routes.py, строки 245–257.

```
245 @refdata_bp.route("/add_regional_district", methods=["GET", "POST"])
246 @login_required
247 def add_regional_district():
248     """ Маршрут для добавления нового субъекта РФ."""
249
250     user = session.get("username", "Неизвестный пользователь")
251     log_to_db(user, "Открыта страница добавления субъекта РФ", entity_type="regional_district")
252
253     # Создание формы
254     form = AddRegionalDistrictForm()
255
256     # Сохранение текущих фильтров и параметров отображения
257     page                        = request.args.get("page", 1, type=int)
```

Маршрут добавления – вызов сервиса. Источник: app/refdata/routes/territories/regional_district_routes.py, строки 302–315.

```
302                             synchronous_area_filter=synchronous_area_filter,
303                     ))
304                 add_regional_district_all_versions_service(payload, user)
305                 flash("Новая запись добавлена во всех версиях БД (общий ref_uuid).", "success")
306             else:
307                 add_regional_district_service(payload, user)
308                 flash("Новая запись успешно добавлена.", "success")
309
310             # Перенаправление на список с сохранением параметров и переходом к новой записи
311             total_records = regional_district_query(
312                                 regional_district_filter,
313                                 federal_district_filter,
314                                 energy_zone_filter,
315                                 synchronous_area_filter).count()
```

Сервис добавления – создание сущности. Источник:
app/refdata/services/territories/regional_district_services.py, строки 457–482.

```
                    raise ValueError(f"Запись с полным наименованием «{name_full}» уже
существует.")

                # Создаем новую запись
                obj = RegionalDistrict(
                    name=name,
                    name_full=name_full or None,
                    name_rp=name_rp or None,
                    name_dp=name_dp or None,
                    id_federal_district=federal_district_id,
                )
                set_db_version_on_create(obj)
                db.session.add(obj)
                db.session.flush()  # получить id без полного коммита

                log_to_db(
                    user,
                    "Создан субъект РФ",
                    (
                        f"Наименование: {name};"
                        f"Полное наименование: {_dash(name_full)};"
                        f"Наименование (в родительном падеже): {_dash(name_rp)};"
                        f"Наименование (в дательном падеже): {_dash(name_dp)};"
                        f"Федеральный округ: {get_federal_district_name(federal_district_id)}",
                    ),
                    entity_type="regional_district",
                    entity_id=obj.id)
```

HTML/Jinja2 формы добавления. Источник:
app/templates/refdata/territories/regional_district/regional_district_add.html, строки 23–37.

```
        <div class="mb-3">
            <label for="name" class="form-label">Наименование субъекта РФ</label>
            {{ form.name(class="form-control", id="name") }}

            <!-- Ошибки валидации -->
            {% if form.name.errors %}
                <div class="text-danger">
                    {{ form.name.errors|join(', ') }}
                </div>
            {% endif %}
        </div>

        <!-- Полное наименование субъекта РФ -->
        <div class="mb-3">
            <label for="name_full" class="form-label">Полное наименование субъекта РФ</label>
```

Взаимодействие при добавлении. Субъекты РФ используются станциями, энергорайонами, энергоузлами, показателями потребности в мощности и территориальными расчетами. Сервис _sync_energy_zone_aggregates_for_ids обновляет территориальные агрегаты энергозон через общий сервис синхронизации при изменении соответствующих связей. При смене внешнего ключа меняется принадлежность объекта к расчетной группировке. После создания запись становится доступна для выбора в зависимых формах при их следующем построении с учетом версии и используемых механизмов кэширования.

В текущей форме создания субъекта РФ вводятся наименования и ФО. Связи с энергозоной и синхронной зоной редактируются в списке; наличие этих полей в модели не означает их наличие в форме добавления.

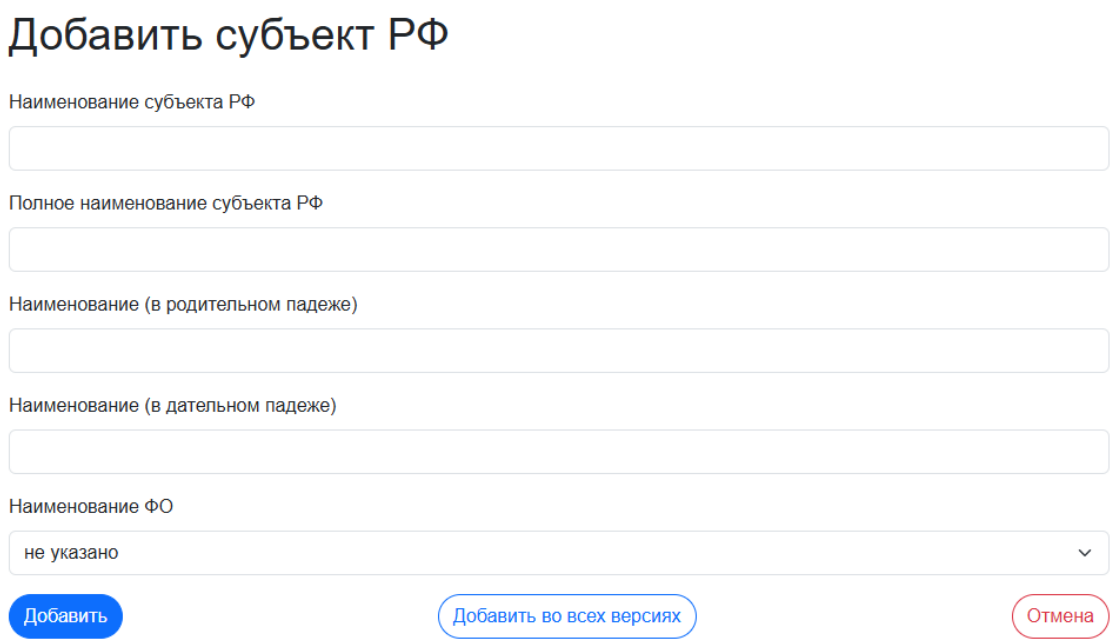



Рисунок 4. Форма добавления субъекта РФ. Действующий сайт, 29.10.2026. Показан фрагмент экранной формы.

4.3.3. Экспорт, импорт и дополнительные операции

| Адрес и метод | Обработчик | Результат |
|---|---|---|
| POST /refdata/import_regional_district | import_regional_district | Обработка загруженного файла и сообщение о результате |
| GET /refdata/export_regional_district | export_regional_district | Файл Excel для скачивания |

Отдельный маршрут импорта в этом справочнике предусмотрен; наличие маршрута не означает наличие видимой кнопки загрузки в текущем шаблоне.

## 4.4. Зарубежные страны, имеющие общие границы с РФ

4.4.1. Страница списка

Адрес: GET, POST /refdata/foreign_border_country. Обработчик: foreign_border_country_list().

Ведет упорядоченный перечень сопредельных зарубежных стран. Список поддерживает редактирование названия и порядка отображения; отдельная форма служит для добавления страны.

Поля и параметры HTML-формы: per_page, page, sort_by, sort_dir, foreign_border_country_filter, all_versions, foreign_border_country_ids[], display_orders[], foreign_border_country_names[], foreign_border_country_delete[], js/refdata/textarea_autoresize.js.

Обработка: маршрут получает параметры отображения, при POST читает изменяемые строки и флажки удаления, вызывает сервисы и возвращает пользователя к списку с параметрами просмотра. Для обычного просмотра строится выборка и передается в шаблон вместе с пагинацией и вариантами выбора связанных справочников.

Модель: ForeignBorderCountry. Поля SQLAlchemy: id, display_order, name, created_at, updated_at, database_version_id.

Модель данных – фрагмент объявления. Источник: app/refdata/models/territories/foreign_border_country_model.py, строки 13–31.

```
  13  class ForeignBorderCountry(db.Model, AuditMixin, VersionedModelMixin, RefdataUuidMixin):
  14      __tablename__ = 'gs_sys_foreign_border_countries'
  15      __table_args__ = (
  16          db.UniqueConstraint(
  17              'database_version_id',
  18              'name',
  19              name='uq_gs_sys_foreign_border_countries_ver_name',
  20          ),
  21          {"schema": SCHEMA_REFDATA},
  22      )
  23
  24      id = db.Column(db.Integer, primary_key=True, autoincrement=True)
  25
  26      # Порядок отображения
  27      display_order = db.Column(db.Integer, nullable=True)
  28
  29      # Наименование
  30      name = db.Column(db.String(255), nullable=False, index=True)
  31
```

Маршрут списка – вход и параметры. Источник: app/refdata/routes/territories/foreign_border_country_routes.py, строки 37–52.

```
  37  @refdata_bp.route("/foreign_border_country", methods=["GET", "POST"])
  38  @login_required
  39  def foreign_border_country_list():
  40      """Маршрут для отображения списка зарубежных стран."""
  41
  42      user = session.get('username', 'Неизвестный пользователь')
  43      log_to_db(
  44          user,
  45          "Открыта страница зарубежных стран, имеющих общие границы с РФ",
  46          entity_type="foreign_border_country",
  47      )
  48
  49      form = ForeignBorderCountryFilterForm()
  50
  51      page = request.args.get("page", 1, type=int)
  52      per_page = request.args.get("per_page", 25, type=int)
```

Маршрут списка – передача изменений сервису. Источник: app/refdata/routes/territories/foreign_border_country_routes.py, строки 147–152.

```
 147                          sort_dir=sort_dir,
 148                      ))
 149                  update_foreign_border_country_all_versions_service(foreign_border_country_data,
user)
 150                  flash("Изменения применены во всех версиях БД (по ref_uuid).", "success")
 151              else:
 152                  update_foreign_border_country_service(foreign_border_country_data, user)
```

Сервис списка – выборка и пагинация. Источник: app/refdata/services/territories/foreign_border_country_services.py, строки 71–86.

```
@no_autoflush
def get_foreign_border_country_list(
    page,
    per_page,
    foreign_border_country_filter=None,
    sort_by="display_order",
    sort_dir="asc",
):
    """Получает список стран с пагинацией, фильтрацией и сортировкой."""

    query = foreign_border_country_query(
        foreign_border_country_filter=foreign_border_country_filter,
        sort_by=sort_by,
        sort_dir=sort_dir,
    )
    return query.paginate(page=page, per_page=per_page, error_out=False)
```

Сервис изменения – сохранение и журналирование. Источник: app/refdata/services/territories/foreign_border_country_services.py, строки 185–198.

```

        db.session.flush()

    try:
        _commit_with_retry()

        if updated_ids:
            log_to_db(
                user,
                "Сохранены изменения по зарубежным странам",
                f"Измененных записей: {len(updated_ids)} (id: {updated_ids})",
                entity_type="foreign_border_country",
                entity_id=foreign_border_country_id,
            )
```

HTML/Jinja2 списка – элементы формы. Источник: app/templates/refdata/territories/foreign_border_country/foreign_border_country.html, строки 164–177.

```

                                <td>
                                    <input type="text" name="foreign_border_country_names[]"
class="form-control" value="{{ country.name }}">
                                </td>

                                <td class="text-center align-middle">
                                    <input type="checkbox" name="foreign_border_country_delete[]"
value="{{ country.id }}" class="form-check-input">
                                </td>
                            </tr>
                            {% else %}
                            <tr>
                                <td colspan="4" class="text-center align-middle">Нет данных для
отображения</td>
                            </tr>
                            {% endfor %}
```

Взаимодействие с другими модулями. Записи используются энергетическими балансами при описании межгосударственных перетоков и импортно-экспортных связей. Связь с конкретным потребителем подтверждается перечнем ссылок на модель в этом разделе; изменение записи справочника не создает автоматически строку баланса.

Подтверждающие ссылки на потребителей модели:
app/energy_balance/models/ee_import_export_oes_value_model.py:68 – foreign_border_country = db.relationship("ForeignBorderCountry")
app/energy_balance/models/regional_energy_system_transfer_model.py:6 – Получатель – RegionalEnergySystem, ForeignBorderCountry, RegionalDistrict или EnergyUnit
app/energy_balance/services/ee_import_export_value_services.py:21 – from app.refdata.models.territories.foreign_border_country_model import ForeignBorderCountry

app/energy_balance/services/ee_transfers_page_services.py:46 – from app.refdata.models.territories.foreign_border_country_model import ForeignBorderCountry
app/energy_balance/services/ee_transfers_resolve_services.py:13 – from app.refdata.models.territories.foreign_border_country_model import ForeignBorderCountry

### 4.4.2. Форма добавления

Адрес: GET, POST /refdata/add_foreign_border_country. Обработчик: add_foreign_border_country().

Назначение: создание записи справочника «Зарубежные страны, имеющие общие границы с РФ». Вводимые поля формы: name. Список полей формы может быть уже модели: дополнительные реквизиты изменяются на странице списка или заполняются сервисом.

GET создает форму и подготавливает варианты выбора. POST с валидной формой формирует данные и передает их сервису добавления. При успехе выполняется переход к списку; сообщения о результате выводятся пользователю. Кнопка отмены возвращает к списку без создания записи. Создаваемая сущность – ForeignBorderCountry, модель и ее связи приведены выше в подразделе 4.4.1.

Поля WTForms и валидаторы: csrf_token = HiddenField(); name = StringField('Наименование страны', validators=[DataRequired(message='Поле «Наименование страны» обязательно.'), Length(min=2, max=255, message='Длина от 2 до 255 символов.')]).

Шаблон содержит кнопку добавления во всех версиях для административного пользователя. Серверная ветвь all_versions=1 вызывает специализированную проверку доступа и межверсионный сервис; при обычном добавлении используется выбранная версия.

Модель добавляемого объекта ForeignBorderCountry – основной реквизит. Источник: app/refdata/models/territories/foreign_border_country_model.py, строки 30–30.

```
30      name = db.Column(db.String(255), nullable=False, index=True)
```

Маршрут добавления – вход в форму. Источник: app/refdata/routes/territories/foreign_border_country_routes.py, строки 189–201.

```
189 @refdata_bp.route("/add_foreign_border_country", methods=["GET", "POST"])
190 @login_required
191 def add_foreign_border_country():
192     """Маршрут для добавления новой зарубежной страны."""
193
194     user = session.get('username', 'Неизвестный пользователь')
195     log_to_db(
196         user,
197         "Открыта страница добавления зарубежных стран",
198         entity_type="foreign_border_country",
199     )
200
201     form = AddForeignBorderCountryForm()
```

Маршрут добавления – вызов сервиса. Источник: app/refdata/routes/territories/foreign_border_country_routes.py, строки 233–242.

```
233                         sort_dir=sort_dir,
234                     ))
235                 add_foreign_border_country_all_versions_service(payload, user)
236                 flash("Новая запись добавлена во всех версиях БД (общий ref_uuid).", "success")
237             else:
238                 add_foreign_border_country_service(payload, user)
239                 flash("Новая запись успешно добавлена.", "success")
240
241             total_records = foreign_border_country_query(foreign_border_country_filter).count()
242             last_page = (total_records + per_page - 1) // per_page
```

Сервис добавления – создание сущности. Источник:
app/refdata/services/territories/foreign_border_country_services.py, строки 272–286.

```
                                f"Запись с порядком отображения «{display_order}» уже существует."
                            )

                obj = ForeignBorderCountry(name=name, display_order=display_order)
                set_db_version_on_create(obj)
                db.session.add(obj)
                db.session.flush()

                log_to_db(
                    user,
                    "Создана запись зарубежной страны",
                    f"Наименование: {name}",
                    entity_type="foreign_border_country",
                    entity_id=obj.id,
                )
```

HTML/Jinja2 формы добавления. Источник:
app/templates/refdata/territories/foreign_border_country/foreign_border_country_add.html, строки 19–33.

```
        <div class="mb-3">
            <label for="name" class="form-label">Наименование страны</label>
            {{ form.name(class="form-control", id="name") }}
            {% if form.name.errors %}
                <div class="text-danger">{{ form.name.errors|join(', ') }}</div>
            {% endif %}
        </div>

        <div class="mt-4 d-flex flex-wrap gap-2 align-items-center">
            <button type="submit" class="btn btn-primary">Добавить</button>
            {% if current_user.has_admin %}<button type="submit" name="all_versions" value="1"
class="btn btn-outline-primary" title="Добавить запись во все версии БД">Добавить во всех
версиях</button>{% endif %}
            <a href="{{ url_for('refdata_bp.foreign_border_country_list',
                                page=page,
                                per_page=per_page,
                                sort_by=sort_by,
```

Взаимодействие при добавлении. Записи используются энергетическими балансами при описании межгосударственных перетоков и импортно-экспортных связей. Связь с конкретным потребителем подтверждается перечнем ссылок на модель в этом разделе; изменение записи справочника не создает автоматически строку баланса. После создания запись становится доступна для выбора в зависимых формах при их следующем построении с учетом версии и используемых механизмов кэширования.

4.4.3. Экспорт, импорт и дополнительные операции

| Адрес и метод | Обработчик | Результат |
|---|---|---|
| GET /refdata/export_foreign_border_country | export_foreign_border_country | Файл Excel для скачивания |

Отдельный маршрут импорта в этом справочнике не зарегистрирован. Поддержка импорта не приписывается всем справочникам по аналогии.

## 4.5. Объединенные энергосистемы

4.5.1. Страница списка

Адрес: GET, POST /refdata/union_energy_system. Обработчик: union_energy_system_list().

Хранит краткое и полное наименования ОЭС, порядок отображения и принадлежность к типу энергосистемы. Предусмотрены фильтрация, табличное редактирование, создание, импорт и экспорт данных.

Поля и параметры HTML-формы: per_page, page, sort_by, sort_dir, energy_system_type_filter, union_energy_system_filter, all_versions, union_energy_system_ids[], display_orders[], union_energy_system_names[], union_energy_system_full_names[], energy_system_types[], union_energy_system_delete[], js/refdata/textarea_autoresize.js.

Обработка: маршрут получает параметры отображения, при POST читает изменяемые строки и флажки удаления, вызывает сервисы и возвращает пользователя к списку с параметрами просмотра. Для обычного просмотра строится выборка и передается в шаблон вместе с пагинацией и вариантами выбора связанных справочников.

Модель: UnionEnergySystem. Поля SQLAlchemy: id, display_order, name, name_full, id_energy_system_type, created_at, updated_at, database_version_id, regional_energy_systems, energy_system_type, demand_parameters.

Модель данных – фрагмент объявления. Источник: app/refdata/models/energy_systems/union_energy_system_model.py, строки 15–33.

```
15  class UnionEnergySystem(db.Model, AuditMixin, VersionedModelMixin, RefdataUuidMixin):
16      __tablename__ = 'gs_sys_union_energy_systems'
17      __table_args__ = {"schema": SCHEMA_REFDATA}
18
19      id = db.Column(db.Integer, primary_key=True, autoincrement=True)
20
21      # Порядок отображения
22      display_order = db.Column(db.Integer, nullable=True)
23
24      # Наименование и полное наименование
25      name = db.Column(db.String(80), nullable=False, index=True)
26
27      # Устанавливаем по умолчанию name_full := name на уровне Python-контекста вставки
28      name_full = db.Column(
29          db.String(80),
30          nullable=False,
31          default=lambda context: context.get_current_parameters().get("name")
32      )
33
```

Связь модели с другой сущностью. Источник: app/refdata/models/energy_systems/union_energy_system_model.py, строки 60–63.

```
60      regional_energy_systems = db.relationship(
61          'RegionalEnergySystem',
62          back_populates='union_energy_system'
63      )
```

Маршрут списка – вход и параметры. Источник: app/refdata/routes/energy_systems/union_energy_system_routes.py, строки 40–55.

```
40  @refdata_bp.route("/union_energy_system", methods=["GET", "POST"])
41  @login_required
42  def union_energy_system_list():
43      """Маршрут для отображения списка ОЭС."""
44
45      user = session.get('username', 'Неизвестный пользователь')
46      log_to_db(
47          user,
48          "Открыта страница ОЭС",
49          entity_type="union_energy_system")
50
51      # Создание формы
52      form = UnionEnergySystemFilterForm()
53
54      # Получение параметров запроса
55      page                        = request.args.get("page", 1, type=int)
```

Маршрут списка – передача изменений сервису. Источник: app/refdata/routes/energy_systems/union_energy_system_routes.py, строки 155–160.

```
                            sort_dir=sort_dir,
                        ))
                update_union_energy_system_all_versions_service(union_energy_system_data, user)
                flash("Изменения применены во всех версиях БД (по ref_uuid).", "success")
            else:
                update_union_energy_system_service(union_energy_system_data, user)
```

Сервис списка – выборка и пагинация. Источник: app/refdata/services/energy_systems/union_energy_system_services.py, строки 184–203.

```
@no_autoflush
def get_union_energy_system_list(
    page,
    per_page,
    union_energy_system_filter=None,
    energy_system_type_filter=None,
    sort_by="display_order",
    sort_dir="asc"):
    """ Получает список ОЭС с пагинацией, фильтрацией и сортировкой. """

    # Базовый запрос
    query = union_energy_system_query(
        union_energy_system_filter=union_energy_system_filter,
        energy_system_type_filter=energy_system_type_filter,
        sort_by=sort_by,
        sort_dir=sort_dir,
    )

    # Пагинация
    return query.paginate(page=page, per_page=per_page, error_out=False)
```

Сервис изменения – сохранение и журналирование. Источник: app/refdata/services/energy_systems/union_energy_system_services.py, строки 318–329.

```

    try:
        # Сохранение изменений в базе данных
        # Фиксация транзакции (устойчивый коммит)
        _commit_with_retry()

        if updated_ids:
            log_to_db(
                user,
                "Сохранены изменения по ОЭС",
                f"Измененных записей: {len(updated_ids)} (id: {updated_ids})",
                entity_type="union_energy_system")
```

HTML/Jinja2 списка – элементы формы. Источник: app/templates/refdata/energy_systems/union_energy_system/union_energy_system.html, строки 9–22.

```
    <h1 class="text-center mb-4">Список объединенных энергосистем</h1>

    <form method="GET"
        action="{{ url_for('refdata_bp.union_energy_system_list') }}"
        class="mb-3"
        id="filterForm">

        <div class="row g-2 align-items-center">
            <!-- ЛЕВАЯ СТОРОНА: поиск + кнопки -->
            <div class="col-12 col-lg-8">
                <div class="d-flex align-items-center gap-2 flex-wrap flex-lg-nowrap">
                    <div class="flex-grow-1" style="min-width: 220px;">
                        {{ form.union_energy_system_filter(
                            class="form-control",
```

Взаимодействие с другими модулями. RegionalEnergySystem.id_union_energy_system задает принадлежность РЭС к ОЭС. Иерархия используется для группировки генерации, электропотребления, потребности в мощности и энергетических балансов. Наименования и внешние

ключи справочника используются повторно, без дублирования классификатора в расчетных модулях.

Подтверждающие ссылки на потребителей модели:
app/energy_balance/models/ee_import_export_oes_value_model.py:67 – union_energy_system = db.relationship("UnionEnergySystem")
app/energy_consumption/models/energy_systems/union_energy_system_energy_consumption_parameter_model.py:26 – "UnionEnergySystem",
app/fuel/models/fue_distribution_parameter_model.py:28 – "UnionEnergySystem",
app/power_demand/models/energy_systems/union_energy_system_demand_parameter_model.py:2 – """Параметры нагрузки (demand) для объединенной энергосистемы (UnionEnergySystem)."""
app/api/services/generation_objects_exchange_services.py:37 – from app.refdata.models.energy_systems.union_energy_system_model import UnionEnergySystem

### 4.5.2. Форма добавления

Адрес: GET, POST /refdata/add_union_energy_system. Обработчик: add_union_energy_system().

Назначение: создание записи справочника «Объединенные энергосистемы». Вводимые поля формы: name, name_full, energy_system_type. Список полей формы может быть уже модели: дополнительные реквизиты изменяются на странице списка или заполняются сервисом.

GET создает форму и подготавливает варианты выбора. POST с валидной формой формирует данные и передает их сервису добавления. При успехе выполняется переход к списку; сообщения о результате выводятся пользователю. Кнопка отмены возвращает к списку без создания записи. Создаваемая сущность – UnionEnergySystem, модель и ее связи приведены выше в подразделе 4.5.1.

Поля WTForms и валидаторы: csrf_token = HiddenField(); name = StringField('Наименование ОЭС', validators=[DataRequired(message='Поле «Наименование ОЭС» обязательно.'), Length(min=3, max=255, message='Длина от 3 до 255 символов.')]); name_full = StringField('Полное наименование ОЭС', validators=[DataRequired(message='Поле «Полное наименование ОЭС» обязательно.'), Length(min=3, max=255, message='Длина от 3 до 255 символов.')]); energy_system_type = SelectField('Тип энергосистемы', choices=[], coerce=int, validators=[DataRequired(message='Выберите тип энергосистемы.')]).

Шаблон содержит кнопку добавления во всех версиях для административного пользователя. Серверная ветвь all_versions=1 вызывает специализированную проверку доступа и межверсионный сервис; при обычном добавлении используется выбранная версия.

Модель добавляемого объекта UnionEnergySystem – основной реквизит. Источник: app/refdata/models/energy_systems/union_energy_system_model.py, строки 25–25.

```
  25      name = db.Column(db.String(80), nullable=False, index=True)
```

Маршрут добавления – вход в форму. Источник: app/refdata/routes/energy_systems/union_energy_system_routes.py, строки 203–215.

```
 203  @refdata_bp.route("/add_union_energy_system", methods=["GET", "POST"])
 204  @login_required
 205  def add_union_energy_system():
 206      """ Маршрут для добавления новой ОЭС. """
 207
 208      user = session.get('username', 'Неизвестный пользователь')
 209      log_to_db(
 210          user,
 211          "Открыта страница добавления ОЭС",
 212          entity_type="union_energy_system")
 213
 214      # Создание формы
 215      form = AddUnionEnergySystemForm()
```

Маршрут добавления – вызов сервиса. Источник: app/refdata/routes/energy_systems/union_energy_system_routes.py, строки 258–269.

```
                            sort_dir=sort_dir,
                        ))
                add_union_energy_system_all_versions_service(payload, user)
                flash("Новая запись добавлена во всех версиях БД (общий ref_uuid).", "success")
            else:
                add_union_energy_system_service(payload, user)
                flash("Новая запись успешно добавлена.", "success")

            # Перенаправление на список с сохранением параметров и переходом к новой записи
            total_records = union_energy_system_query(
                                union_energy_system_filter,
                                energy_system_type_filter).count()
```

Сервис добавления – создание сущности. Источник: app/refdata/services/energy_systems/union_energy_system_services.py, строки 407–430.

```
                        raise ValueError(f"Запись с порядком отображения «{display_order}» уже
существует.")

                # Создаем новую запись
                obj = UnionEnergySystem(
                    display_order=display_order,
                    name=name,
                    name_full=name_full or None,
                    id_energy_system_type=energy_system_type_id,
                )
                set_db_version_on_create(obj)
                db.session.add(obj)
                db.session.flush()  # получить id без полного коммита

                log_to_db(
                    user,
                    "Создана ОЭС",
                    (
                        f"Порядок отображения: {display_order};"
                        f"Наименование: {name}; "
                        f"Полное наименование: {_dash(name_full)};"
                        f"Часть энергосистемы России:
{get_energy_system_type_name(energy_system_type_id)}"
                    ),
                    entity_type="union_energy_system",
                    entity_id=obj.id)
```

HTML/Jinja2 формы добавления. Источник: app/templates/refdata/energy_systems/union_energy_system/union_energy_system_add.html, строки 23–37.

```
        <div class="mb-3">
            <label for="name" class="form-label">Наименование ОЭС</label>
            {{ form.name(class="form-control", id="name") }}

            <!-- Ошибки валидации -->
            {% if form.name.errors %}
                <div class="text-danger">
                    {{ form.name.errors|join(', ') }}
                </div>
            {% endif %}
        </div>

        <!-- Полное наименование ОЭС -->
        <div class="mb-3">
            <label for="name_full" class="form-label">Полное наименование ОЭС</label>
```

Взаимодействие при добавлении. RegionalEnergySystem.id_union_energy_system задает принадлежность РЭС к ОЭС. Иерархия используется для группировки генерации, электропотребления, потребности в мощности и энергетических балансов. Наименования и внешние ключи справочника используются повторно, без дублирования классификатора в расчетных модулях. После создания запись становится доступна для выбора в

зависимых формах при их следующем построении с учетом версии и используемых механизмов кэширования.

4.5.3. Экспорт, импорт и дополнительные операции

| Адрес и метод | Обработчик | Результат |
|---|---|---|
| POST /refdata/import_union_energy_system | import_union_energy_system | Обработка загруженного файла и сообщение о результате |
| GET /refdata/export_union_energy_system | export_union_energy_system | Файл Excel для скачивания |

Отдельный маршрут импорта в этом справочнике предусмотрен; наличие маршрута не означает наличие видимой кнопки загрузки в текущем шаблоне.

## 4.6. Региональные энергосистемы

4.6.1. Страница списка

Адрес: GET, POST /refdata/regional_energy_system. Обработчик: regional_energy_system_list().

Ведет наименования РЭС и их принадлежность к ОЭС. Поддерживает связь одной РЭС с несколькими субъектами РФ через таблицу regional_district_regional_energy_system. Список и форма добавления используют варианты выбора из связанных справочников.

Поля и параметры HTML-формы: per_page, page, sort_by, sort_dir, energy_area_filter, union_energy_system_filter, regional_energy_system_filter, all_versions, regional_energy_system_ids[], regional_energy_system_names[], regional_energy_system_full_names[], regional_energy_system_rp_names[], union_energy_system[], regional_energy_system_delete[], js/refdata/textarea_autoresize.js.

Обработка: маршрут получает параметры отображения, при POST читает изменяемые строки и флажки удаления, вызывает сервисы и возвращает пользователя к списку с параметрами просмотра. Для обычного просмотра строится выборка и передается в шаблон вместе с пагинацией и вариантами выбора связанных справочников.

Модель: RegionalEnergySystem. Поля SQLAlchemy: id, name, name_full, name_rp, id_union_energy_system, union_energy_system, created_at, updated_at, database_version_id, regional_districts, energy_units, demand_parameters.

Модель данных – фрагмент объявления. Источник: app/refdata/models/energy_systems/regional_energy_system_model.py, строки 16–35.

```
  16  class RegionalEnergySystem(db.Model, AuditMixin, VersionedModelMixin, RefdataUuidMixin):
  17      __tablename__ = 'gs_sys_regional_energy_systems'
  18      __table_args__ = {"schema": SCHEMA_REFDATA}
  19
  20      id = db.Column(db.Integer, primary_key=True, autoincrement=True)
  21
  22      # Короткое наименование
  23      name = db.Column(db.String(255), nullable=False, index=True)
  24      # Полное наименование
  25      name_full = db.Column(db.String(255), nullable=False)
  26      # Полное наименование в родительном падеже
  27      name_rp = db.Column(db.String(255), nullable=False)
  28
  29      # FK -> UnionEnergySystem
  30      id_union_energy_system = db.Column(
  31          db.Integer,
  32          db.ForeignKey(f'{SCHEMA_REFDATA}.gs_sys_union_energy_systems.id', ondelete='RESTRICT'),
  33          index=True,
  34          nullable=True
  35      )
```

Связь модели с другой сущностью. Источник: app/refdata/models/energy_systems/regional_energy_system_model.py, строки 36–39.

```
  36      union_energy_system = db.relationship(
  37          'UnionEnergySystem',
  38          back_populates='regional_energy_systems'
  39      )
```

Маршрут списка – вход и параметры. Источник: app/refdata/routes/energy_systems/regional_energy_system_routes.py, строки 44–59.

```
  44  @refdata_bp.route("/regional_energy_system", methods=["GET", "POST"])
  45  @login_required
  46  def regional_energy_system_list():
  47      """Маршрут для отображения списка региональных энергосистем."""
  48
  49      user = session.get('username', 'Неизвестный пользователь')
  50      log_to_db(
  51          user,
  52          "Открыта страница региональных энергосистем",
  53          entity_type="regional_energy_system")
  54
  55      # Создание формы
  56      form = RegionalEnergySystemFilterForm()
  57
  58      # Получение параметров запроса
  59      page                            = request.args.get("page", 1, type=int)
```

Маршрут списка – передача изменений сервису. Источник: app/refdata/routes/energy_systems/regional_energy_system_routes.py, строки 163–168.

```
 163                              union_energy_system_filter=union_energy_system_filter,
 164                      ))
 165                  update_regional_energy_system_all_versions_service(regional_energy_system_data,
user)
 166                  flash("Изменения применены во всех версиях БД (по ref_uuid).", "success")
 167              else:
 168                  update_regional_energy_system_service(regional_energy_system_data, user)
```

Сервис списка – выборка и пагинация. Источник:
app/refdata/services/energy_systems/regional_energy_system_services.py, строки 123–143.

```
123  @no_autoflush
124  def get_regional_energy_system_list(
125      page,
126      per_page,
127      sort_by="id",
128      sort_dir="asc",
129      regional_energy_system_filter=None,
130      union_energy_system_filter=None,
131      ):
132      """ Получает список региональных энергосистем с пагинацией, фильтрацией и сортировкой, загружая
связи с субъектами РФ. """
133
134      # Базовый запрос
135      query = regional_energy_system_query(
136          sort_by=sort_by,
137          sort_dir=sort_dir,
138          regional_energy_system_filter=regional_energy_system_filter,
139          union_energy_system_filter=union_energy_system_filter,
140      )
141
142      # Пагинация
143      return query.paginate(page=page, per_page=per_page, error_out=False)
```

Сервис изменения – сохранение и журналирование. Источник:
app/refdata/services/energy_systems/regional_energy_system_services.py, строки 303–314.

```
303
304      try:
305          # Сохранение изменений в базе данных
306          # Фиксация транзакции (устойчивый коммит)
307          _commit_with_retry()
308
309          if updated_ids:
310              log_to_db(
311                  user,
312                  "Сохранены изменения по региональной энергосистеме",
313                  f"Измененных записей: {len(updated_ids)} (id: {updated_ids})",
314                  entity_type="regional_energy_system")
```

HTML/Jinja2 списка – элементы формы. Источник:
app/templates/refdata/energy_systems/regional_energy_system/regional_energy_system.html, строки 10–23.

```
 10      <h1 class="text-center mb-4">Список региональных энергосистем</h1>
 11
 12      <form method="GET"
 13          action="{{ url_for('refdata_bp.regional_energy_system_list') }}"
 14          class="mb-3"
 15          id="filterForm"
 16          >
 17          <div class="row g-2 align-items-center">
 18              <!-- ЛЕВАЯ СТОРОНА: поиск + кнопки -->
 19              <div class="col-12 col-lg-8">
 20                  <div class="d-flex align-items-center gap-2 flex-wrap flex-lg-nowrap">
 21                      <div class="flex-grow-1" style="min-width: 220px;">
 22                          {{ form.regional_energy_system_filter(
 23                              class="form-control",
```

Взаимодействие с другими модулями. РЭС объединяют энергорайоны и территориальные показатели. Связь РЭС с субъектами является многие-ко-многим: административная и энергетическая иерархии не обязаны совпадать. Эта связь участвует в отборе и агрегировании данных электропотребления и потребности в мощности.

Подтверждающие ссылки на потребителей модели:
app/energy_balance/models/espp_energy_generation_model.py:80 – "RegionalEnergySystem",
app/energy_consumption/models/energy_systems/regional_energy_system_energy_consumption_parameter_model.py:29 – "RegionalEnergySystem",
app/fuel/models/fue_equipment_group_fuel_param_model.py:21 – from app.refdata.models.energy_systems.regional_energy_system_model import RegionalEnergySystem
app/generation/models/station/station_model.py:96 – 'RegionalEnergySystem',

app/power_demand/models/energy_systems/regional_energy_system_demand_parameter_model.py:42 – "RegionalEnergySystem",

### 4.6.2. Форма добавления

Адрес: GET, POST /refdata/add_regional_energy_system. Обработчик: add_regional_energy_system().

Назначение: создание записи справочника «Региональные энергосистемы». Вводимые поля формы: name, name_full, name_rp. Список полей формы может быть уже модели: дополнительные реквизиты изменяются на странице списка или заполняются сервисом.

GET создает форму и подготавливает варианты выбора. POST с валидной формой формирует данные и передает их сервису добавления. При успехе выполняется переход к списку; сообщения о результате выводятся пользователю. Кнопка отмены возвращает к списку без создания записи. Создаваемая сущность – RegionalEnergySystem, модель и ее связи приведены выше в подразделе 4.6.1.

Поля WTForms и валидаторы: csrf_token = HiddenField(); name = StringField('Региональная энергосистема', validators=[DataRequired(message='Поле «Региональная энергосистема» обязательно.'), Length(min=3, max=255, message='Длина от 3 до 255 символов.')]); name_full = StringField('Полное наименование', validators=[DataRequired(message='Поле «Полное наименование» обязательно.'), Length(min=3, max=255, message='Длина от 3 до 255 символов.')]); name_rp = StringField('Наименование (в родительном падеже)', validators=[DataRequired(message='Поле «Наименование (в родительном падеже)» обязательно.'), Length(min=3, max=255, message='Длина от 3 до 255 символов.')]); union_energy_system = SelectField('ОЭС', choices=[], coerce=int, validators=[DataRequired(message='Выберите ОЭС.')]); regional_districts = SelectMultipleField('Субъекты РФ', choices=[], coerce=int, validators=[Optional()], render_kw={'multiple': True}).

Шаблон содержит кнопку добавления во всех версиях для административного пользователя. Серверная ветвь all_versions=1 вызывает специализированную проверку доступа и межверсионный сервис; при обычном добавлении используется выбранная версия.

Модель добавляемого объекта RegionalEnergySystem – основной реквизит. Источник: app/refdata/models/energy_systems/regional_energy_system_model.py, строки 23–23.

```
23      name = db.Column(db.String(255), nullable=False, index=True)
```

Маршрут добавления – вход в форму. Источник: app/refdata/routes/energy_systems/regional_energy_system_routes.py, строки 216–228.

```
216  @refdata_bp.route("/add_regional_energy_system", methods=["GET", "POST"])
217  @login_required
218  def add_regional_energy_system():
219      """ Маршрут для добавления новой региональной энергосистемы. """
220
221      user = session.get('username', 'Неизвестный пользователь')
222      log_to_db(
223          user,
224          "Открыта страница добавления региональной энергосистемы",
225          entity_type="regional_energy_system")
226
227      # Создание формы
228      form = AddRegionalEnergySystemForm()
```

Маршрут добавления – вызов сервиса. Источник:
app/refdata/routes/energy_systems/regional_energy_system_routes.py, строки 265–276.

```
                            union_energy_system_filter=union_energy_system_filter,
                    ))
                add_regional_energy_system_all_versions_service(payload, user)
                flash("Новая запись добавлена во всех версиях БД (общий ref_uuid).", "success")
            else:
                add_regional_energy_system_service(payload, user)
                flash("Новая запись успешно добавлена.", "success")

            # Перенаправление на список с сохранением параметров и переходом к новой записи
            total_records = regional_energy_system_query(
                                regional_energy_system_filter,
                                union_energy_system_filter).count()
```

Сервис добавления – создание сущности. Источник:
app/refdata/services/energy_systems/regional_energy_system_services.py, строки 389–403.

```
                    raise ValueError(f"Запись с полным наименованием «{name_full}» уже
существует.")

                # Создаем новую запись
                obj = RegionalEnergySystem(
                    name=name,
                    name_full=name_full or None,
                    name_rp=name_rp or None,
                    id_union_energy_system=union_energy_system_id,
                )

                # Обновление связей «многие ко многим»
                existing_districts = {district.id for district in obj.regional_districts}
                new_districts = set(regional_district_ids)

                # Добавить новые связи
```

HTML/Jinja2 формы добавления. Источник:
app/templates/refdata/energy_systems/regional_energy_system/regional_energy_system_add.html, строки 93–107.

```
                    <li>
                        <label class="dropdown-item">
                            <input type="checkbox" name="{{ form.regional_districts.name }}"
value="{{ regional_district[0] }}"
                                   data-region-name="{{ regional_district[1] }}"
                                   class="region-checkbox"
                                   {% if form.regional_districts.data and regional_district[0] in
form.regional_districts.data %}checked{% endif %}>
                            {{ regional_district[1] }}
                        </label>
                    </li>
                    {% endfor %}
                </ul>
            </div>

            <!-- Ошибки валидации -->
            {% if form.regional_districts.errors %}
```

Взаимодействие при добавлении. РЭС объединяют энергорайоны и территориальные показатели. Связь РЭС с субъектами является многие-ко-многим: административная и энергетическая иерархии не обязаны совпадать. Эта связь участвует в отборе и агрегировании данных электропотребления и потребности в мощности. После создания запись становится доступна для выбора в зависимых формах при их следующем построении с учетом версии и используемых механизмов кэширования.

### 4.6.3. Экспорт, импорт и дополнительные операции

| Адрес и метод | Обработчик | Результат |
|---|---|---|
| POST /refdata/import_regional_energy_system | import_regional_energy_system | Обработка загруженного файла и |

| Адрес и метод | Обработчик | Результат |
|---|---|---|
| | | сообщение о результате |
| GET /refdata/export_regional_energy_system | export_regional_energy_system | Файл Excel для скачивания |

Отдельный маршрут импорта в этом справочнике предусмотрен; наличие маршрута не означает наличие видимой кнопки загрузки в текущем шаблоне.

## 4.7. Энергорайоны

### 4.7.1. Страница списка

Адрес: GET, POST /refdata/energy_units. Обработчик: energy_unit_list().

Ведет энергорайоны EnergyUnit, их наименования и грамматические формы, принадлежность к субъекту РФ и региональной энергосистеме. Список позволяет отбирать записи по территориальным связям, редактировать их и переходить к созданию новой записи.

Поля и параметры HTML-формы: per_page, page, sort_by, sort_dir, regional_district_filter, regional_energy_system_filter, union_energy_system_filter, energy_unit_filter, all_versions, energy_unit_ids[], energy_unit_names[], energy_unit_names_rp[], energy_unit_names_dp[], energy_unit_names_dat[], regional_district_ids[], regional_energy_system_ids[], energy_unit_delete[], js/refdata/textarea_autoresize.js.

Обработка: маршрут получает параметры отображения, при POST читает изменяемые строки и флажки удаления, вызывает сервисы и возвращает пользователя к списку с параметрами просмотра. Для обычного просмотра строится выборка и передается в шаблон вместе с пагинацией и вариантами выбора связанных справочников.

Модель: EnergyUnit. Поля SQLAlchemy: id, name, name_rp, name_dp, name_dat, id_regional_district, regional_district, id_regional_energy_system, regional_energy_system, created_at, updated_at, database_version_id, stations, demand_parameters.

Модель данных – фрагмент объявления. Источник: app/refdata/models/energy_systems/energy_unit_model.py, строки 13–33.

```
13  class EnergyUnit(db.Model, AuditMixin, VersionedModelMixin, RefdataUuidMixin):
14      __tablename__ = 'gs_sys_energy_units'
15      __table_args__ = {"schema": SCHEMA_REFDATA}
16
17      id = db.Column(db.Integer, primary_key=True, autoincrement=True)
18      name = db.Column(db.String(256), nullable=False, index=True)
19
20      # Полное наименование в родительном падеже
21      name_rp = db.Column(db.String(255), nullable=False)
22      # Полное наименование в предложном падеже
23      name_dp = db.Column(db.String(255), nullable=False)
24      # Наименование в дательном падеже (для «Итого по …»)
25      name_dat = db.Column(db.String(255), nullable=False, server_default="")
26
27      # FK -> RegionalDistrict
28      id_regional_district = db.Column(
29          db.Integer,
30          db.ForeignKey(f'{SCHEMA_REFDATA}.gs_sys_regional_districts.id', ondelete='RESTRICT'),
31          index=True,
32          nullable=False
33      )
```

Связь модели с другой сущностью. Источник: app/refdata/models/energy_systems/energy_unit_model.py, строки 34–37.

```
34      regional_district = db.relationship(
35          'RegionalDistrict',
36          back_populates='energy_units'
37      )
```

Маршрут списка – вход и параметры. Источник: app/refdata/routes/energy_systems/energy_unit_routes.py, строки 47–62.

```
47  @refdata_bp.route("/energy_units", methods=["GET", "POST"])
48  @login_required
49  def energy_unit_list():
50      """Маршрут для отображения списка энергоузлов."""
51
52      user = session.get('username', 'Неизвестный пользователь')
53      log_to_db(
54          user,
55          "Открыта страница энергоузла",
56          entity_type="energy_unit")
57
58      # Создание формы
59      form = EnergyUnitFilterForm()
60
61      # Получение параметров запроса
62      page                            = request.args.get("page", 1, type=int)
```

Маршрут списка – передача изменений сервису. Источник: app/refdata/routes/energy_systems/energy_unit_routes.py, строки 184–189.

```
184                             union_energy_system_filter=union_energy_system_filter,
185                         ))
186                 update_energy_unit_all_versions_service(energy_unit_data, user)
187                 flash("Изменения применены во всех версиях БД (по ref_uuid).", "success")
188             else:
189                 update_energy_unit_service(energy_unit_data, user)
```

Сервис списка – выборка и пагинация. Источник: app/refdata/services/energy_systems/energy_unit_services.py, строки 191–212.

```
191  @no_autoflush
192  def get_energy_unit_list(
193      page,
194      per_page,
195      energy_unit_filter=None,
196      regional_district_filter=None,
197      regional_energy_system_filter=None,
198      union_energy_system_filter=None,
199      sort_by="id",
200      sort_dir="asc"):
201      """ Получает список энергоузлов с пагинацией, фильтрацией и сортировкой. """
202
203      # Базовый запрос
204      query = energy_unit_query(
205          energy_unit_filter=energy_unit_filter,
206          regional_district_filter=regional_district_filter,
207          regional_energy_system_filter=regional_energy_system_filter,
208          union_energy_system_filter=union_energy_system_filter,
209          sort_by=sort_by,
210          sort_dir=sort_dir,
211      )
212
```

Сервис изменения – сохранение и журналирование. Источник: app/refdata/services/energy_systems/energy_unit_services.py, строки 330–341.

```
330
331     try:
332         # Сохранение изменений в базе данных
333         # Фиксация транзакции (устойчивый коммит)
334         _commit_with_retry()
335
336         if updated_ids:
337             log_to_db(
338                 user,
339                 "Сохранены изменения по энергоузлам",
340                 f"Измененных записей: {len(updated_ids)} (id: {updated_ids})",
341                 entity_type="energy_unit")
```

HTML/Jinja2 списка – элементы формы. Источник: app/templates/refdata/energy_systems/energy_unit/energy_unit.html, строки 10–23.

```
10      <h1 class="text-center mb-4">Список энергорайонов</h1>
11
12      <form method="GET"
13          action="{{ url_for('refdata_bp.energy_unit_list') }}"
14          class="mb-3"
15          id="filterForm"
16          >
17          <div class="row g-2 align-items-center">
18              <!-- ЛЕВАЯ СТОРОНА: поиск + кнопки -->
19              <div class="col-12 col-lg-8">
20                  <div class="d-flex align-items-center gap-2 flex-wrap flex-lg-nowrap">
21                      <div class="flex-grow-1" style="min-width: 220px;">
22                          {{ form.energy_unit_filter(
23                              class="form-control",
```

Взаимодействие с другими модулями. Station.id_energy_unit связывает электростанцию с энергорайоном. EnergyUnitDemandParameter хранит связанные показатели потребности в мощности. При группировке станций используются одновременно энергетическая и административная принадлежности. В интерфейсе EnergyUnit называется «Энергорайон», а EnergyArea – «Энергоузел».

Подтверждающие ссылки на потребителей модели:
app/energy_balance/models/regional_energy_system_transfer_model.py:6 – Получатель – RegionalEnergySystem, ForeignBorderCountry, RegionalDistrict или EnergyUnit
app/energy_consumption/models/energy_systems/energy_unit_energy_consumption_parameter_model.py:26 – "EnergyUnit",
app/fuel/models/fue_equipment_group_model.py:133 – energy_unit = db.relationship("EnergyUnit", foreign_keys=[energy_unit_id])
app/generation/models/station/station_model.py:107 – energy_unit = db.relationship('EnergyUnit', back_populates='stations')
app/power_demand/models/energy_systems/energy_unit_demand_parameter_model.py:2 – """Параметры нагрузки (demand) для энергоузла (EnergyUnit)."""

### 4.7.2. Форма добавления

Адрес: GET, POST /refdata/add_energy_unit. Обработчик: add_energy_unit().

Назначение: создание записи справочника «Энергорайоны». Вводимые поля формы: name, name_rp, name_dp, name_dat. Список полей формы может быть уже модели: дополнительные реквизиты изменяются на странице списка или заполняются сервисом.

GET создает форму и подготавливает варианты выбора. POST с валидной формой формирует данные и передает их сервису добавления. При успехе выполняется переход к списку; сообщения о результате выводятся пользователю. Кнопка отмены возвращает к списку без создания записи. Создаваемая сущность – EnergyUnit, модель и ее связи приведены выше в подразделе 4.7.1.

Поля WTForms и валидаторы: csrf_token = HiddenField(); name = StringField('Энергорайон', validators=[DataRequired(message='Поле «Энергорайон» обязательно.'), Length(min=3, max=255, message='Длина от 3 до 255 символов.')]); name_rp = StringField('Энергорайон (род. пад.)', validators=[DataRequired(message='Поле «Энергорайон (род. пад.)» обязательно.'), Length(min=2, max=255, message='Длина от 2 до 255 символов.')]); name_dp = StringField('Энергорайон (предл. пад.)', validators=[DataRequired(message='Поле «Энергорайон (предл. пад.)» обязательно.'), Length(min=2, max=255, message='Длина от 2 до 255 символов.')]); name_dat = StringField('Наименование (в дательном падеже)', validators=[Optional(), Length(max=255, message='Длина до 255 символов.')]); regional_district = SelectField('Субъект РФ', choices=[], coerce=int, validators=[DataRequired(message='Выберите субъект РФ.')]); regional_energy_system = SelectField('Региональная энергосистема', choices=[], coerce=int, validators=[Optional()]); union_energy_system = SelectField('ОЭС', choices=[], coerce=int, validators=[Optional()]).

Шаблон содержит кнопку добавления во всех версиях для административного пользователя. Серверная ветвь all_versions=1 вызывает специализированную проверку доступа и межверсионный сервис; при обычном добавлении используется выбранная версия.

Модель добавляемого объекта EnergyUnit – основной реквизит. Источник: app/refdata/models/energy_systems/energy_unit_model.py, строки 18–18.

```
18      name = db.Column(db.String(256), nullable=False, index=True)
```

Маршрут добавления – вход в форму. Источник: app/refdata/routes/energy_systems/energy_unit_routes.py, строки 252–264.

```
252 @refdata_bp.route("/add_energy_unit", methods=["GET", "POST"])
253 @login_required
254 def add_energy_unit():
255     """ Маршрут для добавления нового энергоузла. """
256
257     user = session.get('username', 'Неизвестный пользователь')
258     log_to_db(
259         user,
260         "Открыта страница добавления энергоузла",
261         entity_type="energy_unit")
262
263     # Создание формы
264     form = AddEnergyUnitForm()
```

Маршрут добавления – вызов сервиса. Источник: app/refdata/routes/energy_systems/energy_unit_routes.py, строки 316–329.

```
316                                 union_energy_system_filter=union_energy_system_filter,
317                         ))
318                 add_energy_unit_all_versions_service(payload, user)
319                 flash("Новая запись добавлена во всех версиях БД (общий ref_uuid).", "success")
320             else:
321                 add_energy_unit_service(payload, user)
322                 flash("Новая запись успешно добавлена.", "success")
323
324             # Перенаправление на список с сохранением параметров и переходом к новой записи
325             total_records = energy_unit_query(
326                                 energy_unit_filter,
327                                 regional_district_filter,
328                                 regional_energy_system_filter,
329                                 union_energy_system_filter).count()
```

Сервис добавления – создание сущности. Источник:
app/refdata/services/energy_systems/energy_unit_services.py, строки 411–435.

```
411                     raise ValueError(f"Запись с наименованием «{name}» уже существует.")
412
413                 # Создаем новую запись
414                 obj = EnergyUnit(
415                     name=name,
416                     name_rp=name_rp,
417                     name_dp=name_dp,
418                     name_dat=name_dat,
419                     id_regional_district=rd_obj.id,
420                     id_regional_energy_system=res_obj.id,
421                 )
422                 set_db_version_on_create(obj)
423                 db.session.add(obj)
424                 db.session.flush()  # получить id без полного коммита
425
426                 log_to_db(
427                     user,
428                     "Создана региональная энергосистема",
429                     (
430                         f"Наименование: {name}; "
431                         f"Субъект РФ: {rd_obj.name}; "
432                         f"Региональная энергосистема: {res_obj.name} "
433                     ),
434                     entity_type="energy_unit",
435                     entity_id=obj.id)
```

HTML/Jinja2 формы добавления. Источник:
app/templates/refdata/energy_systems/energy_unit/energy_unit_add.html, строки 7–21.

```
  7  <div class="container mt-5">
  8      <h1 class="mb-4"><i class="fas fa-plus-circle"></i> Добавить энергорайон</h1>
  9
 10      <!-- Форма ввода данных -->
 11      <form action="{{ url_for('refdata_bp.add_energy_unit') }}" method="POST">
 12          {{ form.hidden_tag() }}
 13
 14          <!-- Скрытые поля для сохранения параметров предыдущей страницы -->
 15          <input type="hidden" name="page"                            value="{{ pagination.page if
pagination else 1 }}">
 16          <input type="hidden" name="per_page"                        value="{{ per_page }}">
 17          <input type="hidden" name="sort_by"                         value="{{ sort_by }}">
 18          <input type="hidden" name="sort_dir"                        value="{{ sort_dir }}">
 19          <input type="hidden" name="energy_unit_filter"              value="{{ energy_unit_filter
}}">
 20          <input type="hidden" name="regional_district_filter"        value="{{
regional_district_filter }}">
 21          <input type="hidden" name="regional_energy_system_filter"   value="{{
regional_energy_system_filter }}">
```

Взаимодействие при добавлении. Station.id_energy_unit связывает электростанцию с энергорайоном. EnergyUnitDemandParameter хранит связанные показатели потребности в мощности. При группировке станций используются одновременно энергетическая и административная принадлежности. В интерфейсе EnergyUnit называется «Энергорайон», а EnergyArea – «Энергоузел». После создания запись становится доступна для выбора в зависимых формах при их следующем построении с учетом версии и используемых механизмов кэширования.

### 4.7.3. Экспорт, импорт и дополнительные операции

| Адрес и метод | Обработчик | Результат |
|---|---|---|
| GET /refdata/export_energy_unit | export_energy_unit | Файл Excel для скачивания |

Отдельный маршрут импорта в этом справочнике не зарегистрирован. Поддержка импорта не приписывается всем справочникам по аналогии.

## 4.8. Энергоузлы

### 4.8.1. Страница списка

Адрес: GET, POST /refdata/energy_areas. Обработчик: energy_area_list().

Ведет энергоузлы EnergyArea с наименованием и привязкой к субъекту РФ. Предусмотрены отбор, редактирование в таблице и отдельная форма добавления; экспорт формирует файл Excel.

Поля и параметры HTML-формы: per_page, page, sort_by, sort_dir, regional_district_filter, regional_energy_system_filter, union_energy_system_filter, energy_area_filter, all_versions, energy_area_ids[], energy_area_names[], regional_district_ids[], energy_area_delete[], js/refdata/textarea_autoresize.js.

Обработка: маршрут получает параметры отображения, при POST читает изменяемые строки и флажки удаления, вызывает сервисы и возвращает пользователя к списку с параметрами просмотра. Для обычного просмотра строится выборка и передается в шаблон вместе с пагинацией и вариантами выбора связанных справочников.

Модель: EnergyArea. Поля SQLAlchemy: id, name, id_regional_district, regional_district, created_at, updated_at, database_version_id, machines, demand_parameters.

Модель данных – фрагмент объявления. Источник: app/refdata/models/energy_systems/energy_area_model.py, строки 13–31.

```
  13  class EnergyArea(db.Model, AuditMixin, VersionedModelMixin, RefdataUuidMixin):
  14      __tablename__ = 'gs_sys_energy_areas'
  15      __table_args__ = {"schema": SCHEMA_REFDATA}
  16
  17      id = db.Column(db.Integer, primary_key=True, autoincrement=True)
  18      name = db.Column(db.String(256), nullable=False, index=True)
  19
  20      # FK -> RegionalDistrict (обязательный)
  21      id_regional_district = db.Column(
  22          db.Integer,
  23          db.ForeignKey(f'{SCHEMA_REFDATA}.gs_sys_regional_districts.id', ondelete='RESTRICT'),
  24          index=True,
  25          nullable=False
  26      )
  27      regional_district = db.relationship('RegionalDistrict', back_populates='energy_areas')
  28
  29      created_at = db.Column(db.DateTime(timezone=True), server_default=func.now(), nullable=False)
  30      updated_at = db.Column(db.DateTime(timezone=True), server_default=func.now(),
onupdate=func.now(), nullable=False)
  31
```

Маршрут списка – вход и параметры. Источник: app/refdata/routes/energy_systems/energy_area_routes.py, строки 45–60.

```
  45  @refdata_bp.route("/energy_areas", methods=["GET", "POST"])
  46  def energy_area_list():
  47      """Маршрут для отображения списка энергорайонов"""
  48
  49      user = session.get('username', 'Неизвестный пользователь')
  50      log_to_db(
  51          user,
  52          "Открыта страница энергорайонов",
  53          entity_type="energy_area")
  54
  55      # Создание формы
  56      form = EnergyAreaFilterForm()
  57
  58      # Получение параметров запроса
  59      page                            = request.args.get("page", 1, type=int)
  60      per_page                        = request.args.get("per_page", 25, type=int)
```

Маршрут списка – передача изменений сервису. Источник: app/refdata/routes/energy_systems/energy_area_routes.py, строки 164–169.

```
164                             union_energy_system_filter=union_energy_system_filter,
165                         ))
166                     update_energy_area_all_versions_service(energy_area_data, user)
167                     flash("Изменения применены во всех версиях БД (по ref_uuid).", "success")
168                 else:
169                     update_energy_area_service(energy_area_data, user)
```

Сервис списка – выборка и пагинация. Источник: app/refdata/services/energy_systems/energy_area_services.py, строки 199–220.

```
199 @no_autoflush
200 def get_energy_area_list(
201         page,
202         per_page,
203         energy_area_filter=None,
204         regional_district_filter=None,
205         regional_energy_system_filter=None,
206         union_energy_system_filter=None,
207         sort_by="id",
208         sort_dir="asc"):
209     """ Получает список энергорайонов с пагинацией, фильтрацией и сортировкой. """
210
211     # Базовый запрос
212     query = energy_area_query(
213         energy_area_filter=energy_area_filter,
214         regional_district_filter=regional_district_filter,
215         regional_energy_system_filter=regional_energy_system_filter,
216         union_energy_system_filter=union_energy_system_filter,
217         sort_by=sort_by,
218         sort_dir=sort_dir,
219     )
220
```

Сервис изменения – сохранение и журналирование. Источник: app/refdata/services/energy_systems/energy_area_services.py, строки 311–322.

```
311
312     try:
313         # Сохранение изменений в базе данных
314         # Фиксация транзакции (устойчивый коммит)
315         _commit_with_retry()
316
317         if updated_ids:
318             log_to_db(
319                 user,
320                 "Сохранены изменения по энергорайонам",
321                 f"Измененных записей: {len(updated_ids)} (id: {updated_ids})",
322                 entity_type="energy_area")
```

HTML/Jinja2 списка – элементы формы. Источник: app/templates/refdata/energy_systems/energy_area/energy_area.html, строки 10–23.

```
 10     <h1 class="text-center mb-4">Список энергоузлов</h1>
 11
 12     <form method="GET"
 13         action="{{ url_for('refdata_bp.energy_area_list') }}"
 14         class="mb-3"
 15         id="filterForm"
 16         >
 17         <div class="row g-2 align-items-center">
 18             <!-- ЛЕВАЯ СТОРОНА: поиск + кнопки -->
 19             <div class="col-12 col-lg-8">
 20                 <div class="d-flex align-items-center gap-2 flex-wrap flex-lg-nowrap">
 21                     <div class="flex-grow-1" style="min-width: 220px;">
 22                         {{ form.energy_area_filter(
 23                             class="form-control",
```

Взаимодействие с другими модулями. Machine.id_energy_area связывает агрегат с энергоузлом. EnergyAreaDemandParameter содержит параметры потребности в мощности этого уровня. Энергоузел не следует отождествлять с энергорайоном EnergyUnit: модели и точки подключения различаются.

Подтверждающие ссылки на потребителей модели:
app/energy_consumption/models/energy_systems/energy_area_energy_consumption_parameter_model.py:2 – """Параметры потребления для энергорайона (EnergyArea)."""
app/generation/models/machine/machine_model.py:82 – energy_area = db.relationship('EnergyArea', back_populates='machines')
app/power_demand/models/energy_systems/energy_area_demand_parameter_model.py:2 – """Параметры нагрузки (demand) для энергоузла (EnergyArea)."""
app/energy_consumption/routes/energy_consumption_parameter_routes.py:67 – from app.refdata.models.energy_systems.energy_area_model import EnergyArea
app/generation/services/machine_services/machine_services.py:27 – from app.refdata.models.energy_systems.energy_area_model import EnergyArea

### 4.8.2. Форма добавления

Адрес: GET, POST /refdata/add_energy_area. Обработчик: add_energy_area().

Назначение: создание записи справочника «Энергоузлы». Вводимые поля формы: name. Список полей формы может быть уже модели: дополнительные реквизиты изменяются на странице списка или заполняются сервисом.

GET создает форму и подготавливает варианты выбора. POST с валидной формой формирует данные и передает их сервису добавления. При успехе выполняется переход к списку; сообщения о результате выводятся пользователю. Кнопка отмены возвращает к списку без создания записи. Создаваемая сущность – EnergyArea, модель и ее связи приведены выше в подразделе 4.8.1.

Поля WTForms и валидаторы: csrf_token = HiddenField(); name = StringField('Энергоузел', validators=[DataRequired(message='Поле «Энергоузел» обязательно.'), Length(min=3, max=255, message='Длина от 3 до 255 символов.')]); regional_district = SelectField('Субъект РФ', choices=[], coerce=int, validators=[DataRequired(message='Выберите субъект РФ.')]); regional_energy_system = SelectField('Региональная энергосистема', choices=[], coerce=int, validators=[Optional()]); union_energy_system = SelectField('ОЭС', choices=[], coerce=int, validators=[Optional()]).

Шаблон содержит кнопку добавления во всех версиях для административного пользователя. Серверная ветвь all_versions=1 вызывает специализированную проверку доступа и межверсионный сервис; при обычном добавлении используется выбранная версия.

Модель добавляемого объекта EnergyArea – основной реквизит. Источник: app/refdata/models/energy_systems/energy_area_model.py, строки 18–18.

```
18      name = db.Column(db.String(256), nullable=False, index=True)
```

Маршрут добавления – вход в форму. Источник: app/refdata/routes/energy_systems/energy_area_routes.py, строки 228–240.

```
228 @refdata_bp.route("/add_energy_area", methods=["GET", "POST"])
229 def add_energy_area():
230     """Маршрут для добавления нового энергорайона."""
231
232     user = session.get('username', 'Неизвестный пользователь')
233     log_to_db(
234         user,
235         "Открыта страница добавления энергорайона",
236         entity_type="energy_area")
237
238     # Создание формы
239     form = AddEnergyAreaForm()
240
```

Маршрут добавления – вызов сервиса. Источник: app/refdata/routes/energy_systems/energy_area_routes.py, строки 284–297.

```
284                             union_energy_system_filter=union_energy_system_filter,
285                         ))
286                     add_energy_area_all_versions_service(payload, user)
287                     flash("Новая запись добавлена во всех версиях БД (общий ref_uuid).", "success")
288                 else:
289                     add_energy_area_service(payload, user)
290                     flash("Новая запись успешно добавлена.", "success")
291 
292                 # Перенаправление на список с сохранением параметров и переходом к новой записи
293                 total_records = energy_area_query(
294                                     energy_area_filter,
295                                     regional_district_filter,
296                                     regional_energy_system_filter,
297                                     union_energy_system_filter).count()
```

Сервис добавления – создание сущности. Источник: app/refdata/services/energy_systems/energy_area_services.py, строки 382–401.

```
382                     raise ValueError(f"Запись с именем «{name}» уже существует.")
383 
384                 # Создаем новую запись
385                 obj = EnergyArea(
386                     name=name,
387                     id_regional_district=regional_district_id,
388                 )
389                 set_db_version_on_create(obj)
390                 db.session.add(obj)
391                 db.session.flush()  # получить id без полного коммита
392 
393                 log_to_db(
394                     user,
395                     "Создан энергорайон",
396                     (
397                         f"Наименование: {name}; "
398                         f"Субъект РФ: {get_regional_district_name(regional_district_id)}"
399                     ),
400                     entity_type="energy_area",
401                     entity_id=obj.id)
```

HTML/Jinja2 формы добавления. Источник: app/templates/refdata/energy_systems/energy_area/energy_area_add.html, строки 7–21.

```
  7  <div class="container mt-5">
  8      <h1 class="mb-4"><i class="fas fa-plus-circle"></i> Добавить энергоузел</h1>
  9 
 10      <!-- Форма ввода данных -->
 11      <form action="{{ url_for('refdata_bp.add_energy_area') }}" method="POST">
 12          {{ form.hidden_tag() }}
 13 
 14          <!-- Скрытые поля для сохранения параметров предыдущей страницы -->
 15          <input type="hidden" name="page"                              value="{{ pagination.page if
pagination else 1 }}">
 16          <input type="hidden" name="per_page"                          value="{{ per_page }}">
 17          <input type="hidden" name="sort_by"                           value="{{ sort_by }}">
 18          <input type="hidden" name="sort_dir"                          value="{{ sort_dir }}">
 19          <input type="hidden" name="energy_area_filter"                value="{{ energy_area_filter
}}">
 20          <input type="hidden" name="regional_district_filter"          value="{{
regional_district_filter }}">
 21          <input type="hidden" name="regional_energy_system_filter"     value="{{
regional_energy_system_filter }}">
```

Взаимодействие при добавлении. Machine.id_energy_area связывает агрегат с энергоузлом. EnergyAreaDemandParameter содержит параметры потребности в мощности этого уровня. Энергоузел не следует отождествлять с энергорайоном EnergyUnit: модели и точки подключения различаются. После создания запись становится доступна для выбора в зависимых формах при их следующем построении с учетом версии и используемых механизмов кэширования.

4.8.3. Экспорт, импорт и дополнительные операции

| Адрес и метод | Обработчик | Результат |
|---|---|---|
| GET /refdata/export_energy_area_to_excel | export_energy_area | Файл Excel для скачивания |

Отдельный маршрут импорта в этом справочнике не зарегистрирован. Поддержка импорта не приписывается всем справочникам по аналогии.

## 4.9. Типы энергосистем

4.9.1. Страница списка

Адрес: GET, POST /refdata/energy_system_type. Обработчик: energy_system_type_list().

Содержит классификатор типов энергосистем. Наименование редактируется в общей таблице; новый тип вводится через отдельную форму. Тип служит признаком классификации ОЭС.

Поля и параметры HTML-формы: per_page, page, sort_by, sort_dir, energy_system_type_filter, all_versions, energy_system_type_ids[], energy_system_type_names[], energy_system_type_delete[], js/refdata/textarea_autoresize.js.

Обработка: маршрут получает параметры отображения, при POST читает изменяемые строки и флажки удаления, вызывает сервисы и возвращает пользователя к списку с параметрами просмотра. Для обычного просмотра строится выборка и передается в шаблон вместе с пагинацией и вариантами выбора связанных справочников.

Модель: EnergySystemType. Поля SQLAlchemy: id, name, created_at, updated_at, database_version_id, union_energy_systems, demand_parameters.

Модель данных – фрагмент объявления. Источник: app/refdata/models/energy_systems/energy_system_type_model.py, строки 14–32.

```
  14  class EnergySystemType(db.Model, AuditMixin, VersionedModelMixin, RefdataUuidMixin):
  15      __tablename__ = "gs_sys_energy_system_types"
  16      __table_args__ = {"schema": SCHEMA_REFDATA}
  17
  18      id = db.Column(db.Integer, primary_key=True, autoincrement=True)
  19      name = db.Column(db.String(255), unique=True, nullable=False, index=True)
  20
  21      created_at = db.Column(db.DateTime(timezone=True), server_default=func.now(), nullable=False)
  22      updated_at = db.Column(db.DateTime(timezone=True), server_default=func.now(),
onupdate=func.now(), nullable=False)
  23
  24      # Поле для связи с версией БД
  25      database_version_id = db.Column(
  26          db.Integer,
  27          db.ForeignKey(f"{SCHEMA_REFDATA}.gs_database_versions.id", ondelete="SET NULL"),
  28          nullable=True,
  29          index=True
  30      )
  31
  32      # двусторонняя связь с UnionEnergySystem
```

Связь модели с другой сущностью. Источник: app/refdata/models/energy_systems/energy_system_type_model.py, строки 33–37.

```
  33      union_energy_systems = db.relationship(
  34          "UnionEnergySystem",
  35          back_populates="energy_system_type",
  36          foreign_keys="UnionEnergySystem.id_energy_system_type"
  37      )
```

Маршрут списка – вход и параметры. Источник: app/refdata/routes/energy_systems/energy_system_type_routes.py, строки 35–50.

```
@refdata_bp.route("/energy_system_type", methods=["GET", "POST"])
@login_required
def energy_system_type_list():
    """Маршрут для отображения списка типов частей энергосистемы России."""

    user = session.get('username', 'Неизвестный пользователь')
    log_to_db(
        user,
        "Открыта страница типов частей энергосистемы России",
        entity_type="energy_system_type")

    # Создание формы
    form = EnergySystemTypeFilterForm()

    # Получение параметров запроса
    page                        = request.args.get("page", 1, type=int)
```

Маршрут списка – передача изменений сервису. Источник: app/refdata/routes/energy_systems/energy_system_type_routes.py, строки 135–140.

```
                            sort_dir=sort_dir,
                    ))
                update_energy_system_type_all_versions_service(energy_system_type_data, user)
                flash("Изменения применены во всех версиях БД (по ref_uuid).", "success")
            else:
                update_energy_system_type_service(energy_system_type_data, user)
```

Сервис списка – выборка и пагинация. Источник: app/refdata/services/energy_systems/energy_system_type_services.py, строки 82–99.

```
@no_autoflush
def get_energy_system_type_list(
    page,
    per_page,
    energy_system_type_filter=None,
    sort_by="id",
    sort_dir="asc"):
    """ Получает список частей энергосистемы России с пагинацией, фильтрацией и сортировкой. """

    # Базовый запрос
    query = energy_system_type_query(
        energy_system_type_filter=energy_system_type_filter,
        sort_by=sort_by,
        sort_dir=sort_dir,
    )

    # Пагинация
    return query.paginate(page=page, per_page=per_page, error_out=False)
```

Сервис изменения – сохранение и журналирование. Источник: app/refdata/services/energy_systems/energy_system_type_services.py, строки 173–184.

```

    try:
        # Сохранение изменений в базе данных
        # Фиксация транзакции (устойчивый коммит)
        _commit_with_retry()

        if updated_ids:
            log_to_db(
                user,
                "Сохранены изменения по частям энергосистемы России",
                f"Измененных записей: {len(updated_ids)} (id: {updated_ids})",
                entity_type="energy_system_type")
```

HTML/Jinja2 списка – элементы формы. Источник: app/templates/refdata/energy_systems/energy_system_type/energy_system_type.html, строки 164–177.

```
 164                                <!-- Строки таблицы. Столбец "Тип части энергосистемы России" -->
 165                                <td>
 166                                    <input type="text" name="energy_system_type_names[]" class="form-
control" value="{{ energy_system_type.name|e }}" maxlength="255">
 167                                </td>
 168
 169                                <!-- Строки таблицы. Столбец "Чекмарк для удаления" -->
 170                                <td class="text-center align-middle">
 171                                    <input type="checkbox" name="energy_system_type_delete[]" value="{{
energy_system_type.id }}" class="form-check-input" aria-label="Удалить запись {{ energy_system_type.id
}}">
 172                                </td>
 173                            </tr>
 174                        {% else %}
 175                            <tr>
 176                                <td colspan="3" class="text-center align-middle">Нет данных для
отображения</td>
 177                            </tr>
```

Взаимодействие с другими модулями. UnionEnergySystem.id_energy_system_type ссылается на этот классификатор. EnergySystemTypeDemandParameter связывает тип с параметрами потребности в мощности. Расчетные модули используют тип при отборе и группировке энергетических объектов.

Подтверждающие ссылки на потребителей модели:
app/energy_consumption/models/energy_systems/energy_system_type_energy_consumption_parameter_model.py:26 – "EnergySystemType",
app/power_demand/models/energy_systems/energy_system_type_demand_parameter_model.py:2 – """Параметры нагрузки (demand) для типа энергосистемы (EnergySystemType)."""
app/api/services/gentypes_info_exchange_services.py:94 – from app.refdata.models.energy_systems.energy_system_type_model import EnergySystemType
app/energy_balance/services/ee_generation_hierarchy_control_services.py:264 – EnergySystemType as entity_model,
app/fuel/services/stations/stations_equipment_groups_services.py:3681 – from app.refdata.models.energy_systems.energy_system_type_model import EnergySystemType

### 4.9.2. Форма добавления

Адрес: GET, POST /refdata/add_energy_system_type. Обработчик: add_energy_system_type().

Назначение: создание записи справочника «Типы энергосистем». Вводимые поля формы: name. Список полей формы может быть уже модели: дополнительные реквизиты изменяются на странице списка или заполняются сервисом.

GET создает форму и подготавливает варианты выбора. POST с валидной формой формирует данные и передает их сервису добавления. При успехе выполняется переход к списку; сообщения о результате выводятся пользователю. Кнопка отмены возвращает к списку без создания записи. Создаваемая сущность – EnergySystemType, модель и ее связи приведены выше в подразделе 4.9.1.

Поля WTForms и валидаторы: name = StringField('Наименование типа части энергосистемы России', validators=[DataRequired(message='Укажите наименование типа части энергосистемы России.'), Length(max=255)], render_kw={'placeholder': 'Наименование типа части энергосистемы России'}); submit = SubmitField('Сохранить').

Шаблон содержит кнопку добавления во всех версиях для административного пользователя. Серверная ветвь all_versions=1 вызывает специализированную проверку доступа и межверсионный сервис; при обычном добавлении используется выбранная версия.

Модель добавляемого объекта EnergySystemType – основной реквизит. Источник: app/refdata/models/energy_systems/energy_system_type_model.py, строки 19–19.

```
  19      name = db.Column(db.String(255), unique=True, nullable=False, index=True)
```

Маршрут добавления – вход в форму. Источник: app/refdata/routes/energy_systems/energy_system_type_routes.py, строки 174–186.

```
 174  @refdata_bp.route("/add_energy_system_type", methods=["GET", "POST"])
 175  @login_required
 176  def add_energy_system_type():
 177      """ Маршрут для добавления новой части энергосистемы России. """
 178
 179      user = session.get('username', 'Неизвестный пользователь')
 180      log_to_db(
 181          user,
 182          "Открыта страница добавления части энергосистемы России",
 183          entity_type="energy_system_type")
 184
 185      # Создание формы
 186      form = AddEnergySystemTypeForm()
```

Маршрут добавления – вызов сервиса. Источник: app/refdata/routes/energy_systems/energy_system_type_routes.py, строки 219–229.

```
 219                                 sort_dir=sort_dir,
 220                         ))
 221                 add_energy_system_type_all_versions_service(payload, user)
 222                 flash("Новая запись добавлена во всех версиях БД (общий ref_uuid).", "success")
 223             else:
 224                 add_energy_system_type_service(payload, user)
 225                 flash("Новая запись успешно добавлена.", "success")
 226
 227             # Перенаправление на список с сохранением параметров и переходом к новой записи
 228             total_records = energy_system_type_query(
 229                                 energy_system_type_filter).count()
```

Сервис добавления – создание сущности. Источник: app/refdata/services/energy_systems/energy_system_type_services.py, строки 238–253.

```
 238                     raise ValueError(f"Запись с именем «{name}» уже существует.")
 239
 240                 # Создаем новую запись
 241                 obj = EnergySystemType(
 242                     name=name,
 243                 )
 244                 set_db_version_on_create(obj)
 245                 db.session.add(obj)
 246                 db.session.flush()  # получить id без полного коммита
 247
 248                 log_to_db(
 249                     user,
 250                     "Создана часть энергосистемы России",
 251                     (f"Наименование: {name};"),
 252                     entity_type="energy_system_type",
 253                     entity_id=obj.id)
```

HTML/Jinja2 формы добавления. Источник: app/templates/refdata/energy_systems/energy_system_type/energy_system_type_add.html, строки 22–36.

```
  22          <div class="mb-3">
  23              <label for="name" class="form-label">Наименование типа энергосистемы</label>
  24              {{ form.name(class="form-control", id="name") }}
  25          </div>
  26
  27          <!-- Кнопки сохранения записи и возвращения назад -->
  28          <div class="mt-4 d-flex flex-wrap gap-2 align-items-center">
  29              <button type="submit" class="btn btn-primary">Добавить</button>
  30              {% if current_user.has_admin %}<button type="submit" name="all_versions" value="1"
 class="btn btn-outline-primary" title="Добавить запись во все версии БД">Добавить во всех
 версиях</button>{% endif %}
  31
  32              <a href="{{ url_for('refdata_bp.energy_system_type_list',
  33                                  page=page,
  34                                  per_page=per_page,
  35                                  sort_by=sort_by,
  36                                  sort_dir=sort_dir,
```

Взаимодействие при добавлении. UnionEnergySystem.id_energy_system_type ссылается на этот классификатор. EnergySystemTypeDemandParameter связывает тип с параметрами потребности в мощности. Расчетные модули используют тип при отборе и группировке энергетических объектов. После создания запись становится доступна для выбора в зависимых формах при их следующем построении с учетом версии и используемых механизмов кэширования.

4.9.3. Экспорт, импорт и дополнительные операции

| Адрес и метод | Обработчик | Результат |
|---|---|---|
| GET /refdata/export_energy_system_type | export_energy_system_type | Файл Excel для скачивания |

Отдельный маршрут импорта в этом справочнике не зарегистрирован. Поддержка импорта не приписывается всем справочникам по аналогии.

## 4.10. Синхронные зоны

4.10.1. Страница списка

Адрес: GET, POST /refdata/synchronous_area. Обработчик: synchronous_area_list().

Ведет номер, краткое и полное наименования синхронной зоны и порядок ее отображения. Список позволяет искать, сортировать и изменять записи; отдельная форма добавляет зону.

Поля и параметры HTML-формы: per_page, page, sort_by, sort_dir, synchronous_area_filter, all_versions, synchronous_area_ids[], synchronous_area_numbers[], display_orders[], synchronous_area_names[], synchronous_area_full_names[], synchronous_area_delete[], js/refdata/textarea_autoresize.js.

Обработка: маршрут получает параметры отображения, при POST читает изменяемые строки и флажки удаления, вызывает сервисы и возвращает пользователя к списку с параметрами просмотра. Для обычного просмотра строится выборка и передается в шаблон вместе с пагинацией и вариантами выбора связанных справочников.

Модель: SynchronousArea. Поля SQLAlchemy: id, display_order, number, name, name_full, created_at, updated_at, database_version_id, regional_districts, demand_parameters.

Модель данных – фрагмент объявления. Источник: app/refdata/models/energy_systems/synchronous_area_model.py, строки 13–33.

```
class SynchronousArea(db.Model, AuditMixin, VersionedModelMixin, RefdataUuidMixin):
    __tablename__ = 'gs_sys_synchronous_areas'
    __table_args__ = {"schema": SCHEMA_REFDATA}

    id = db.Column(db.Integer, primary_key=True, autoincrement=True)
    # Порядок отображения в справочнике
    display_order = db.Column(db.Integer, nullable=True)
    number = db.Column(db.String(256), unique=True, nullable=True, index=True)
    name = db.Column(db.String(256), unique=True, nullable=False, index=True)
    name_full = db.Column(db.String(256), nullable=True, index=True)

    created_at = db.Column(db.DateTime(timezone=True), server_default=func.now(), nullable=False)
    updated_at = db.Column(db.DateTime(timezone=True), server_default=func.now(),
onupdate=func.now(), nullable=False)

    # Поле для связи с версией БД
    database_version_id = db.Column(
        db.Integer,
        db.ForeignKey(f"{SCHEMA_REFDATA}.gs_database_versions.id", ondelete="SET NULL"),
        nullable=True,
        index=True
    )
```

Связь модели с другой сущностью. Источник: app/refdata/models/energy_systems/synchronous_area_model.py, строки 36–40.

```
    regional_districts = db.relationship(
        'RegionalDistrict',
        back_populates='synchronous_area',
        foreign_keys='RegionalDistrict.id_synchronous_area',
    )
```

Маршрут списка – вход и параметры. Источник: app/refdata/routes/energy_systems/synchronous_area_routes.py, строки 35–50.

```
@refdata_bp.route("/synchronous_area", methods=["GET", "POST"])
@login_required
def synchronous_area_list():
    """Маршрут для отображения списка синхронных зон."""

    user = session.get('username', 'Неизвестный пользователь')
    log_to_db(
        user,
        "Открыта страница синхронных зон",
        entity_type="synchronous_area")

    # Создание формы
    form = SynchronousAreaFilterForm()

    # Получение параметров запроса
    page                        = request.args.get("page", 1, type=int)
```

Маршрут списка – передача изменений сервису. Источник: app/refdata/routes/energy_systems/synchronous_area_routes.py, строки 167–172.

```
                            sort_dir=sort_dir,
                        ))
                update_synchronous_area_all_versions_service(synchronous_area_data, user)
                flash("Изменения применены во всех версиях БД (по ref_uuid).", "success")
            else:
                update_synchronous_area_service(synchronous_area_data, user)
```

Сервис списка – выборка и пагинация. Источник: app/refdata/services/energy_systems/synchronous_area_services.py, строки 97–114.

```
 97  @no_autoflush
 98  def get_synchronous_area_list(
 99      page,
100      per_page,
101      synchronous_area_filter=None,
102      sort_by="display_order",
103      sort_dir="asc"):
104      """ Получает список синхронных зон с пагинацией, фильтрацией и сортировкой."""
105
106      # Базовый запрос
107      query = synchronous_area_query(
108          synchronous_area_filter=synchronous_area_filter,
109          sort_by=sort_by,
110          sort_dir=sort_dir,
111      )
112
113      # Пагинация
114      return query.paginate(page=page, per_page=per_page, error_out=False)
```

Сервис изменения – сохранение и журналирование. Источник: app/refdata/services/energy_systems/synchronous_area_services.py, строки 236–247.

```
236
237      try:
238          # Сохранение изменений в базе данных
239          # Фиксация транзакции (устойчивый коммит)
240          _commit_with_retry()
241
242          if updated_ids:
243              log_to_db(
244                  user,
245                  "Сохранены изменения по синхронным зонам",
246                  f"Измененных записей: {len(updated_ids)} (id: {updated_ids})",
247                  entity_type="synchronous_area")
```

HTML/Jinja2 списка – элементы формы. Источник: app/templates/refdata/energy_systems/synchronous_area/synchronous_area.html, строки 202–215.

```
202                                  <!-- Строки таблицы. Столбец "Наименование синхронной зоны" -->
203                                  <td>
204                                      <input type="text" name="synchronous_area_names[]" class="form-
control" value="{{ synchronous_area.name }}">
205                                  </td>
206
207                                  <!-- Строки таблицы. Столбец "Полное наименование синхронной зоны"
-->
208                                  <td>
209                                      <input type="text" name="synchronous_area_full_names[]"
class="form-control" value="{{ synchronous_area.name_full or '' }}">
210                                  </td>
211
212
213                                  <!-- Строки таблицы. Столбец "Чекмарк для удаления" -->
214                                  <td class="text-center align-middle">
215                                      <input type="checkbox" name="synchronous_area_delete[]"
value="{{ synchronous_area.id }}" class="form-check-input">
```

Взаимодействие с другими модулями. RegionalDistrict.id_synchronous_area определяет принадлежность субъекта к синхронной зоне. SynchronousAreaDemandParameter связывает зону с показателями потребности в мощности; территориальная агрегация опирается на соответствующие внешние ключи.

Подтверждающие ссылки на потребителей модели:
app/energy_consumption/models/energy_systems/synchronous_area_energy_consumption_parameter_model.py:26 – "SynchronousArea",
app/power_demand/models/energy_systems/synchronous_area_demand_parameter_model.py:2 – """Параметры нагрузки (demand) для синхронной зоны (SynchronousArea)."""
app/api/services/gentypes_info_exchange_services.py:99 – from app.refdata.models.energy_systems.synchronous_area_model import SynchronousArea

app/energy_balance/services/ee_generation_hierarchy_control_services.py:268 – SynchronousArea as entity_model, app/generation/services/station_changes_services/station_changes_services.py:1705 – from app.refdata.models.energy_systems.synchronous_area_model import SynchronousArea

### 4.10.2. Форма добавления

Адрес: GET, POST /refdata/add_synchronous_area. Обработчик: add_synchronous_area().

Назначение: создание записи справочника «Синхронные зоны». Вводимые поля формы: number, name, name_full. Список полей формы может быть уже модели: дополнительные реквизиты изменяются на странице списка или заполняются сервисом.

GET создает форму и подготавливает варианты выбора. POST с валидной формой формирует данные и передает их сервису добавления. При успехе выполняется переход к списку; сообщения о результате выводятся пользователю. Кнопка отмены возвращает к списку без создания записи. Создаваемая сущность – SynchronousArea, модель и ее связи приведены выше в подразделе 4.10.1.

Поля WTForms и валидаторы: csrf_token = HiddenField(); number = StringField('Номер синхронной зоны', validators=[DataRequired(message='Поле «Номер синхронной зоны» обязательно.'), Length(min=1, max=255, message='Длина от 1 до 255 символов.')]); name = StringField('Наименование синхронной зоны', validators=[Optional(), Length(max=100)], render_kw={'placeholder': 'Введите наименование синхронной зоны'}); name_full = StringField('Полное наименование синхронной зоны', validators=[Optional(), Length(max=256)], render_kw={'placeholder': 'Введите полное наименование синхронной зоны'}).

Шаблон содержит кнопку добавления во всех версиях для административного пользователя. Серверная ветвь all_versions=1 вызывает специализированную проверку доступа и межверсионный сервис; при обычном добавлении используется выбранная версия.

Модель добавляемого объекта SynchronousArea – основной реквизит. Источник: app/refdata/models/energy_systems/synchronous_area_model.py, строки 21–21.

```
21      name = db.Column(db.String(256), unique=True, nullable=False, index=True)
```

Маршрут добавления – вход в форму. Источник: app/refdata/routes/energy_systems/synchronous_area_routes.py, строки 206–218.

```
206 @refdata_bp.route("/add_synchronous_area", methods=["GET", "POST"])
207 @login_required
208 def add_synchronous_area():
209     """ Маршрут для добавления новой синхронной зоны. """
210
211     user = session.get('username', 'Неизвестный пользователь')
212     log_to_db(
213         user,
214         "Открыта страница добавления синхронной зоны",
215         entity_type="synchronous_area")
216
217     # Создание формы
218     form = AddSynchronousAreaForm()
```

Маршрут добавления – вызов сервиса. Источник:
app/refdata/routes/energy_systems/synchronous_area_routes.py, строки 254–264.

```
                            sort_dir=sort_dir,
                        ))
                add_synchronous_area_all_versions_service(payload, user)
                flash("Новая запись добавлена во всех версиях БД (общий ref_uuid).", "success")
            else:
                add_synchronous_area_service(payload, user)
                flash("Новая запись успешно добавлена.", "success")

             # Перенаправление на список с сохранением параметров и переходом к новой записи
            total_records = synchronous_area_query(
                                synchronous_area_filter).count()
```

Сервис добавления – создание сущности. Источник:
app/refdata/services/energy_systems/synchronous_area_services.py, строки 334–356.

```
                        )

                # Создаем новую запись
                obj = SynchronousArea(
                    name=name,
                    name_full=name_full,
                    number=number or None,
                    display_order=display_order,
                )
                set_db_version_on_create(obj)
                db.session.add(obj)
                db.session.flush()  # получить id без полного коммита

                log_to_db(
                    user,
                    "Создана синхронная зона",
                    (
                        f"Номер: {_dash(number)}; "
                        f"Наименование: {name}; "
                        f"Полное наименование: {_dash(name_full)}; "
                    ),
                    entity_type="synchronous_area",
                    entity_id=obj.id)
```

HTML/Jinja2 формы добавления. Источник:
app/templates/refdata/energy_systems/synchronous_area/synchronous_area_add.html, строки 22–36.

```
        <div class="mb-3">
            <label for="name" class="form-label">Номер синхронной зоны</label>
            {{ form.number(class="form-control", id="number") }}

            <label for="name" class="form-label">Наименование синхронной зоны</label>
            {{ form.name(class="form-control", id="name") }}

            <label for="name_full" class="form-label">Полное наименование синхронной зоны</label>
            {{ form.name_full(class="form-control", id="name_full") }}

        </div>

        <!-- Кнопки сохранения записи и возвращения назад -->
        <div class="mt-4 d-flex flex-wrap gap-2 align-items-center">
            <button type="submit" class="btn btn-primary">Добавить</button>
```

Взаимодействие при добавлении. RegionalDistrict.id_synchronous_area определяет принадлежность субъекта к синхронной зоне. SynchronousAreaDemandParameter связывает зону с показателями потребности в мощности; территориальная агрегация опирается на соответствующие внешние ключи. После создания запись становится доступна для выбора в зависимых формах при их следующем построении с учетом версии и используемых механизмов кэширования.

### 4.10.3. Экспорт, импорт и дополнительные операции

| Адрес и метод | Обработчик | Результат |
|---|---|---|

| Адрес и метод | Обработчик | Результат |
|---|---|---|
| GET /refdata/export_synchronous_area | export_synchronous_area | Файл Excel для скачивания |

Отдельный маршрут импорта в этом справочнике не зарегистрирован. Поддержка импорта не приписывается всем справочникам по аналогии.

## 4.11. Энергозоны

### 4.11.1. Страница списка

Адрес: GET, POST /refdata/energy_zone. Обработчик: energy_zone_list().

Ведет номера и наименования энергозон. В таблице выполняется поиск и изменение записей, отдельная форма создает новую зону. Модель содержит ограничения уникальности номера и наименования внутри версии БД.

Поля и параметры HTML-формы: per_page, page, sort_by, sort_dir, energy_zone_filter, all_versions, energy_zone_ids[], energy_zone_numbers[], energy_zone_names[], energy_zone_delete[], js/refdata/textarea_autoresize.js.

Обработка: маршрут получает параметры отображения, при POST читает изменяемые строки и флажки удаления, вызывает сервисы и возвращает пользователя к списку с параметрами просмотра. Для обычного просмотра строится выборка и передается в шаблон вместе с пагинацией и вариантами выбора связанных справочников.

Модель: EnergyZone. Поля SQLAlchemy: id, number, name, created_at, updated_at, database_version_id, demand_parameters, regional_districts.

Модель данных – фрагмент объявления. Источник: app/refdata/models/energy_systems/energy_zone_model.py, строки 13–31.

```
 13  class EnergyZone(db.Model, AuditMixin, VersionedModelMixin, RefdataUuidMixin):
 14      __tablename__ = 'gs_sys_energy_zones'
 15      __table_args__ = {"schema": SCHEMA_REFDATA}
 16
 17      id = db.Column(db.Integer, primary_key=True, autoincrement=True)
 18      number = db.Column(db.String(256), nullable=False, index=True)
 19      name = db.Column(db.String(256), nullable=False, index=True)
 20
 21      created_at = db.Column(db.DateTime(timezone=True), server_default=func.now(), nullable=False)
 22      updated_at = db.Column(db.DateTime(timezone=True), server_default=func.now(),
onupdate=func.now(), nullable=False)
 23
 24      # Поле для связи с версией БД
 25      database_version_id = db.Column(
 26          db.Integer,
 27          db.ForeignKey(f"{SCHEMA_REFDATA}.gs_database_versions.id", ondelete="SET NULL"),
 28          nullable=True,
 29          index=True
 30      )
 31
```

Связь модели с другой сущностью. Источник: app/refdata/models/energy_systems/energy_zone_model.py, строки 38–42.

```
 38      demand_parameters = db.relationship(
 39          "EnergyZoneDemandParameter",
 40          back_populates="energy_zone",
 41          cascade="all, delete-orphan",
 42      )
```

Маршрут списка – вход и параметры. Источник: app/refdata/routes/energy_systems/energy_zone_routes.py, строки 33–48.

```
@refdata_bp.route("/energy_zone", methods=["GET", "POST"])
@login_required
def energy_zone_list():
    """Маршрут для отображения списка энергозон."""

    user = session.get('username', 'Неизвестный пользователь')
    log_to_db(
        user,
        "Открыта страница: энергозоны",
        entity_type="energy_zone")

    # Создание формы
    form = EnergyZoneFilterForm()

    # Получение параметров запроса
    page                = request.args.get("page", 1, type=int)
```

Маршрут списка – передача изменений сервису. Источник: app/refdata/routes/energy_systems/energy_zone_routes.py, строки 136–141.

```
                            sort_dir=sort_dir,
                    ))
                update_energy_zone_all_versions_service(energy_zone_data, user)
                flash("Изменения применены во всех версиях БД (по ref_uuid).", "success")
            else:
                update_energy_zone_service(energy_zone_data, user)
```

Сервис списка – выборка и пагинация. Источник: app/refdata/services/energy_systems/energy_zone_services.py, строки 99–116.

```
@no_autoflush
def get_energy_zone_list(
    page,
    per_page,
    energy_zone_filter=None,
    sort_by="id",
    sort_dir="asc"):
    """ Получает список энергозон с пагинацией, фильтрацией и сортировкой. """

    # Базовый запрос
    query = energy_zone_query(
        energy_zone_filter=energy_zone_filter,
        sort_by=sort_by,
        sort_dir=sort_dir,
    )

    # Пагинация
    return query.paginate(page=page, per_page=per_page, error_out=False)
```

Сервис изменения – сохранение и журналирование. Источник: app/refdata/services/energy_systems/energy_zone_services.py, строки 199–210.

```

    try:
        # Сохранение изменений в базе данных
        # Фиксация транзакции (устойчивый коммит)
        _commit_with_retry()

        if updated_ids:
            log_to_db(
                user,
                "Сохранены изменения по энергозонам",
                f"Измененных записей: {len(updated_ids)} (id: {updated_ids})",
                entity_type="energy_zone")
```

HTML/Jinja2 списка – элементы формы. Источник:
app/templates/refdata/energy_systems/energy_zone/energy_zone.html, строки 169–182.

```
 169                                    <!-- Строки таблицы. Столбец "Номер энергозоны" -->
 170                                    <td class="text-center align-middle">
 171                                        <input type="text" name="energy_zone_numbers[]" class="form-
control" value="{{ energy_zone.number }}">
 172                                    </td>
 173
 174                                    <!-- Строки таблицы. Столбец "Наименование энергозоны" -->
 175                                    <td>
 176                                        <input type="text" name="energy_zone_names[]" class="form-
control" value="{{ energy_zone.name }}">
 177                                    </td>
 178
 179
 180                                    <!-- Строки таблицы. Столбец "Чекмарк для удаления" -->
 181                                    <td class="text-center align-middle">
 182                                        <input type="checkbox" name="energy_zone_delete[]" value="{{
energy_zone.id }}" class="form-check-input">
```

Взаимодействие с другими модулями. RegionalDistrict.id_energy_zone связывает субъекты РФ с энергозонами; EnergyZoneDemandParameter хранит показатели потребности в мощности по этим зонам. Согласованность территориальных агрегатов обеспечивается общими сервисами и сервисом субъектов РФ, а не копированием названий в расчетные таблицы.

Подтверждающие ссылки на потребителей модели:
app/energy_consumption/models/energy_systems/energy_zone_energy_consumption_parameter_model.py:26 – "EnergyZone",
app/power_demand/models/energy_systems/energy_zone_demand_parameter_model.py:2 – """Параметры нагрузки (demand) для энергозоны (EnergyZone)."""
app/rational_structure/models/territory_technology_model.py:18 – energy_zone = db.relationship("EnergyZone")
app/fuel/routes/fuel_routes.py:394 – from app.refdata.models.energy_systems.energy_zone_model import EnergyZone
app/generation/services/station_changes_services/station_changes_services.py:16 – from app.refdata.models.energy_systems.energy_zone_model import EnergyZone

### 4.11.2. Форма добавления

Адрес: GET, POST /refdata/add_energy_zone. Обработчик: add_energy_zone().

Назначение: создание записи справочника «Энергозоны». Вводимые поля формы: number, name. Список полей формы может быть уже модели: дополнительные реквизиты изменяются на странице списка или заполняются сервисом.

GET создает форму и подготавливает варианты выбора. POST с валидной формой формирует данные и передает их сервису добавления. При успехе выполняется переход к списку; сообщения о результате выводятся пользователю. Кнопка отмены возвращает к списку без создания записи. Создаваемая сущность – EnergyZone, модель и ее связи приведены выше в подразделе 4.11.1.

Поля WTForms и валидаторы: csrf_token = HiddenField(); number = StringField('Номер энергозоны', validators=[DataRequired(message='Поле «Номер энергозоны» обязательно.'), Length(min=1, max=255, message='Длина от 1 до 255 символов.')]); name = StringField('Наименование энергозоны', validators=[Optional(), Length(max=100)], render_kw={'placeholder': 'Введите наименование энергозоны'}).

Шаблон содержит кнопку добавления во всех версиях для административного пользователя. Серверная ветвь all_versions=1 вызывает специализированную проверку доступа и межверсионный сервис; при обычном добавлении используется выбранная версия.

Модель добавляемого объекта EnergyZone – основной реквизит. Источник: app/refdata/models/energy_systems/energy_zone_model.py, строки 19–19.

```
19      name = db.Column(db.String(256), nullable=False, index=True)
```

Маршрут добавления – вход в форму. Источник: app/refdata/routes/energy_systems/energy_zone_routes.py, строки 174–186.

```
174 @refdata_bp.route("/add_energy_zone", methods=["GET", "POST"])
175 @login_required
176 def add_energy_zone():
177     """ Маршрут для добавления новой энергозоны». """
178 
179     user = session.get('username', 'Неизвестный пользователь')
180     log_to_db(
181         user,
182         "Открыта страница добавления энергозоны",
183         entity_type="energy_zone")
184 
185     # Создание формы
186     form = AddEnergyZoneForm()
```

Маршрут добавления – вызов сервиса. Источник: app/refdata/routes/energy_systems/energy_zone_routes.py, строки 219–229.

```
219                                 sort_dir=sort_dir,
220                     ))
221                 add_energy_zone_all_versions_service(payload, user)
222                 flash("Новая запись добавлена во всех версиях БД (общий ref_uuid).", "success")
223             else:
224                 add_energy_zone_service(payload, user)
225                 flash("Новая запись успешно добавлена.", "success")
226 
227             # Перенаправление на список с сохранением параметров и переходом к новой записи
228             total_records = energy_zone_query(
229                                     energy_zone_filter).count()
```

Сервис добавления – создание сущности. Источник: app/refdata/services/energy_systems/energy_zone_services.py, строки 272–291.

```
272                     raise ValueError(f"Запись с полным наименованием «{number}» уже существует.")
273 
274                 # Создаем новую запись
275                 obj = EnergyZone(
276                     name=name,
277                     number=number or None,
278                 )
279                 set_db_version_on_create(obj)
280                 db.session.add(obj)
281                 db.session.flush()  # получить id без полного коммита
282 
283                 log_to_db(
284                     user,
285                     "Создана энергозона",
286                     (
287                         f"Номер: {_dash(number)};"
288                         f"Наименование: {name}. "
289                     ),
290                     entity_type="energy_zone",
291                     entity_id=obj.id)
```

HTML/Jinja2 формы добавления. Источник:
app/templates/refdata/energy_systems/energy_zone/energy_zone_add.html, строки 22–36.

```
  22          <div class="mb-3">
  23              <label for="name" class="form-label">Номер энергозоны</label>
  24              {{ form.number(class="form-control", id="number") }}
  25
  26              <label for="name" class="form-label">Наименование энергозоны</label>
  27              {{ form.name(class="form-control", id="name") }}
  28          </div>
  29
  30          <!-- Кнопки сохранения записи и возвращения назад -->
  31          <div class="mt-4 d-flex flex-wrap gap-2 align-items-center">
  32              <button type="submit" class="btn btn-primary">Добавить</button>
  33              {% if current_user.has_admin %}<button type="submit" name="all_versions" value="1"
class="btn btn-outline-primary" title="Добавить запись во все версии БД">Добавить во всех
версиях</button>{% endif %}
  34
  35              <a href="{{ url_for('refdata_bp.energy_zone_list',
  36                                  page=page,
```

Взаимодействие при добавлении. RegionalDistrict.id_energy_zone связывает субъекты РФ с энергозонами; EnergyZoneDemandParameter хранит показатели потребности в мощности по этим зонам. Согласованность территориальных агрегатов обеспечивается общими сервисами и сервисом субъектов РФ, а не копированием названий в расчетные таблицы. После создания запись становится доступна для выбора в зависимых формах при их следующем построении с учетом версии и используемых механизмов кэширования.

4.11.3. Экспорт, импорт и дополнительные операции

| Адрес и метод | Обработчик | Результат |
|---|---|---|
| GET /refdata/export_energy_zone | export_energy_zone | Файл Excel для скачивания |

Отдельный маршрут импорта в этом справочнике не зарегистрирован. Поддержка импорта не приписывается всем справочникам по аналогии.

## 4.12. Генерирующие компании

4.12.1. Страница списка

Адрес: GET, POST /refdata/gen_company. Обработчик: gen_company_list().

Ведет полное и сокращенное наименования генерирующих компаний. Реализованы табличное редактирование, создание записи, импорт и экспорт Excel.

Поля и параметры HTML-формы: per_page, page, sort_by, sort_dir, gen_company_filter, all_versions, gen_company_ids[], gen_company_names[], gen_company_delete[], js/refdata/textarea_autoresize.js.

Обработка: маршрут получает параметры отображения, при POST читает изменяемые строки и флажки удаления, вызывает сервисы и возвращает пользователя к списку с параметрами просмотра. Для обычного просмотра строится выборка и передается в шаблон вместе с пагинацией и вариантами выбора связанных справочников.

Модель: GenCompany. Поля SQLAlchemy: id, name, name_short, created_at, updated_at, database_version_id, machines.

Модель данных – фрагмент объявления. Источник: app/refdata/models/gen_companies/gen_company_model.py, строки 13–31.

```
class GenCompany(db.Model, AuditMixin, VersionedModelMixin, RefdataUuidMixin):
    __tablename__ = 'gs_sys_companies'
    __table_args__ = {"schema": SCHEMA_REFDATA}

    id = db.Column(db.Integer, primary_key=True, autoincrement=True)
    name = db.Column(db.String(80), unique=True, nullable=False, index=True)
    name_short = db.Column(db.String(80), unique=False, nullable=True, index=True)

    created_at = db.Column(db.DateTime(timezone=True), server_default=func.now(), nullable=False)
    updated_at = db.Column(db.DateTime(timezone=True), server_default=func.now(),
onupdate=func.now(), nullable=False)

    # Поле для связи с версией БД
    database_version_id = db.Column(
        db.Integer,
        db.ForeignKey(f"{SCHEMA_REFDATA}.gs_database_versions.id", ondelete="SET NULL"),
        nullable=True,
        index=True
    )

```

Связь модели с другой сущностью. Источник: app/refdata/models/gen_companies/gen_company_model.py, строки 33–33.

```
    machines = db.relationship('Machine', back_populates='gen_company')
```

Маршрут списка – вход и параметры. Источник: app/refdata/routes/gen_companies/gen_company_routes.py, строки 36–51.

```
@refdata_bp.route("/gen_company", methods=["GET", "POST"])
@login_required
def gen_company_list():
    """Маршрут для отображения генерирующих компаний"""

    user = session.get('username', 'Неизвестный пользователь')
    log_to_db(
        user,
        "Открыта страница генерирующих компаний",
        entity_type="gen_company")

    # Создание формы
    form = GenCompanyFilterForm()

    # Получение параметров запроса
    page                = request.args.get("page", 1, type=int)
```

Маршрут списка – передача изменений сервису. Источник: app/refdata/routes/gen_companies/gen_company_routes.py, строки 127–132.

```
                               sort_dir=sort_dir,
                      ))
                update_gen_company_all_versions_service(gen_company_data, user)
                flash("Изменения применены во всех версиях БД (по ref_uuid).", "success")
            else:
                update_gen_company_service(gen_company_data, user)
```

Сервис списка – выборка и пагинация. Источник: app/refdata/services/gen_companies/gen_company_services.py, строки 90–107.

```
 90  @no_autoflush
 91  def get_gen_company_list(
 92      page,
 93      per_page,
 94      gen_company_filter=None,
 95      sort_by="id",
 96      sort_dir="asc"):
 97      """ Получает список генерирующих компаний с пагинацией, фильтрацией и сортировкой. """
 98
 99      # Базовый запрос
100      query = gen_company_query(
101          gen_company_filter=gen_company_filter,
102          sort_by=sort_by,
103          sort_dir=sort_dir,
104      )
105
106      # Пагинация
107      return query.paginate(page=page, per_page=per_page, error_out=False)
```

Сервис изменения – сохранение и журналирование. Источник: app/refdata/services/gen_companies/gen_company_services.py, строки 176–187.

```
176
177      try:
178          # Сохранение изменений в базе данных
179          # Фиксация транзакции (устойчивый коммит)
180          _commit_with_retry()
181
182          if updated_ids:
183              log_to_db(
184                  user,
185                  "Сохранены изменения по генерирующим компаниям",
186                  f"Измененных записей: {len(updated_ids)} (id: {updated_ids})",
187                  entity_type="gen_company")
```

HTML/Jinja2 списка – элементы формы. Источник: app/templates/refdata/gen_companies/gen_company.html, строки 159–172.

```
159                                 <!-- Строки таблицы. Столбец "Генерирующая копмания" -->
160                                 <td>
161                                     <input type="text" name="gen_company_names[]" class="form-
control" value="{{ gen_company.name }}">
162                                 </td>
163
164                                 <!-- Строки таблицы. Столбец "Чекмарк для удаления" -->
165                                 <td class="text-center align-middle">
166                                     <input type="checkbox" name="gen_company_delete[]" value="{{
gen_company.id }}" class="form-check-input">
167                                 </td>
168                             </tr>
169                         {% else %}
170                             <tr>
171                                 <td colspan="4" class="text-center align-middle">Нет данных для
отображения</td>
172                             </tr>
```

Взаимодействие с другими модулями. Machine.id_gen_company связывает агрегат с компанией. Данные используются в разделе генерирующих мощностей для представления владельца и группировки оборудования; изменение справочника меняет отображаемое наименование компании при чтении связанной модели.

Подтверждающие ссылки на потребителей модели:
app/fuel/models/external_mapping/fue_em_gen_company_branch_model.py:6 – Поле `name` используется для поиска и сопоставления с GenCompany.name.
app/generation/models/machine/machine_model.py:64 – gen_company = db.relationship('GenCompany', back_populates='machines')
app/rational_structure/services/technical_characteristic_machine_services.py:12 – from app.refdata.models.gen_companies.gen_company_model import GenCompany
app/fuel/routes/fuel_routes.py:385 – from app.refdata.models.gen_companies.gen_company_model import GenCompany

app/fuel/services/fuel_imports/import_fuel_db_equipment_groups_services.py:26 – from app.refdata.models.gen_companies.gen_company_model import GenCompany

### 4.12.2. Форма добавления

Адрес: GET, POST /refdata/add_gen_company. Обработчик: add_gen_company().

Назначение: создание записи справочника «Генерирующие компании». Вводимые поля формы: name. Список полей формы может быть уже модели: дополнительные реквизиты изменяются на странице списка или заполняются сервисом.

GET создает форму и подготавливает варианты выбора. POST с валидной формой формирует данные и передает их сервису добавления. При успехе выполняется переход к списку; сообщения о результате выводятся пользователю. Кнопка отмены возвращает к списку без создания записи. Создаваемая сущность – GenCompany, модель и ее связи приведены выше в подразделе 4.12.1.

Поля WTForms и валидаторы: csrf_token = HiddenField(); name = StringField('Наименование генерирующей компании', validators=[DataRequired(message='Поле «Наименование генерирующей компании» обязательно.'), Length(min=3, max=255, message='Длина от 3 до 255 символов.')]).

Шаблон содержит кнопку добавления во всех версиях для административного пользователя. Серверная ветвь all_versions=1 вызывает специализированную проверку доступа и межверсионный сервис; при обычном добавлении используется выбранная версия.

Модель добавляемого объекта GenCompany – основной реквизит. Источник: app/refdata/models/gen_companies/gen_company_model.py, строки 18–18.

```
18      name = db.Column(db.String(80), unique=True, nullable=False, index=True)
```

Маршрут добавления – вход в форму. Источник: app/refdata/routes/gen_companies/gen_company_routes.py, строки 166–178.

```
166 @refdata_bp.route("/add_gen_company", methods=["GET", "POST"])
167 @login_required
168 def add_gen_company():
169     """ Маршрут для добавления новой генерирующей компании. """
170
171     user = session.get('username', 'Неизвестный пользователь')
172     log_to_db(
173         user,
174         "Открыта страница добавления генерирующей компании",
175         entity_type="gen_company")
176
177     # Создание формы
178     form = AddGenCompanyForm()
```

Маршрут добавления – вызов сервиса. Источник: app/refdata/routes/gen_companies/gen_company_routes.py, строки 212–222.

```
212                                 sort_dir=sort_dir,
213                     ))
214                 add_gen_company_all_versions_service(payload, user)
215                 flash("Новая запись добавлена во всех версиях БД (общий ref_uuid).", "success")
216             else:
217                 add_gen_company_service(payload, user)
218                 flash("Новая запись успешно добавлена.", "success")
219
220             # Перенаправление на список с сохранением параметров и переходом к новой записи
221             total_records = gen_company_query(
222                                     gen_company_filter).count()
```

Сервис добавления – создание сущности. Источник: app/refdata/services/gen_companies/gen_company_services.py, строки 244–259.

```
244                     raise ValueError(f"Запись с наименованием «{name}» уже существует.")
245
246                 # Создаем новую запись
247                 obj = GenCompany(
248                     name=name,
249                 )
250                 set_db_version_on_create(obj)
251                 db.session.add(obj)
252                 db.session.flush()  # получить id без полного коммита
253
254                 log_to_db(
255                     user,
256                     "Создана генерирующая компания",
257                     f"Наименование: {name}.",
258                     entity_type="gen_company",
259                     entity_id=obj.id)
```

HTML/Jinja2 формы добавления. Источник: app/templates/refdata/gen_companies/gen_company_add.html, строки 22–36.

```
22          <div class="mb-3">
23              <label for="name" class="form-label">Наименование генерирующей компании</label>
24              {{ form.name(class="form-control", id="name") }}
25              {% if form.name.errors %}
26                  <div class="text-danger">
27                      {{ form.name.errors|join(', ') }}
28                  </div>
29              {% endif %}
30          </div>
31
32          <!-- Кнопки сохранения записи и возвращения назад -->
33          <div class="mt-4 d-flex flex-wrap gap-2 align-items-center">
34              <button type="submit" class="btn btn-primary">Добавить</button>
35              {% if current_user.has_admin %}<button type="submit" name="all_versions" value="1"
class="btn btn-outline-primary" title="Добавить запись во все версии БД">Добавить во всех
версиях</button>{% endif %}
36
```

Взаимодействие при добавлении. Machine.id_gen_company связывает агрегат с компанией. Данные используются в разделе генерирующих мощностей для представления владельца и группировки оборудования; изменение справочника меняет отображаемое наименование компании при чтении связанной модели. После создания запись становится доступна для выбора в зависимых формах при их следующем построении с учетом версии и используемых механизмов кэширования.

4.12.3. Экспорт, импорт и дополнительные операции

| Адрес и метод | Обработчик | Результат |
|---|---|---|
| POST /refdata/import_gen_company | import_gen_company | Обработка загруженного файла и сообщение о результате |
| GET /refdata/export_gen_company | export_gen_company | Файл Excel для скачивания |

Отдельный маршрут импорта в этом справочнике предусмотрен; наличие маршрута не означает наличие видимой кнопки загрузки в текущем шаблоне.

## 4.13. Типы топлива

4.13.1. Страница списка

Адрес: GET, POST /refdata/fuel_type. Обработчик: fuel_type_list().

Хранит тип топлива, порядок отображения и внешнее наименование nazvl. Предусмотрены редактирование, добавление, импорт и экспорт. Тип топлива является родительским классификатором для видов топлива.

Поля и параметры HTML-формы: per_page, page, sort_by, sort_dir, nazvl_filter, fuel_type_filter, all_versions, fuel_ids[], display_orders[], fuel_type_names[], fuel_type_nazvl[], fuel_type_delete[], js/refdata/textarea_autoresize.js.

Обработка: маршрут получает параметры отображения, при POST читает изменяемые строки и флажки удаления, вызывает сервисы и возвращает пользователя к списку с параметрами просмотра. Для обычного просмотра строится выборка и передается в шаблон вместе с пагинацией и вариантами выбора связанных справочников.

Модель: FuelType. Поля SQLAlchemy: id, display_order, name, nazvl, created_at, updated_at, database_version_id, fuels.

Модель данных – фрагмент объявления. Источник: app/refdata/models/fuels/fuel_type_model.py, строки 13–34.

```
class FuelType(db.Model, AuditMixin, VersionedModelMixin, RefdataUuidMixin):
    __tablename__ = 'gs_sys_fuel_types'
    __table_args__ = {"schema": SCHEMA_REFDATA}

    id = db.Column(db.Integer, primary_key=True, autoincrement=True)
    # Порядок отображения в справочнике
    display_order = db.Column(db.Integer, nullable=True)
    name = db.Column(db.String(80), unique=True, nullable=False, index=True)

    # Название типа топлива из базы Топливо
    nazvl = db.Column(db.String(80), nullable=True)

    created_at = db.Column(db.DateTime(timezone=True), server_default=func.now(), nullable=False)
    updated_at = db.Column(db.DateTime(timezone=True), server_default=func.now(),
onupdate=func.now(), nullable=False)

    # Поле для связи с версией БД
    database_version_id = db.Column(
        db.Integer,
        db.ForeignKey(f"{SCHEMA_REFDATA}.gs_database_versions.id", ondelete="SET NULL"),
        nullable=True,
        index=True
    )
```

Связь модели с другой сущностью. Источник: app/refdata/models/fuels/fuel_type_model.py, строки 37–37.

```
    fuels = db.relationship('Fuel', back_populates='fuel_type')
```

Маршрут списка – вход и параметры. Источник: app/refdata/routes/fuels/fuel_type_routes.py, строки 37–52.

```
@refdata_bp.route("/fuel_type", methods=["GET", "POST"])
@login_required
def fuel_type_list():
    """Маршрут для отображения списка видов топлива."""

    def _normalize_filter(value):
        if value in (None, "", "None"):
            return None
        return value

    user = session.get('username', 'Неизвестный пользователь')
    log_to_db(user, "Открыта страница видов топлива", entity_type="fuel_type")

    # Создание формы
    form = FuelTypeFilterForm()

```

Маршрут списка – передача изменений сервису. Источник: app/refdata/routes/fuels/fuel_type_routes.py, строки 151–156.

```
                            sort_dir=sort_dir,
                    ))
                update_fuel_type_all_versions_service(fuel_type_data, user)
                flash("Изменения применены во всех версиях БД (по ref_uuid).", "success")
            else:
                update_fuel_type_service(fuel_type_data, user)
```

Сервис списка – выборка и пагинация. Источник: app/refdata/services/fuels/fuel_type_services.py, строки 110–129.

```
@no_autoflush
def get_fuel_type_list(
    page,
    per_page,
    fuel_type_filter=None,
    nazvl_filter=None,
    sort_by="display_order",
    sort_dir="asc"):
    """ Получает список видов топлива с пагинацией, фильтрацией и сортировкой. """

    # Базовый запрос
    query = fuel_type_query(
        fuel_type_filter=fuel_type_filter,
        nazvl_filter=nazvl_filter,
        sort_by=sort_by,
        sort_dir=sort_dir,
    )

    # Пагинация
    return query.paginate(page=page, per_page=per_page, error_out=False)
```

Сервис изменения – сохранение и журналирование. Источник: app/refdata/services/fuels/fuel_type_services.py, строки 232–244.

```

    try:
        # Сохранение изменений в базе данных
        # Фиксация транзакции (устойчивый коммит)
        _commit_with_retry()
        _invalidate_fuel_type_caches()

        if updated_ids:
            log_to_db(
                user,
                "Сохранены изменения по видам топлива",
                f"Измененных записей: {len(updated_ids)} (id: {updated_ids})",
                entity_type="fuel_type")
```

HTML/Jinja2 списка – элементы формы. Источник: app/templates/refdata/fuels/fuel_type/fuel_type.html, строки 261–274.

```
                            <!-- Строки таблицы. Столбец "Тип топлива" -->
                            <td>
                                <input type="text" name="fuel_type_names[]" class="form-control"
value="{{ fuel_type.name }}">
                            </td>

                            <!-- Строки таблицы. Столбец "Короткое наименование" -->
                            <td>
                                <input type="text" name="fuel_type_nazvl[]" class="form-control"
value="{{ fuel_type.nazvl or '' }}">
                            </td>

                            <!-- Строки таблицы. Столбец "Чекмарк для удаления" -->
                            <td class="text-center align-middle">
                                <input type="checkbox" name="fuel_type_delete[]" value="{{
fuel_type.id }}" class="form-check-input">
                            </td>
```

Взаимодействие с другими модулями. Fuel.id_fuel_type ссылается на FuelType. Модули топлива, генерации и экологии используют эту классификацию для отбора и сопоставления исходных данных. Поле nazvl служит для согласования обозначений с внешними источниками данных.

Подтверждающие ссылки на потребителей модели:
app/api/services/generation_objects_exchange_services.py:39 – from app.refdata.models.fuels.fuel_type_model import FuelType
app/fuel/refdata/routes/fuel_routes.py:25 – from app.refdata.models.fuels.fuel_type_model import FuelType

app/generation/routes/totals_summary_routes.py:193 – from app.refdata.models.fuels.fuel_type_model import FuelType
app/fuel/refdata/routes/fuel_type_routes.py:21 – from app.refdata.models.fuels.fuel_type_model import FuelType
app/generation/services/station_changes_services/station_changes_services.py:18 – from app.refdata.models.fuels.fuel_type_model import FuelType

### 4.13.2. Форма добавления

Адрес: GET, POST /refdata/add_fuel_type. Обработчик: add_fuel_type().

Назначение: создание записи справочника «Типы топлива». Вводимые поля формы: name. Список полей формы может быть уже модели: дополнительные реквизиты изменяются на странице списка или заполняются сервисом.

GET создает форму и подготавливает варианты выбора. POST с валидной формой формирует данные и передает их сервису добавления. При успехе выполняется переход к списку; сообщения о результате выводятся пользователю. Кнопка отмены возвращает к списку без создания записи. Создаваемая сущность – FuelType, модель и ее связи приведены выше в подразделе 4.13.1.

Поля WTForms и валидаторы: csrf_token = HiddenField(); name = StringField('Вид топлива', validators=[DataRequired(message='Поле Вид топлива» обязательно.'), Length(min=3, max=255, message='Длина от 3 до 255 символов.')]).

Шаблон содержит кнопку добавления во всех версиях для административного пользователя. Серверная ветвь all_versions=1 вызывает специализированную проверку доступа и межверсионный сервис; при обычном добавлении используется выбранная версия.

Модель добавляемого объекта FuelType – основной реквизит. Источник: app/refdata/models/fuels/fuel_type_model.py, строки 20–20.

```
20      name = db.Column(db.String(80), unique=True, nullable=False, index=True)
```

Маршрут добавления – вход в форму. Источник: app/refdata/routes/fuels/fuel_type_routes.py, строки 212–224.

```
212 @refdata_bp.route("/add_fuel_type", methods=["GET", "POST"])
213 @login_required
214 def add_fuel_type():
215     """ Маршрут для добавления нового вида топлива. """
216
217     user = session.get('username', 'Неизвестный пользователь')
218     log_to_db(
219         user,
220         "Открыта страница добавления видов топлива",
221         entity_type="fuel_type")
222
223     # Создание формы
224     form = AddFuelTypeForm()
```

Маршрут добавления – вызов сервиса. Источник: app/refdata/routes/fuels/fuel_type_routes.py, строки 260–270.

```
260                                 sort_dir=sort_dir,
261                     ))
262                 add_fuel_type_all_versions_service(payload, user)
263                 flash("Новая запись добавлена во всех версиях БД (общий ref_uuid).", "success")
264             else:
265                 add_fuel_type_service(payload, user)
266                 flash("Новая запись успешно добавлена.", "success")
267
268             # Перенаправление на список с сохранением параметров и переходом к новой записи
269             total_records = fuel_type_query(
270                                 fuel_type_filter).count()
```

Сервис добавления – создание сущности. Источник: app/refdata/services/fuels/fuel_type_services.py, строки 307–320.

```
                                f"Запись с порядком отображения «{display_order}» уже существует."
                            )

                obj = FuelType(name=name, display_order=display_order)
                set_db_version_on_create(obj)
                db.session.add(obj)
                db.session.flush()

                log_to_db(
                    user,
                    "Создан вид топлива",
                    f"Наименование: {name}",
                    entity_type="fuel_type",
                    entity_id=obj.id)
```

HTML/Jinja2 формы добавления. Источник: app/templates/refdata/fuels/fuel_type/fuel_type_add.html, строки 20–34.

```
        <div class="mb-3">
            <label for="name" class="form-label">Тип топлива</label>
            {{ form.name(class="form-control", id="name") }}
            {% if form.name.errors %}
                <div class="text-danger">{{ form.name.errors|join(', ') }}</div>
            {% endif %}
        </div>

        <div class="mt-4 d-flex flex-wrap gap-2 align-items-center">
            <button type="submit" class="btn btn-primary">Добавить</button>
            {% if current_user.has_admin %}<button type="submit" name="all_versions" value="1"
class="btn btn-outline-primary" title="Добавить запись во все версии БД">Добавить во всех
версиях</button>{% endif %}
            <a href="{{ url_for('refdata_bp.fuel_type_list',
                                page=page,
                                per_page=per_page,
                                sort_by=sort_by,
```

Взаимодействие при добавлении. Fuel.id_fuel_type ссылается на FuelType. Модули топлива, генерации и экологии используют эту классификацию для отбора и сопоставления исходных данных. Поле nazvl служит для согласования обозначений с внешними источниками данных. После создания запись становится доступна для выбора в зависимых формах при их следующем построении с учетом версии и используемых механизмов кэширования.

### 4.13.3. Экспорт, импорт и дополнительные операции

| Адрес и метод | Обработчик | Результат |
|---|---|---|
| POST /refdata/import_fuel_type | import_fuel_type | Обработка загруженного файла и сообщение о результате |
| GET /refdata/export_fuel_type | export_fuel_type | Файл Excel для скачивания |

Отдельный маршрут импорта в этом справочнике предусмотрен; наличие маршрута не означает наличие видимой кнопки загрузки в текущем шаблоне.

## 4.14. Виды топлива

### 4.14.1. Страница списка

Адрес: GET, POST /refdata/fuel. Обработчик: fuel_list().

Хранит наименование, код kod, внешнее наименование nazvl, признак угольного топлива kmbur, тип топлива и родительский вид parent_id. Список поддерживает иерархическое представление; служебный GET /fuel/hierarchy

возвращает дерево в JSON. При выборе родителя сервис проверяет допустимость связи, включая недопустимость циклической иерархии.

Поля и параметры HTML-формы: per_page, page, sort_by, sort_dir, fuel_type_filter, nazvl_filter, kmbur_filter, fuel_filter, all_versions, fuel_ids[], fuel_kods[], fuel_names[], fuel_types[], fuel_parents[], fuel_nazvl[], fuel_kmbur[], fuel_delete[], js/refdata/textarea_autoresize.js.

Обработка: маршрут получает параметры отображения, при POST читает изменяемые строки и флажки удаления, вызывает сервисы и возвращает пользователя к списку с параметрами просмотра. Для обычного просмотра строится выборка и передается в шаблон вместе с пагинацией и вариантами выбора связанных справочников.

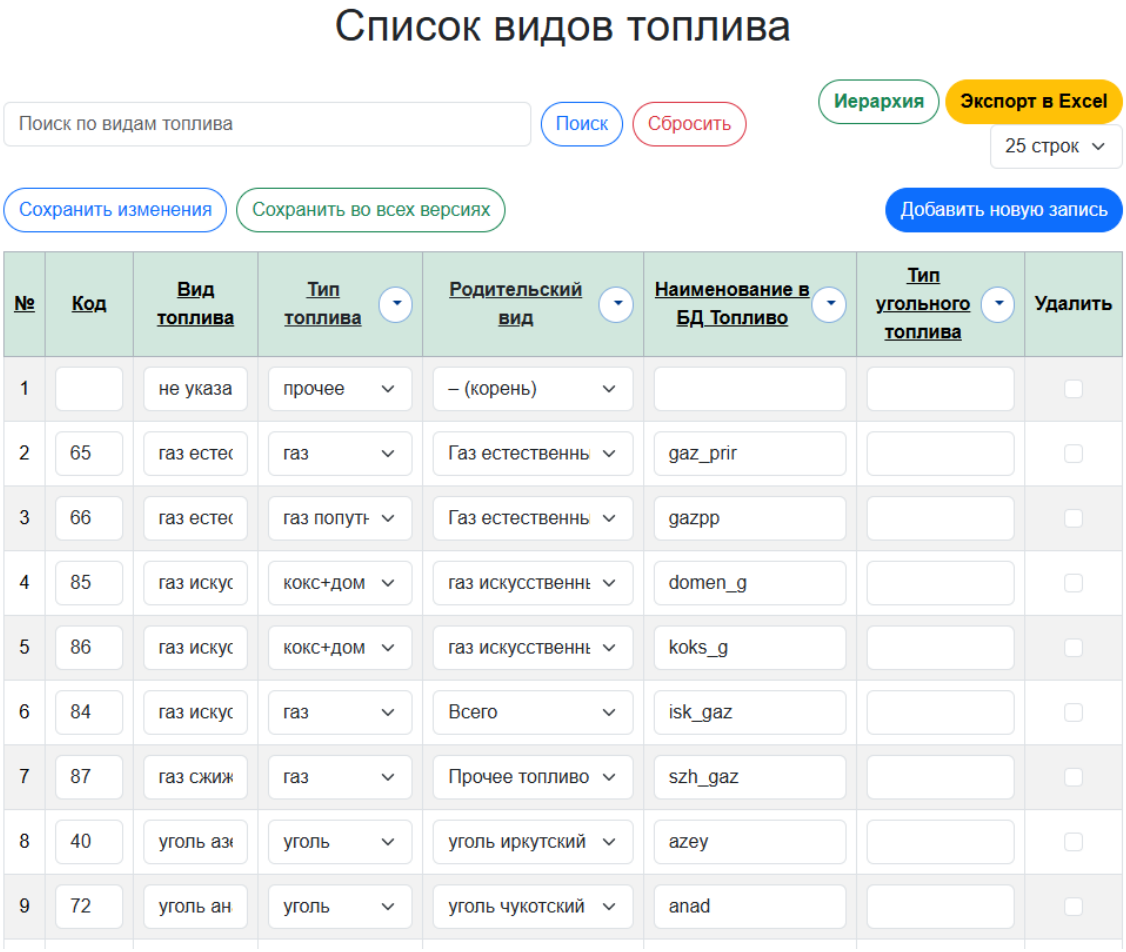



Рисунок 5. Виды топлива. Действующий сайт, 29.10.2026. Показан фрагмент экранной формы.

Модель: Fuel. Поля SQLAlchemy: id, name, kod, nazvl, kmbur, parent_id, parent, id_fuel_type, fuel_type, created_at, updated_at, database_version_id, machine_fuels.

Модель данных – фрагмент объявления. Источник: app/refdata/models/fuels/fuel_model.py, строки 13–35.

```
class Fuel(db.Model, AuditMixin, VersionedModelMixin, RefdataUuidMixin):
    __tablename__ = 'gs_sys_fuels'
    __table_args__ = {"schema": SCHEMA_REFDATA}

    id = db.Column(db.Integer, primary_key=True, autoincrement=True)
    name = db.Column(db.String(80), unique=True, nullable=False, index=True)

    # Код вида топлива из Excel «Состав видов топлива» (столбец Kod)
    kod = db.Column(db.Integer, nullable=True, index=True)

    # Название вида топлива из базы Топливо
    nazvl = db.Column(db.String(80), nullable=True)

    # Название типа угольного топлива из базы Топливо (каменный/бурый)
    kmbur = db.Column(db.String(80), nullable=True)

    # FK -> Fuel (родительский вид в иерархии состава)
    parent_id = db.Column(
        db.Integer,
        db.ForeignKey(f'{SCHEMA_REFDATA}.gs_sys_fuels.id', ondelete='SET NULL'),
        nullable=True,
        index=True,
    )
```

Связь модели с другой сущностью. Источник: app/refdata/models/fuels/fuel_model.py, строки 36–41.

```
    parent = db.relationship(
        'Fuel',
        remote_side=[id],
        backref=db.backref('children', lazy='dynamic'),
        foreign_keys=[parent_id],
    )
```

Маршрут списка – вход и параметры. Источник: app/refdata/routes/fuels/fuel_routes.py, строки 62–77.

```
@refdata_bp.route("/fuel", methods=["GET", "POST"])
@login_required
def fuel_list():
    """Маршрут для отображения списка типов топлива."""

    def _normalize_filter(value):
        if value in (None, "", "None"):
            return None
        return value

    user = session.get('username', 'Неизвестный пользователь')
    log_to_db(user, "Открыта страница типов топлива", entity_type="fuel")

    # Создание формы
    form = FuelFilterForm()

```

Маршрут списка – передача изменений сервису. Источник: app/refdata/routes/fuels/fuel_routes.py, строки 196–201.

```
                               sort_dir=sort_dir,
                    ))
                update_fuel_all_versions_service(fuel_data, user)
                flash("Изменения применены во всех версиях БД (по ref_uuid).", "success")
            else:
                update_fuel_service(fuel_data, user)
```

Сервис списка – выборка и пагинация. Источник: app/refdata/services/fuels/fuel_services.py, строки 145–167.

```
145  @no_autoflush
146  def get_fuel_list(
147      page,
148      per_page,
149      fuel_filter=None,
150      fuel_type_filter=None,
151      nazvl_filter=None,
152      kmbur_filter=None,
153      sort_by="id",
154      sort_dir="asc",
155      parent_filter=None):
156      """ Получает список типов топлива с пагинацией, фильтрацией и сортировкой. """
157
158      # Базовый запрос
159      query = fuel_query(
160          fuel_filter=fuel_filter,
161          fuel_type_filter=fuel_type_filter,
162          nazvl_filter=nazvl_filter,
163          kmbur_filter=kmbur_filter,
164          parent_filter=parent_filter,
165          sort_by=sort_by,
166          sort_dir=sort_dir,
167      ).options(joinedload(Fuel.fuel_type), joinedload(Fuel.parent))
```

Сервис изменения – сохранение и журналирование. Источник: app/refdata/services/fuels/fuel_services.py, строки 374–385.

```
374
375      try:
376          # Сохранение изменений в базе данных
377          # Фиксация транзакции (устойчивый коммит)
378          _commit_with_retry()
379
380          if updated_ids:
381              log_to_db(
382                  user,
383                  "Сохранены изменения по типам топлива",
384                  f"Измененных записей: {len(updated_ids)} (id: {updated_ids})",
385                  entity_type="fuel")
```

HTML/Jinja2 списка – элементы формы. Источник: app/templates/refdata/fuels/fuel/fuel.html, строки 326–339.

```
326                                          <ul class="dropdown-menu dropdown-menu-end fuel-parent-
filter-menu" aria-labelledby="parentFilterDropdownFuelAdmin">
327                                              <li class="fuel-parent-filter-search-wrap"
onclick="event.stopPropagation()">
328                                                  <input type="text"
329                                                         class="form-control form-control-sm fuel-
parent-filter-search"
330                                                         placeholder="Поиск..."
331                                                         autocomplete="off">
332                                              </li>
333                                              <li>
334                                                  <a href="{{ url_for('refdata_bp.fuel_list',
335                                                              page=pagination.page,
336                                                              fuel_filter=fuel_filter,
337                                                              fuel_type_filter=fuel_type_filter,
338                                                              parent_filter='',
339                                                              nazvl_filter=nazvl_filter,
```

Взаимодействие с другими модулями. MachineFuel связывает виды топлива с агрегатами и годами. Модули топлива и экологии используют единый классификатор, коды и внешние обозначения при импорте, сопоставлении и расчетах. Изменение parent_id меняет структуру состава топлива, но не подменяет исходные значения расхода топлива.

Подтверждающие ссылки на потребителей модели:

app/generation/models/machine/machine_fuel_model.py:4 – - Связи сохранены: year (Year.machine_fuels), machine_fuel (Machine.machine_fuels), fuel (Fuel.machine_fuels).

app/api/services/generation_objects_exchange_services.py:38 – from app.refdata.models.fuels.fuel_model import Fuel

app/energy_balance/services/station_ee_generation_page_services.py:54 – from app.refdata.models.fuels.fuel_model import Fuel
app/fuel/refdata/routes/fuel_routes.py:26 – from app.refdata.models.fuels.fuel_model import Fuel
app/fuel/routes/equipment_group_fuel_batch_ui_routes.py:454 – from app.refdata.models.fuels.fuel_model import Fuel

### 4.14.2. Форма добавления

Адрес: GET, POST /refdata/add_fuel. Обработчик: add_fuel().

Назначение: создание записи справочника «Виды топлива». Вводимые поля формы: name, kod, fuel_type, parent. Список полей формы может быть уже модели: дополнительные реквизиты изменяются на странице списка или заполняются сервисом.

GET создает форму и подготавливает варианты выбора. POST с валидной формой формирует данные и передает их сервису добавления. При успехе выполняется переход к списку; сообщения о результате выводятся пользователю. Кнопка отмены возвращает к списку без создания записи. Создаваемая сущность – Fuel, модель и ее связи приведены выше в подразделе 4.14.1.

Поля WTForms и валидаторы: csrf_token = HiddenField(); name = StringField('Тип топлива', validators=[DataRequired(message='Поле «Тип топлива» обязательно.'), Length(min=3, max=255, message='Длина от 3 до 255 символов.')]); kod = StringField('Код', validators=[Optional(), Length(max=20)]); fuel_type = SelectField('Вид топлива', choices=[], coerce=int, validators=[DataRequired(message='Выберите вид топлива.')]); parent = SelectField('Родительский вид', choices=[], coerce=int, validators=[Optional()]).

Шаблон содержит кнопку добавления во всех версиях для административного пользователя. Серверная ветвь all_versions=1 вызывает специализированную проверку доступа и межверсионный сервис; при обычном добавлении используется выбранная версия.

Модель добавляемого объекта Fuel – основной реквизит. Источник: app/refdata/models/fuels/fuel_model.py, строки 18–18.

```
18      name = db.Column(db.String(80), unique=True, nullable=False, index=True)
```

Маршрут добавления – вход в форму. Источник: app/refdata/routes/fuels/fuel_routes.py, строки 290–302.

```
290 @refdata_bp.route("/add_fuel", methods=["GET", "POST"])
291 @login_required
292 def add_fuel():
293     """ Маршрут для добавления нового топлива. """
294
295     user = session.get('username', 'Неизвестный пользователь')
296     log_to_db(
297         user,
298         "Открыта страница добавления типов топлива",
299         entity_type="fuel")
300
301     # Создание формы
302     form = AddFuelForm()
```

Маршрут добавления – вызов сервиса. Источник: app/refdata/routes/fuels/fuel_routes.py, строки 341–354.

```
                            sort_dir=sort_dir,
                        ))
                add_fuel_all_versions_service(payload, user)
                flash("Новая запись добавлена во всех версиях БД (общий ref_uuid).", "success")
            else:
                add_fuel_service(payload, user)
                flash("Новая запись успешно добавлена.", "success")

            # Перенаправление на список с сохранением параметров и переходом к новой записи
            total_records = fuel_query(
                                fuel_filter,
                                fuel_type_filter,
                                nazvl_filter,
                                kmbur_filter).count()
```

Сервис добавления – создание сущности. Источник: app/refdata/services/fuels/fuel_services.py, строки 449–465.

```
                    raise ValueError(f"Запись с наименованием «{name}» уже существует.")

                # Создаем новую запись
                obj = Fuel(
                    name=name,
                    kod=kod,
                    nazvl=_normalize_fuel_nazvl(record.get("nazvl")),
                    id_fuel_type=fuel_type_id,
                    parent_id=parent_id,
                )
                set_db_version_on_create(obj)
                db.session.add(obj)
                db.session.flush()  # получить id без полного коммита

                parent_name = (
                    db.session.get(Fuel, parent_id).name if parent_id else "не указано"
                )
```

HTML/Jinja2 формы добавления. Источник: app/templates/refdata/fuels/fuel/fuel_add.html, строки 21–35.

```
        <div class="mb-3">
            <label for="name" class="form-label">Вид топлива</label>
            {{ form.name(class="form-control", id="name") }}
            {% if form.name.errors %}
                <div class="text-danger">{{ form.name.errors|join(', ') }}</div>
            {% endif %}
        </div>


        <div class="mb-3">
            <label for="kod" class="form-label">Код</label>
            {{ form.kod(class="form-control", id="kod") }}
            {% if form.kod.errors %}
                <div class="text-danger">{{ form.kod.errors|join(', ') }}</div>
            {% endif %}
```

Взаимодействие при добавлении. MachineFuel связывает виды топлива с агрегатами и годами. Модули топлива и экологии используют единый классификатор, коды и внешние обозначения при импорте, сопоставлении и расчетах. Изменение parent_id меняет структуру состава топлива, но не подменяет исходные значения расхода топлива. После создания запись становится доступна для выбора в зависимых формах при их следующем построении с учетом версии и используемых механизмов кэширования.

### 4.14.3. Экспорт, импорт и дополнительные операции

| Адрес и метод | Обработчик | Результат |
|---|---|---|
| GET /refdata/fuel/hierarchy | fuel_hierarchy | JSON-дерево видов топлива |
| POST /refdata/import_fuel | import_fuel | Обработка загруженного |

| Адрес и метод | Обработчик | Результат |
|---|---|---|
| | | файла и сообщение о результате |
| GET /refdata/export_fuel | export_fuel | Файл Excel для скачивания |

Отдельный маршрут импорта в этом справочнике предусмотрен; наличие маршрута не означает наличие видимой кнопки загрузки в текущем шаблоне.

Кнопка «Иерархия» открывает модальное окно на странице видов топлива, а не отдельную HTML-страницу. Дерево загружается через GET /refdata/fuel/hierarchy. Пользователь может искать по названию, коду и NazvL, раскрывать и сворачивать ветви. Сервис строит дерево текущей версии; parent_id определяет отношение родитель–потомок.

Маршрут JSON-дерева топлива. Источник: app/refdata/routes/fuels/fuel_routes.py, строки 43–59.

```
43  @refdata_bp.route("/fuel/hierarchy", methods=["GET"])
44  @login_required
45  def fuel_hierarchy():
46      """JSON-дерево иерархии видов топлива текущей версии БД."""
47      user = session.get("username", "Неизвестный пользователь")
48      try:
49          tree = get_fuel_hierarchy_tree()
50          log_to_db(
51              user,
52              "Открыта визуализация иерархии видов топлива",
53              f"Узлов: {tree.get('total', 0)}; корней: {tree.get('roots_count', 0)}",
54              entity_type="fuel",
55          )
56          return jsonify(ok=True, **tree)
57      except Exception as e:
58          current_app.logger.exception("Ошибка построения иерархии топлива")
59          return jsonify(ok=False, error=str(e) or "Ошибка построения иерархии."), 500
```

Проверка допустимости родительского вида топлива. Источник: app/refdata/services/fuels/fuel_services.py, строки 121–142.

```
121  def _assert_valid_fuel_parent(fuel_id, parent_id):
122      """Проверяет, что parent_id существует и не создает цикл."""
123      if parent_id is None:
124          return
125
126      parent_obj = db.session.get(Fuel, parent_id)
127      if not parent_obj:
128          raise ValueError(f"Родительский вид топлива с id={parent_id} не найден.")
129
130      if fuel_id is not None and int(parent_id) == int(fuel_id):
131          raise ValueError("Вид топлива не может быть родителем самого себя.")
132
133      seen = {int(fuel_id)} if fuel_id is not None else set()
134      current_id = int(parent_id)
135      while current_id is not None:
136          if current_id in seen:
137              raise ValueError("Обнаружена циклическая ссылка в иерархии видов топлива.")
138          seen.add(current_id)
139          current_obj = db.session.get(Fuel, current_id)
140          if not current_obj:
141              raise ValueError(f"Родительский вид топлива с id={current_id} не найден.")
142          current_id = current_obj.parent_id
```

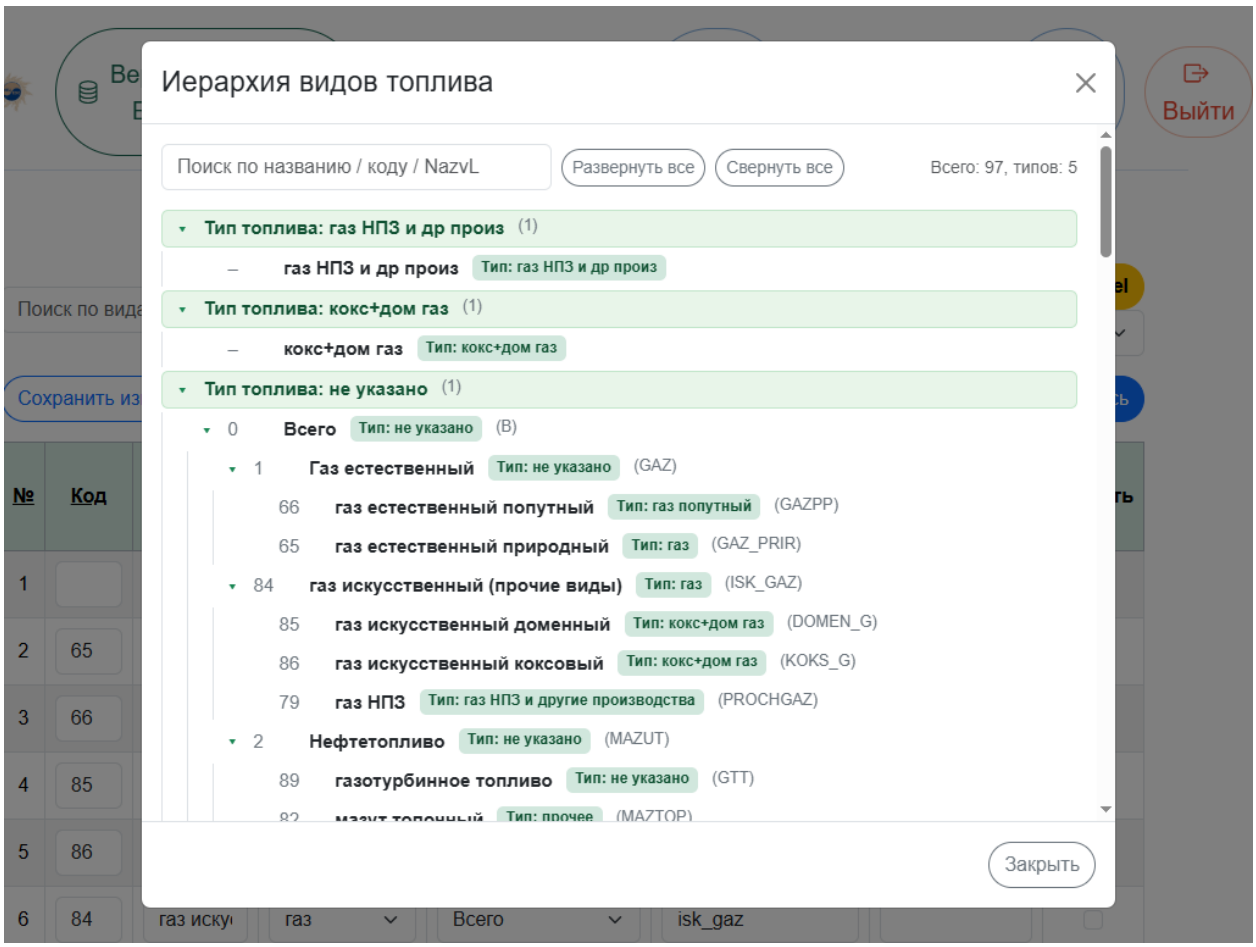



Рисунок 6. Модальное окно иерархии топлива. Действующий сайт, 29.10.2026. Показан фрагмент экранной формы.

## 4.15. Типы электростанций

### 4.15.1. Страница списка

Адрес: GET, POST /refdata/station_type. Обработчик: station_type_list().

Ведет перечень типов электростанций и порядок их отображения. Список позволяет изменять классификационные записи, отдельная форма добавляет новый тип.

Поля и параметры HTML-формы: per_page, page, sort_by, sort_dir, station_type_filter, all_versions, station_ids[], display_orders[], station_type_names[], station_type_delete[], js/refdata/textarea_autoresize.js.

Обработка: маршрут получает параметры отображения, при POST читает изменяемые строки и флажки удаления, вызывает сервисы и возвращает пользователя к списку с параметрами просмотра. Для обычного просмотра строится выборка и передается в шаблон вместе с пагинацией и вариантами выбора связанных справочников.

Модель: StationType. Поля SQLAlchemy: id, display_order, name, created_at, updated_at, database_version_id, stations.

Модель данных – фрагмент объявления. Источник: app/refdata/models/refdata_for_stations/station/station_type_model.py, строки 13–31.

```
  13  class StationType(db.Model, AuditMixin, VersionedModelMixin, RefdataUuidMixin):
  14      __tablename__ = 'gs_sys_station_types'
  15      __table_args__ = {"schema": SCHEMA_REFDATA}
  16
  17      id = db.Column(db.Integer, primary_key=True, autoincrement=True)
  18      # Порядок отображения в справочнике
  19      display_order = db.Column(db.Integer, nullable=True)
  20      name = db.Column(db.String(255), unique=True, nullable=False, index=True)
  21
  22      created_at = db.Column(db.DateTime(timezone=True), server_default=func.now(), nullable=False)
  23      updated_at = db.Column(db.DateTime(timezone=True), server_default=func.now(),
onupdate=func.now(), nullable=False)
  24
  25      # Поле для связи с версией БД
  26      database_version_id = db.Column(
  27          db.Integer,
  28          db.ForeignKey(f"{SCHEMA_REFDATA}.gs_database_versions.id", ondelete="SET NULL"),
  29          nullable=True,
  30          index=True
  31      )
```

Связь модели с другой сущностью. Источник: app/refdata/models/refdata_for_stations/station/station_type_model.py, строки 33–33.

```
  33      stations = db.relationship('Station', back_populates='station_type')
```

Маршрут списка – вход и параметры. Источник: app/refdata/routes/refdata_for_stations/stations/station_type_routes.py, строки 42–57.

```
  42  @refdata_bp.route("/station_type", methods=["GET", "POST"])
  43  @login_required
  44  def station_type_list():
  45      """Маршрут для отображения списка типов электростанций."""
  46
  47      user = session.get('username', 'Неизвестный пользователь')
  48      log_to_db(
  49          user,
  50          "Открыта страница типов электростанций",
  51          entity_type="station_type")
  52
  53      # Создание формы
  54      form = StationTypeFilterForm()
  55
  56      # Получение параметров запроса
  57      page                = request.args.get("page", 1, type=int)
```

Маршрут списка – передача изменений сервису. Источник: app/refdata/routes/refdata_for_stations/stations/station_type_routes.py, строки 145–150.

```
 145                              sort_dir=sort_dir,
 146                      ))
 147                  update_station_type_all_versions_service(station_type_data, user)
 148                  flash("Изменения применены во всех версиях БД (по ref_uuid).", "success")
 149              else:
 150                  update_station_type_service(station_type_data, user)
```

Сервис списка – выборка и пагинация. Источник:
app/refdata/services/refdata_for_stations/stations/station_type_services.py, строки 81–98.

```
 81  @no_autoflush
 82  def get_station_type_list(
 83      page,
 84      per_page,
 85      station_type_filter=None,
 86      sort_by="display_order",
 87      sort_dir="asc"):
 88      """ Получает список типов электростанций с пагинацией, фильтрацией и сортировкой. """
 89
 90      # Базовый запрос
 91      query = station_type_query(
 92          station_type_filter=station_type_filter,
 93          sort_by=sort_by,
 94          sort_dir=sort_dir,
 95      )
 96
 97      # Пагинация
 98      return query.paginate(page=page, per_page=per_page, error_out=False)
```

Сервис изменения – сохранение и журналирование. Источник:
app/refdata/services/refdata_for_stations/stations/station_type_services.py, строки 187–199.

```
187
188      try:
189          # Сохранение изменений в базе данных
190          # Фиксация транзакции (устойчивый коммит)
191          _commit_with_retry()
192
193          if updated_ids:
194              log_to_db(
195                  user,
196                  "Сохранены изменения по типам электростанций",
197                  f"Измененных записей: {len(updated_ids)} (id: {updated_ids})",
198                  entity_type="station_type",
199                  entity_id=station_type_id)
```

HTML/Jinja2 списка – элементы формы. Источник:
app/templates/refdata/refdata_for_stations/stations/station_type/station_type.html, строки 192–205.

```
192                              <!-- Строки таблицы. Столбец "Тип  электростанции" -->
193                              <td>
194                                  <input type="text" name="station_type_names[]" class="form-control"
value="{{ station_type.name }}">
195                              </td>
196
197                              <!-- Строки таблицы. Столбец "Чекмарк для удаления" -->
198                              <td class="text-center align-middle">
199                                  <input type="checkbox" name="station_type_delete[]" value="{{
station_type.id }}" class="form-check-input">
200                              </td>
201                          </tr>
202                          {% else %}
203                          <tr>
204                              <td colspan="3" class="text-center align-middle">Нет данных для
отображения</td>
205                          </tr>
```

Взаимодействие с другими модулями. Station.id_station_type связывает станцию с типом. Модуль генерации и энергетические балансы используют тип станции для отбора и группировки объектов. Справочник хранит классификацию, а мощности и выработка хранятся в профильных моделях.

Подтверждающие ссылки на потребителей модели:
app/generation/models/station/station_model.py:116 – station_type = db.relationship('StationType', back_populates='stations')
app/energy_balance/services/power_balance_installed_capacity_services.py:886 – from app.refdata.models.refdata_for_stations.station.station_type_model import StationType
app/energy_consumption/services/energy_consumption_summary_services.py:125 – from app.refdata.models.refdata_for_stations.station.station_type_model import StationType

app/fuel/routes/fuel_routes.py:317 – from app.refdata.models.refdata_for_stations.station.station_type_model import StationType
app/rational_structure/services/equipment_runtime_machine_services.py:25 – from app.refdata.models.refdata_for_stations.station.station_type_model import StationType

### 4.15.2. Форма добавления

Адрес: GET, POST /refdata/add_station_type. Обработчик: add_station_type().

Назначение: создание записи справочника «Типы электростанций». Вводимые поля формы: name. Список полей формы может быть уже модели: дополнительные реквизиты изменяются на странице списка или заполняются сервисом.

GET создает форму и подготавливает варианты выбора. POST с валидной формой формирует данные и передает их сервису добавления. При успехе выполняется переход к списку; сообщения о результате выводятся пользователю. Кнопка отмены возвращает к списку без создания записи. Создаваемая сущность – StationType, модель и ее связи приведены выше в подразделе 4.15.1.

Поля WTForms и валидаторы: csrf_token = HiddenField(); name = StringField('Тип электростанции', validators=[DataRequired(message='Поле «Тип электростанции» обязательно.'), Length(min=2, max=255, message='Длина от 2 до 255 символов.')]).

Шаблон содержит кнопку добавления во всех версиях для административного пользователя. Серверная ветвь all_versions=1 вызывает специализированную проверку доступа и межверсионный сервис; при обычном добавлении используется выбранная версия.

Модель добавляемого объекта StationType – основной реквизит. Источник: app/refdata/models/refdata_for_stations/station/station_type_model.py, строки 20–20.

```
20      name = db.Column(db.String(255), unique=True, nullable=False, index=True)
```

Маршрут добавления – вход в форму. Источник: app/refdata/routes/refdata_for_stations/stations/station_type_routes.py, строки 185–197.

```
185  @refdata_bp.route("/add_station_type", methods=["GET", "POST"])
186  @login_required
187  def add_station_type():
188      """ Маршрут для добавления нового типа  электростанции. """
189
190      user = session.get('username', 'Неизвестный пользователь')
191      log_to_db(
192          user,
193          "Открыта страница добавления типов электростанций",
194          entity_type="station_type")
195
196      # Создание формы
197      form = AddStationTypeForm()
```

Маршрут добавления – вызов сервиса. Источник: app/refdata/routes/refdata_for_stations/stations/station_type_routes.py, строки 231–241.

```
231                                sort_dir=sort_dir,
232                            ))
233                    add_station_type_all_versions_service(payload, user)
234                    flash("Новая запись добавлена во всех версиях БД (общий ref_uuid).", "success")
235                else:
236                    add_station_type_service(payload, user)
237                    flash("Новая запись успешно добавлена.", "success")
238
239                # Перенаправление на список с сохранением параметров и переходом к новой записи
240                total_records = station_type_query(
241                                    station_type_filter).count()
```

Сервис добавления – создание сущности. Источник:
app/refdata/services/refdata_for_stations/stations/station_type_services.py, строки 260–273.

```
260                             f"Запись с порядком отображения «{display_order}» уже существует."
261                         )
262
263                 obj = StationType(name=name, display_order=display_order)
264                 set_db_version_on_create(obj)
265                 db.session.add(obj)
266                 db.session.flush()
267
268                 log_to_db(
269                     user,
270                     "Создан тип  электростанции",
271                     f"Наименование: {name}",
272                     entity_type="station_type",
273                     entity_id=obj.id)
```

HTML/Jinja2 формы добавления. Источник:
app/templates/refdata/refdata_for_stations/stations/station_type/station_type_add.html, строки 20–34.

```
20          <div class="mb-3">
21              <label for="name" class="form-label">Тип  электростанции</label>
22              {{ form.name(class="form-control", id="name") }}
23              {% if form.name.errors %}
24                  <div class="text-danger">{{ form.name.errors|join(', ') }}</div>
25              {% endif %}
26          </div>
27
28          <div class="mt-4 d-flex flex-wrap gap-2 align-items-center">
29              <button type="submit" class="btn btn-primary">Добавить</button>
30              {% if current_user.has_admin %}<button type="submit" name="all_versions" value="1"
class="btn btn-outline-primary" title="Добавить запись во все версии БД">Добавить во всех
версиях</button>{% endif %}
31              <a href="{{ url_for('refdata_bp.station_type_list',
32                                  page=page,
33                                  per_page=per_page,
34                                  sort_by=sort_by,
```

Взаимодействие при добавлении. Station.id_station_type связывает станцию с типом. Модуль генерации и энергетические балансы используют тип станции для отбора и группировки объектов. Справочник хранит классификацию, а мощности и выработка хранятся в профильных моделях. После создания запись становится доступна для выбора в зависимых формах при их следующем построении с учетом версии и используемых механизмов кэширования.

4.15.3. Экспорт, импорт и дополнительные операции

| Адрес и метод | Обработчик | Результат |
|---|---|---|
| GET /refdata/export_station_type | export_station_type | Файл Excel для скачивания |

Отдельный маршрут импорта в этом справочнике не зарегистрирован. Поддержка импорта не приписывается всем справочникам по аналогии.

## 4.16. Типы состояния оборудования

4.16.1. Страница списка

Адрес: GET, POST /refdata/condition_type. Обработчик: condition_type_list().

Ведет классификатор состояний оборудования. Пользователь просматривает, ищет и редактирует наименования состояний либо добавляет новое состояние в отдельной форме.

Поля и параметры HTML-формы: per_page, page, sort_by, sort_dir, condition_type_filter, all_versions, condition_type_ids[], condition_type_names[], condition_type_delete[].

Обработка: маршрут получает параметры отображения, при POST читает изменяемые строки и флажки удаления, вызывает сервисы и возвращает пользователя к списку с параметрами просмотра. Для обычного просмотра строится выборка и передается в шаблон вместе с пагинацией и вариантами выбора связанных справочников.

Модель: ConditionType. Поля SQLAlchemy: id, name, created_at, updated_at, database_version_id, stations, machines, pgu_machines.

Модель данных – фрагмент объявления. Источник: app/refdata/models/refdata_for_stations/condition_type_model.py, строки 13–35.

```
  13  class ConditionType(db.Model, AuditMixin, VersionedModelMixin, RefdataUuidMixin):
  14      __tablename__ = 'gs_sys_condition_types'
  15      __table_args__ = {"schema": SCHEMA_REFDATA}
  16
  17      id = db.Column(db.Integer, primary_key=True, autoincrement=True)
  18      name = db.Column(db.String(80), unique=True, nullable=False, index=True)
  19
  20      created_at = db.Column(db.DateTime(timezone=True), server_default=func.now(), nullable=False)
  21      updated_at = db.Column(db.DateTime(timezone=True), server_default=func.now(),
onupdate=func.now(), nullable=False)
  22
  23      # Поле для связи с версией БД
  24      database_version_id = db.Column(
  25          db.Integer,
  26          db.ForeignKey(f"{SCHEMA_REFDATA}.gs_database_versions.id", ondelete="SET NULL"),
  27          nullable=True,
  28          index=True
  29      )
  30
  31      stations = db.relationship(
  32          'Station',
  33          back_populates='condition_type',
  34          primaryjoin="ConditionType.id == Station.id_condition_type"
  35      )
```

Маршрут списка – вход и параметры. Источник: app/refdata/routes/refdata_for_stations/condition_type_routes.py, строки 35–50.

```
  35  @refdata_bp.route("/condition_type", methods=["GET", "POST"])
  36  @login_required
  37  def condition_type_list():
  38      """Маршрут для отображения списка типов состояния."""
  39
  40      user = session.get('username', 'Неизвестный пользователь')
  41      log_to_db(
  42          user,
  43          "Открыта страница типов состояния",
  44          entity_type="condition_type"
  45      )
  46
  47      # Создание формы
  48      form = ConditionTypeFilterForm()
  49
  50      # Получение параметров запроса
```

Маршрут списка – передача изменений сервису. Источник: app/refdata/routes/refdata_for_stations/condition_type_routes.py, строки 124–129.

```
 124                                              sort_by=sort_by,
 125                                              sort_dir=sort_dir))
 126                  update_condition_type_all_versions_service(condition_type_payload, user)
 127                  flash("Изменения применены во всех версиях БД (по ref_uuid).", "success")
 128              else:
 129                  update_condition_type_service(condition_type_payload, user)
```

Сервис списка – выборка и пагинация. Источник: app/refdata/services/refdata_for_stations/condition_type_services.py, строки 65–81.

```
@no_autoflush
def get_condition_type_list(
    page: int,
    per_page: int,
    condition_type_filter: str | None = None,
    sort_by: str = "id",
    sort_dir: str = "asc"):
    """ Получает список типов состояния с пагинацией."""

    # Базовый запрос
    query = condition_type_query(
        condition_type_filter=condition_type_filter,
        sort_by=sort_by,
        sort_dir=sort_dir)

    # Пагинация
    return query.paginate(page=page, per_page=per_page, error_out=False)
```

Сервис изменения – сохранение и журналирование. Источник: app/refdata/services/refdata_for_stations/condition_type_services.py, строки 149–161.

```

    try:
        # Сохранение изменений в базе данных
        # Фиксация транзакции (устойчивый коммит)
        _commit_with_retry()

        if updated_ids:
            log_to_db(
                user,
                "Сохранены изменения по типам состояния",
                f"Изменено записей: {len(updated_ids)}",
                entity_type="condition_type"
                )
```

HTML/Jinja2 списка – элементы формы. Источник: app/templates/refdata/refdata_for_stations/condition_type/condition_type.html, строки 158–171.

```
                            <!-- Строки таблицы. Столбец "Тип состояния" -->
                            <td>
                                <input type="text" name="condition_type_names[]" class="form-
control" value="{{ condition_type.name }}">
                            </td>

                            <!-- Строки таблицы. Столбец "Чекмарк для удаления" -->
                            <td class="text-center align-middle">
                                <input type="checkbox" name="condition_type_delete[]" value="{{
condition_type.id }}" class="form-check-input">
                            </td>
                        </tr>
                        {% else %}
                        <tr>
                            <td colspan="3" class="text-center align-middle">Нет данных для
отображения</td>
                        </tr>
```

Взаимодействие с другими модулями. Классификатор применяется в модуле генерации при хранении состояний оборудования. Он задает значение признака, а не даты ввода, демонтажа или ремонтные интервалы; конкретные ссылки потребителей приведены ниже.

Подтверждающие ссылки на потребителей модели:

app/generation/models/machine/machine_model.py:55 – condition_type = db.relationship('ConditionType', back_populates='machines')

app/generation/models/pgu_machine/pgu_machine_model.py:32 – condition_type = db.relationship('ConditionType', back_populates='pgu_machines')

app/generation/models/station/station_model.py:70 – condition_type = db.relationship('ConditionType', back_populates='stations')

app/generation/routes/stations/station_details_routes.py:38 – from app.refdata.models.refdata_for_stations.condition_type_model import ConditionType

app/generation/services/machine_services/machine_all_versions_services.py:61 – from app.refdata.models.refdata_for_stations.condition_type_model import ConditionType

### 4.16.2. Форма добавления

Адрес: GET, POST /refdata/add_condition_type. Обработчик: add_condition_type().

Назначение: создание записи справочника «Типы состояния оборудования». Вводимые поля формы: name. Список полей формы может быть уже модели: дополнительные реквизиты изменяются на странице списка или заполняются сервисом.

GET создает форму и подготавливает варианты выбора. POST с валидной формой формирует данные и передает их сервису добавления. При успехе выполняется переход к списку; сообщения о результате выводятся пользователю. Кнопка отмены возвращает к списку без создания записи. Создаваемая сущность – ConditionType, модель и ее связи приведены выше в подразделе 4.16.1.

Поля WTForms и валидаторы: csrf_token = HiddenField(); name = StringField('Тип состояния', validators=[DataRequired(message='Поле «Тип состояния» обязательно.'), Length(min=2, max=255, message='Длина от 2 до 255 символов.')]).

Шаблон содержит кнопку добавления во всех версиях для административного пользователя. Серверная ветвь all_versions=1 вызывает специализированную проверку доступа и межверсионный сервис; при обычном добавлении используется выбранная версия.

Модель добавляемого объекта ConditionType – основной реквизит. Источник: app/refdata/models/refdata_for_stations/condition_type_model.py, строки 18–18.

```
18      name = db.Column(db.String(80), unique=True, nullable=False, index=True)
```

Маршрут добавления – вход в форму. Источник: app/refdata/routes/refdata_for_stations/condition_type_routes.py, строки 165–177.

```
165 @refdata_bp.route("/add_condition_type", methods=["GET", "POST"])
166 @login_required
167 def add_condition_type():
168     """ Маршрут для добавления нового вида топлива. """
169 
170     user = session.get('username', 'Неизвестный пользователь')
171     log_to_db(
172         user,
173         "Открыта страница добавления типа состояния",
174         entity_type="condition_type"
175         )
176 
177     # Создание формы
```

Маршрут добавления – вызов сервиса. Источник: app/refdata/routes/refdata_for_stations/condition_type_routes.py, строки 212–222.

```
212                                 sort_dir=sort_dir,
213                     ))
214                 add_condition_type_all_versions_service(payload, user)
215                 flash("Новая запись добавлена во всех версиях БД (общий ref_uuid).", "success")
216             else:
217                 add_condition_type_service(payload, user)
218                 flash("Новая запись успешно добавлена.", "success")
219 
220             # Перенаправление на список с сохранением параметров и переходом к новой записи
221             total_records = condition_type_query(
222                                 condition_type_filter).count()
```

Сервис добавления – создание сущности. Источник:
app/refdata/services/refdata_for_stations/condition_type_services.py, строки 215–230.

```
                    raise ValueError(f"Запись с наименованием «{name}» уже существует.")

                # Создаем новую запись
                obj = ConditionType(
                    name=name
                    )
                set_db_version_on_create(obj)
                db.session.add(obj)
                db.session.flush()  # получить id без полного коммита

                log_to_db(
                    user,
                    "Создан тип состояния",
                    f"Наименование: {name}",
                    entity_type="condition_type",
                    entity_id=obj.id)
```

HTML/Jinja2 формы добавления. Источник:
app/templates/refdata/refdata_for_stations/condition_type/condition_type_add.html, строки 22–36.

```
        <div class="mb-3">
            <label for="name" class="form-label">Наименование типа состояния</label>
            {{ form.name(class="form-control", id="name") }}
            {% if form.name.errors %}
                <div class="text-danger">{{ form.name.errors|join(', ') }}</div>
            {% endif %}
        </div>

        <!-- Кнопки сохранения записи и возвращения назад -->
        <div class="mt-4 d-flex flex-wrap gap-2 align-items-center">
            <button type="submit" class="btn btn-primary">Добавить</button>
            {% if current_user.has_admin %}<button type="submit" name="all_versions" value="1"
class="btn btn-outline-primary" title="Добавить запись во все версии БД">Добавить во всех
версиях</button>{% endif %}

            <a href="{{ url_for('refdata_bp.condition_type_list',
                                page=page,
```

Взаимодействие при добавлении. Классификатор применяется в модуле генерации при хранении состояний оборудования. Он задает значение признака, а не даты ввода, демонтажа или ремонтные интервалы; конкретные ссылки потребителей приведены ниже. После создания запись становится доступна для выбора в зависимых формах при их следующем построении с учетом версии и используемых механизмов кэширования.

4.16.3. Экспорт, импорт и дополнительные операции

| Адрес и метод | Обработчик | Результат |
|---|---|---|
| GET /refdata/export_condition_type | export_condition_type | Файл Excel для скачивания |

Отдельный маршрут импорта в этом справочнике не зарегистрирован. Поддержка импорта не приписывается всем справочникам по аналогии.

## 4.17. Типы агрегатов

4.17.1. Страница списка

Адрес: GET, POST /refdata/machine_type. Обработчик: machine_type_list().

Содержит общий классификатор типов агрегатов. Предусмотрены табличное редактирование и отдельная форма добавления типа.

Поля и параметры HTML-формы: per_page, page, sort_by, sort_dir, machine_type_filter, all_versions, machine_ids[], machine_type_names[], machine_type_delete[], js/refdata/textarea_autoresize.js.

Обработка: маршрут получает параметры отображения, при POST читает изменяемые строки и флажки удаления, вызывает сервисы и возвращает пользователя к списку с параметрами просмотра. Для обычного просмотра строится выборка и передается в шаблон вместе с пагинацией и вариантами выбора связанных справочников.

Модель: MachineType. Поля SQLAlchemy: id, name, created_at, updated_at, database_version_id, machines.

Модель данных – фрагмент объявления. Источник: app/refdata/models/refdata_for_stations/machine/machine_type_model.py, строки 13–31.

```
  13  class MachineType(db.Model, AuditMixin, VersionedModelMixin, RefdataUuidMixin):
  14      __tablename__ = 'gs_sys_machine_types'
  15      __table_args__ = {"schema": SCHEMA_REFDATA}
  16
  17      id = db.Column(db.Integer, primary_key=True, autoincrement=True)
  18      # Сохраняем исходную nullable=True, чтобы не трогать существующие данные.
  19      name = db.Column(db.String(80), unique=True, nullable=True, index=True)
  20
  21      created_at = db.Column(db.DateTime(timezone=True), server_default=func.now(), nullable=False)
  22      updated_at = db.Column(db.DateTime(timezone=True), server_default=func.now(),
onupdate=func.now(), nullable=False)
  23
  24      # Поле для связи с версией БД
  25      database_version_id = db.Column(
  26          db.Integer,
  27          db.ForeignKey(f"{SCHEMA_REFDATA}.gs_database_versions.id", ondelete="SET NULL"),
  28          nullable=True,
  29          index=True
  30      )
  31
```

Связь модели с другой сущностью. Источник: app/refdata/models/refdata_for_stations/machine/machine_type_model.py, строки 32–32.

```
  32      machines = db.relationship('Machine', back_populates='machine_type')
```

Маршрут списка – вход и параметры. Источник: app/refdata/routes/refdata_for_stations/machines/machine_type_routes.py, строки 35–50.

```
  35  @refdata_bp.route("/machine_type", methods=["GET", "POST"])
  36  @login_required
  37  def machine_type_list():
  38      """Маршрут для отображения списка типов агрегатов."""
  39
  40      user = session.get('username', 'Неизвестный пользователь')
  41      log_to_db(
  42          user,
  43          "Открыта страница типов агрегатов",
  44          entity_type="machine_type")
  45
  46      # Создание формы
  47      form = MachineTypeFilterForm()
  48
  49      # Получение параметров запроса
  50      page                = request.args.get("page", 1, type=int)
```

Маршрут списка – передача изменений сервису. Источник: app/refdata/routes/refdata_for_stations/machines/machine_type_routes.py, строки 124–129.

```
 124                              sort_dir=sort_dir,
 125                      ))
 126                  update_machine_type_all_versions_service(machine_type_data, user)
 127                  flash("Изменения применены во всех версиях БД (по ref_uuid).", "success")
 128              else:
 129                  update_machine_type_service(machine_type_data, user)
```

Сервис списка – выборка и пагинация. Источник:
app/refdata/services/refdata_for_stations/machines/machine_type_services.py, строки 69–86.

```
 69  @no_autoflush
 70  def get_machine_type_list(
 71      page,
 72      per_page,
 73      machine_type_filter=None,
 74      sort_by="id",
 75      sort_dir="asc"):
 76      """ Получает список типов агрегатов с пагинацией, фильтрацией и сортировкой. """
 77
 78      # Базовый запрос
 79      query = machine_type_query(
 80          machine_type_filter=machine_type_filter,
 81          sort_by=sort_by,
 82          sort_dir=sort_dir,
 83      )
 84
 85      # Пагинация
 86      return query.paginate(page=page, per_page=per_page, error_out=False)
```

Сервис изменения – сохранение и журналирование. Источник:
app/refdata/services/refdata_for_stations/machines/machine_type_services.py, строки 154–166.

```
154
155      try:
156          # Сохранение изменений в базе данных
157          # Фиксация транзакции (устойчивый коммит)
158          _commit_with_retry()
159
160          if updated_ids:
161              log_to_db(
162                  user,
163                  "Сохранены изменения по типам агрегатов",
164                  f"Измененных записей: {len(updated_ids)} (id: {updated_ids})",
165                  entity_type="machine_type",
166                  entity_id=machine_type_id)
```

HTML/Jinja2 списка – элементы формы. Источник:
app/templates/refdata/refdata_for_stations/machines/machine_type/machine_type.html, строки 160–173.

```
160                              <!-- Строки таблицы. Столбец "Тип агрегата" -->
161                              <td>
162                                  <input type="text" name="machine_type_names[]" class="form-control"
value="{{ machine_type.name }}">
163                              </td>
164
165                              <!-- Строки таблицы. Столбец "Чекмарк для удаления" -->
166                              <td class="text-center align-middle">
167                                  <input type="checkbox" name="machine_type_delete[]" value="{{
machine_type.id }}" class="form-check-input">
168                              </td>
169                          </tr>
170                          {% else %}
171                          <tr>
172                              <td colspan="3" class="text-center align-middle">Нет данных для
отображения</td>
173                          </tr>
```

Взаимодействие с другими модулями. Machine.id_machine_type связывает агрегат с общим типом. Модуль генерации использует эту классификацию при представлении состава оборудования; специализированные типы ТЭС и ПГУ ведутся в самостоятельных справочниках.

Подтверждающие ссылки на потребителей модели:
app/generation/models/machine/machine_model.py:97 – machine_type = db.relationship('MachineType', back_populates='machines')
app/generation/services/machine_services/machine_services.py:31 – from app.refdata.models.refdata_for_stations.machine.machine_type_model import MachineType
app/generation/services/station_services/import_station_services.py:34 – from app.refdata.models.refdata_for_stations.machine.machine_type_model import MachineType

4.17.2. Форма добавления

Адрес: GET, POST /refdata/add_machine_type. Обработчик: add_machine_type().

Назначение: создание записи справочника «Типы агрегатов». Вводимые поля формы: name. Список полей формы может быть уже модели: дополнительные реквизиты изменяются на странице списка или заполняются сервисом.

GET создает форму и подготавливает варианты выбора. POST с валидной формой формирует данные и передает их сервису добавления. При успехе выполняется переход к списку; сообщения о результате выводятся пользователю. Кнопка отмены возвращает к списку без создания записи. Создаваемая сущность – MachineType, модель и ее связи приведены выше в подразделе 4.17.1.

Поля WTForms и валидаторы: csrf_token = HiddenField(); name = StringField('Тип агрегата', validators=[DataRequired(message='Поле «Тип агрегата» обязательно.'), Length(min=2, max=255, message='Длина от 2 до 255 символов.')]).

Шаблон содержит кнопку добавления во всех версиях для административного пользователя. Серверная ветвь all_versions=1 вызывает специализированную проверку доступа и межверсионный сервис; при обычном добавлении используется выбранная версия.

Модель добавляемого объекта MachineType – основной реквизит. Источник: app/refdata/models/refdata_for_stations/machine/machine_type_model.py, строки 19–19.

```
19      name = db.Column(db.String(80), unique=True, nullable=True, index=True)
```

Маршрут добавления – вход в форму. Источник: app/refdata/routes/refdata_for_stations/machines/machine_type_routes.py, строки 164–176.

```
164  @refdata_bp.route("/add_machine_type", methods=["GET", "POST"])
165  @login_required
166  def add_machine_type():
167      """ Маршрут для добавления нового типа агрегата. """
168
169      user = session.get('username', 'Неизвестный пользователь')
170      log_to_db(
171          user,
172          "Открыта страница добавления типов агрегатов",
173          entity_type="machine_type")
174
175      # Создание формы
176      form = AddMachineTypeForm()
```

Маршрут добавления – вызов сервиса. Источник: app/refdata/routes/refdata_for_stations/machines/machine_type_routes.py, строки 210–220.

```
210                                sort_dir=sort_dir,
211                            ))
212                    add_machine_type_all_versions_service(payload, user)
213                    flash("Новая запись добавлена во всех версиях БД (общий ref_uuid).", "success")
214                else:
215                    add_machine_type_service(payload, user)
216                    flash("Новая запись успешно добавлена.", "success")
217
218                # Перенаправление на список с сохранением параметров и переходом к новой записи
219                total_records = machine_type_query(
220                                    machine_type_filter).count()
```

Сервис добавления – создание сущности. Источник:
app/refdata/services/refdata_for_stations/machines/machine_type_services.py, строки 213–226.

```
213                 if dup:
214                     raise ValueError(f"Запись с наименованием «{name}» уже существует.")
215
216                 obj = MachineType(name=name)
217                 set_db_version_on_create(obj)
218                 db.session.add(obj)
219                 db.session.flush()
220
221                 log_to_db(
222                     user,
223                     "Создан тип агрегата",
224                     f"Наименование: {name}",
225                     entity_type="machine_type",
226                     entity_id=obj.id)
```

HTML/Jinja2 формы добавления. Источник:
app/templates/refdata/refdata_for_stations/machines/machine_type/machine_type_add.html, строки 20–34.

```
20          <div class="mb-3">
21              <label for="name" class="form-label">Тип агрегата</label>
22              {{ form.name(class="form-control", id="name") }}
23              {% if form.name.errors %}
24                  <div class="text-danger">{{ form.name.errors|join(', ') }}</div>
25              {% endif %}
26          </div>
27
28          <div class="mt-4 d-flex flex-wrap gap-2 align-items-center">
29              <button type="submit" class="btn btn-primary">Добавить</button>
30              {% if current_user.has_admin %}<button type="submit" name="all_versions" value="1"
class="btn btn-outline-primary" title="Добавить запись во все версии БД">Добавить во всех
версиях</button>{% endif %}
31              <a href="{{ url_for('refdata_bp.machine_type_list',
32                                  page=page,
33                                  per_page=per_page,
34                                  sort_by=sort_by,
```

Взаимодействие при добавлении. Machine.id_machine_type связывает агрегат с общим типом. Модуль генерации использует эту классификацию при представлении состава оборудования; специализированные типы ТЭС и ПГУ ведутся в самостоятельных справочниках. После создания запись становится доступна для выбора в зависимых формах при их следующем построении с учетом версии и используемых механизмов кэширования.

4.17.3. Экспорт, импорт и дополнительные операции

| Адрес и метод | Обработчик | Результат |
|---|---|---|
| GET /refdata/export_machine_type | export_machine_type | Файл Excel для скачивания |

Отдельный маршрут импорта в этом справочнике не зарегистрирован. Поддержка импорта не приписывается всем справочникам по аналогии.

## 4.18. Типы ТЭС

4.18.1. Страница списка

Адрес: GET, POST /refdata/tes_type. Обработчик: tes_type_list().

Ведет типы тепловых электростанций с порядком отображения. Классификатор редактируется в списке; новые записи вводятся в отдельной форме.

Поля и параметры HTML-формы: per_page, page, sort_by, sort_dir, tes_type_filter, all_versions, tes_ids[], display_orders[], tes_type_names[], tes_type_delete[], js/refdata/textarea_autoresize.js.

Обработка: маршрут получает параметры отображения, при POST читает изменяемые строки и флажки удаления, вызывает сервисы и возвращает пользователя к списку с параметрами просмотра. Для обычного просмотра строится выборка и передается в шаблон вместе с пагинацией и вариантами выбора связанных справочников.

Модель: TesType. Поля SQLAlchemy: id, display_order, name, created_at, updated_at, database_version_id, machine_tes_types.

Модель данных – фрагмент объявления. Источник: app/refdata/models/refdata_for_stations/machine/tes_type_model.py, строки 13–31.

```
class TesType(db.Model, AuditMixin, VersionedModelMixin, RefdataUuidMixin):
    __tablename__ = 'gs_sys_tes_types'
    __table_args__ = {"schema": SCHEMA_REFDATA}

    id = db.Column(db.Integer, primary_key=True, autoincrement=True)
    # Порядок отображения в справочнике
    display_order = db.Column(db.Integer, nullable=True)
    name = db.Column(db.String(80), unique=True, nullable=False, index=True)

    created_at = db.Column(db.DateTime(timezone=True), server_default=func.now(), nullable=False)
    updated_at = db.Column(db.DateTime(timezone=True), server_default=func.now(),
onupdate=func.now(), nullable=False)

    # Поле для связи с версией БД
    database_version_id = db.Column(
        db.Integer,
        db.ForeignKey(f"{SCHEMA_REFDATA}.gs_database_versions.id", ondelete="SET NULL"),
        nullable=True,
        index=True
    )
```

Связь модели с другой сущностью. Источник: app/refdata/models/refdata_for_stations/machine/tes_type_model.py, строки 33–37.

```
    machine_tes_types = db.relationship(
        'MachineTesType',
        back_populates='tes_type',
        cascade="all, delete-orphan"
    )
```

Маршрут списка – вход и параметры. Источник: app/refdata/routes/refdata_for_stations/machines/tes_type_routes.py, строки 42–57.

```
@refdata_bp.route("/tes_type", methods=["GET", "POST"])
@login_required
def tes_type_list():
    """Маршрут для отображения списка типов ТЭС."""

    user = session.get('username', 'Неизвестный пользователь')
    log_to_db(
        user,
        "Открыта страница типов ТЭС",
        entity_type="tes_type")

    # Создание формы
    form = TesTypeFilterForm()

    # Получение параметров запроса
    page                = request.args.get("page", 1, type=int)
```

Маршрут списка – передача изменений сервису. Источник: app/refdata/routes/refdata_for_stations/machines/tes_type_routes.py, строки 145–150.

```
                               sort_dir=sort_dir,
                           ))
                update_tes_type_all_versions_service(tes_type_data, user)
                flash("Изменения применены во всех версиях БД (по ref_uuid).", "success")
            else:
                update_tes_type_service(tes_type_data, user)
```

Сервис списка – выборка и пагинация. Источник: app/refdata/services/refdata_for_stations/machines/tes_type_services.py, строки 82–99.

```
 82  @no_autoflush
 83  def get_tes_type_list(
 84      page,
 85      per_page,
 86      tes_type_filter=None,
 87      sort_by="display_order",
 88      sort_dir="asc"):
 89      """ Получает список типов ТЭС с пагинацией, фильтрацией и сортировкой. """
 90
 91      # Базовый запрос
 92      query = tes_type_query(
 93          tes_type_filter=tes_type_filter,
 94          sort_by=sort_by,
 95          sort_dir=sort_dir,
 96      )
 97
 98      # Пагинация
 99      return query.paginate(page=page, per_page=per_page, error_out=False)
```

Сервис изменения – сохранение и журналирование. Источник: app/refdata/services/refdata_for_stations/machines/tes_type_services.py, строки 188–200.

```
188
189      try:
190          # Сохранение изменений в базе данных
191          # Фиксация транзакции (устойчивый коммит)
192          _commit_with_retry()
193
194          if updated_ids:
195              log_to_db(
196                  user,
197                  "Сохранены изменения по типам ТЭС",
198                  f"Измененных записей: {len(updated_ids)} (id: {updated_ids})",
199                  entity_type="tes_type",
200                  entity_id=tes_type_id)
```

HTML/Jinja2 списка – элементы формы. Источник: app/templates/refdata/refdata_for_stations/machines/tes_type/tes_type.html, строки 192–205.

```
192                              <!-- Строки таблицы. Столбец "Тип ТЭС" -->
193                              <td>
194                                  <input type="text" name="tes_type_names[]" class="form-control"
value="{{ tes_type.name }}">
195                              </td>
196
197                              <!-- Строки таблицы. Столбец "Чекмарк для удаления" -->
198                              <td class="text-center align-middle">
199                                  <input type="checkbox" name="tes_type_delete[]" value="{{
tes_type.id }}" class="form-check-input">
200                              </td>
201                          </tr>
202                          {% else %}
203                          <tr>
204                              <td colspan="3" class="text-center align-middle">Нет данных для
отображения</td>
205                          </tr>
```

Взаимодействие с другими модулями. MachineTesType использует классификатор для привязки типа ТЭС к оборудованию во времени. Это позволяет модулю генерации учитывать годовую классификацию без хранения типа только в текстовом наименовании агрегата.

Подтверждающие ссылки на потребителей модели:
app/generation/models/machine/machine_tes_type_model.py:4 – - Связи сохранены: year (Year.machine_tes_types), machine_tes_type (Machine.machine_tes_types), tes_type (TesType.machine_tes_types).
app/generation/routes/totals_summary_routes.py:189 – from app.refdata.models.refdata_for_stations.machine.tes_type_model import TesType
app/generation/services/machine_services/machine_services.py:30 – from app.refdata.models.refdata_for_stations.machine.tes_type_model import TesType

app/generation/services/station_changes_services/station_changes_services.py:23 – from app.refdata.models.refdata_for_stations.machine.tes_type_model import TesType
app/generation/services/station_services/export_totals_summary_services.py:30 – from app.refdata.models.refdata_for_stations.machine.tes_type_model import TesType

### 4.18.2. Форма добавления

Адрес: GET, POST /refdata/add_tes_type. Обработчик: add_tes_type().

Назначение: создание записи справочника «Типы ТЭС». Вводимые поля формы: name. Список полей формы может быть уже модели: дополнительные реквизиты изменяются на странице списка или заполняются сервисом.

GET создает форму и подготавливает варианты выбора. POST с валидной формой формирует данные и передает их сервису добавления. При успехе выполняется переход к списку; сообщения о результате выводятся пользователю. Кнопка отмены возвращает к списку без создания записи. Создаваемая сущность – TesType, модель и ее связи приведены выше в подразделе 4.18.1.

Поля WTForms и валидаторы: csrf_token = HiddenField(); name = StringField('Тип ТЭС', validators=[DataRequired(message='Поле «Тип ТЭС обязательно.'), Length(min=2, max=255, message='Длина от 2 до 255 символов.')]).

Шаблон содержит кнопку добавления во всех версиях для административного пользователя. Серверная ветвь all_versions=1 вызывает специализированную проверку доступа и межверсионный сервис; при обычном добавлении используется выбранная версия.

Модель добавляемого объекта TesType – основной реквизит. Источник: app/refdata/models/refdata_for_stations/machine/tes_type_model.py, строки 20–20.

```
20      name = db.Column(db.String(80), unique=True, nullable=False, index=True)
```

Маршрут добавления – вход в форму. Источник: app/refdata/routes/refdata_for_stations/machines/tes_type_routes.py, строки 185–197.

```
185  @refdata_bp.route("/add_tes_type", methods=["GET", "POST"])
186  @login_required
187  def add_tes_type():
188      """ Маршрут для добавления нового типа ТЭС. """
189
190      user = session.get('username', 'Неизвестный пользователь')
191      log_to_db(
192          user,
193          "Открыта страница добавления типов ТЭС",
194          entity_type="tes_type")
195
196      # Создание формы
197      form = AddTesTypeForm()
```

Маршрут добавления – вызов сервиса. Источник: app/refdata/routes/refdata_for_stations/machines/tes_type_routes.py, строки 231–241.

```
231                                 sort_dir=sort_dir,
232                         ))
233                     add_tes_type_all_versions_service(payload, user)
234                     flash("Новая запись добавлена во всех версиях БД (общий ref_uuid).", "success")
235                 else:
236                     add_tes_type_service(payload, user)
237                     flash("Новая запись успешно добавлена.", "success")
238
239                 # Перенаправление на список с сохранением параметров и переходом к новой записи
240                 total_records = tes_type_query(
241                                     tes_type_filter).count()
```

Сервис добавления – создание сущности. Источник:
app/refdata/services/refdata_for_stations/machines/tes_type_services.py, строки 261–274.

```
                                f"Запись с порядком отображения «{display_order}» уже существует."
                            )

                obj = TesType(name=name, display_order=display_order)
                set_db_version_on_create(obj)
                db.session.add(obj)
                db.session.flush()

                log_to_db(
                    user,
                    "Создан тип ТЭС",
                    f"Наименование: {name}",
                    entity_type="tes_type",
                    entity_id=obj.id)
```

HTML/Jinja2 формы добавления. Источник:
app/templates/refdata/refdata_for_stations/machines/tes_type/tes_type_add.html, строки 20–34.

```
        <div class="mb-3">
            <label for="name" class="form-label">Тип ТЭС</label>
            {{ form.name(class="form-control", id="name") }}
            {% if form.name.errors %}
                <div class="text-danger">{{ form.name.errors|join(', ') }}</div>
            {% endif %}
        </div>

        <div class="mt-4 d-flex flex-wrap gap-2 align-items-center">
            <button type="submit" class="btn btn-primary">Добавить</button>
            {% if current_user.has_admin %}<button type="submit" name="all_versions" value="1"
class="btn btn-outline-primary" title="Добавить запись во все версии БД">Добавить во всех
версиях</button>{% endif %}
            <a href="{{ url_for('refdata_bp.tes_type_list',
                                page=page,
                                per_page=per_page,
                                sort_by=sort_by,
```

Взаимодействие при добавлении. MachineTesType использует классификатор для привязки типа ТЭС к оборудованию во времени. Это позволяет модулю генерации учитывать годовую классификацию без хранения типа только в текстовом наименовании агрегата. После создания запись становится доступна для выбора в зависимых формах при их следующем построении с учетом версии и используемых механизмов кэширования.

4.18.3. Экспорт, импорт и дополнительные операции

| Адрес и метод | Обработчик | Результат |
|---|---|---|
| GET /refdata/export_tes_type | export_tes_type | Файл Excel для скачивания |

Отдельный маршрут импорта в этом справочнике не зарегистрирован. Поддержка импорта не приписывается всем справочникам по аналогии.

## 4.19. Типы агрегатов ТЭС

4.19.1. Страница списка

Адрес: GET, POST /refdata/tes_machine_type. Обработчик: tes_machine_type_list().

Ведет специализированные типы агрегатов тепловых электростанций и порядок отображения. Страница поддерживает поиск и табличное редактирование, форма добавления создает новый классификационный элемент.

Поля и параметры HTML-формы: per_page, page, sort_by, sort_dir, tes_machine_type_filter, all_versions, tes_machine_ids[], display_orders[], tes_machine_type_names[], tes_machine_type_delete[], js/refdata/textarea_autoresize.js.

Обработка: маршрут получает параметры отображения, при POST читает изменяемые строки и флажки удаления, вызывает сервисы и возвращает пользователя к списку с параметрами просмотра. Для обычного просмотра строится выборка и передается в шаблон вместе с пагинацией и вариантами выбора связанных справочников.

Модель: TesMachineType. Поля SQLAlchemy: id, display_order, name, created_at, updated_at, database_version_id, machines.

Модель данных – фрагмент объявления. Источник: app/refdata/models/refdata_for_stations/machine/tes_machine_type_model.py, строки 13–31.

```
  13  class TesMachineType(db.Model, AuditMixin, VersionedModelMixin, RefdataUuidMixin):
  14      __tablename__ = 'gs_sys_tes_machine_types'
  15      __table_args__ = {"schema": SCHEMA_REFDATA}
  16
  17      id = db.Column(db.Integer, primary_key=True, autoincrement=True)
  18      # Порядок отображения в справочнике
  19      display_order = db.Column(db.Integer, nullable=True)
  20      name = db.Column(db.String(80), unique=True, nullable=False, index=True)
  21
  22      created_at = db.Column(db.DateTime(timezone=True), server_default=func.now(), nullable=False)
  23      updated_at = db.Column(db.DateTime(timezone=True), server_default=func.now(),
onupdate=func.now(), nullable=False)
  24
  25      # Поле для связи с версией БД
  26      database_version_id = db.Column(
  27          db.Integer,
  28          db.ForeignKey(f"{SCHEMA_REFDATA}.gs_database_versions.id", ondelete="SET NULL"),
  29          nullable=True,
  30          index=True
  31      )
```

Связь модели с другой сущностью. Источник: app/refdata/models/refdata_for_stations/machine/tes_machine_type_model.py, строки 33–37.

```
  33      machines = db.relationship(
  34          'Machine',
  35          back_populates='tes_machine_type',
  36          cascade="all, delete-orphan"
  37      )
```

Маршрут списка – вход и параметры. Источник: app/refdata/routes/refdata_for_stations/machines/tes_machine_type_routes.py, строки 42–57.

```
  42  @refdata_bp.route("/tes_machine_type", methods=["GET", "POST"])
  43  @login_required
  44  def tes_machine_type_list():
  45      """Маршрут для отображения списка типов агрегатов ТЭС."""
  46
  47      user = session.get('username', 'Неизвестный пользователь')
  48      log_to_db(
  49          user,
  50          "Открыта страница типов агрегатов ТЭС",
  51          entity_type="tes_machine_type")
  52
  53      # Создание формы
  54      form = TesMachineTypeFilterForm()
  55
  56      # Получение параметров запроса
  57      page                = request.args.get("page", 1, type=int)
```

Маршрут списка – передача изменений сервису. Источник: app/refdata/routes/refdata_for_stations/machines/tes_machine_type_routes.py, строки 145–150.

```
 145                              sort_dir=sort_dir,
 146                      ))
 147                  update_tes_machine_type_all_versions_service(tes_machine_type_data, user)
 148                  flash("Изменения применены во всех версиях БД (по ref_uuid).", "success")
 149              else:
 150                  update_tes_machine_type_service(tes_machine_type_data, user)
```

Сервис списка – выборка и пагинация. Источник: app/refdata/services/refdata_for_stations/machines/tes_machine_type_services.py, строки 81–98.

```
 81  @no_autoflush
 82  def get_tes_machine_type_list(
 83      page,
 84      per_page,
 85      tes_machine_type_filter=None,
 86      sort_by="display_order",
 87      sort_dir="asc"):
 88      """ Получает список типов агрегатов ТЭС с пагинацией, фильтрацией и сортировкой. """
 89
 90      # Базовый запрос
 91      query = tes_machine_type_query(
 92          tes_machine_type_filter=tes_machine_type_filter,
 93          sort_by=sort_by,
 94          sort_dir=sort_dir,
 95      )
 96
 97      # Пагинация
 98      return query.paginate(page=page, per_page=per_page, error_out=False)
```

Сервис изменения – сохранение и журналирование. Источник: app/refdata/services/refdata_for_stations/machines/tes_machine_type_services.py, строки 187–199.

```
187
188      try:
189          # Сохранение изменений в базе данных
190          # Фиксация транзакции (устойчивый коммит)
191          _commit_with_retry()
192
193          if updated_ids:
194              log_to_db(
195                  user,
196                  "Сохранены изменения по типам агрегатов ТЭС",
197                  f"Измененных записей: {len(updated_ids)} (id: {updated_ids})",
198                  entity_type="tes_machine_type",
199                  entity_id=tes_machine_type_id)
```

HTML/Jinja2 списка – элементы формы. Источник: app/templates/refdata/refdata_for_stations/machines/tes_machine_type/tes_machine_type.html, строки 192–205.

```
192                              <!-- Строки таблицы. Столбец "Тип агрегата" -->
193                              <td>
194                                  <input type="text" name="tes_machine_type_names[]" class="form-
control" value="{{ tes_machine_type.name }}">
195                              </td>
196
197                              <!-- Строки таблицы. Столбец "Чекмарк для удаления" -->
198                              <td class="text-center align-middle">
199                                  <input type="checkbox" name="tes_machine_type_delete[]" value="{{
tes_machine_type.id }}" class="form-check-input">
200                              </td>
201                          </tr>
202                          {% else %}
203                          <tr>
204                              <td colspan="3" class="text-center align-middle">Нет данных для
отображения</td>
205                          </tr>
```

Взаимодействие с другими модулями. Специализированные модели оборудования генерации используют TesMachineType в дополнение к общему MachineType. Классификация участвует в представлении теплового оборудования; конкретные модели и сервисы-потребители указаны в ссылках ниже.

Подтверждающие ссылки на потребителей модели:
app/generation/models/machine/machine_model.py:106 – tes_machine_type = db.relationship('TesMachineType', back_populates='machines')
app/generation/routes/totals_summary_routes.py:191 – TesMachineType,
app/generation/models/pgu_machine/pgu_machine_model.py:53 – tes_machine_type = db.relationship('TesMachineType', backref='pgu_machines')

app/generation/services/machine_services/machine_services.py:29 – from app.refdata.models.refdata_for_stations.machine.tes_machine_type_model import TesMachineType
app/generation/services/station_changes_services/station_changes_services.py:24 – from app.refdata.models.refdata_for_stations.machine.tes_machine_type_model import TesMachineType

### 4.19.2. Форма добавления

Адрес: GET, POST /refdata/add_tes_machine_type. Обработчик: add_tes_machine_type().

Назначение: создание записи справочника «Типы агрегатов ТЭС». Вводимые поля формы: name. Список полей формы может быть уже модели: дополнительные реквизиты изменяются на странице списка или заполняются сервисом.

GET создает форму и подготавливает варианты выбора. POST с валидной формой формирует данные и передает их сервису добавления. При успехе выполняется переход к списку; сообщения о результате выводятся пользователю. Кнопка отмены возвращает к списку без создания записи. Создаваемая сущность – TesMachineType, модель и ее связи приведены выше в подразделе 4.19.1.

Поля WTForms и валидаторы: csrf_token = HiddenField(); name = StringField('Тип агрегата ТЭС', validators=[DataRequired(message='Поле «Тип агрегата ТЭС» обязательно.'), Length(min=2, max=255, message='Длина от 2 до 255 символов.')]).

Шаблон содержит кнопку добавления во всех версиях для административного пользователя. Серверная ветвь all_versions=1 вызывает специализированную проверку доступа и межверсионный сервис; при обычном добавлении используется выбранная версия.

Модель добавляемого объекта TesMachineType – основной реквизит. Источник: app/refdata/models/refdata_for_stations/machine/tes_machine_type_model.py, строки 20–20.

```
20      name = db.Column(db.String(80), unique=True, nullable=False, index=True)
```

Маршрут добавления – вход в форму. Источник: app/refdata/routes/refdata_for_stations/machines/tes_machine_type_routes.py, строки 185–197.

```
185  @refdata_bp.route("/add_tes_machine_type", methods=["GET", "POST"])
186  @login_required
187  def add_tes_machine_type():
188      """ Маршрут для добавления нового типа агрегата ТЭС. """
189
190      user = session.get('username', 'Неизвестный пользователь')
191      log_to_db(
192          user,
193          "Открыта страница добавления типов агрегатов ТЭС",
194          entity_type="tes_machine_type")
195
196      # Создание формы
197      form = AddTesMachineTypeForm()
```

Маршрут добавления – вызов сервиса. Источник: app/refdata/routes/refdata_for_stations/machines/tes_machine_type_routes.py, строки 231–241.

```
231                              sort_dir=sort_dir,
232                          ))
233                  add_tes_machine_type_all_versions_service(payload, user)
234                  flash("Новая запись добавлена во всех версиях БД (общий ref_uuid).", "success")
235              else:
236                  add_tes_machine_type_service(payload, user)
237                  flash("Новая запись успешно добавлена.", "success")
238
239              # Перенаправление на список с сохранением параметров и переходом к новой записи
240              total_records = tes_machine_type_query(
241                                  tes_machine_type_filter).count()
```

Сервис добавления – создание сущности. Источник: app/refdata/services/refdata_for_stations/machines/tes_machine_type_services.py, строки 263–276.

```
263                         )
264
265                 # Создаем новую запись
266                 obj = TesMachineType(name=name, display_order=display_order)
267                 set_db_version_on_create(obj)
268                 db.session.add(obj)
269                 db.session.flush()
270
271                 log_to_db(
272                     user,
273                     "Создан тип агрегата ТЭС",
274                     f"Наименование: {name}",
275                     entity_type="tes_machine_type",
276                     entity_id=obj.id)
```

HTML/Jinja2 формы добавления. Источник: app/templates/refdata/refdata_for_stations/machines/tes_machine_type/tes_machine_type_add.html, строки 20–34.

```
20          <div class="mb-3">
21              <label for="name" class="form-label">Тип агрегата ТЭС</label>
22              {{ form.name(class="form-control", id="name") }}
23              {% if form.name.errors %}
24                  <div class="text-danger">{{ form.name.errors|join(', ') }}</div>
25              {% endif %}
26          </div>
27
28          <div class="mt-4 d-flex flex-wrap gap-2 align-items-center">
29              <button type="submit" class="btn btn-primary">Добавить</button>
30              {% if current_user.has_admin %}<button type="submit" name="all_versions" value="1"
class="btn btn-outline-primary" title="Добавить запись во все версии БД">Добавить во всех
версиях</button>{% endif %}
31              <a href="{{ url_for('refdata_bp.tes_machine_type_list',
32                                  page=page,
33                                  per_page=per_page,
34                                  sort_by=sort_by,
```

Взаимодействие при добавлении. Специализированные модели оборудования генерации используют TesMachineType в дополнение к общему MachineType. Классификация участвует в представлении теплового оборудования; конкретные модели и сервисы-потребители указаны в ссылках ниже. После создания запись становится доступна для выбора в зависимых формах при их следующем построении с учетом версии и используемых механизмов кэширования.

### 4.19.3. Экспорт, импорт и дополнительные операции

| Адрес и метод | Обработчик | Результат |
|---|---|---|
| GET /refdata/export_tes_machine_type | export_tes_machine_type | Файл Excel для скачивания |

Отдельный маршрут импорта в этом справочнике не зарегистрирован. Поддержка импорта не приписывается всем справочникам по аналогии.

## 4.20. Типы агрегатов ПГУ

### 4.20.1. Страница списка

Адрес: GET, POST /refdata/pgu_tes_machine_type. Обработчик: pgu_tes_machine_type_list().

Ведет классификатор типов агрегатов парогазовых установок. Страница списка используется для просмотра и изменения наименований; отдельная форма добавляет тип.

Поля и параметры HTML-формы: per_page, page, sort_by, sort_dir, pgu_tes_machine_type_filter, all_versions, pgu_tes_machine_type_ids[], pgu_tes_machine_type_names[], pgu_tes_machine_type_delete[], js/refdata/textarea_autoresize.js.

Обработка: маршрут получает параметры отображения, при POST читает изменяемые строки и флажки удаления, вызывает сервисы и возвращает пользователя к списку с параметрами просмотра. Для обычного просмотра строится выборка и передается в шаблон вместе с пагинацией и вариантами выбора связанных справочников.

Модель: PGUTesMachineType. Поля SQLAlchemy: id, name, created_at, updated_at, database_version_id, pgu_machines.

Модель данных – фрагмент объявления. Источник: app/refdata/models/refdata_for_stations/machine/pgu_tes_machine_type_model.py, строки 13–35.

```
13  class PGUTesMachineType(db.Model, AuditMixin, VersionedModelMixin, RefdataUuidMixin):
14      __tablename__ = 'gs_sys_pgu_tes_machine_types'
15      __table_args__ = {"schema": SCHEMA_REFDATA}
16
17      id = db.Column(db.Integer, primary_key=True, autoincrement=True)
18      name = db.Column(db.String(80), unique=True, nullable=False, index=True)
19
20      created_at = db.Column(db.DateTime(timezone=True), server_default=func.now(), nullable=False)
21      updated_at = db.Column(db.DateTime(timezone=True), server_default=func.now(),
onupdate=func.now(), nullable=False)
22
23      # Поле для связи с версией БД
24      database_version_id = db.Column(
25          db.Integer,
26          db.ForeignKey(f"{SCHEMA_REFDATA}.gs_database_versions.id", ondelete="SET NULL"),
27          nullable=True,
28          index=True
29      )
30
31      pgu_machines = db.relationship(
32          'PGUMachine',
33          back_populates='pgu_tes_machine_type',
34          cascade="all, delete-orphan"
35      )
```

Маршрут списка – вход и параметры. Источник: app/refdata/routes/refdata_for_stations/machines/pgu_tes_machine_type_routes.py, строки 35–50.

```
35  @refdata_bp.route("/pgu_tes_machine_type", methods=["GET", "POST"])
36  @login_required
37  def pgu_tes_machine_type_list():
38      """Маршрут для отображения списка типов агрегатов ПГУ."""
39
40      user = session.get('username', 'Неизвестный пользователь')
41      log_to_db(
42          user,
43          "Открыта страница типов агрегатов ПГУ",
44          entity_type="pgu_tes_machine_type")
45
46      # Создание формы
47      form = PGUTesMachineTypeFilterForm()
48
49      # Получение параметров запроса
50      page                            = request.args.get("page", 1, type=int)
```

Маршрут списка – передача изменений сервису. Источник: app/refdata/routes/refdata_for_stations/machines/pgu_tes_machine_type_routes.py, строки 124–129.

```
124                                             sort_dir=sort_dir,
125                     ))
126                 update_pgu_tes_machine_type_all_versions_service(pgu_tes_machine_type_data, user)
127                 flash("Изменения применены во всех версиях БД (по ref_uuid).", "success")
128             else:
129                 update_pgu_tes_machine_type_service(pgu_tes_machine_type_data, user)
```

Сервис списка – выборка и пагинация. Источник: app/refdata/services/refdata_for_stations/machines/pgu_tes_machine_type_services.py, строки 69–86.

```
 69  @no_autoflush
 70  def get_pgu_tes_machine_type_list(
 71      page,
 72      per_page,
 73      pgu_tes_machine_type_filter=None,
 74      sort_by="id",
 75      sort_dir="asc"):
 76      """ Получает список типов агрегатов ПГУ с пагинацией, фильтрацией и сортировкой. """
 77
 78      # Базовый запрос
 79      query = pgu_tes_machine_type_query(
 80          pgu_tes_machine_type_filter=pgu_tes_machine_type_filter,
 81          sort_by=sort_by,
 82          sort_dir=sort_dir,
 83      )
 84
 85      # Пагинация
 86      return query.paginate(page=page, per_page=per_page, error_out=False)
```

Сервис изменения – сохранение и журналирование. Источник: app/refdata/services/refdata_for_stations/machines/pgu_tes_machine_type_services.py, строки 154–166.

```
154
155      try:
156          # Сохранение изменений в базе данных
157          # Фиксация транзакции (устойчивый коммит)
158          _commit_with_retry()
159
160          if updated_ids:
161              log_to_db(
162                  user,
163                  "Сохранены изменения по типам агрегатов",
164                  f"Измененных записей: {len(updated_ids)} (id: {updated_ids})",
165                  entity_type="pgu_tes_machine_type",
166                  entity_id=pgu_tes_machine_type_id)
```

HTML/Jinja2 списка – элементы формы. Источник: app/templates/refdata/refdata_for_stations/machines/pgu_tes_machine_type/pgu_tes_machine_type.html, строки 160–173.

```
160                              <!-- Строки таблицы. Столбец "Тип агрегата" -->
161                              <td>
162                                  <input type="text" name="pgu_tes_machine_type_names[]" class="form-
control" value="{{ pgu_tes_machine_type.name }}">
163                              </td>
164
165                              <!-- Строки таблицы. Столбец "Чекмарк для удаления" -->
166                              <td class="text-center align-middle">
167                                  <input type="checkbox" name="pgu_tes_machine_type_delete[]"
value="{{ pgu_tes_machine_type.id }}" class="form-check-input">
168                              </td>
169                          </tr>
170                          {% else %}
171                          <tr>
172                              <td colspan="3" class="text-center align-middle">Нет данных для
отображения</td>
173                          </tr>
```

Взаимодействие с другими модулями. PGUMachine.pgu_tes_machine_type связывает состав ПГУ с классификатором. Тип используется при отображении и обработке оборудования в модуле генерации; он не заменяет связи агрегатов ПГУ с их техническими характеристиками.

Подтверждающие ссылки на потребителей модели:
app/generation/models/pgu_machine/pgu_machine_model.py:62 – pgu_tes_machine_type = db.relationship('PGUTesMachineType', back_populates='pgu_machines')
app/generation/services/machine_services/machine_services.py:32 – from app.refdata.models.refdata_for_stations.machine.pgu_tes_machine_type_model import PGUTesMachineType
app/generation/services/station_changes_services/station_changes_services.py:25 – from app.refdata.models.refdata_for_stations.machine.pgu_tes_machine_type_model import PGUTesMachineType

app/generation/services/station_services/filters_services.py:35 – PGUTesMachineType,
app/generation/services/station_services/station_services.py:3548 – from
app.refdata.models.refdata_for_stations.machine.pgu_tes_machine_type_model import PGUTesMachineType

### 4.20.2. Форма добавления

Адрес: GET, POST /refdata/add_pgu_tes_machine_type. Обработчик: add_pgu_tes_machine_type().

Назначение: создание записи справочника «Типы агрегатов ПГУ». Вводимые поля формы: name. Список полей формы может быть уже модели: дополнительные реквизиты изменяются на странице списка или заполняются сервисом.

GET создает форму и подготавливает варианты выбора. POST с валидной формой формирует данные и передает их сервису добавления. При успехе выполняется переход к списку; сообщения о результате выводятся пользователю. Кнопка отмены возвращает к списку без создания записи. Создаваемая сущность – PGUTesMachineType, модель и ее связи приведены выше в подразделе 4.20.1.

Поля WTForms и валидаторы: csrf_token = HiddenField(); name = StringField('Тип агрегата', validators=[DataRequired(message='Поле «Тип агрегата» обязательно.'), Length(min=2, max=255, message='Длина от 2 до 255 символов.')]).

Шаблон содержит кнопку добавления во всех версиях для административного пользователя. Серверная ветвь all_versions=1 вызывает специализированную проверку доступа и межверсионный сервис; при обычном добавлении используется выбранная версия.

Модель добавляемого объекта PGUTesMachineType – основной реквизит. Источник: app/refdata/models/refdata_for_stations/machine/pgu_tes_machine_type_model.py, строки 18–18.

```
18      name = db.Column(db.String(80), unique=True, nullable=False, index=True)
```

Маршрут добавления – вход в форму. Источник: app/refdata/routes/refdata_for_stations/machines/pgu_tes_machine_type_routes.py, строки 164–176.

```
164  @refdata_bp.route("/add_pgu_tes_machine_type", methods=["GET", "POST"])
165  @login_required
166  def add_pgu_tes_machine_type():
167      """ Маршрут для добавления нового типа агрегата ПГУ. """
168
169      user = session.get('username', 'Неизвестный пользователь')
170      log_to_db(
171          user,
172          "Открыта страница добавления типов агрегатов ПГУ",
173          entity_type="pgu_tes_machine_type")
174
175      # Создание формы
176      form = AddPGUTesMachineTypeForm()
```

Маршрут добавления – вызов сервиса. Источник: app/refdata/routes/refdata_for_stations/machines/pgu_tes_machine_type_routes.py, строки 210–220.

```
210                                             sort_dir=sort_dir,
211                     ))
212                 add_pgu_tes_machine_type_all_versions_service(payload, user)
213                 flash("Новая запись добавлена во всех версиях БД (общий ref_uuid).", "success")
214             else:
215                 add_pgu_tes_machine_type_service(payload, user)
216                 flash("Новая запись типа агрегата ПГУ успешно добавлена.", "success")
217
218             # Перенаправление на список с сохранением параметров и переходом к новой записи
219             total_records = pgu_tes_machine_type_query(
220                                 pgu_tes_machine_type_filter).count()
```

Сервис добавления – создание сущности. Источник: app/refdata/services/refdata_for_stations/machines/pgu_tes_machine_type_services.py, строки 218–231.

```
218                 if dup:
219                     raise ValueError(f"Запись с наименованием «{name}» уже существует.")
220
221                 obj = PGUTesMachineType(name=name)
222                 set_db_version_on_create(obj)
223                 db.session.add(obj)
224                 db.session.flush()
225
226                 log_to_db(
227                     user,
228                     "Создан тип агрегата ПГУ",
229                     f"Наименование: {name}",
230                     entity_type="pgu_tes_machine_type",
231                     entity_id=obj.id)
```

HTML/Jinja2 формы добавления. Источник: app/templates/refdata/refdata_for_stations/machines/pgu_tes_machine_type/pgu_tes_machine_type_add.html, строки 20–34.

```
  20          <div class="mb-3">
  21              <label for="name" class="form-label">Тип агрегата ПГУ</label>
  22              {{ form.name(class="form-control", id="name") }}
  23              {% if form.name.errors %}
  24                  <div class="text-danger">{{ form.name.errors|join(', ') }}</div>
  25              {% endif %}
  26          </div>
  27
  28          <div class="mt-4 d-flex flex-wrap gap-2 align-items-center">
  29              <button type="submit" class="btn btn-primary">Добавить ПГУ</button>
  30              {% if current_user.has_admin %}<button type="submit" name="all_versions" value="1"
class="btn btn-outline-primary" title="Добавить запись во все версии БД">Добавить во всех
версиях</button>{% endif %}
  31              <a href="{{ url_for('refdata_bp.pgu_tes_machine_type_list',
  32                                  page=page,
  33                                  per_page=per_page,
  34                                  sort_by=sort_by,
```

Взаимодействие при добавлении. PGUMachine.pgu_tes_machine_type связывает состав ПГУ с классификатором. Тип используется при отображении и обработке оборудования в модуле генерации; он не заменяет связи агрегатов ПГУ с их техническими характеристиками. После создания запись становится доступна для выбора в зависимых формах при их следующем построении с учетом версии и используемых механизмов кэширования.

4.20.3. Экспорт, импорт и дополнительные операции

| Адрес и метод | Обработчик | Результат |
|---|---|---|
| GET /refdata/export_pgu_tes_machine_type | export_pgu_tes_machine_type | Файл Excel для скачивания |

Отдельный маршрут импорта в этом справочнике не зарегистрирован. Поддержка импорта не приписывается всем справочникам по аналогии.

## 4.21. Типы групп оборудования

4.21.1. Страница списка

Адрес: GET, POST /refdata/equipment_group. Обработчик: equipment_group_list().

Хранит наименование и порядок отображения типа группы оборудования, ссылки на тип технологии и ее доступность. В списке имеются фильтры по связанным классификаторам. Сервисы обрабатывают совпадающие типы групп и

перенос связей на каноническую запись; удаление выполняется с учетом зависимостей.

Поля и параметры HTML-формы: per_page, page, sort_by, sort_dir, technology_type_filter, technology_availability_filter, equipment_group_filter, all_versions, equipment_group_ids[], display_orders[], equipment_group_names[], technology_types[], technology_availabilities[], equipment_group_delete[], js/refdata/textarea_autoresize.js.

Обработка: маршрут получает параметры отображения, при POST читает изменяемые строки и флажки удаления, вызывает сервисы и возвращает пользователя к списку с параметрами просмотра. Для обычного просмотра строится выборка и передается в шаблон вместе с пагинацией и вариантами выбора связанных справочников.

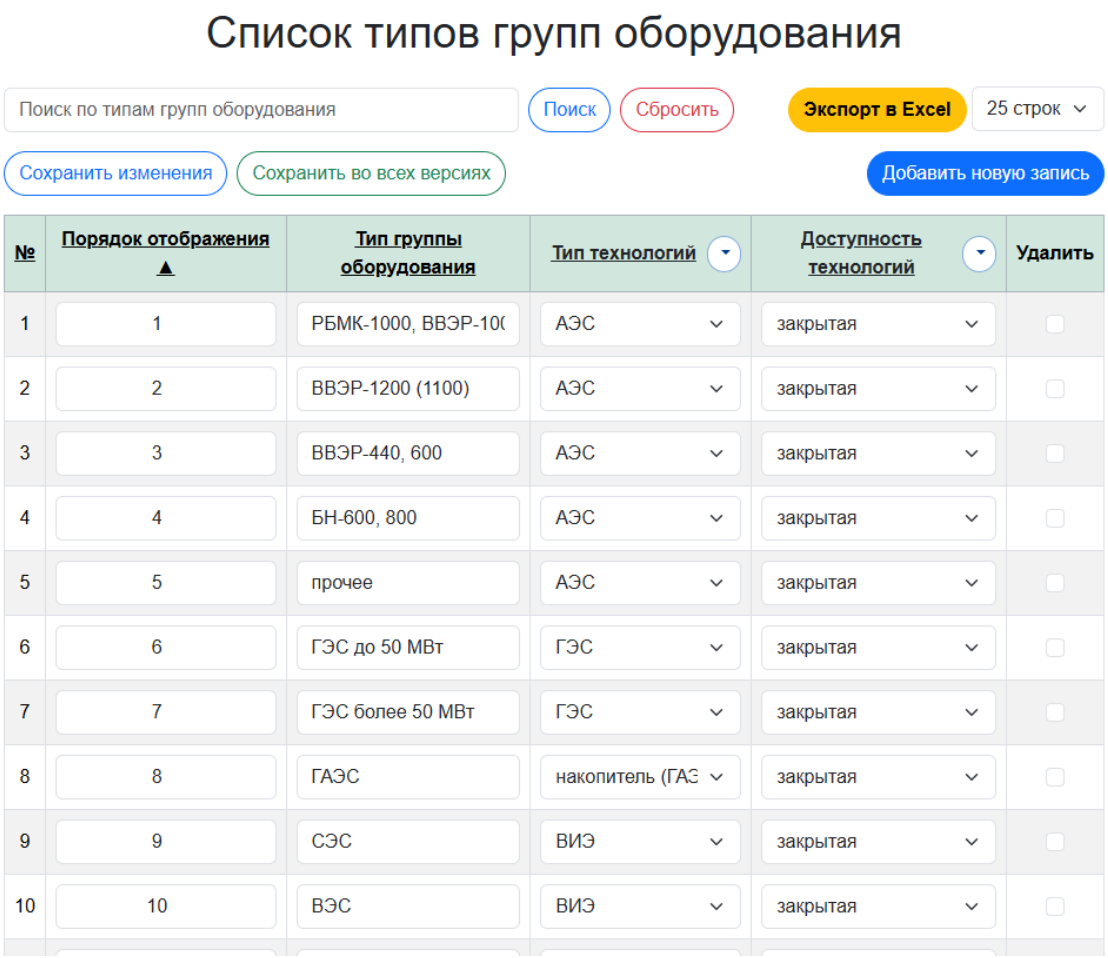



Рисунок 7. Типы групп оборудования. Действующий сайт, 29.10.2026. Показан фрагмент экранной формы.

Модель: EquipmentGroupType. Поля SQLAlchemy: id, display_order, name, id_technology_availability, technology_availability, id_technology_type, technology_type, created_at, updated_at, database_version_id, machines.

Модель данных – фрагмент объявления. Источник: app/refdata/models/refdata_for_stations/technologies/equipment_group_model.py, строки 14–32.

```
class EquipmentGroupType(db.Model, AuditMixin, VersionedModelMixin, RefdataUuidMixin):
    __tablename__ = 'gs_sys_equipment_groups'
    __table_args__ = {"schema": SCHEMA_REFDATA}

    id = db.Column(db.Integer, primary_key=True, autoincrement=True)
    # Порядок отображения
    display_order = db.Column(db.Integer, nullable=True)
    name = db.Column(db.String(80), unique=True, nullable=False, index=True)

    # FK -> TechnologyAvailability
    id_technology_availability = db.Column(
        db.Integer,
        db.ForeignKey(f'{SCHEMA_REFDATA}.gs_sys_technology_availabilities.id', ondelete='SET
NULL'),
        nullable=True,
        index=True,
    )
    technology_availability = db.relationship('TechnologyAvailability',
back_populates='equipment_groups')

    # FK -> TechnologyType
```

Маршрут списка – вход и параметры. Источник: app/refdata/routes/refdata_for_stations/technologies/equipment_group_routes.py, строки 58–73.

```
@refdata_bp.route("/equipment_group", methods=["GET", "POST"])
@login_required
def equipment_group_list():
    """Маршрут для отображения списка типов групп оборудования."""

    user = session.get('username', 'Неизвестный пользователь')
    log_to_db(
        user,
        "Открыта страница типов групп оборудования",
        entity_type="equipment_group")

    # Создание формы
    form = EquipmentGroupTypeFilterForm()

    # Получение параметров запроса
    page                            = request.args.get("page", 1, type=int)
```

Маршрут списка – передача изменений сервису. Источник: app/refdata/routes/refdata_for_stations/technologies/equipment_group_routes.py, строки 186–193.

```
                                            sort_by=sort_by,
                                            sort_dir=sort_dir))
                update_equipment_group_all_versions_service(
                    equipment_group_data, user
                )
                flash("Изменения применены во всех версиях БД (по ref_uuid).", "success")
            else:
                update_equipment_group_service(equipment_group_data, user)
```

Сервис списка – выборка и пагинация. Источник:
app/refdata/services/refdata_for_stations/technologies/equipment_group_services.py, строки 345–366.

```
@no_autoflush
def get_equipment_group_list(
    page,
    per_page,
    equipment_group_filter=None,
    technology_type_filter=None,
    technology_availability_filter=None,
    sort_by="display_order",
    sort_dir="asc"):
    """ Получает список типов групп оборудования с пагинацией, фильтрацией и сортировкой. """

    # Базовый запрос
    query = equipment_group_query(
        equipment_group_filter=equipment_group_filter,
        technology_type_filter=technology_type_filter,
        technology_availability_filter=technology_availability_filter,
        sort_by=sort_by,
        sort_dir=sort_dir,
    )

    # Пагинация
    return query.paginate(page=page, per_page=per_page, error_out=False)
```

Сервис изменения – сохранение и журналирование. Источник:
app/refdata/services/refdata_for_stations/technologies/equipment_group_services.py, строки 496–508.

```
                    updated_ids.append(equipment_group_id)

            db.session.flush()

        _commit_with_retry()

        if updated_ids:
            log_to_db(
                user,
                "Сохранены изменения по типам групп оборудования",
                f"Измененных записей: {len(updated_ids)} (id: {updated_ids})",
                entity_type="equipment_group",
                entity_id=last_entity_id)
```

HTML/Jinja2 списка – элементы формы. Источник:
app/templates/refdata/refdata_for_stations/technologies/equipment_group/equipment_group.html, строки 333–346.

```
                            <!-- Строки таблицы. Столбец "Группа оборудования" -->
                            <td>
                                <input type="text" name="equipment_group_names[]" class="form-
control" value="{{ equipment_group.name }}">
                            </td>

                            <!-- Строки таблицы. Столбец "Тип технологий" -->
                            <td>
                                <select name="technology_types[]" class="form-select">
                                    {% for technology_type in technology_types %}
                                    <option value="{{ technology_type.0 }}"
                                        {% if technology_type.0 ==
equipment_group.id_technology_type %}selected{% endif %}>
                                        {{ technology_type.1 }}
                                    </option>
                                    {% endfor %}
```

Взаимодействие с другими модулями. Machine.id_equipment_group ссылается на EquipmentGroupType. Классификатор используется генерацией, топливным модулем и моделями рациональной структуры. Это именно тип группы, а не экземпляр топливной группы оборудования: назначение конкретных агрегатов и расчетные параметры ведутся в профильных модулях.

Подтверждающие ссылки на потребителей модели:
app/fuel/models/fue_equipment_group_set_station_model.py:43 – "EquipmentGroupType",
app/generation/models/machine/machine_model.py:133 – equipment_group =
db.relationship('EquipmentGroupType', back_populates='machines')

app/rational_structure/models/territory_technology_model.py:26 – equipment_group_type = db.relationship("EquipmentGroupType")
app/api/services/equipment_group_exchange_services.py:29 – EquipmentGroupType,
app/ecology/services/gis_tes_pg_from_fuel_services.py:113 – «Котельная» – это поле «Тип группы оборудования» (EquipmentGroupType.name

### 4.21.2. Форма добавления

Адрес: GET, POST /refdata/add_equipment_group. Обработчик: add_equipment_group().

Назначение: создание записи справочника «Типы групп оборудования». Вводимые поля формы: name, technology_type, technology_availability. Список полей формы может быть уже модели: дополнительные реквизиты изменяются на странице списка или заполняются сервисом.

GET создает форму и подготавливает варианты выбора. POST с валидной формой формирует данные и передает их сервису добавления. При успехе выполняется переход к списку; сообщения о результате выводятся пользователю. Кнопка отмены возвращает к списку без создания записи. Создаваемая сущность – EquipmentGroupType, модель и ее связи приведены выше в подразделе 4.21.1.

Поля WTForms и валидаторы: csrf_token = HiddenField(); name = StringField('Тип группы оборудования', validators=[DataRequired(message='Поле «Тип группы оборудования» обязательно.'), Length(min=2, max=255, message='Длина от 2 до 255 символов.')]); technology_type = SelectField('Тип технологии', choices=[], coerce=int, validators=[Optional()]); technology_availability = SelectField('Доступность технологии', choices=[], coerce=int, validators=[Optional()]).

Шаблон содержит кнопку добавления во всех версиях для административного пользователя. Серверная ветвь all_versions=1 вызывает специализированную проверку доступа и межверсионный сервис; при обычном добавлении используется выбранная версия.

Модель добавляемого объекта EquipmentGroupType – основной реквизит. Источник: app/refdata/models/refdata_for_stations/technologies/equipment_group_model.py, строки 21–21.

```
21      name = db.Column(db.String(80), unique=True, nullable=False, index=True)
```

Маршрут добавления – вход в форму. Источник: app/refdata/routes/refdata_for_stations/technologies/equipment_group_routes.py, строки 244–256.

```
244 @refdata_bp.route("/add_equipment_group", methods=["GET", "POST"])
245 @login_required
246 def add_equipment_group():
247     """ Маршрут для добавления нового типа группы оборудования. """
248
249     user = session.get('username', 'Неизвестный пользователь')
250     log_to_db(
251         user,
252         "Открыта страница добавления типов групп оборудования",
253         entity_type="equipment_group")
254
255     # Создание формы
256     form = AddEquipmentGroupTypeForm()
```

Маршрут добавления – вызов сервиса. Источник: app/refdata/routes/refdata_for_stations/technologies/equipment_group_routes.py, строки 290–302.

```
290                                             sort_by=sort_by,
291                                             sort_dir=sort_dir))
292                 add_equipment_group_all_versions_service(payload, user)
293                 flash("Новая запись добавлена во всех версиях БД (общий ref_uuid).", "success")
294             else:
295                 add_equipment_group_service(payload, user)
296                 flash("Новая запись успешно добавлена.", "success")
297
298             # Перенаправление на список с сохранением параметров и переходом к новой записи
299             total_records = equipment_group_query(
300                                 equipment_group_filter,
301                                 technology_type_filter,
302                                 technology_availability_filter).count()
```

Сервис добавления – создание сущности. Источник: app/refdata/services/refdata_for_stations/technologies/equipment_group_services.py, строки 590–610.

```
590                     if dup_display:
591                         raise ValueError(f"Запись с порядком отображения «{display_order}» уже
существует.")
592
593                 obj = EquipmentGroupType(
594                     display_order=display_order,
595                     name=name,
596                     id_technology_type=technology_type_id,
597                     id_technology_availability=technology_availability_id
598                 )
599                 set_db_version_on_create(obj)
600                 db.session.add(obj)
601                 db.session.flush()
602
603                 log_to_db(
604                     user,
605                     "Создан тип группы оборудования",
606                     f"Порядок отображения: {_dash(display_order)}; "
607                     f"Тип технологии: {get_technology_type_name(technology_type_id)} "
608                     f"Тип доступности технологии:
{get_technology_availability_name(technology_availability_id)} ",
609                     entity_type="equipment_group",
610                     entity_id=obj.id)
```

HTML/Jinja2 формы добавления. Источник: app/templates/refdata/refdata_for_stations/technologies/equipment_group/equipment_group_add.html, строки 22–36.

```
22          <div class="mb-3">
23              <label for="name" class="form-label">Тип группы оборудования</label>
24              {{ form.name(class="form-control", id="name") }}
25              {% if form.name.errors %}
26                  <div class="text-danger">{{ form.name.errors|join(', ') }}</div>
27              {% endif %}
28          </div>
29
30          <div class="mb-3">
31              <label for="technology_type" class="form-label">Тип технологии</label>
32              {{ form.technology_type(class="form-control", id="technology_type") }}
33              {% if form.technology_type.errors %}
34                  <div class="text-danger">{{ form.technology_type.errors|join(', ') }}</div>
35              {% endif %}
36          </div>
```

Взаимодействие при добавлении. Machine.id_equipment_group ссылается на EquipmentGroupType. Классификатор используется генерацией, топливным модулем и моделями рациональной структуры. Это именно тип группы, а не экземпляр топливной группы оборудования: назначение конкретных агрегатов и расчетные параметры ведутся в профильных модулях. После создания запись становится доступна для выбора в зависимых формах при их следующем построении с учетом версии и используемых механизмов кэширования.

4.21.3. Экспорт, импорт и дополнительные операции

| Адрес и метод | Обработчик | Результат |
|---|---|---|
| GET /refdata/export_equipment_group | export_equipment_group | Файл Excel для скачивания |

Отдельный маршрут импорта в этом справочнике не зарегистрирован. Поддержка импорта не приписывается всем справочникам по аналогии.

## 4.22. Типы технологий

4.22.1. Страница списка

Адрес: GET, POST /refdata/technology_type. Обработчик: technology_type_list().

Ведет типы технологий и порядок отображения. Страница позволяет искать и редактировать записи; новая технология добавляется через самостоятельную форму.

Поля и параметры HTML-формы: per_page, page, sort_by, sort_dir, technology_type_filter, all_versions, technology_ids[], display_orders[], technology_type_names[], technology_type_delete[], js/refdata/textarea_autoresize.js.

Обработка: маршрут получает параметры отображения, при POST читает изменяемые строки и флажки удаления, вызывает сервисы и возвращает пользователя к списку с параметрами просмотра. Для обычного просмотра строится выборка и передается в шаблон вместе с пагинацией и вариантами выбора связанных справочников.

Модель: TechnologyType. Поля SQLAlchemy: id, display_order, name, created_at, updated_at, database_version_id, equipment_groups, machines.

Модель данных – фрагмент объявления. Источник: app/refdata/models/refdata_for_stations/technologies/technology_type_model.py, строки 13–32.

```
  13  class TechnologyType(db.Model, AuditMixin, VersionedModelMixin, RefdataUuidMixin):
  14      __tablename__ = 'gs_sys_technology_types'
  15      __table_args__ = {"schema": SCHEMA_REFDATA}
  16
  17      id = db.Column(db.Integer, primary_key=True, autoincrement=True)
  18      # Порядок отображения
  19      display_order = db.Column(db.Integer, nullable=True)
  20      # Сохраняем исходную nullable=True, чтобы не трогать существующие данные.
  21      name = db.Column(db.String(80), unique=True, nullable=True, index=True)
  22
  23      created_at = db.Column(db.DateTime(timezone=True), server_default=func.now(), nullable=False)
  24      updated_at = db.Column(db.DateTime(timezone=True), server_default=func.now(),
onupdate=func.now(), nullable=False)
  25
  26      # Поле для связи с версией БД
  27      database_version_id = db.Column(
  28          db.Integer,
  29          db.ForeignKey(f"{SCHEMA_REFDATA}.gs_database_versions.id", ondelete="SET NULL"),
  30          nullable=True,
  31          index=True
  32      )
```

Связь модели с другой сущностью. Источник: app/refdata/models/refdata_for_stations/technologies/technology_type_model.py, строки 34–34.

```
  34      equipment_groups = db.relationship('EquipmentGroupType', back_populates='technology_type')
```

Маршрут списка – вход и параметры. Источник: app/refdata/routes/refdata_for_stations/technologies/technology_type_routes.py, строки 35–50.

```
@refdata_bp.route("/technology_type", methods=["GET", "POST"])
@login_required
def technology_type_list():
    """Маршрут для отображения списка типов технологий."""

    user = session.get('username', 'Неизвестный пользователь')
    log_to_db(
        user,
        "Открыта страница типов технологий",
        entity_type="technology_type")

    # Создание формы
    form = TechnologyTypeFilterForm()

    # Получение параметров запроса
    page                = request.args.get("page", 1, type=int)
```

Маршрут списка – передача изменений сервису. Источник: app/refdata/routes/refdata_for_stations/technologies/technology_type_routes.py, строки 137–142.

```
                            sort_dir=sort_dir,
                        ))
                update_technology_type_all_versions_service(technology_type_data, user)
                flash("Изменения применены во всех версиях БД (по ref_uuid).", "success")
            else:
                update_technology_type_service(technology_type_data, user)
```

Сервис списка – выборка и пагинация. Источник: app/refdata/services/refdata_for_stations/technologies/technology_type_services.py, строки 80–97.

```
@no_autoflush
def get_technology_type_list(
    page,
    per_page,
    technology_type_filter=None,
    sort_by="display_order",
    sort_dir="asc"):
    """ Получает список типов технологий с пагинацией, фильтрацией и сортировкой. """

    # Базовый запрос
    query = technology_type_query(
        technology_type_filter=technology_type_filter,
        sort_by=sort_by,
        sort_dir=sort_dir,
    )

    # Пагинация
    return query.paginate(page=page, per_page=per_page, error_out=False)
```

Сервис изменения – сохранение и журналирование. Источник: app/refdata/services/refdata_for_stations/technologies/technology_type_services.py, строки 223–235.

```

    try:
        # Сохранение изменений в базе данных
        # Фиксация транзакции (устойчивый коммит)
        _commit_with_retry()

        if updated_ids:
            log_to_db(
                user,
                "Сохранены изменения по типам технологий",
                f"Измененных записей: {len(updated_ids)} (id: {updated_ids})",
                entity_type="technology_type",
                entity_id=technology_type_id)
```

HTML/Jinja2 списка – элементы формы. Источник: app/templates/refdata/refdata_for_stations/technologies/technology_type/technology_type.html, строки 193–206.

```
 193                                <!-- Строки таблицы. Столбец "Тип технологии" -->
 194                                <td>
 195                                    <input type="text" name="technology_type_names[]" class="form-
control" value="{{ technology_type.name }}">
 196                                </td>
 197
 198                                <!-- Строки таблицы. Столбец "Чекмарк для удаления" -->
 199                                <td class="text-center align-middle">
 200                                    <input type="checkbox" name="technology_type_delete[]" value="{{
technology_type.id }}" class="form-check-input">
 201                                </td>
 202                            </tr>
 203                            {% else %}
 204                            <tr>
 205                                <td colspan="4" class="text-center align-middle">Нет данных для
отображения</td>
 206                            </tr>
```

Взаимодействие с другими модулями. EquipmentGroupType.id_technology_type и связи оборудования используют классификатор для технологической группировки. Модуль рациональной структуры и топливные сервисы обращаются к тем же сущностям в местах, подтвержденных ссылками на исходный код ниже.

Подтверждающие ссылки на потребителей модели:
app/generation/models/machine/machine_model.py:124 – technology_type = db.relationship('TechnologyType', back_populates='machines')
app/rational_structure/models/territory_technology_model.py:25 – technology_type = db.relationship("TechnologyType")
app/rational_structure/services/territory_equipment_services.py:38 – from app.refdata.models.refdata_for_stations.technologies.technology_type_model import TechnologyType
app/rational_structure/services/territory_technology_matching.py:8 – from app.refdata.models.refdata_for_stations.technologies.technology_type_model import TechnologyType

### 4.22.2. Форма добавления

Адрес: GET, POST /refdata/add_technology_type. Обработчик: add_technology_type().

Назначение: создание записи справочника «Типы технологий». Вводимые поля формы: name, display_order. Список полей формы может быть уже модели: дополнительные реквизиты изменяются на странице списка или заполняются сервисом.

GET создает форму и подготавливает варианты выбора. POST с валидной формой формирует данные и передает их сервису добавления. При успехе выполняется переход к списку; сообщения о результате выводятся пользователю. Кнопка отмены возвращает к списку без создания записи. Создаваемая сущность – TechnologyType, модель и ее связи приведены выше в подразделе 4.22.1.

Поля WTForms и валидаторы: csrf_token = HiddenField(); name = StringField('Тип технологии', validators=[DataRequired(message='Поле «Тип технологии» обязательно.'), Length(min=2, max=255, message='Длина от 2 до 255 символов.')]); display_order = IntegerField('Порядок отображения', validators=[Optional(), NumberRange(min=0)]).

Шаблон содержит кнопку добавления во всех версиях для административного пользователя. Серверная ветвь all_versions=1 вызывает специализированную проверку доступа и межверсионный сервис; при обычном добавлении используется выбранная версия.

Модель добавляемого объекта TechnologyType – основной реквизит. Источник: app/refdata/models/refdata_for_stations/technologies/technology_type_model.py, строки 21–21.

```
21      name = db.Column(db.String(80), unique=True, nullable=True, index=True)
```

Маршрут добавления – вход в форму. Источник: app/refdata/routes/refdata_for_stations/technologies/technology_type_routes.py, строки 177–189.

```
177 @refdata_bp.route("/add_technology_type", methods=["GET", "POST"])
178 @login_required
179 def add_technology_type():
180     """ Маршрут для добавления нового типа технологии. """
181
182     user = session.get('username', 'Неизвестный пользователь')
183     log_to_db(
184         user,
185         "Открыта страница добавления типов технологий",
186         entity_type="technology_type")
187
188     # Создание формы
189     form = AddTechnologyTypeForm()
```

Маршрут добавления – вызов сервиса. Источник: app/refdata/routes/refdata_for_stations/technologies/technology_type_routes.py, строки 230–240.

```
230                             sort_dir=sort_dir,
231                     ))
232                 add_technology_type_all_versions_service(payload, user)
233                 flash("Новая запись добавлена во всех версиях БД (общий ref_uuid).", "success")
234             else:
235                 add_technology_type_service(payload, user)
236                 flash("Новая запись успешно добавлена.", "success")
237
238             # Перенаправление на список с сохранением параметров и переходом к новой записи
239             total_records = technology_type_query(
240                                 technology_type_filter=technology_type_filter).count()
```

Сервис добавления – создание сущности. Источник: app/refdata/services/refdata_for_stations/technologies/technology_type_services.py, строки 312–325.

```
312                             f"Порядок «{display_order}» уже задан у записи «{other}»
(id={dup_order.id})."
313                         )
314
315                 obj = TechnologyType(name=name, display_order=display_order)
316                 set_db_version_on_create(obj)
317                 db.session.add(obj)
318                 db.session.flush()
319
320                 log_to_db(
321                     user,
322                     "Создан тип технологии",
323                     f"Наименование: {name}",
324                     entity_type="technology_type",
325                     entity_id=obj.id)
```

HTML/Jinja2 формы добавления. Источник: app/templates/refdata/refdata_for_stations/technologies/technology_type/technology_type_add.html, строки 20–34.

```
20          <div class="mb-3">
21              <label for="name" class="form-label">Тип технологии</label>
22              {{ form.name(class="form-control", id="name") }}
23              {% if form.name.errors %}
24                  <div class="text-danger">{{ form.name.errors|join(', ') }}</div>
25              {% endif %}
26          </div>
27
28          <div class="mb-3">
29              <label for="display_order" class="form-label">Порядок отображения</label>
30              {{ form.display_order(class="form-control", id="display_order", type="number", min="0",
placeholder="") }}
31              {% if form.display_order.errors %}
32                  <div class="text-danger">{{ form.display_order.errors|join(', ') }}</div>
33              {% endif %}
34          </div>
```

Взаимодействие при добавлении. EquipmentGroupType.id_technology_type и связи оборудования используют классификатор для технологической группировки. Модуль рациональной структуры и топливные сервисы обращаются к тем же сущностям в местах, подтвержденных ссылками на исходный код ниже. После создания запись становится доступна для выбора в зависимых формах при их следующем построении с учетом версии и используемых механизмов кэширования.

4.22.3. Экспорт, импорт и дополнительные операции

| Адрес и метод | Обработчик | Результат |
|---|---|---|
| GET /refdata/export_technology_type | export_technology_type | Файл Excel для скачивания |

Отдельный маршрут импорта в этом справочнике не зарегистрирован. Поддержка импорта не приписывается всем справочникам по аналогии.

## 4.23. Доступность технологии

4.23.1. Страница списка

Адрес: GET, POST /refdata/technology_availability. Обработчик: technology_availability_list().

Содержит классификатор доступности технологий. Список поддерживает просмотр и изменение записей; отдельная форма создает новое значение доступности.

Поля и параметры HTML-формы: per_page, page, sort_by, sort_dir, technology_availability_filter, all_versions, technology_availability_ids[], technology_availability_names[], technology_availability_delete[], js/refdata/textarea_autoresize.js.

Обработка: маршрут получает параметры отображения, при POST читает изменяемые строки и флажки удаления, вызывает сервисы и возвращает пользователя к списку с параметрами просмотра. Для обычного просмотра строится выборка и передается в шаблон вместе с пагинацией и вариантами выбора связанных справочников.

Модель: TechnologyAvailability. Поля SQLAlchemy: id, name, created_at, updated_at, database_version_id, equipment_groups, machines.

Модель данных – фрагмент объявления. Источник: app/refdata/models/refdata_for_stations/technologies/technology_availability_model.py, строки 13–31.

```
  13  class TechnologyAvailability(db.Model, AuditMixin, VersionedModelMixin, RefdataUuidMixin):
  14      __tablename__ = 'gs_sys_technology_availabilities'
  15      __table_args__ = {"schema": SCHEMA_REFDATA}
  16
  17      id = db.Column(db.Integer, primary_key=True, autoincrement=True)
  18      # Сохраняем исходную nullable=True, чтобы не трогать существующие данные.
  19      name = db.Column(db.String(80), unique=True, nullable=True, index=True)
  20
  21      created_at = db.Column(db.DateTime(timezone=True), server_default=func.now(), nullable=False)
  22      updated_at = db.Column(db.DateTime(timezone=True), server_default=func.now(),
onupdate=func.now(), nullable=False)
  23
  24      # Поле для связи с версией БД
  25      database_version_id = db.Column(
  26          db.Integer,
  27          db.ForeignKey(f"{SCHEMA_REFDATA}.gs_database_versions.id", ondelete="SET NULL"),
  28          nullable=True,
  29          index=True
  30      )
  31
```

Связь модели с другой сущностью. Источник: app/refdata/models/refdata_for_stations/technologies/technology_availability_model.py, строки 32–32.

```
    equipment_groups = db.relationship('EquipmentGroupType',
back_populates='technology_availability')
```

Маршрут списка – вход и параметры. Источник: app/refdata/routes/refdata_for_stations/technologies/technology_availability_routes.py, строки 35–50.

```
@refdata_bp.route("/technology_availability", methods=["GET", "POST"])
@login_required
def technology_availability_list():
    """Маршрут для отображения списка доступности технологий."""

    user = session.get('username', 'Неизвестный пользователь')
    log_to_db(
        user,
        "Открыта страница доступности технологий",
        entity_type="technology_availability")

    # Создание формы
    form = TechnologyAvailabilityFilterForm()

    # Получение параметров запроса
    page                = request.args.get("page", 1, type=int)
```

Маршрут списка – передача изменений сервису. Источник: app/refdata/routes/refdata_for_stations/technologies/technology_availability_routes.py, строки 124–129.

```
                               sort_dir=sort_dir,
                    ))
                update_technology_availability_all_versions_service(technology_availability_data,
user)
                flash("Изменения применены во всех версиях БД (по ref_uuid).", "success")
            else:
                update_technology_availability_service(technology_availability_data, user)
```

Сервис списка – выборка и пагинация. Источник: app/refdata/services/refdata_for_stations/technologies/technology_availability_services.py, строки 69–86.

```
@no_autoflush
def get_technology_availability_list(
    page,
    per_page,
    technology_availability_filter=None,
    sort_by="id",
    sort_dir="asc"):
    """ Получает список доступности технологий с пагинацией, фильтрацией и сортировкой. """

    # Базовый запрос
    query = technology_availability_query(
        technology_availability_filter=technology_availability_filter,
        sort_by=sort_by,
        sort_dir=sort_dir,
    )

    # Пагинация
    return query.paginate(page=page, per_page=per_page, error_out=False)
```

Сервис изменения – сохранение и журналирование. Источник: app/refdata/services/refdata_for_stations/technologies/technology_availability_services.py, строки 154–166.

```

    try:
        # Сохранение изменений в базе данных
        # Фиксация транзакции (устойчивый коммит)
        _commit_with_retry()

        if updated_ids:
            log_to_db(
                user,
                "Сохранены изменения по доступности технологий",
                f"Измененных записей: {len(updated_ids)} (id: {updated_ids})",
                entity_type="technology_availability",
                entity_id=technology_availability_id)
```

HTML/Jinja2 списка – элементы формы. Источник: app/templates/refdata/refdata_for_stations/technologies/technology_availability/technology_availability.html, строки 160–173.

```
 160                                <!-- Строки таблицы. Столбец "Доступность технологии" -->
 161                                <td>
 162                                    <input type="text" name="technology_availability_names[]"
class="form-control" value="{{ technology_availability.name }}">
 163                                </td>
 164
 165                                <!-- Строки таблицы. Столбец "Чекмарк для удаления" -->
 166                                <td class="text-center align-middle">
 167                                    <input type="checkbox" name="technology_availability_delete[]"
value="{{ technology_availability.id }}" class="form-check-input">
 168                                </td>
 169                            </tr>
 170                            {% else %}
 171                            <tr>
 172                                <td colspan="3" class="text-center align-middle">Нет данных для
отображения</td>
 173                            </tr>
```

Взаимодействие с другими модулями. EquipmentGroupType.id_technology_availability и Machine.technology_availability связывают признак доступности с типами групп и агрегатами. Признак используется профильными модулями при классификации оборудования и технологий; сам справочник не рассчитывает техническую или экономическую доступность.

Подтверждающие ссылки на потребителей модели: app/generation/models/machine/machine_model.py:115 – technology_availability = db.relationship('TechnologyAvailability', back_populates='machines')

### 4.23.2. Форма добавления

Адрес: GET, POST /refdata/add_technology_availability. Обработчик: add_technology_availability().

Назначение: создание записи справочника «Доступность технологии». Вводимые поля формы: name. Список полей формы может быть уже модели: дополнительные реквизиты изменяются на странице списка или заполняются сервисом.

GET создает форму и подготавливает варианты выбора. POST с валидной формой формирует данные и передает их сервису добавления. При успехе выполняется переход к списку; сообщения о результате выводятся пользователю. Кнопка отмены возвращает к списку без создания записи. Создаваемая сущность – TechnologyAvailability, модель и ее связи приведены выше в подразделе 4.23.1.

Поля WTForms и валидаторы: csrf_token = HiddenField(); name = StringField('Технология', validators=[DataRequired(message='Поле «Технология» обязательно.'), Length(min=2, max=255, message='Длина от 2 до 255 символов.')]).

Шаблон содержит кнопку добавления во всех версиях для административного пользователя. Серверная ветвь all_versions=1 вызывает специализированную проверку доступа и межверсионный сервис; при обычном добавлении используется выбранная версия.

Модель добавляемого объекта TechnologyAvailability – основной реквизит. Источник: app/refdata/models/refdata_for_stations/technologies/technology_availability_model.py, строки 19–19.

```
  19      name = db.Column(db.String(80), unique=True, nullable=True, index=True)
```

Маршрут добавления – вход в форму. Источник: app/refdata/routes/refdata_for_stations/technologies/technology_availability_routes.py, строки 164–176.

```
@refdata_bp.route("/add_technology_availability", methods=["GET", "POST"])
@login_required
def add_technology_availability():
    """ Маршрут для добавления новой доступности технологии. """

    user = session.get('username', 'Неизвестный пользователь')
    log_to_db(
        user,
        "Открыта страница добавления доступности технологий",
        entity_type="technology_availability")

    # Создание формы
    form = AddTechnologyAvailabilityForm()
```

Маршрут добавления – вызов сервиса. Источник: app/refdata/routes/refdata_for_stations/technologies/technology_availability_routes.py, строки 210–220.

```
                            sort_dir=sort_dir,
                        ))
                add_technology_availability_all_versions_service(payload, user)
                flash("Новая запись добавлена во всех версиях БД (общий ref_uuid).", "success")
            else:
                add_technology_availability_service(payload, user)
                flash("Новая запись успешно добавлена.", "success")

            # Перенаправление на список с сохранением параметров и переходом к новой записи
            total_records = technology_availability_query(
                                technology_availability_filter).count()
```

Сервис добавления – создание сущности. Источник: app/refdata/services/refdata_for_stations/technologies/technology_availability_services.py, строки 213–226.

```
                if dup:
                    raise ValueError(f"Запись с наименованием «{name}» уже существует.")

                obj = TechnologyAvailability(name=name)
                set_db_version_on_create(obj)
                db.session.add(obj)
                db.session.flush()

                log_to_db(
                    user,
                    "Создана доступность технологии",
                    f"Наименование: {name}",
                    entity_type="technology_availability",
                    entity_id=obj.id)
```

HTML/Jinja2 формы добавления. Источник: app/templates/refdata/refdata_for_stations/technologies/technology_availability/technology_availability_add.html, строки 20–34.

```
        <div class="mb-3">
            <label for="name" class="form-label">Доступность технологии</label>
            {{ form.name(class="form-control", id="name") }}
            {% if form.name.errors %}
                <div class="text-danger">{{ form.name.errors|join(', ') }}</div>
            {% endif %}
        </div>

        <div class="mt-4 d-flex flex-wrap gap-2 align-items-center">
            <button type="submit" class="btn btn-primary">Добавить</button>
            {% if current_user.has_admin %}<button type="submit" name="all_versions" value="1"
class="btn btn-outline-primary" title="Добавить запись во все версии БД">Добавить во всех
версиях</button>{% endif %}
            <a href="{{ url_for('refdata_bp.technology_availability_list',
                                page=page,
                                per_page=per_page,
                                sort_by=sort_by,
```

Взаимодействие при добавлении. EquipmentGroupType.id_technology_availability и Machine.technology_availability связывают признак доступности с типами групп и агрегатами. Признак

используется профильными модулями при классификации оборудования и технологий; сам справочник не рассчитывает техническую или экономическую доступность. После создания запись становится доступна для выбора в зависимых формах при их следующем построении с учетом версии и используемых механизмов кэширования.

4.23.3. Экспорт, импорт и дополнительные операции

| Адрес и метод | Обработчик | Результат |
|---|---|---|
| GET /refdata/export_technology_availability | export_technology_availability | Файл Excel для скачивания |

Отдельный маршрут импорта в этом справочнике не зарегистрирован. Поддержка импорта не приписывается всем справочникам по аналогии.

## 4.24. Типы затрат на производство ЭЭ и ТЭ

4.24.1. Страница списка

Адрес: GET, POST /refdata/electricity_production_cost_type. Обработчик: electricity_production_cost_type_list().

Ведет код cost_code и наименование вида затрат на производство электрической и тепловой энергии. Список позволяет изменять классификацию, отдельная форма добавляет запись; предусмотрен экспорт в Excel.

Поля и параметры HTML-формы: per_page, page, sort_by, sort_dir, electricity_production_cost_type_filter, all_versions, electricity_production_cost_type_ids[], cost_codes[], electricity_production_cost_type_names[], electricity_production_cost_type_delete[], js/refdata/textarea_autoresize.js.

Обработка: маршрут получает параметры отображения, при POST читает изменяемые строки и флажки удаления, вызывает сервисы и возвращает пользователя к списку с параметрами просмотра. Для обычного просмотра строится выборка и передается в шаблон вместе с пагинацией и вариантами выбора связанных справочников.

Модель: ElectricityProductionCostType. Поля SQLAlchemy: id, cost_code, name, created_at, updated_at, database_version_id.

Модель данных – фрагмент объявления. Источник: app/refdata/models/electricity_production_cost/electricity_production_cost_type_model.py, строки 14–37.

```
class ElectricityProductionCostType(db.Model, AuditMixin, VersionedModelMixin, RefdataUuidMixin):
    __tablename__ = 'gs_sys_electricity_production_cost_types'
    __table_args__ = (
        db.UniqueConstraint(
            'database_version_id',
            'name',
            name='uq_gs_sys_electricity_production_cost_types_ver_name',
        ),
        {"schema": SCHEMA_REFDATA},
    )

    id = db.Column(db.Integer, primary_key=True, autoincrement=True)
    cost_code = db.Column(db.Integer, nullable=True)
    name = db.Column(db.String(255), nullable=True, index=True)

    created_at = db.Column(db.DateTime(timezone=True), server_default=func.now(), nullable=False)
    updated_at = db.Column(db.DateTime(timezone=True), server_default=func.now(),
onupdate=func.now(), nullable=False)

    database_version_id = db.Column(
        db.Integer,
        db.ForeignKey(f"{SCHEMA_REFDATA}.gs_database_versions.id", ondelete="SET NULL"),
        nullable=True,
        index=True
    )
```

Маршрут списка – вход и параметры. Источник: app/refdata/routes/electricity_production_cost/electricity_production_cost_type_routes.py, строки 31–46.

```
@refdata_bp.route("/electricity_production_cost_type", methods=["GET", "POST"])
@login_required
def electricity_production_cost_type_list():
    user = session.get('username', 'Неизвестный пользователь')
    log_to_db(
        user,
        "Открыта страница затрат на производство и реализацию электрической энергии (мощности)",
        entity_type="electricity_production_cost_type",
    )

    form = ElectricityProductionCostTypeFilterForm()

    page = request.args.get("page", 1, type=int)
    per_page = request.args.get("per_page", 25, type=int)
    sort_by = request.args.get("sort_by", "cost_code")
    sort_dir = request.args.get("sort_dir", "asc")
```

Маршрут списка – передача изменений сервису. Источник: app/refdata/routes/electricity_production_cost/electricity_production_cost_type_routes.py, строки 132–137.

```
                           sort_dir=sort_dir,
                       ))
                update_electricity_production_cost_type_all_versions_service(record_data, user)
                flash("Изменения применены во всех версиях БД (по ref_uuid).", "success")
            else:
                update_electricity_production_cost_type_service(record_data, user)
```

Сервис списка – выборка и пагинация. Источник: app/refdata/services/electricity_production_cost/electricity_production_cost_type_services.py, строки 78–91.

```
@no_autoflush
def get_electricity_production_cost_type_list(
    page,
    per_page,
    electricity_production_cost_type_filter=None,
    sort_by="cost_code",
    sort_dir="asc",
):
    query = electricity_production_cost_type_query(
        electricity_production_cost_type_filter=electricity_production_cost_type_filter,
        sort_by=sort_by,
        sort_dir=sort_dir,
    )
    return query.paginate(page=page, per_page=per_page, error_out=False)
```

Сервис изменения – сохранение и журналирование. Источник: app/refdata/services/electricity_production_cost/electricity_production_cost_type_services.py, строки 221–234.

```
221
222         db.session.flush()
223
224     try:
225         _commit_with_retry()
226
227         if updated_ids:
228             log_to_db(
229                 user,
230                 "Сохранены изменения по затратам на производство ЭЭ (мощности)",
231                 f"Измененных записей: {len(updated_ids)} (id: {updated_ids})",
232                 entity_type="electricity_production_cost_type",
233                 entity_id=record_id,
234             )
```

HTML/Jinja2 списка – элементы формы. Источник: app/templates/refdata/electricity_production_cost/electricity_production_cost_type/electricity_production_cost_type.html, строки 165–178.

```
165
166                             <td>
167                                 <input type="text" name="electricity_production_cost_type_names[]"
class="form-control" value="{{ item.name }}">
168                             </td>
169
170                             <td class="text-center align-middle">
171                                 <input type="checkbox"
name="electricity_production_cost_type_delete[]" value="{{ item.id }}" class="form-check-input">
172                             </td>
173                         </tr>
174                         {% else %}
175                         <tr>
176                             <td colspan="4" class="text-center align-middle">Нет данных для
отображения</td>
177                         </tr>
178                         {% endfor %}
```

Взаимодействие с другими модулями. Классификатор используется моделями технико-экономических показателей затрат и профильными сервисами. Он задает вид показателя; денежные значения, временная привязка и расчетные коэффициенты хранятся отдельно. Прямые ссылки на потребителей приведены в этом разделе.

Подтверждающие ссылки на потребителей модели:
app/fuel/models/fue_equipment_group_electricity_production_cost_model.py:16 – code_zatr хранится в cost_code; наименование подгружается из ElectricityProductionCostType.
app/fuel/services/equipment_groups/equipment_group_electricity_production_cost_services.py:24 – ElectricityProductionCostType,

### 4.24.2. Форма добавления

Адрес: GET, POST /refdata/add_electricity_production_cost_type. Обработчик: add_electricity_production_cost_type().

Назначение: создание записи справочника «Типы затрат на производство ЭЭ и ТЭ». Вводимые поля формы: name, cost_code. Список полей формы может быть уже модели: дополнительные реквизиты изменяются на странице списка или заполняются сервисом.

GET создает форму и подготавливает варианты выбора. POST с валидной формой формирует данные и передает их сервису добавления. При успехе выполняется переход к списку; сообщения о результате выводятся пользователю. Кнопка отмены возвращает к списку без создания записи. Создаваемая сущность – ElectricityProductionCostType, модель и ее связи приведены выше в подразделе 4.24.1.

Поля WTForms и валидаторы: csrf_token = HiddenField(); name = StringField('Наименование', validators=[DataRequired(message='Поле «Наименование» обязательно.'), Length(min=2, max=255, message='Длина от 2 до 255 символов.')]); cost_code = IntegerField('Код затрат', validators=[Optional(), NumberRange(min=0)]).

Шаблон содержит кнопку добавления во всех версиях для административного пользователя. Серверная ветвь all_versions=1 вызывает специализированную проверку доступа и межверсионный сервис; при обычном добавлении используется выбранная версия.

Модель добавляемого объекта ElectricityProductionCostType – основной реквизит. Источник: app/refdata/models/electricity_production_cost/electricity_production_cost_type_model.py, строки 27–27.

```
27      name = db.Column(db.String(255), nullable=True, index=True)
```

Маршрут добавления – вход в форму. Источник: app/refdata/routes/electricity_production_cost/electricity_production_cost_type_routes.py, строки 174–186.

```
174  @refdata_bp.route("/add_electricity_production_cost_type", methods=["GET", "POST"])
175  @login_required
176  def add_electricity_production_cost_type():
177      user = session.get('username', 'Неизвестный пользователь')
178      log_to_db(
179          user,
180          "Открыта страница добавления затрат на производство ЭЭ (мощности)",
181          entity_type="electricity_production_cost_type",
182      )
183
184      form = AddElectricityProductionCostTypeForm()
185
186      page = request.args.get("page", 1, type=int)
```

Маршрут добавления – вызов сервиса. Источник: app/refdata/routes/electricity_production_cost/electricity_production_cost_type_routes.py, строки 227–237.

```
227                          sort_dir=sort_dir,
228                      ))
229                  add_electricity_production_cost_type_all_versions_service(payload, user)
230                  flash("Новая запись добавлена во всех версиях БД (общий ref_uuid).", "success")
231              else:
232                  add_electricity_production_cost_type_service(payload, user)
233                  flash("Новая запись успешно добавлена.", "success")
234
235              total_records = electricity_production_cost_type_query(
236                  electricity_production_cost_type_filter=electricity_production_cost_type_filter,
237              ).count()
```

Сервис добавления – создание сущности. Источник: app/refdata/services/electricity_production_cost/electricity_production_cost_type_services.py, строки 331–345.

```
331                              f"Код затрат «{cost_code}» уже задан у записи «{other}»
(id={dup_code.id})."
332                          )
333
334                  obj = ElectricityProductionCostType(name=name, cost_code=cost_code)
335                  set_db_version_on_create(obj)
336                  db.session.add(obj)
337                  db.session.flush()
338
339                  log_to_db(
340                      user,
341                      "Создана запись затрат на производство ЭЭ (мощности)",
342                      f"Наименование: {name}",
343                      entity_type="electricity_production_cost_type",
344                      entity_id=obj.id,
345                  )
```

HTML/Jinja2 формы добавления. Источник: app/templates/refdata/electricity_production_cost/electricity_production_cost_type/electricity_production_cost_type_add.html, строки 19–33.

```
  19          <div class="mb-3">
  20              <label for="name" class="form-label">Наименование</label>
  21              {{ form.name(class="form-control", id="name") }}
  22              {% if form.name.errors %}
  23                  <div class="text-danger">{{ form.name.errors|join(', ') }}</div>
  24              {% endif %}
  25          </div>
  26
  27          <div class="mb-3">
  28              <label for="cost_code" class="form-label">Код затрат</label>
  29              {{ form.cost_code(class="form-control", id="cost_code", type="number", min="0",
placeholder="") }}
  30              {% if form.cost_code.errors %}
  31                  <div class="text-danger">{{ form.cost_code.errors|join(', ') }}</div>
  32              {% endif %}
  33          </div>
```

Взаимодействие при добавлении. Классификатор используется моделями технико-экономических показателей затрат и профильными сервисами. Он задает вид показателя; денежные значения, временная привязка и расчетные коэффициенты хранятся отдельно. Прямые ссылки на потребителей приведены в этом разделе. После создания запись становится доступна для выбора в зависимых формах при их следующем построении с учетом версии и используемых механизмов кэширования.

4.24.3. Экспорт, импорт и дополнительные операции

| Адрес и метод | Обработчик | Результат |
|---|---|---|
| GET /refdata/export_electricity_production_cost_type | export_electricity_production_cost_type | Файл Excel для скачивания |

Отдельный маршрут импорта в этом справочнике не зарегистрирован. Поддержка импорта не приписывается всем справочникам по аналогии.

## 4.25. Виды экономической деятельности (ВЭД)

4.25.1. Страница списка

Адрес: GET, POST /refdata/ved. Обработчик: economic_activity_type_list().

Ведет наименование ВЭД, дополнительное наименование name_2 и порядок отображения. Адрес страницы – /refdata/ved, форма создания – /refdata/add_ved. Список поддерживает редактирование, фильтрацию и экспорт.

Поля и параметры HTML-формы: per_page, page, sort_by, sort_dir, economic_activity_type_filter, all_versions, economic_activity_ids[], display_orders[], economic_activity_type_names[], economic_activity_type_names_2[], economic_activity_type_delete[], js/refdata/textarea_autoresize.js.

Обработка: маршрут получает параметры отображения, при POST читает изменяемые строки и флажки удаления, вызывает сервисы и возвращает пользователя к списку с параметрами просмотра. Для обычного просмотра строится выборка и передается в шаблон вместе с пагинацией и вариантами выбора связанных справочников.

Модель: EconomicActivityType. Поля SQLAlchemy: id, display_order, name, name_2, created_at, updated_at, database_version_id.

Модель данных – фрагмент объявления. Источник: app/refdata/models/economic_activity/economic_activity_type_model.py, строки 13–31.

```
  13  class EconomicActivityType(db.Model, AuditMixin, VersionedModelMixin, RefdataUuidMixin):
  14      __tablename__ = 'gs_sys_economic_activity_types'
  15      __table_args__ = (
  16          db.UniqueConstraint(
  17              'database_version_id',
  18              'name',
  19              name='uq_gs_sys_economic_activity_types_ver_name',
  20          ),
  21          {"schema": SCHEMA_REFDATA},
  22      )
  23
  24      id = db.Column(db.Integer, primary_key=True, autoincrement=True)
  25
  26      # Порядок отображения
  27      display_order = db.Column(db.Integer, nullable=True)
  28
  29      # Полное наименование (страницы «Экономики»)
  30      name = db.Column(db.String(255), nullable=True, index=True)
  31
```

Маршрут списка – вход и параметры. Источник: app/refdata/routes/economic_activity/economic_activity_type_routes.py, строки 35–50.

```
  35  @refdata_bp.route("/ved", methods=["GET", "POST"])
  36  @login_required
  37  def economic_activity_type_list():
  38      """Маршрут для отображения списка видов экономической деятельности."""
  39
  40      user = session.get('username', 'Неизвестный пользователь')
  41      log_to_db(
  42          user,
  43          "Открыта страница видов экономической деятельности",
  44          entity_type="economic_activity_type")
  45
  46      # Создание формы
  47      form = EconomicActivityTypeFilterForm()
  48
  49      # Получение параметров запроса
  50      page                = request.args.get("page", 1, type=int)
```

Маршрут списка – передача изменений сервису. Источник: app/refdata/routes/economic_activity/economic_activity_type_routes.py, строки 166–171.

```
 166                              sort_dir=sort_dir,
 167                          ))
 168                  update_economic_activity_type_all_versions_service(economic_activity_type_data,
user)
 169                  flash("Изменения применены во всех версиях БД (по ref_uuid).", "success")
 170              else:
 171                  update_economic_activity_type_service(economic_activity_type_data, user)
```

Сервис списка – выборка и пагинация. Источник: app/refdata/services/economic_activity/economic_activity_type_services.py, строки 111–128.

```
111  @no_autoflush
112  def get_economic_activity_type_list(
113      page,
114      per_page,
115      economic_activity_type_filter=None,
116      sort_by="display_order",
117      sort_dir="asc"):
118      """ Получает список видов экономической деятельности с пагинацией, фильтрацией и сортировкой.
"""
119
120      # Базовый запрос
121      query = economic_activity_type_query(
122          economic_activity_type_filter=economic_activity_type_filter,
123          sort_by=sort_by,
124          sort_dir=sort_dir,
125      )
126
127      # Пагинация
128      return query.paginate(page=page, per_page=per_page, error_out=False)
```

Сервис изменения – сохранение и журналирование. Источник: app/refdata/services/economic_activity/economic_activity_type_services.py, строки 273–285.

```
273
274      try:
275          # Сохранение изменений в базе данных
276          # Фиксация транзакции (устойчивый коммит)
277          _commit_with_retry()
278
279          if updated_ids:
280              log_to_db(
281                  user,
282                  "Сохранены изменения по видам экономической деятельности",
283                  f"Измененных записей: {len(updated_ids)} (id: {updated_ids})",
284                  entity_type="economic_activity_type",
285                  entity_id=economic_activity_type_id)
```

HTML/Jinja2 списка – элементы формы. Источник: app/templates/refdata/economic_activity/economic_activity_type/economic_activity_type.html, строки 214–227.

```
214                                 <!-- Строки таблицы. Столбец "Вид экономической деятельности" -->
215                                 <td>
216                                     <input type="text" name="economic_activity_type_names[]"
class="form-control" value="{{ economic_activity_type.name }}">
217                                 </td>
218
219                                 <td>
220                                     <input type="text" name="economic_activity_type_names_2[]"
class="form-control" value="{{ economic_activity_type.name_2 or '' }}">
221                                 </td>
222
223                                 <!-- Строки таблицы. Столбец "Чекмарк для удаления" -->
224                                 <td class="text-center align-middle">
225                                     <input type="checkbox" name="economic_activity_type_delete[]"
value="{{ economic_activity_type.id }}" class="form-check-input">
226                                 </td>
227                             </tr>
```

Взаимодействие с другими модулями. ВЭД используются экономическими моделями выпуска продукции, накопления основного капитала и электропотребления, а также расчетами электроемкости. Классификатор связывает отраслевые строки разных модулей по идентификатору; изменение названия не выполняет перерасчет экономических показателей автоматически.

Подтверждающие ссылки на потребителей модели:
app/economics/models/federal_district_accum_fixed_capital_parameter_model.py:46 – "EconomicActivityType",
app/electrical_intensity/models/federal_district_electrical_intensity_coefficient_model.py:35 – "EconomicActivityType",
app/economics/models/federal_district_eat_consumption_parameter_model.py:46 – "EconomicActivityType",
app/economics/models/federal_district_product_output_parameter_model.py:46 – "EconomicActivityType",

app/economics/models/russia_federation_accum_fixed_capital_parameter_model.py:35 – "EconomicActivityType",

### 4.25.2. Форма добавления

Адрес: GET, POST /refdata/add_ved. Обработчик: add_economic_activity_type().

Назначение: создание записи справочника «Виды экономической деятельности (ВЭД)». Вводимые поля формы: name, name_2, display_order. Список полей формы может быть уже модели: дополнительные реквизиты изменяются на странице списка или заполняются сервисом.

GET создает форму и подготавливает варианты выбора. POST с валидной формой формирует данные и передает их сервису добавления. При успехе выполняется переход к списку; сообщения о результате выводятся пользователю. Кнопка отмены возвращает к списку без создания записи. Создаваемая сущность – EconomicActivityType, модель и ее связи приведены выше в подразделе 4.25.1.

Поля WTForms и валидаторы: csrf_token = HiddenField(); name = StringField('Полное наименование', validators=[DataRequired(message='Поле «Полное наименование» обязательно.'), Length(min=2, max=255, message='Длина от 2 до 255 символов.')]); name_2 = StringField('Краткое наименование', validators=[Optional(), Length(max=255, message='Длина не более 255 символов.')]); display_order = IntegerField('Порядок отображения', validators=[Optional(), NumberRange(min=0)]).

Шаблон содержит кнопку добавления во всех версиях для административного пользователя. Серверная ветвь all_versions=1 вызывает специализированную проверку доступа и межверсионный сервис; при обычном добавлении используется выбранная версия.

Модель добавляемого объекта EconomicActivityType – основной реквизит. Источник: app/refdata/models/economic_activity/economic_activity_type_model.py, строки 30–30.

```
30      name = db.Column(db.String(255), nullable=True, index=True)
```

Маршрут добавления – вход в форму. Источник: app/refdata/routes/economic_activity/economic_activity_type_routes.py, строки 206–218.

```
206 @refdata_bp.route("/add_ved", methods=["GET", "POST"])
207 @login_required
208 def add_economic_activity_type():
209     """ Маршрут для добавления нового вида экономической деятельности. """
210
211     user = session.get('username', 'Неизвестный пользователь')
212     log_to_db(
213         user,
214         "Открыта страница добавления видов экономической деятельности",
215         entity_type="economic_activity_type")
216
217     # Создание формы
218     form = AddEconomicActivityTypeForm()
```

Маршрут добавления – вызов сервиса. Источник: app/refdata/routes/economic_activity/economic_activity_type_routes.py, строки 260–270.

```
260                                 sort_dir=sort_dir,
261                     ))
262                 add_economic_activity_type_all_versions_service(payload, user)
263                 flash("Новая запись добавлена во всех версиях БД (общий ref_uuid).", "success")
264             else:
265                 add_economic_activity_type_service(payload, user)
266                 flash("Новая запись успешно добавлена.", "success")
267
268             # Перенаправление на список с сохранением параметров и переходом к новой записи
269             total_records = economic_activity_type_query(
270
economic_activity_type_filter=economic_activity_type_filter).count()
```

Сервис добавления – создание сущности. Источник:
app/refdata/services/economic_activity/economic_activity_type_services.py, строки 368–381.

```
                            f"Порядок «{display_order}» уже задан у записи «{other}»
(id={dup_order.id})."
                        )

                obj = EconomicActivityType(name=name, name_2=name_2, display_order=display_order)
                set_db_version_on_create(obj)
                db.session.add(obj)
                db.session.flush()

                log_to_db(
                    user,
                    "Создан вид экономической деятельности",
                    f"Наименование: {name}",
                    entity_type="economic_activity_type",
                    entity_id=obj.id)
```

HTML/Jinja2 формы добавления. Источник:
app/templates/refdata/economic_activity/economic_activity_type/economic_activity_type_add.html, строки 20–34.

```
        <div class="mb-3">
            <label for="name" class="form-label">Полное наименование</label>
            <div class="form-text">Отображается на страницах «Экономики».</div>
            {{ form.name(class="form-control", id="name") }}
            {% if form.name.errors %}
                <div class="text-danger">{{ form.name.errors|join(', ') }}</div>
            {% endif %}
        </div>

        <div class="mb-3">
            <label for="name_2" class="form-label">Краткое наименование</label>
            <div class="form-text">Отображается на страницах «Электроемкости».</div>
            {{ form.name_2(class="form-control", id="name_2") }}
            {% if form.name_2.errors %}
                <div class="text-danger">{{ form.name_2.errors|join(', ') }}</div>
```

Взаимодействие при добавлении. ВЭД используются экономическими моделями выпуска продукции, накопления основного капитала и электропотребления, а также расчетами электроемкости. Классификатор связывает отраслевые строки разных модулей по идентификатору; изменение названия не выполняет перерасчет экономических показателей автоматически. После создания запись становится доступна для выбора в зависимых формах при их следующем построении с учетом версии и используемых механизмов кэширования.

### 4.25.3. Экспорт, импорт и дополнительные операции

| Адрес и метод | Обработчик | Результат |
|---|---|---|
| GET /refdata/export_ved | export_economic_activity_type | Файл Excel для скачивания |

Отдельный маршрут импорта в этом справочнике не зарегистрирован. Поддержка импорта не приписывается всем справочникам по аналогии.

**5. Взаимодействие с другими модулями**

Обмен внутри АРМ преимущественно осуществляется через общие модели и внешние ключи в одной БД, а не через переходы пользователя между страницами. Маршруты справочников вызывают собственные сервисы; расчетные модули читают соответствующие сущности напрямую либо через общие get-сервисы. REST-адреса обмена упоминаются только как подтвержденные потребители моделей, а не как самостоятельные страницы справочников.

| Модуль/подсистема | Получаемые данные и логика связи |
|---|---|
| Генерирующие мощности | Территории, энергосистемы, компании, типы станций/агрегатов/ТЭС/ПГУ, состояния, технологии, топливо и годы. Station, Machine, PGUMachine и годовые модели хранят ссылки на классификаторы. |
| Топливо | Единые Fuel/FuelType, типы групп оборудования и затрат, компании, территории и годы. Классификатор типа группы отделен от экземпляров топливных групп и их расчетных параметров. |
| Экономика | Федеральные округа, ВЭД и годы для отраслевых и территориальных показателей. Используются идентификаторы ВЭД и версионные/годовые ключи. |
| Электропотребление и электроемкость | Территориально-энергетическая иерархия, ВЭД и временная ось. Изменение привязки субъекта к энергозоне сопровождается синхронизацией агрегатов потребления через сервис. |
| Потребность в мощности | Уровни ФО, субъект РФ, ОЭС, РЭС, энергорайон, энергоузел, тип энергосистемы, синхронная зона и энергозона. Модели параметров связывают показатели с соответствующим уровнем справочника. |
| Энергетические балансы | Энергосистемы, территории, страны, типы станций и годы. Используются для группировки генерации, межсистемных передач и импортно-экспортных значений. |
| Экология | Территории, энергосистемы, оборудование, топливо и годы для сопоставления исходных данных и построения расчетных представлений. Расчет выбросов не реализуется в |

| Модуль/подсистема | Получаемые данные и логика связи |
|---|---|
| | страницах справочников. |
| Рациональная структура | Типы технологий и групп оборудования для территориально-технологических моделей и сопоставления оборудования. |
| Версии БД | Выбор контекста database_version_id; межверсионная идентичность ref_uuid и разрешение ссылок на записи соответствующей версии. |
| Авторизация, журнал и кэши | flask_login/current_user и декораторы доступа; log_to_db для событий; варианты выбора и справочные get-сервисы для зависимых форм. |

### 5.1. Пример сквозного взаимодействия: субъект РФ → энергозона → потребление

Пользователь изменяет энергозону субъекта в списке. POST передает energy_zone_id в сервис субъектов РФ. После изменения принадлежности сервис собирает затронутые зоны, обновляет состояние ORM-связи regional_districts и вызывает sync_energy_zone_consumption_aggregates. В результате данные территориальной привязки используются модулем электропотребления без повторного ручного ввода классификатора.

Вызов синхронизации модуля электропотребления. Источник: app/refdata/services/territories/regional_district_services.py, строки 48–69.

```
48  def _sync_energy_zone_aggregates_for_ids(
49      energy_zone_ids,
50      *,
51      database_version_id=None,
52  ):
53      """Пересчет агрегатов потребления ЭЗ после смены привязки субъектов."""
54      ids = {int(ez_id) for ez_id in energy_zone_ids if ez_id is not None}
55      if not ids:
56          return
57
58      from app.energy_consumption.services.energy_consumption_parameter_services import (
59          sync_energy_zone_consumption_aggregates,
60      )
61
62      for ez_id in sorted(ids):
63          ez = db.session.get(EnergyZone, ez_id)
64          if ez is not None:
65              db.session.expire(ez, ["regional_districts"])
66          sync_energy_zone_consumption_aggregates(
67              database_version_id=database_version_id,
68              id_energy_zone=ez_id,
69          )
```

### 5.2. Пример сквозного взаимодействия: тип группы → технология → оборудование

Тип группы оборудования связан с TechnologyType и TechnologyAvailability. Модель Machine содержит ссылку на тип группы, а топливные модели используют тот же классификатор в собственных связях. Изменение классификационной записи влияет на последующее чтение зависимых объектов; перенос связей при объединении дубликатов реализуется специализированными сервисами. Это не равнозначно изменению индивидуальных расчетных параметров всех групп.

Объединение дублирующихся типов групп – фрагмент. Источник: app/refdata/services/refdata_for_stations/technologies/equipment_group_services.py, строки 84–109.

```
  84  def _merge_equipment_group_type_into_canonical(*, from_id: int, to_id: int, user) -> None:
  85      """
  86      Переносит все ссылки с типа группы оборудования from_id на to_id и удаляет строку from_id.
  87      Используется при сохранении, когда редактируемая запись дублирует пару (name,
id_technology_type).
  88      """
  89      if from_id == to_id:
  90          return
  91
  92      from_obj = db.session.get(EquipmentGroupType, from_id)
  93      to_obj = db.session.get(EquipmentGroupType, to_id)
  94      if not from_obj or not to_obj:
  95          raise ValueError(f"Слияние типов групп: запись from_id={from_id} или to_id={to_id} не
найдена.")
  96
  97      Machine.query.filter(Machine.id_equipment_group == from_id).update(
  98          {Machine.id_equipment_group: to_id},
  99          synchronize_session=False,
 100      )
 101
 102      # По одной строке + flush: при нескольких записях с (station, from_id, NULL) в UNIQUE
 103      # подзапрос «уже есть to_id» не видит только что обновленные строки до flush.
 104      while True:
 105          old_link = (
 106              EquipmentGroupSetStation.query.filter_by(
 107                  equipment_group_type_id=from_id
 108              ).first()
 109          )
```

## 5.3. Граница модуля: управление версиями БД

Файл app/refdata/routes/database_versions_routes.py физически расположен рядом со справочниками, но регистрирует страницы на station_bp, в пространстве адресов /generation/. Ссылка «Версии» относится к общей подсистеме версий АРМ. Ее страницы не включены в перечень HTML-страниц /refdata/: в настоящем документе описана именно логика взаимодействия с ними.

**6. Полнота материалов и перечень страниц**

Описаны 51 самостоятельная HTML-страница: главная страница плюс 25 списков и 25 форм добавления. Сортировка, поиск и пагинация не создают новых страниц по назначению: это состояния соответствующего списка. Модальное окно иерархии топлива описано отдельно внутри справочника. Экспорт, импорт и JSON API перечислены как служебные операции.

| № раздела | Справочник | Список | Добавление |
|---|---|---|---|
| 4.1 | Годы | /refdata/years | /refdata/add_year |
| 4.2 | Федеральные округа | /refdata/federal_district | /refdata/add_federal_district |
| 4.3 | Субъекты РФ | /refdata/regional_district | /refdata/add_regional_district |
| 4.4 | Зарубежные страны, имеющие общие границы с РФ | /refdata/foreign_border_country | /refdata/add_foreign_border_country |
| 4.5 | Объединенные энергосистемы | /refdata/union_energy_system | /refdata/add_union_energy_system |
| 4.6 | Региональные энергосистемы | /refdata/regional_energy_system | /refdata/add_regional_energy_system |
| 4.7 | Энергорайоны | /refdata/energy_units | /refdata/add_energy_unit |
| 4.8 | Энергоузлы | /refdata/energy_areas | /refdata/add_energy_area |
| 4.9 | Типы энергосистем | /refdata/energy_system_type | /refdata/add_energy_system_type |
| 4.10 | Синхронные зоны | /refdata/synchronous_area | /refdata/add_synchronous_area |
| 4.11 | Энергозоны | /refdata/energy_zone | /refdata/add_energy_zone |
| 4.12 | Генерирующие компании | /refdata/gen_company | /refdata/add_gen_company |
| 4.13 | Типы топлива | /refdata/fuel_type | /refdata/add_fuel_type |
| 4.14 | Виды топлива | /refdata/fuel | /refdata/add_fuel |
| 4.15 | Типы электроста | /refdata/station_type | /refdata/add_station_type |

| № раздела | Справочник | Список | Добавление |
|---|---|---|---|
| | нций | | |
| 4.16 | Типы состояния оборудования | /refdata/condition_type | /refdata/add_condition_type |
| 4.17 | Типы агрегатов | /refdata/machine_type | /refdata/add_machine_type |
| 4.18 | Типы ТЭС | /refdata/tes_type | /refdata/add_tes_type |
| 4.19 | Типы агрегатов ТЭС | /refdata/tes_machine_type | /refdata/add_tes_machine_type |
| 4.20 | Типы агрегатов ПГУ | /refdata/pgu_tes_machine_type | /refdata/add_pgu_tes_machine_type |
| 4.21 | Типы групп оборудования | /refdata/equipment_group | /refdata/add_equipment_group |
| 4.22 | Типы технологий | /refdata/technology_type | /refdata/add_technology_type |
| 4.23 | Доступность технологии | /refdata/technology_availability | /refdata/add_technology_availability |
| 4.24 | Типы затрат на производство ЭЭ и ТЭ | /refdata/electricity_production_cost_type | /refdata/add_electricity_production_cost_type |
| 4.25 | Виды экономической деятельности (ВЭД) | /refdata/ved | /refdata/add_ved |

Вне перечня самостоятельных страниц: модели организаций business_unit, department, economic_region, признаки годов YearFeature, параметры YearService, коэффициенты пересчета цен и другие вспомогательные сущности, не имеющие собственной страницы в навигации /refdata. Их наличие в каталоге моделей не является основанием приписывать модулю дополнительные веб-страницы.

**6.1. Сведения об источниках и проверке**

Шаблон заявки: «Сведения для регистрации программы.doc», предоставлен пользователем. Структура 12 пунктов сохранена в разделе 1; отдельные листинги и экранные формы размещены в последующих разделах, как допускает шаблон.

Снимки экрана получены с действующего сайта 29.10.2026. Для печатного документа общий навигационный заголовок с именем учетной записи исключен кадрированием; длинные таблицы представлены верхним фрагментом.

Изображения не являются макетами и не заменяют описание полей, скрытых горизонтальной или вертикальной прокруткой.

Функциональные возможности и листинги сверены со структурой маршрутов, моделей, форм, сервисов и шаблонов текущей рабочей копии. Просмотр сайта выполнялся без отправки форм сохранения, добавления, удаления или импорта. Исполнение всех изменяющих операций на рабочей БД не проверялось и в рамках подготовки регистрационного описания не требовалось.

Документ является описанием двух модулей АРМ: «Справочники» и «Экология» и не заменяет окончательную заявку на регистрацию всей программы. До подачи заявитель подтверждает официальное наименование программы, авторский состав и вклад, контакты, дату завершения, объем заявляемой поставки и сведения об операционной системе.

## 7. Модуль «Экология»: назначение и архитектура

Модуль предназначен для хранения и просмотра отчетных данных о парниковых газах и загрязняющих веществах, выверки их относительно топливно-энергетических показателей, расчета выбросов действующих и новых ТЭС и подготовки сводных таблиц. Основной адрес: http://msk-arm-gs01.ntcees.ru/ecology/. В этом дополнении описывается текущая рабочая копия исходников; совпадение ее сборки с действующим сервером не установлено.

Различаются два контура. Отчетно-аналитический контур использует сведения ГИС ТЭК, выверенные параметры анализа и агрегационные суммы. Расчетный контур использует принятые топливные снимки, характеристики, коэффициенты и нормативы для действующих и новых ТЭС. Итоговые суммы анализа и сводные расчетные выбросы имеют разное происхождение и не заменяют друг друга.

### 7.1. Страницы, вкладки и границы описания

Описаны 30 функциональных экранных представлений: главная страница, 2 отчета, 4 листа анализа, 10 вкладок действующих ТЭС, 3 вкладки новых ТЭС, 4 сводные вкладки, 4 агрегированных листа, редактор подсказок и рассмотрение топливного пакета. Дополнительно перечислены 4 скрытые совместимые вкладки сводного расчета. Это не 30 отдельных HTML-шаблонов: используются 7 полноценных шаблонов с общими включаемыми фрагментами. Территория, год, мощностная категория, фильтр и пагинация являются параметрами представления. Все 34 объявления маршрутов, включая служебные операции и варианты адресов со слешем, перечислены в разделе 17.

На каждой странице приведены назначение, доступные действия, источники данных, связи с другими модулями и фрагменты реализации с точными путями и строками. Для страниц без отдельной модели указано отсутствие собственного хранилища. Фрагменты являются выборочным листингом, а не полным исходным текстом модуля.

### 7.2. Программные слои и модели данных

Flask Blueprint ecology_bp связывает маршруты с сервисами. ORM-модели SQLAlchemy хранят первичные и выверенные данные, а шаблоны Jinja и JavaScript выводят таблицы, фильтры, подсказки и интерактивный ввод. Для исходных числовых полей используется Numeric(36, 16); расчетные сервисы применяют Decimal и отдельные правила отображения. Округление для вывода не следует трактовать как изменение сохраненной точности.

| Модель | Таблица | Ключи и разрезы |
|---|---|---|
| OperatingTesFormulaText | gs_eco_operating_tes_formula_texts | formula_key |
| EcologyFuelSnapshot | gs_eco_fuel_snapshots | database_version_id, year_number, scope |
| EcologyFuelSubmission | gs_eco_fuel_submissions | database_version_id, year_number, scope, status |
| NewTesNorm | gs_eco_new_tes_norms | database_version_id, year_number, fuel_key, capacity_class |
| OperatingTesCoefficient | gs_eco_operating_tes_coefficients | database_version_id, year_number, territory_slug, affiliation, kind |

| Модель | Таблица | Ключи и разрезы |
|---|---|---|
| OperatingTesFuelCharacteristic | gs_eco_operating_tes_fuel_chars | database_version_id, year_number, territory_slug, fuel_key |
| OperatingTesInputValue | gs_eco_operating_tes_inputs | database_version_id, year_number, territory_slug, row_key |
| PGParamCalc | gs_eco_analysis_pg_params | database_version_id, sheet_slug, station_numb, year_number |
| PGReportGisTek | gs_eco_gis_tek_greenhouse_gas_params | database_version_id, station_numb, equipment_group_id, year_number |
| ZVParamCalc | gs_eco_analysis_zv_params | database_version_id, sheet_slug, station_numb, year_number |
| ZVReportGisTek | gs_eco_gis_tek_pollutant_params | database_version_id, station_numb, equipment_group_id, year_number |

Отчетные PGReportGisTek/ZVReportGisTek привязаны к коду группы оборудования, году и версии БД; отдельный equipment_group_id обеспечивает связь с группой «Топлива». Код numb не уникален сам по себе, поэтому версия и разрешение связи существенны. PGParamCalc/ZVParamCalc дополнительно разделены по sheet_slug. Составной внешний ключ года учитывает Year.number и Year.database_version_id. EcologyFuelSnapshot уникален по версии, году и scope. OperatingTesFormulaText, в отличие от числовых моделей, не имеет поля database_version_id: это общее хранилище текстов подсказок.

Модель отчетных ПГ: уникальность и составной ключ года. Источник: app/ecology/models/pg_report_gis_tek_model.py, строки 19–37.

```
19      __tablename__ = "gs_eco_gis_tek_greenhouse_gas_params"
20      __table_args__ = (
21          UniqueConstraint(
22              "station_numb",
23              "year_number",
24              "database_version_id",
25              name="uq_eco_gis_tek_pg_numb_year_ver",
26          ),
27          ForeignKeyConstraint(
28              ["year_number", "database_version_id"],
29              [
30                  f"{SCHEMA_REFDATA}.gs_sys_years.number",
31                  f"{SCHEMA_REFDATA}.gs_sys_years.database_version_id",
32              ],
33              ondelete="RESTRICT",
34              name="fk_eco_gis_tek_pg_year_ver",
35          ),
36          {"schema": SCHEMA_ECOLOGY},
37      )
```

Модель отчетных ПГ: исходные числовые поля. Источник: app/ecology/models/pg_report_gis_tek_model.py, строки 90–106.

```
  90      # Расход газа, т у.т.
  91      gas_consumption_tce = db.Column(Numeric(36, 16), nullable=True)
  92
  93      # Выбросы CO2 от сжигания газа, т
  94      co2_from_gas_t = db.Column(Numeric(36, 16), nullable=True)
  95
  96      # Расход мазута, т у.т.
  97      oil_fuel_consumption_tce = db.Column(Numeric(36, 16), nullable=True)
  98
  99      # Выбросы CO2 от сжигания мазута, т
 100      co2_from_oil_fuel_t = db.Column(Numeric(36, 16), nullable=True)
 101
 102      # Расход твердого топлива, т у.т.
 103      solid_fuel_consumption_tce = db.Column(Numeric(36, 16), nullable=True)
 104
 105      # Выбросы CO2 от сжигания твердого топлива, т
 106      co2_from_solid_fuel_t = db.Column(Numeric(36, 16), nullable=True)
```

Выверенные параметры анализа ЗВ: хранимые компоненты. Источник: app/ecology/models/zv_param_calc_model.py, строки 76–95.

```
  76      # Твердые вещества, т (TVERD_V)
  77      solid_substances = db.Column(Numeric(36, 16), nullable=True)
  78
  79      # Диоксид серы, т (SO2_gaz)
  80      sulfur_dioxide = db.Column(Numeric(36, 16), nullable=True)
  81
  82      # Оксид углерода [CO], т (CO_gaz)
  83      carbon_monoxide = db.Column(Numeric(36, 16), nullable=True)
  84
  85      # Оксиды азота [в пересчете на NO2], т (NOx_gaz)
  86      nitrogen_oxides = db.Column(Numeric(36, 16), nullable=True)
  87
  88      # Углеводороды [без летучих органических соединений], т (UGLEVOD)
  89      hydrocarbons = db.Column(Numeric(36, 16), nullable=True)
  90
  91      # Летучие органические соединения [ЛОС], т (LOS)
  92      voc = db.Column(Numeric(36, 16), nullable=True)
  93
  94      # В т.ч. прочие газообразные и жидкие, т (PROCH_ekol)
  95      other_gaseous_and_liquid = db.Column(Numeric(36, 16), nullable=True)
```

Модель принятого топливного снимка. Источник: app/ecology/models/fuel_exchange_model.py, строки 9–21.

```
   9  class EcologyFuelSnapshot(db.Model, AuditMixin):
  10      __tablename__ = "gs_eco_fuel_snapshots"
  11      __table_args__ = (
  12          db.UniqueConstraint("database_version_id", "year_number", "scope",
name="uq_eco_fuel_snapshot_version_year_scope"),
  13          {"schema": SCHEMA_ECOLOGY},
  14      )
  15
  16      id = db.Column(db.Integer, primary_key=True)
  17      database_version_id = db.Column(db.Integer,
db.ForeignKey(f"{SCHEMA_REFDATA}.gs_database_versions.id", ondelete="CASCADE"), nullable=False)
  18      year_number = db.Column(db.Integer, nullable=False)
  19      scope = db.Column(db.String(20), nullable=False, default="operating",
server_default="operating")
  20      payload = db.Column(db.JSON, nullable=False)
  21      updated_at = db.Column(db.DateTime(timezone=True), server_default=func.now(),
onupdate=func.now(), nullable=False)
```

## 7.3. Права, версия БД и журнал

Все внешние маршруты защищены login_required. Для записи can_edit_ecology требует аутентификацию и признак has_admin либо роль, имя которой содержит ecology. Отдельная проверка административной роли регулирует видимость служебных идентификационных столбцов. Недостаток прав блокирует импорт, PATCH, принятие пакета и сохранение текстов на сервере, независимо от того, показана ли кнопка в HTML.

Выбор версии БД используется в запросах отчетов, параметров и снимков. Передача топлива без выбранной версии завершается ошибкой. Журнал «Экология» регистрирует посещения, импорт и выгрузки, изменение значений и решения по пакетам. Дата, пользователь и контекст действия фиксируются существующими сервисами приложения; настоящее описание не подменяет эти записи.

Проверка права изменения экологических данных. Источник: app/ecology/services/gis_tek_page_services.py, строки 115–121.

```
115  def can_edit_ecology(user) -> bool:
116      if not user or not getattr(user, "is_authenticated", False):
117          return False
118      if getattr(user, "has_admin", False):
119          return True
120      names = [str(n or "") for n in (getattr(user, "role_names", None) or [])]
121      return any("ecology" in n.lower() for n in names)
```

Регистрация просмотра страницы. Источник: app/ecology/services/ecology_logging_services.py, строки 184–201.

```
184  def log_page_visit(
185      page_title: str,
186      *,
187      tab: Optional[str] = None,
188      details: Optional[Iterable[str]] = None,
189  ) -> None:
190      """Открытие страницы или вкладки модуля (дубль middleware подавляется)."""
191      action = f"{PAGE_VIEW_ACTION_PREFIX}: {MODULE_TITLE} – {page_title}"
192      if tab:
193          action += f", вкладка «{tab}»"
194      parts: list[str] = []
195      if has_request_context():
196          parts.append(f"url={request.full_path}")
197      parts.append(f"раздел={MODULE_TITLE}")
198      if tab:
199          parts.append(f"вкладка={tab}")
200      parts.extend(item for item in (details or ()) if item)
201      log_to_db(_actor(), action, "; ".join(parts), entity_type=ENTITY_TYPE)
```

## 7.4. Условия получения экранных форм

Снимки получены 29 октября 2026 года с действующего сайта. Общая панель с именем пользователя исключена кадрированием; длинные и широкие таблицы представлены видимым фрагментом. Предметные данные не изменялись: формы импорта, PATCH, сохранение подсказок и решения по пакетам не отправлялись. Посещения могут регистрироваться штатным журналом приложения.

Безопасно просмотрены главная страница, два отчета, редактор текстов, принятые топливные показатели действующих и новых ТЭС и коэффициенты. Страницы, вызывающие ensure_default_persisted_rows, ensure_default_norm_rows или seed_missing_params_from_reports, для иллюстраций не открывались: GET в этих ветвях способен создавать отсутствующие исходные записи. Их поведение описано по исходникам, без утверждения о проверке изменяющих операций на рабочей БД.

Расчетный GET может сохранять недостающие значения по умолчанию. Источник: app/ecology/services/operating_tes_page_services.py, строки 812–818.

```
812      elif layout == "chars":
813          ensure_default_persisted_rows(years)
814          char_overrides = load_characteristic_overrides(years)
815      else:
816          inputs = load_accepted_fuel_inputs(years)
817          ensure_default_persisted_rows(years)
818          input_overrides = {} if sheet["slug"] in {"zola", "sera", "co", "azot", "pg", "pg-tj"} else
load_input_overrides(years)
```

Разделение безопасного просмотра топлива и инициализации нормативов. Источник: app/ecology/services/new_tes_page_services.py, строки 275–291.

```
    char_overrides = load_characteristic_overrides(years) if layout != "fuel" else {}
    if layout == "fuel":
        raw = load_accepted_new_tes_fuel_inputs(years)
        inputs = {
            year: {
                slug: by_class.get(capacity_class) or empty_fuel_bucket()
                for slug, by_class in (by_terr or {}).items()
            }
            for year, by_terr in raw.items()
        }
    elif layout == "chars":
        ensure_default_norm_rows(years)
        norm_overrides = load_norm_overrides(years)
    else:
        raw = load_accepted_new_tes_fuel_inputs(years)
        ensure_default_norm_rows(years)
        norm_overrides = load_norm_overrides(years)
```

Инициализация параметров анализа при формировании годовых строк. Источник: app/ecology/services/gis_tes_pg_from_fuel_services.py, строки 790–810.

```
    slug = str(sheet_slug or "").strip()
    if slug not in ANALYSIS_SHEET_SLUGS or not groups:
        return
    years: list[int] = list(range(int(start_year), int(end_year) + 1))
    numbs: set[int] = set()
    for group in groups:
        for numb in group.get("member_numbs") or [group.get("numb")]:
            if isinstance(numb, int):
                numbs.add(numb)
    if slug in PG_ANALYSIS_SHEET_SLUGS:
        seed_missing_analysis_pg_params_from_reports(
            slug,
            station_numbs=numbs or None,
            years=years,
        )
    elif slug in ZV_ANALYSIS_SHEET_SLUGS:
        seed_missing_analysis_zv_params_from_reports(
            slug,
            station_numbs=numbs or None,
            years=years,
        )
```

## 8. Главная страница и отчетные данные ГИС ТЭК

### 8.1. Главная страница модуля

Адрес: /ecology/

**Функциональные возможности и действия пользователя**

Страница объединяет четыре группы переходов: отчетные данные, анализ данных, расчетные значения и агрегированные значения. Отдельные кнопки открывают ПГ и ЗВ, расчет действующих и новых ТЭС, сводные расчеты и итоговые суммы. Для пользователя с правом редактирования доступен редактор текстов подсказок формул. Главная страница не содержит собственной формы ввода экологических показателей.

**Взаимодействие и источники данных**

Навигация ведет к отчетным и расчетным подсистемам «Экологии». Общая панель связывает модуль с главной страницей АРМ, справочниками, версиями БД и журналом. Состав кнопок анализа формируется по COMPARISON_GROUPS, а доступ к редактору определяется can_edit_ecology.




Рисунок 8. Главная страница модуля «Экология». Действующий сайт, 29.10.2026. Показан фрагмент экранной формы.

**Элементы реализации**

Собственной модели этой страницы нет: маршрут собирает навигацию и права, не создавая отдельную таблицу показателей.

Маршрут страницы. Источник: app/ecology/routes/ecology_routes.py, строки 141–149.

```
@ecology_bp.route("/")
@login_required
def hub():
    log_page_visit("Главная страница модуля")
    return render_template(
        "ecology/ecology_start.html",
        can_edit_ecology=can_edit_ecology(current_user),
        comparison_groups=COMPARISON_GROUPS,
    )
```

Сервис и расчетная логика. Источник: app/ecology/services/gis_tek_page_services.py, строки 115–121.

```
def can_edit_ecology(user) -> bool:
    if not user or not getattr(user, "is_authenticated", False):
        return False
    if getattr(user, "has_admin", False):
        return True
    names = [str(n or "") for n in (getattr(user, "role_names", None) or [])]
    return any("ecology" in n.lower() for n in names)
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/ecology_start.html, строки 9–42.

```
  <section class="home-outline__section">
    <h2 class="home-outline__section-title">Отчетные данные</h2>
    <div class="home-outline__tiles">
      {{ hub_tile(url_for('ecology_bp.gis_tek_pg'), 'Парниковые
      газы (ПГ)') }} {{ hub_tile(url_for('ecology_bp.gis_tek_pollutants'),
      'Загрязняющие вещества (ЗВ)') }}
    </div>
  </section>

  <section class="home-outline__section">
    <h2 class="home-outline__section-title">Анализ данных</h2>
    <div class="home-outline__tiles">
      {% for group in comparison_groups %} {% call
      hub_tile(url_for('ecology_bp.gis_tes_fuel_comparison',
      sheet_slug=group.default_sheet_slug)) %} {% if group.slug == 'pg' %}
      Парниковые газы (ПГ)<br />анализ отчетных данных {% elif group.slug ==
      'zv' %} Загрязняющие вещества (ЗВ)<br />анализ отчетных данных {% else %}
      {{ group.title }} {% endif %} {% endcall %} {% endfor %}
    </div>
  </section>

  <section class="home-outline__section">
    <h2 class="home-outline__section-title">Расчетные значения</h2>
    <div class="home-outline__tiles">
      {{ hub_tile(url_for('ecology_bp.operating_tes_table',
      territory_slug='vsego', sheet_slug='obemy'), 'Расчет по действующим ТЭС',
      'is-accent') }} {{ hub_tile(url_for('ecology_bp.new_tes_table',
      territory_slug='vsego', sheet_slug='obemy'), 'Расчет по новым ТЭС',
      'is-accent') }}
    </div>
    <div class="home-outline__tiles mt-2">
      {{ hub_tile(url_for('ecology_bp.calculation_summary_index'),
      'Сводные данные расчетов', 'is-accent') }}
    </div>
```

## 8.2. Парниковые газы: отчетные данные ГИС ТЭС

Адрес: /ecology/gis_tek/pg/

**Функциональные возможности и действия пользователя**

Таблица показывает отчетные показатели по группам оборудования за выбранный год. Строки организованы по ОЭС и региональным энергосистемам. Доступны поиск по станции, UUID и компании, территориальные фильтры, отбор по принадлежности и коду numb1120, фильтры в шапке, пагинация, показ пустых строк, округление и суммы по регионам. Для газа, мазута и твердого топлива показаны расход в т у.т. и масса CO2 в тоннах. Два итоговых столбца суммируют

расход топлива и выбросы по трем группам; они вычисляются программой, а не загружаются как независимые исходные показатели. Панель импорта и экспорта содержит загрузку Excel для уполномоченного пользователя и выгрузку текущей выборки. Самостоятельной страницы добавления или карточки редактирования отчета нет.

**Взаимодействие и источники данных**

Группа оборудования и ее код поступают из «Топлива», наименование станции и ссылка на карточку берутся из «Генерации». Год и территория согласуются со справочниками и выбранной версией БД. Отчетные строки используются для первоначального заполнения отдельных параметров анализа; уже исправленные параметры анализа не должны автоматически заменяться повторным просмотром отчета.

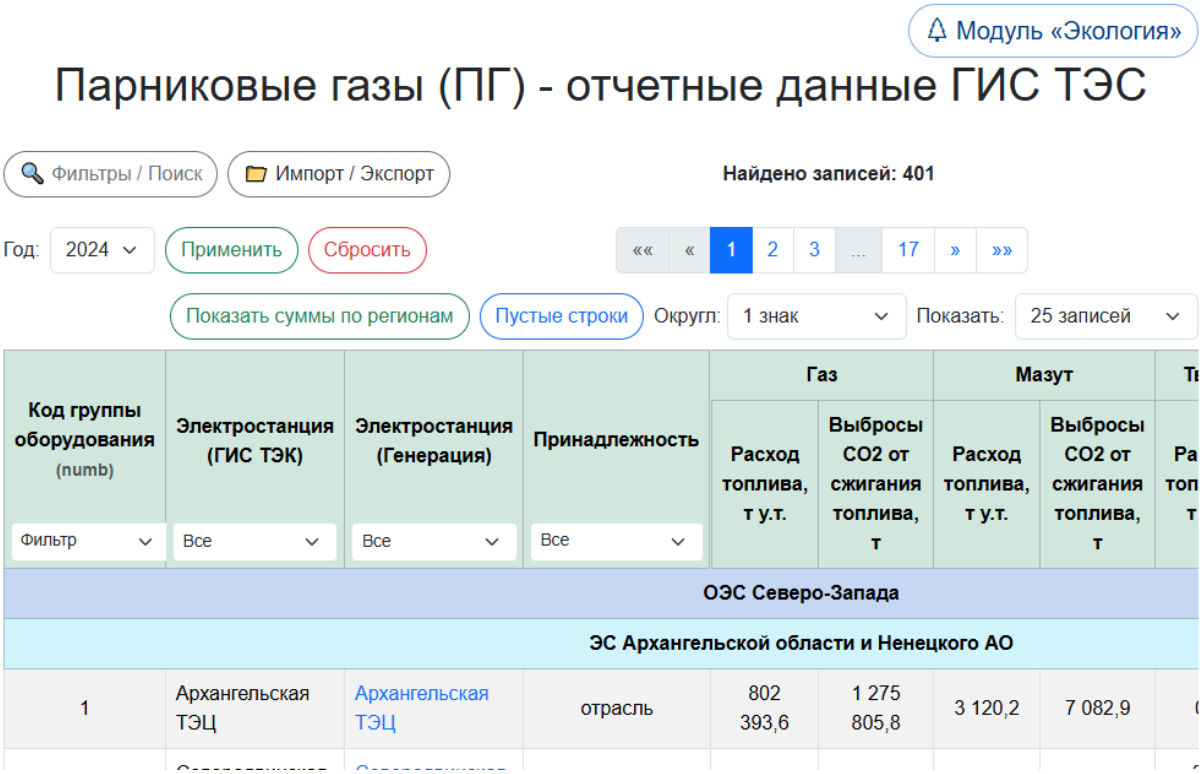



Рисунок 9. Парниковые газы: отчетные данные ГИС ТЭС. Действующий сайт, 29.10.2026. Показан фрагмент экранной формы.

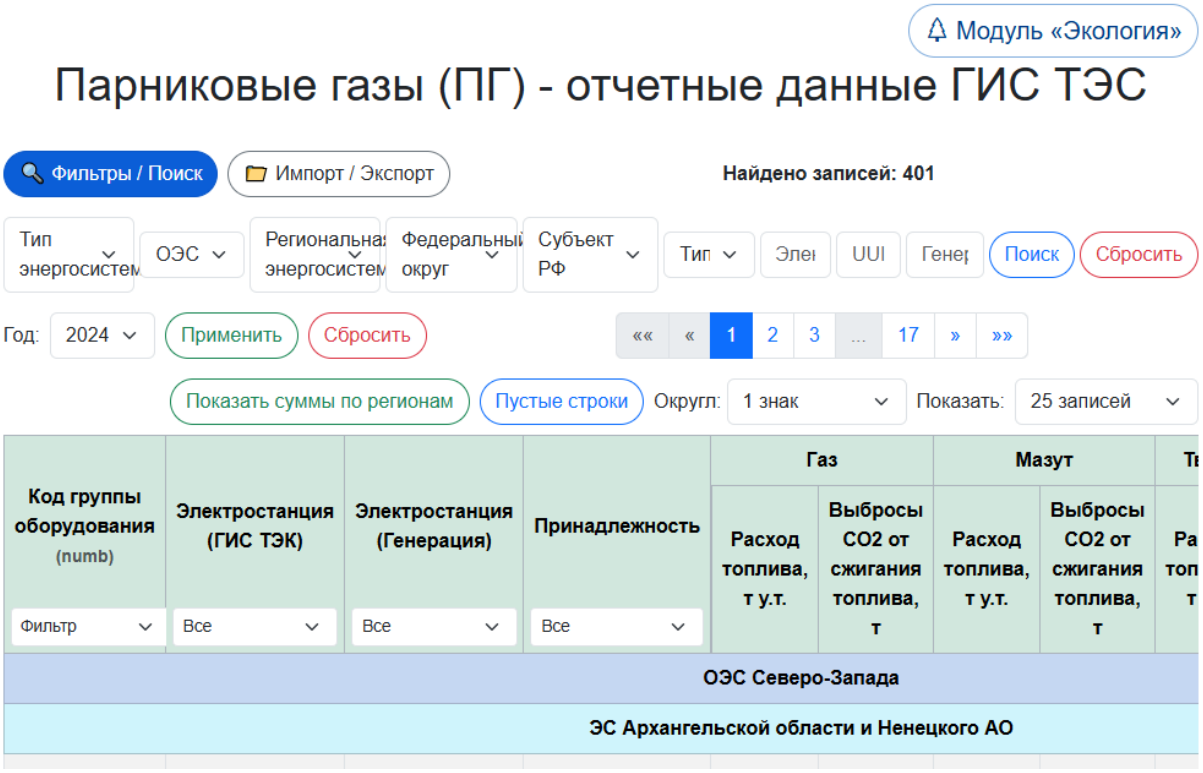



Рисунок 10. Раскрытая панель фильтров отчетных данных. Действующий сайт, 29.10.2026. Показан фрагмент экранной формы.

## Элементы реализации

Модель PGReportGisTek. Источник: app/ecology/models/pg_report_gis_tek_model.py, строки 11–19.

```
11  class PGReportGisTek(db.Model, AuditMixin):
12      """Строка листа «СБОРКА ПГ»: выбросы CO2 по видам топлива и году.
13
14      Связь с EquipmentGroup через equipment_group_id и station_numb = EquipmentGroup.numb
15      (без FK на numb – поле не уникально).
16      year_number + database_version_id → Year.number + Year.database_version_id.
17      """
18
19      __tablename__ = "gs_eco_gis_tek_greenhouse_gas_params"
```

Маршрут страницы. Источник: app/ecology/routes/ecology_routes.py, строки 219–222.

```
219  @ecology_bp.route("/gis_tek/pg/", methods=["GET"])
220  @login_required
221  def gis_tek_pg():
222      return _render_report(REPORT_PG)
```

Сервис и расчетная логика. Источник: app/ecology/services/gis_tek_page_services.py, строки 621–626.

```
621  def apply_pg_derived_metrics(values: dict) -> dict:
622      """Суммы расхода топлива и выбросов CO2 по всем видам топлива."""
623      values = dict(values)
624      for attr, parts in PG_TOTAL_PARTS.items():
625          values[attr] = _sum_parts(values, parts)
626      return values
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/gis_tek_report.html, строки 228–257.

```
            </thead>
            <tbody>
                {% if hierarchy and hierarchy|length > 0 %}
                    {% for ues_block in hierarchy %}
                        <tr class="table-primary text-center" style="vertical-align: middle; font-
weight: bold;">
                            <td colspan="{{ column_count }}">{{ ues_block.ues_name }}</td>
                        </tr>
                        {% for res_block in ues_block.res_list %}
                            <tr class="table-info text-center" style="vertical-align: middle; font-
weight: bold;">
                                <td colspan="{{ column_count }}">{{ res_block.res_name }}</td>
                            </tr>
                            {% for row in res_block.rows %}
                            <tr class="text-center align-middle{% if row.is_empty %} ot-row-empty-
hidden{% endif %}">
                                {% if show_numb_column %}
                                <td>{{ row.station_numb if row.station_numb is not none else '–'
}}</td>
                                {% endif %}
                                <td class="text-start">{{ row.gis_tek_station_name }}</td>
                                <td class="text-start">
                                    {% if row.generation_stations %}
                                        {% for st in row.generation_stations %}
                                            {% if not loop.first %}<br>{% endif %}
                                            <a href="{{ url_for('station_bp.station_details',
station_id=st.id) }}"
                                               class="text-decoration-none"
                                               target="_blank"
                                               rel="noopener noreferrer">{{ st.name }}</a>
                                        {% endfor %}
                                    {% else %}
                                        –
                                    {% endif %}
                                </td>
```

Сервис формирования страницы. Источник: app/ecology/services/gis_tek_page_services.py, строки 942–970.

```
def build_gis_tek_page_context(kind: str, args, user=None) -> dict:
    model = REPORT_MODELS[kind]
    version_id = get_current_db_version_id()
    years = _filter_years()
    data_years = _data_years(model, version_id)
    if data_years:
        default_year = data_years[-1]
    else:
        from config import START_YEAR
        default_year = START_YEAR if START_YEAR in years else (years[-1] if years else None)
    selected_year = _resolve_year(args, years, default_year)

    per_page = _parse_per_page(args.get("per_page"))
    try:
        page = int(args.get("page") or 1)
    except (TypeError, ValueError):
        page = 1
    if page < 1:
        page = 1

    q = (args.get("q") or "").strip()
    station_type_filter = (args.get("station_type") or "").strip()
    numb1120_filter = normalize_numb_filter(_args_getlist(args, "numb1120_filter"))
    territory_filters = _parse_gis_tek_territory_filters(args)
    q = str(territory_filters.get("station_name_filter") or q or "").strip()
    show_totals = parse_show_totals_arg(args)
    show_empty_rows = parse_show_empty_rows_arg(args)
    rounding_digits = resolve_rounding_digits(args.get("rounding_digits"))
    norm_zv_selected = parse_norm_zv_parts(args) if kind == REPORT_ZV else []
```

Общий маршрут вывода отчета. Источник: app/ecology/routes/ecology_routes.py, строки 80–91.

```
80      context["import_endpoint"] = (
81          "ecology_bp.gis_tek_pg_import"
82          if kind == REPORT_PG
83          else "ecology_bp.gis_tek_pollutants_import"
84      )
85      context["export_endpoint"] = (
86          "ecology_bp.gis_tek_pg_export"
87          if kind == REPORT_PG
88          else "ecology_bp.gis_tek_pollutants_export"
89      )
90      log_page_visit(context["page_title"])
91      return render_template("ecology/gis_tek_report.html", **context)
```

## 8.3. Загрязняющие вещества: отчетные данные ГИС ТЭС

Адрес: /ecology/gis_tek/pollutants/

**Функциональные возможности и действия пользователя**

Таблица показывает отчетные показатели по группам оборудования за выбранный год. Строки организованы по ОЭС и региональным энергосистемам. Доступны поиск по станции, UUID и компании, территориальные фильтры, отбор по принадлежности и коду numb1120, фильтры в шапке, пагинация, показ пустых строк, округление и суммы по регионам. Показаны твердые вещества, диоксид серы, CO, оксиды азота, углеводороды, ЛОС и прочие газообразные и жидкие вещества в тоннах за год. Газообразные и жидкие вещества суммируются из компонентов, общий итог складывается из них и твердых веществ. Состав столбца «Нормируемые ЗВ» выбирается в шапке; по умолчанию включены твердые вещества, SO2, CO и NOx. Панель импорта и экспорта содержит загрузку Excel для уполномоченного пользователя и выгрузку текущей выборки. Самостоятельной страницы добавления или карточки редактирования отчета нет.

**Взаимодействие и источники данных**

Группа оборудования и ее код поступают из «Топлива», наименование станции и ссылка на карточку берутся из «Генерации». Год и территория согласуются со справочниками и выбранной версией БД. Отчетные строки используются для первоначального заполнения отдельных параметров анализа; уже исправленные параметры анализа не должны автоматически заменяться повторным просмотром отчета.

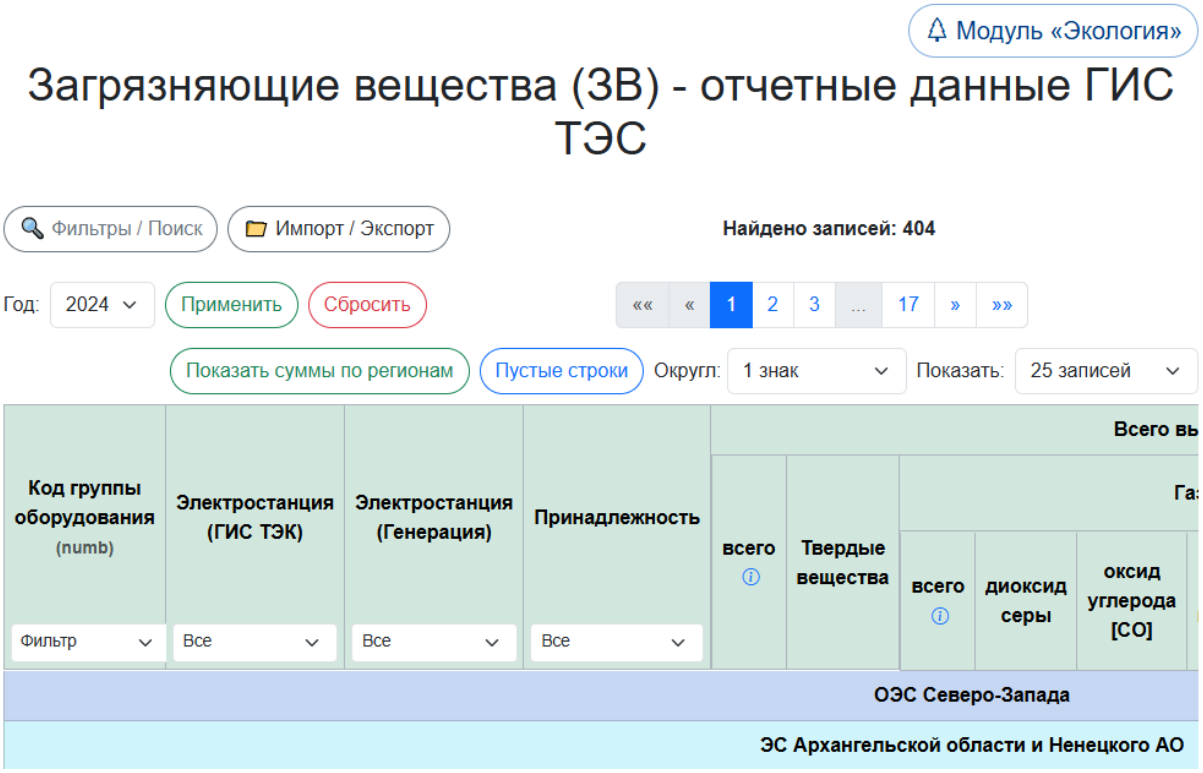



Рисунок 11. Загрязняющие вещества: отчетные данные ГИС ТЭС. Действующий сайт, 29.10.2026. Показан фрагмент экранной формы.

## Элементы реализации

Модель ZVReportGisTek. Источник: app/ecology/models/zv_report_gis_tek_model.py, строки 11–19.

```
11  class ZVReportGisTek(db.Model, AuditMixin):
12      """Строка листа «СБОРКА ЗВ»: выбросы загрязняющих веществ по году.
13
14      Связь с EquipmentGroup через equipment_group_id и station_numb = EquipmentGroup.numb
15      (без FK на numb – поле не уникально).
16      year_number + database_version_id → Year.number + Year.database_version_id.
17      """
18
19      __tablename__ = "gs_eco_gis_tek_pollutant_params"
```

Маршрут страницы. Источник: app/ecology/routes/ecology_routes.py, строки 237–240.

```
237  @ecology_bp.route("/gis_tek/pollutants/", methods=["GET"])
238  @login_required
239  def gis_tek_pollutants():
240      return _render_report(REPORT_ZV)
```

Сервис и расчетная логика. Источник: app/ecology/services/gis_tek_page_services.py, строки 613–618.

```
613  def apply_zv_derived_metrics(values: dict) -> dict:
614      """Подставляет расчетные столбцы ЗВ (газообразные и жидкие, всего)."""
615      values = dict(values or {})
616      values["gaseous_and_liquid"] = sum_gaseous_and_liquid(values)
617      values["total"] = sum_zv_total(values)
618      return values
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/gis_tek_report.html, строки 228–257.

```
            </thead>
            <tbody>
                {% if hierarchy and hierarchy|length > 0 %}
                    {% for ues_block in hierarchy %}
                        <tr class="table-primary text-center" style="vertical-align: middle; font-
weight: bold;">
                            <td colspan="{{ column_count }}">{{ ues_block.ues_name }}</td>
                        </tr>
                        {% for res_block in ues_block.res_list %}
                            <tr class="table-info text-center" style="vertical-align: middle; font-
weight: bold;">
                                <td colspan="{{ column_count }}">{{ res_block.res_name }}</td>
                            </tr>
                            {% for row in res_block.rows %}
                            <tr class="text-center align-middle{% if row.is_empty %} ot-row-empty-
hidden{% endif %}">
                                {% if show_numb_column %}
                                <td>{{ row.station_numb if row.station_numb is not none else '–'
}}</td>
                                {% endif %}
                                <td class="text-start">{{ row.gis_tek_station_name }}</td>
                                <td class="text-start">
                                    {% if row.generation_stations %}
                                        {% for st in row.generation_stations %}
                                            {% if not loop.first %}<br>{% endif %}
                                            <a href="{{ url_for('station_bp.station_details',
station_id=st.id) }}"
                                               class="text-decoration-none"
                                               target="_blank"
                                               rel="noopener noreferrer">{{ st.name }}</a>
                                        {% endfor %}
                                    {% else %}
                                        –
                                    {% endif %}
                                </td>
```

Сервис формирования страницы. Источник: app/ecology/services/gis_tek_page_services.py, строки 942–970.

```
def build_gis_tek_page_context(kind: str, args, user=None) -> dict:
    model = REPORT_MODELS[kind]
    version_id = get_current_db_version_id()
    years = _filter_years()
    data_years = _data_years(model, version_id)
    if data_years:
        default_year = data_years[-1]
    else:
        from config import START_YEAR
        default_year = START_YEAR if START_YEAR in years else (years[-1] if years else None)
    selected_year = _resolve_year(args, years, default_year)

    per_page = _parse_per_page(args.get("per_page"))
    try:
        page = int(args.get("page") or 1)
    except (TypeError, ValueError):
        page = 1
    if page < 1:
        page = 1

    q = (args.get("q") or "").strip()
    station_type_filter = (args.get("station_type") or "").strip()
    numb1120_filter = normalize_numb_filter(_args_getlist(args, "numb1120_filter"))
    territory_filters = _parse_gis_tek_territory_filters(args)
    q = str(territory_filters.get("station_name_filter") or q or "").strip()
    show_totals = parse_show_totals_arg(args)
    show_empty_rows = parse_show_empty_rows_arg(args)
    rounding_digits = resolve_rounding_digits(args.get("rounding_digits"))
    norm_zv_selected = parse_norm_zv_parts(args) if kind == REPORT_ZV else []
```

Общий маршрут вывода отчета. Источник: app/ecology/routes/ecology_routes.py, строки 80–91.

```
    context["import_endpoint"] = (
        "ecology_bp.gis_tek_pg_import"
        if kind == REPORT_PG
        else "ecology_bp.gis_tek_pollutants_import"
    )
    context["export_endpoint"] = (
        "ecology_bp.gis_tek_pg_export"
        if kind == REPORT_PG
        else "ecology_bp.gis_tek_pollutants_export"
    )
    log_page_visit(context["page_title"])
    return render_template("ecology/gis_tek_report.html", **context)
```

## 9. Анализ отчетных данных и сопоставление с модулем «Топливо»

### 9.1. Парниковые газы: анализ с котельными

Адрес: /ecology/analysis/pg-vse/

**Функциональные возможности и действия пользователя**

Лист «pg-vse» предназначен для анализа с котельными. Сопоставляются расход газа, мазута и твердого топлива по топливным данным и выверенным данным экологии. Программа вычисляет разности, коэффициенты эмиссии CO2, суммарную массу ПГ и удельные выбросы на топливо и приведенную энергию. Отсутствующий знаменатель или ноль дают пустой расчетный результат, а не деление на ноль. Доступны период лет, территориальные и станционные фильтры, принадлежность, код группы оборудования, пагинация, округление отдельных групп показателей, суммы по регионам и Excel. Уполномоченный пользователь редактирует только исходные экологические ячейки; зависимые столбцы пересчитываются. Состав включает группы котельных, участвующие в исходной выборке.

**Взаимодействие и источники данных**

Источники: параметры анализа PGParamCalc/ZVParamCalc, отчетные PGReportGisTek/ZVReportGisTek, показатели групп оборудования из «Топлива» и сведения о станциях из «Генерации». Параметры анализа хранятся отдельно для каждого sheet_slug, кода группы, года и версии БД. При первом заполнении недостающие строки копируются из отчетов без перезаписи существующих исправлений. Разметка столбцов и исходный макет читаются из Excel; числовые значения дополняются данными БД. Итоги используются в агрегированных представлениях.

**Элементы реализации**

Модель PGParamCalc. Источник: app/ecology/models/pg_param_calc_model.py, строки 11–21.

```
11  class PGParamCalc(db.Model, AuditMixin):
12      """Строка параметров страниц /ecology/analysis/pg-vse/ и pg-bez-kot/.
13
14      Первоначально копируется из PGReportGisTek; далее может
15      редактироваться отдельно от отчетных данных.
16      Связь с EquipmentGroup: station_numb (numb1120) = EquipmentGroup.numb
17      (без FK на numb – поле не уникально).
18      year_number + database_version_id → Year.number + Year.database_version_id.
19      """
20
21      __tablename__ = "gs_eco_analysis_pg_params"
```

Маршрут страницы. Источник: app/ecology/routes/gis_tes_fuel_comparison_routes.py, строки 43–66.

```
@ecology_bp.route("/analysis/<sheet_slug>/")
@login_required
def gis_tes_fuel_comparison(sheet_slug: str):
    if request.args.get("reset_filters") == "1":
        keep = {
            key: value
            for key in (
                "start_year",
                "end_year",
                "rounding_digits",
                "emission_rounding_digits",
                "specific_rounding_digits",
                "per_page",
                "show_totals",
                "show_empty_rows",
                "show_reported",
            )
            if (value := request.args.get(key)) not in (None, "")
        }
        if "norm_zv" in request.args:
            keep["norm_zv"] = request.args.getlist("norm_zv")
        return redirect(
            url_for("ecology_bp.gis_tes_fuel_comparison", sheet_slug=sheet_slug, **keep)
        )
```

Сервис и расчетная логика. Источник: app/ecology/services/gis_tes_fuel_comparison_services.py, строки 2028–2050.

```
def _compute_emission_coefficients(
    filled: list[Any],
    source_columns: list[dict[str, Any]],
) -> list[Any]:
    """Коэффициенты эмиссии: выбросы CO2 / расход топлива."""
    out = list(filled)
    for col_key, co2_key, fuel_key in [
        ("K_gaz_ek", "CO2_gaz", "GAZ_ek"),
        ("K_mazut_ek", "CO2_mazut", "MAZUT_ek"),
        ("K_ugol_ek", "CO2_ugol", "TvTopl_ek"),
    ]:
        idx = _column_index(source_columns, {col_key})
        if idx is None:
            continue
        co2_idx = _column_index(source_columns, {co2_key})
        fuel_idx = _column_index(source_columns, {fuel_key})
        if co2_idx is None or fuel_idx is None:
            continue
        value = _safe_divide(filled[co2_idx], filled[fuel_idx])
        if len(out) <= idx:
            out.extend([None] * (idx - len(out) + 1))
        out[idx] = value
    return out
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/analysis/gis_tes_fuel_comparison.html, строки 588–610.

```
 588            class="table table-hover table-striped table-bordered power-demand-summary-table{% if not
show_empty_rows|default(false) %} ot-hide-empty{% endif %}"
 589            id="gisTesFuelTable"
 590            data-sheet-slug="{{ sheet.slug }}"
 591            data-can-edit="{{ '1' if can_edit|default(false) else '0' }}"
 592            data-update-url="{% if can_edit|default(false) %}{{
url_for('ecology_bp.gis_tes_fuel_comparison_update_value', sheet_slug=sheet.slug) }}{% endif %}"
 593            data-rounding-digits="{{ rounding_digits }}"
 594            data-emission-rounding-digits="{{ emission_rounding_digits }}"
 595            data-specific-rounding-digits="{{ specific_rounding_digits }}"
 596          >
 597            <thead class="table-success">
 598              {% for header_row in header_rows %}
 599              {% set row_is_key = header_row and header_row[0].is_key %}
 600              <tr class="text-center align-middle{% if row_is_key %} cmp-key-row{% endif %}">
 601              {% for cell in header_row %}
 602              {% if cell.column_key == 'numb1120' and not cell.is_key %}
 603              {{ fuel_param_numb1120_header_th(
 604                   cell.rowspan if cell.rowspan else 1,
 605                   'Код группы оборудования',
 606                   numb1120_filter|default([]),
 607                   numb1120_filter_choices|default([]),
 608                   extra_class='cmp-sticky text-center align-middle gis-tes-filter-th',
 609                   sticky_col=cell.col_index
 610                 ) }}
```

Обновление ячейки анализа. Источник: app/ecology/routes/gis_tes_fuel_comparison_routes.py, строки 153–175.

```
 153  @ecology_bp.route("/analysis/<sheet_slug>/value/", methods=["PATCH"])
 154  @login_required
 155  def gis_tes_fuel_comparison_update_value(sheet_slug: str):
 156      if not can_edit_ecology(current_user):
 157          return jsonify(ok=False, error="Недостаточно прав"), 403
 158      if sheet_slug not in ANALYSIS_SHEET_SLUGS:
 159          return jsonify(ok=False, error="Лист только для просмотра"), 400
 160      data = request.get_json(silent=True) or {}
 161      column_key = str(data.get("field") or "").strip()
 162      try:
 163          station_numb = int(data.get("numb"))
 164          year_number = int(data.get("year"))
 165      except (TypeError, ValueError):
 166          return jsonify(ok=False, error="Некорректные станция или год"), 400
 167      raw = data.get("value")
 168      try:
 169          amount = parse_decimal_from_display(raw) if raw not in (None, "") else None
 170      except Exception:
 171          return jsonify(ok=False, error="Некорректное число"), 400
 172      try:
 173          values = save_analysis_ecology_cell(
 174              sheet_slug,
 175              station_numb,
```

## 9.2. Парниковые газы: анализ без котельных

Адрес: /ecology/analysis/pg-bez-kot/

**Функциональные возможности и действия пользователя**

Лист «pg-bez-kot» предназначен для анализа без котельных. Сопоставляются расход газа, мазута и твердого топлива по топливным данным и выверенным данным экологии. Программа вычисляет разности, коэффициенты эмиссии CO2, суммарную массу ПГ и удельные выбросы на топливо и приведенную энергию. Отсутствующий знаменатель или ноль дают пустой расчетный результат, а не деление на ноль. Доступны период лет, территориальные и станционные фильтры, принадлежность, код группы оборудования, пагинация, округление отдельных групп показателей, суммы по регионам и Excel. Уполномоченный пользователь редактирует только исходные экологические ячейки; зависимые столбцы пересчитываются. Из состава исключаются группы котельных; остальные показатели рассчитываются по той же схеме, что и на листе с котельными.

**Взаимодействие и источники данных**

Источники: параметры анализа PGParamCalc/ZVParamCalc, отчетные PGReportGisTek/ZVReportGisTek, показатели групп оборудования из «Топлива» и сведения о станциях из «Генерации». Параметры анализа хранятся отдельно для каждого sheet_slug, кода группы, года и версии БД. При первом заполнении недостающие строки копируются из отчетов без перезаписи существующих исправлений. Разметка столбцов и исходный макет читаются из Excel; числовые значения дополняются данными БД. Итоги используются в агрегированных представлениях.

## Элементы реализации

Модель PGParamCalc. Источник: app/ecology/models/pg_param_calc_model.py, строки 11–21.

```
11  class PGParamCalc(db.Model, AuditMixin):
12      """Строка параметров страниц /ecology/analysis/pg-vse/ и pg-bez-kot/.
13
14      Первоначально копируется из PGReportGisTek; далее может
15      редактироваться отдельно от отчетных данных.
16      Связь с EquipmentGroup: station_numb (numb1120) = EquipmentGroup.numb
17      (без FK на numb – поле не уникально).
18      year_number + database_version_id → Year.number + Year.database_version_id.
19      """
20
21      __tablename__ = "gs_eco_analysis_pg_params"
```

Маршрут страницы. Источник: app/ecology/routes/gis_tes_fuel_comparison_routes.py, строки 43–66.

```
43  @ecology_bp.route("/analysis/<sheet_slug>/")
44  @login_required
45  def gis_tes_fuel_comparison(sheet_slug: str):
46      if request.args.get("reset_filters") == "1":
47          keep = {
48              key: value
49              for key in (
50                  "start_year",
51                  "end_year",
52                  "rounding_digits",
53                  "emission_rounding_digits",
54                  "specific_rounding_digits",
55                  "per_page",
56                  "show_totals",
57                  "show_empty_rows",
58                  "show_reported",
59              )
60              if (value := request.args.get(key)) not in (None, "")
61          }
62          if "norm_zv" in request.args:
63              keep["norm_zv"] = request.args.getlist("norm_zv")
64          return redirect(
65              url_for("ecology_bp.gis_tes_fuel_comparison", sheet_slug=sheet_slug, **keep)
66          )
```

Сервис и расчетная логика. Источник: app/ecology/services/gis_tes_fuel_comparison_services.py, строки 2028–2050.

```
def _compute_emission_coefficients(
    filled: list[Any],
    source_columns: list[dict[str, Any]],
) -> list[Any]:
    """Коэффициенты эмиссии: выбросы CO2 / расход топлива."""
    out = list(filled)
    for col_key, co2_key, fuel_key in [
        ("K_gaz_ek", "CO2_gaz", "GAZ_ek"),
        ("K_mazut_ek", "CO2_mazut", "MAZUT_ek"),
        ("K_ugol_ek", "CO2_ugol", "TvTopl_ek"),
    ]:
        idx = _column_index(source_columns, {col_key})
        if idx is None:
            continue
        co2_idx = _column_index(source_columns, {co2_key})
        fuel_idx = _column_index(source_columns, {fuel_key})
        if co2_idx is None or fuel_idx is None:
            continue
        value = _safe_divide(filled[co2_idx], filled[fuel_idx])
        if len(out) <= idx:
            out.extend([None] * (idx - len(out) + 1))
        out[idx] = value
    return out
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/analysis/gis_tes_fuel_comparison.html, строки 588–610.

```
            class="table table-hover table-striped table-bordered power-demand-summary-table{% if not
show_empty_rows|default(false) %} ot-hide-empty{% endif %}"
            id="gisTesFuelTable"
            data-sheet-slug="{{ sheet.slug }}"
            data-can-edit="{{ '1' if can_edit|default(false) else '0' }}"
            data-update-url="{% if can_edit|default(false) %}{{
url_for('ecology_bp.gis_tes_fuel_comparison_update_value', sheet_slug=sheet.slug) }}{% endif %}"
            data-rounding-digits="{{ rounding_digits }}"
            data-emission-rounding-digits="{{ emission_rounding_digits }}"
            data-specific-rounding-digits="{{ specific_rounding_digits }}"
          >
            <thead class="table-success">
              {% for header_row in header_rows %}
              {% set row_is_key = header_row and header_row[0].is_key %}
              <tr class="text-center align-middle{% if row_is_key %} cmp-key-row{% endif %}">
              {% for cell in header_row %}
              {% if cell.column_key == 'numb1120' and not cell.is_key %}
              {{ fuel_param_numb1120_header_th(
                   cell.rowspan if cell.rowspan else 1,
                   'Код группы оборудования',
                   numb1120_filter|default([]),
                   numb1120_filter_choices|default([]),
                   extra_class='cmp-sticky text-center align-middle gis-tes-filter-th',
                   sticky_col=cell.col_index
                 ) }}
```

Обновление ячейки анализа. Источник: app/ecology/routes/gis_tes_fuel_comparison_routes.py, строки 153–175.

```
@ecology_bp.route("/analysis/<sheet_slug>/value/", methods=["PATCH"])
@login_required
def gis_tes_fuel_comparison_update_value(sheet_slug: str):
    if not can_edit_ecology(current_user):
        return jsonify(ok=False, error="Недостаточно прав"), 403
    if sheet_slug not in ANALYSIS_SHEET_SLUGS:
        return jsonify(ok=False, error="Лист только для просмотра"), 400
    data = request.get_json(silent=True) or {}
    column_key = str(data.get("field") or "").strip()
    try:
        station_numb = int(data.get("numb"))
        year_number = int(data.get("year"))
    except (TypeError, ValueError):
        return jsonify(ok=False, error="Некорректные станция или год"), 400
    raw = data.get("value")
    try:
        amount = parse_decimal_from_display(raw) if raw not in (None, "") else None
    except Exception:
        return jsonify(ok=False, error="Некорректное число"), 400
    try:
        values = save_analysis_ecology_cell(
            sheet_slug,
            station_numb,
```

## 9.3. Загрязняющие вещества: анализ с котельными

Адрес: /ecology/analysis/zv-vse/

**Функциональные возможности и действия пользователя**

Лист «zv-vse» предназначен для анализа с котельными. Сопоставляются массы загрязняющих веществ, расход топлива, выработка электроэнергии и отпуск тепла. Вычисляются общий выброс, сумма нормируемых ингредиентов, приведенная энергия и удельные показатели на энергию и топливо. Можно менять состав нормируемых ЗВ и показывать дополнительные столбцы отчитавшихся станций. Доступны период лет, территориальные и станционные фильтры, принадлежность, код группы оборудования, пагинация, округление отдельных групп показателей, суммы по регионам и Excel. Уполномоченный пользователь редактирует только исходные экологические ячейки; зависимые столбцы пересчитываются. Состав включает группы котельных, участвующие в исходной выборке.

**Взаимодействие и источники данных**

Источники: параметры анализа PGParamCalc/ZVParamCalc, отчетные PGReportGisTek/ZVReportGisTek, показатели групп оборудования из «Топлива» и сведения о станциях из «Генерации». Параметры анализа хранятся отдельно для каждого sheet_slug, кода группы, года и версии БД. При первом заполнении недостающие строки копируются из отчетов без перезаписи существующих исправлений. Разметка столбцов и исходный макет читаются из Excel; числовые значения дополняются данными БД. Итоги используются в агрегированных представлениях.

**Элементы реализации**

Модель ZVParamCalc. Источник: app/ecology/models/zv_param_calc_model.py, строки 11–21.

```
class ZVParamCalc(db.Model, AuditMixin):
    """Строка параметров страниц /ecology/analysis/zv-vse/ и zv-bez-kot/.

    Первоначально копируется из ZVReportGisTek; далее может редактироваться
    отдельно от отчетных данных.
    Связь с EquipmentGroup: station_numb (numb1120) = EquipmentGroup.numb
    (без FK на numb – поле не уникально).
    year_number + database_version_id → Year.number + Year.database_version_id.
    """

    __tablename__ = "gs_eco_analysis_zv_params"
```

Маршрут страницы. Источник: app/ecology/routes/gis_tes_fuel_comparison_routes.py, строки 43–66.

```
@ecology_bp.route("/analysis/<sheet_slug>/")
@login_required
def gis_tes_fuel_comparison(sheet_slug: str):
    if request.args.get("reset_filters") == "1":
        keep = {
            key: value
            for key in (
                "start_year",
                "end_year",
                "rounding_digits",
                "emission_rounding_digits",
                "specific_rounding_digits",
                "per_page",
                "show_totals",
                "show_empty_rows",
                "show_reported",
            )
            if (value := request.args.get(key)) not in (None, "")
        }
        if "norm_zv" in request.args:
            keep["norm_zv"] = request.args.getlist("norm_zv")
        return redirect(
            url_for("ecology_bp.gis_tes_fuel_comparison", sheet_slug=sheet_slug, **keep)
        )
```

Сервис и расчетная логика. Источник: app/ecology/services/gis_tes_fuel_comparison_services.py, строки 2200–2222.

```
def _compute_zv_specific_metrics(
    filled: list[Any],
    source_columns: list[dict[str, Any]],
    norm_zv_parts: list[str] | tuple[str, ...] = ZV_NORM_DEFAULT_PARTS,
) -> list[Any]:
    """Приведенная энергия и удельные ЗВ: на энергию, на выработку и на топливо."""
    out = list(filled)
    solid = _filled_at(out, source_columns, {ZV_SOLID_KEY})
    so2 = _filled_at(out, source_columns, {"SO2_gaz"})
    nox = _filled_at(out, source_columns, {"NOx_gaz"})
    zv_normir = _safe_sum(*(
        _filled_at(out, source_columns, {key})
        for key, attr in ZV_ECOLOGY_OVERLAY_BY_KEY.items()
        if attr in norm_zv_parts
    ))
    _assign_filled(out, source_columns, "ZV_normir", zv_normir)

    w_en = _filled_at(out, source_columns, {"W_en"})
    teplo = _filled_at(out, source_columns, {"Teplo"})
    energy = _brought_energy_mln_kwh(w_en, teplo)
    _assign_filled(out, source_columns, "energy", energy)

    total_zv = _filled_at(out, source_columns, {ZV_TOTAL_KEY})
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/analysis/gis_tes_fuel_comparison.html, строки 588–610.

```
 588            class="table table-hover table-striped table-bordered power-demand-summary-table{% if not
show_empty_rows|default(false) %} ot-hide-empty{% endif %}"
 589            id="gisTesFuelTable"
 590            data-sheet-slug="{{ sheet.slug }}"
 591            data-can-edit="{{ '1' if can_edit|default(false) else '0' }}"
 592            data-update-url="{% if can_edit|default(false) %}{{
url_for('ecology_bp.gis_tes_fuel_comparison_update_value', sheet_slug=sheet.slug) }}{% endif %}"
 593            data-rounding-digits="{{ rounding_digits }}"
 594            data-emission-rounding-digits="{{ emission_rounding_digits }}"
 595            data-specific-rounding-digits="{{ specific_rounding_digits }}"
 596          >
 597            <thead class="table-success">
 598              {% for header_row in header_rows %}
 599              {% set row_is_key = header_row and header_row[0].is_key %}
 600              <tr class="text-center align-middle{% if row_is_key %} cmp-key-row{% endif %}">
 601              {% for cell in header_row %}
 602              {% if cell.column_key == 'numb1120' and not cell.is_key %}
 603              {{ fuel_param_numb1120_header_th(
 604                   cell.rowspan if cell.rowspan else 1,
 605                   'Код группы оборудования',
 606                   numb1120_filter|default([]),
 607                   numb1120_filter_choices|default([]),
 608                   extra_class='cmp-sticky text-center align-middle gis-tes-filter-th',
 609                   sticky_col=cell.col_index
 610                 ) }}
```

Обновление ячейки анализа. Источник: app/ecology/routes/gis_tes_fuel_comparison_routes.py, строки 153–175.

```
 153  @ecology_bp.route("/analysis/<sheet_slug>/value/", methods=["PATCH"])
 154  @login_required
 155  def gis_tes_fuel_comparison_update_value(sheet_slug: str):
 156      if not can_edit_ecology(current_user):
 157          return jsonify(ok=False, error="Недостаточно прав"), 403
 158      if sheet_slug not in ANALYSIS_SHEET_SLUGS:
 159          return jsonify(ok=False, error="Лист только для просмотра"), 400
 160      data = request.get_json(silent=True) or {}
 161      column_key = str(data.get("field") or "").strip()
 162      try:
 163          station_numb = int(data.get("numb"))
 164          year_number = int(data.get("year"))
 165      except (TypeError, ValueError):
 166          return jsonify(ok=False, error="Некорректные станция или год"), 400
 167      raw = data.get("value")
 168      try:
 169          amount = parse_decimal_from_display(raw) if raw not in (None, "") else None
 170      except Exception:
 171          return jsonify(ok=False, error="Некорректное число"), 400
 172      try:
 173          values = save_analysis_ecology_cell(
 174              sheet_slug,
 175              station_numb,
```

## 9.4. Загрязняющие вещества: анализ без котельных

Адрес: /ecology/analysis/zv-bez-kot/

**Функциональные возможности и действия пользователя**

Лист «zv-bez-kot» предназначен для анализа без котельных. Сопоставляются массы загрязняющих веществ, расход топлива, выработка электроэнергии и отпуск тепла. Вычисляются общий выброс, сумма нормируемых ингредиентов, приведенная энергия и удельные показатели на энергию и топливо. Можно менять состав нормируемых ЗВ и показывать дополнительные столбцы отчитавшихся станций. Доступны период лет, территориальные и станционные фильтры, принадлежность, код группы оборудования, пагинация, округление отдельных групп показателей, суммы по регионам и Excel. Уполномоченный пользователь редактирует только исходные экологические ячейки; зависимые столбцы пересчитываются. Из состава исключаются группы котельных; остальные показатели рассчитываются по той же схеме, что и на листе с котельными.

**Взаимодействие и источники данных**

Источники: параметры анализа PGParamCalc/ZVParamCalc, отчетные PGReportGisTek/ZVReportGisTek, показатели групп оборудования из «Топлива» и сведения о станциях из «Генерации». Параметры анализа хранятся отдельно для каждого sheet_slug, кода группы, года и версии БД. При первом заполнении недостающие строки копируются из отчетов без перезаписи существующих исправлений. Разметка столбцов и исходный макет читаются из Excel; числовые значения дополняются данными БД. Итоги используются в агрегированных представлениях.

**Элементы реализации**

Модель ZVParamCalc. Источник: app/ecology/models/zv_param_calc_model.py, строки 11–21.

```
11  class ZVParamCalc(db.Model, AuditMixin):
12      """Строка параметров страниц /ecology/analysis/zv-vse/ и zv-bez-kot/.
13
14      Первоначально копируется из ZVReportGisTek; далее может редактироваться
15      отдельно от отчетных данных.
16      Связь с EquipmentGroup: station_numb (numb1120) = EquipmentGroup.numb
17      (без FK на numb – поле не уникально).
18      year_number + database_version_id → Year.number + Year.database_version_id.
19      """
20
21      __tablename__ = "gs_eco_analysis_zv_params"
```

Маршрут страницы. Источник: app/ecology/routes/gis_tes_fuel_comparison_routes.py, строки 43–66.

```
43  @ecology_bp.route("/analysis/<sheet_slug>/")
44  @login_required
45  def gis_tes_fuel_comparison(sheet_slug: str):
46      if request.args.get("reset_filters") == "1":
47          keep = {
48              key: value
49              for key in (
50                  "start_year",
51                  "end_year",
52                  "rounding_digits",
53                  "emission_rounding_digits",
54                  "specific_rounding_digits",
55                  "per_page",
56                  "show_totals",
57                  "show_empty_rows",
58                  "show_reported",
59              )
60              if (value := request.args.get(key)) not in (None, "")
61          }
62          if "norm_zv" in request.args:
63              keep["norm_zv"] = request.args.getlist("norm_zv")
64          return redirect(
65              url_for("ecology_bp.gis_tes_fuel_comparison", sheet_slug=sheet_slug, **keep)
66          )
```

Сервис и расчетная логика. Источник: app/ecology/services/gis_tes_fuel_comparison_services.py, строки 2200–2222.

```
2200  def _compute_zv_specific_metrics(
2201      filled: list[Any],
2202      source_columns: list[dict[str, Any]],
2203      norm_zv_parts: list[str] | tuple[str, ...] = ZV_NORM_DEFAULT_PARTS,
2204  ) -> list[Any]:
2205      """Приведенная энергия и удельные ЗВ: на энергию, на выработку и на топливо."""
2206      out = list(filled)
2207      solid = _filled_at(out, source_columns, {ZV_SOLID_KEY})
2208      so2 = _filled_at(out, source_columns, {"SO2_gaz"})
2209      nox = _filled_at(out, source_columns, {"NOx_gaz"})
2210      zv_normir = _safe_sum(*(
2211          _filled_at(out, source_columns, {key})
2212          for key, attr in ZV_ECOLOGY_OVERLAY_BY_KEY.items()
2213          if attr in norm_zv_parts
2214      ))
2215      _assign_filled(out, source_columns, "ZV_normir", zv_normir)
2216
2217      w_en = _filled_at(out, source_columns, {"W_en"})
2218      teplo = _filled_at(out, source_columns, {"Teplo"})
2219      energy = _brought_energy_mln_kwh(w_en, teplo)
2220      _assign_filled(out, source_columns, "energy", energy)
2221
2222      total_zv = _filled_at(out, source_columns, {ZV_TOTAL_KEY})
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/analysis/gis_tes_fuel_comparison.html, строки 588–610.

```
 588            class="table table-hover table-striped table-bordered power-demand-summary-table{% if not
show_empty_rows|default(false) %} ot-hide-empty{% endif %}"
 589            id="gisTesFuelTable"
 590            data-sheet-slug="{{ sheet.slug }}"
 591            data-can-edit="{{ '1' if can_edit|default(false) else '0' }}"
 592            data-update-url="{% if can_edit|default(false) %}{{
url_for('ecology_bp.gis_tes_fuel_comparison_update_value', sheet_slug=sheet.slug) }}{% endif %}"
 593            data-rounding-digits="{{ rounding_digits }}"
 594            data-emission-rounding-digits="{{ emission_rounding_digits }}"
 595            data-specific-rounding-digits="{{ specific_rounding_digits }}"
 596          >
 597            <thead class="table-success">
 598              {% for header_row in header_rows %}
 599              {% set row_is_key = header_row and header_row[0].is_key %}
 600              <tr class="text-center align-middle{% if row_is_key %} cmp-key-row{% endif %}">
 601              {% for cell in header_row %}
 602              {% if cell.column_key == 'numb1120' and not cell.is_key %}
 603              {{ fuel_param_numb1120_header_th(
 604                   cell.rowspan if cell.rowspan else 1,
 605                   'Код группы оборудования',
 606                   numb1120_filter|default([]),
 607                   numb1120_filter_choices|default([]),
 608                   extra_class='cmp-sticky text-center align-middle gis-tes-filter-th',
 609                   sticky_col=cell.col_index
 610                 ) }}
```

Обновление ячейки анализа. Источник: app/ecology/routes/gis_tes_fuel_comparison_routes.py, строки 153–175.

```
@ecology_bp.route("/analysis/<sheet_slug>/value/", methods=["PATCH"])
@login_required
def gis_tes_fuel_comparison_update_value(sheet_slug: str):
    if not can_edit_ecology(current_user):
        return jsonify(ok=False, error="Недостаточно прав"), 403
    if sheet_slug not in ANALYSIS_SHEET_SLUGS:
        return jsonify(ok=False, error="Лист только для просмотра"), 400
    data = request.get_json(silent=True) or {}
    column_key = str(data.get("field") or "").strip()
    try:
        station_numb = int(data.get("numb"))
        year_number = int(data.get("year"))
    except (TypeError, ValueError):
        return jsonify(ok=False, error="Некорректные станция или год"), 400
    raw = data.get("value")
    try:
        amount = parse_decimal_from_display(raw) if raw not in (None, "") else None
    except Exception:
        return jsonify(ok=False, error="Некорректное число"), 400
    try:
        values = save_analysis_ecology_cell(
            sheet_slug,
            station_numb,
```

## 10. Расчет по действующим ТЭС

Территориальные разрезы обычных расчетных форм: Всего; станции отрасли; промпредприятия; ОЭС Северо-Запада, Центра, Средней Волги, Юга, Урала, Сибири и Востока; ТИТЭС Сибири и Востока; децентрализованная зона. На коэффициентах используются отдельные территориальные ключи tites-sibir, tites-vostok и decentralized-zone, а принадлежность выбирается параметром affiliation. Большинство годовых форм показывают территории столбцами, коэффициенты показывают выбранную территорию и годы столбцами.

Каталоги территорий и принадлежностей. Источник: app/ecology/services/operating_tes_constants.py, строки 20–47.

```
  20  TERRITORIES: tuple[dict[str, Any], ...] = (
  21      {'slug': 'vsego', 'sheet_name': 'Всего', 'group': 'territory'},
  22      {'slug': 'minen', 'sheet_name': 'Станции отрасли', 'group': 'special'},
  23      {'slug': 'prompred', 'sheet_name': 'Пром-предприятия', 'group': 'special'},
  24      {'slug': 'severo-zapad', 'sheet_name': 'ОЭС Северо-Запада', 'group': 'territory'},
  25      {'slug': 'centr', 'sheet_name': 'ОЭС Центра', 'group': 'territory'},
  26      {'slug': 'srednyaya-volga', 'sheet_name': 'ОЭС Средней Волги', 'group': 'territory'},
  27      {'slug': 'yug', 'sheet_name': 'ОЭС Юга', 'group': 'territory'},
  28      {'slug': 'ural', 'sheet_name': 'ОЭС Урала', 'group': 'territory'},
  29      {'slug': 'sibir', 'sheet_name': 'ОЭС Сибири', 'group': 'territory'},
  30      {'slug': 'vostok', 'sheet_name': 'ОЭС Востока', 'group': 'territory'},
  31      {'slug': 'norilsk', 'sheet_name': 'ТИТЭС Сибири', 'group': 'special'},
  32      {'slug': 'isolated-dv', 'sheet_name': 'ТИТЭС Востока', 'group': 'special'},
  33      {'slug': 'nikolaevsky', 'sheet_name': 'Децентрализованная зона (Николаевский энергорайон)',
'group': 'special'},
  34  )
  35
  36  COEFFICIENT_TERRITORIES: tuple[dict[str, Any], ...] = (
  37      *(item for item in TERRITORIES if item["slug"] not in {
  38          "minen", "prompred", "isolated-dv", "nikolaevsky", "norilsk"
  39      }),
  40      {"slug": "tites-sibir", "sheet_name": "ТИТЭС Сибири"},
  41      {"slug": "tites-vostok", "sheet_name": "ТИТЭС Востока"},
  42      {"slug": "decentralized-zone", "sheet_name": "Децентрализованная зона (Николаевский
энергорайон)"},
  43  )
  44  COEFFICIENT_AFFILIATIONS = (
  45      {"slug": "minen", "label": "Станции отрасли"},
  46      {"slug": "prompred", "label": "Промпредприятия"},
  47  )
```

### 10.1. Данные по модулю Топливо

Адрес: /ecology/operating_tes/vsego/toplivo/

**Функциональные возможности и действия пользователя**

Просмотр принятых топливных показателей действующих ТЭС: выработка электроэнергии, отпуск тепла, расход топлива и его видовой состав по территориям. Выбираются период и год; индикатор новых данных ведет к рассмотрению переданного пакета. Эта вкладка не разрешает прямую правку показателей. Общие средства: выбор периода, отображаемого года там, где предусмотрен годовой лист, округление, переход к другим листам и отбор отображаемых строк или территорий. Выгрузка действующих ТЭС собирает расчетные вкладки за выбранные годы.

**Взаимодействие и источники данных**

Расход топлива и энергетические показатели поступают из принятых снимков «Топлива», а не из еще не рассмотренных пакетов. Физические характеристики и коэффициенты берутся из моделей «Экологии» с учетом года и версии БД. Территории согласованы со справочниками ОЭС и группами станций. Полученные результаты входят в сводные расчеты действующих и новых ТЭС.

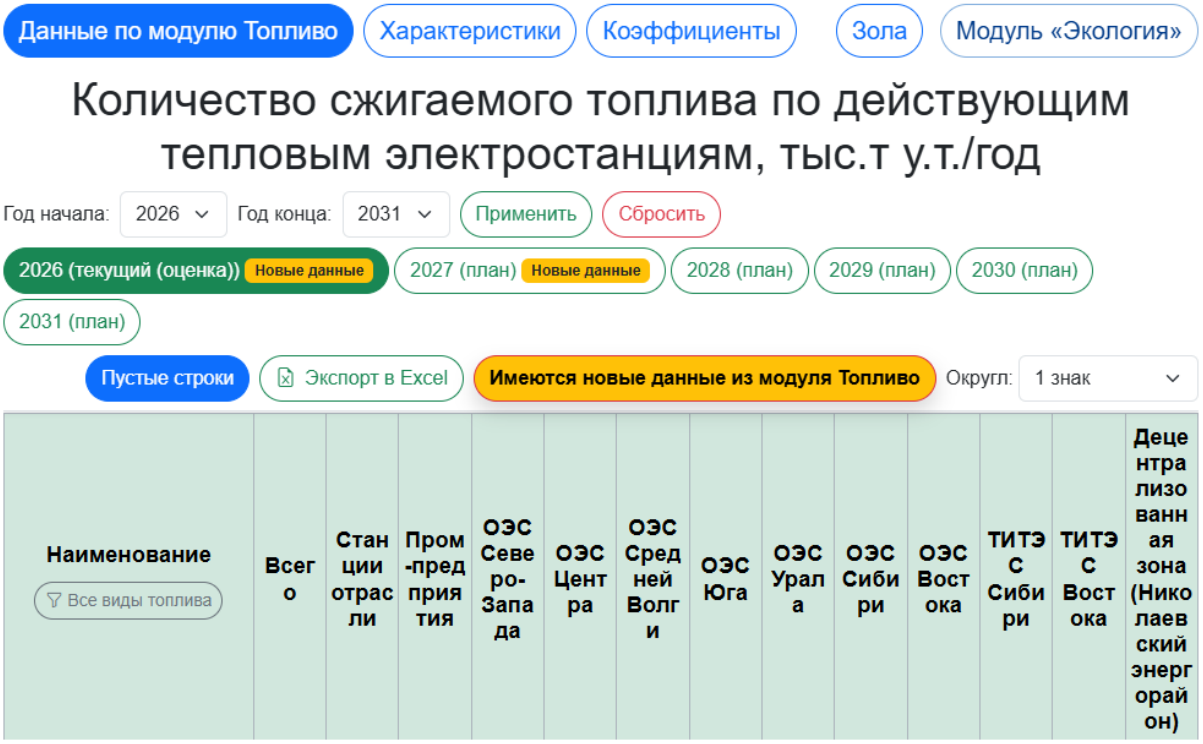



Рисунок 12. Принятые данные по модулю «Топливо» для действующих ТЭС. Действующий сайт, 29.10.2026. Показан фрагмент экранной формы.

## Элементы реализации

Модель EcologyFuelSnapshot. Источник: app/ecology/models/fuel_exchange_model.py, строки 9–21.

```
  9  class EcologyFuelSnapshot(db.Model, AuditMixin):
 10      __tablename__ = "gs_eco_fuel_snapshots"
 11      __table_args__ = (
 12          db.UniqueConstraint("database_version_id", "year_number", "scope",
name="uq_eco_fuel_snapshot_version_year_scope"),
 13          {"schema": SCHEMA_ECOLOGY},
 14      )
 15
 16      id = db.Column(db.Integer, primary_key=True)
 17      database_version_id = db.Column(db.Integer,
db.ForeignKey(f"{SCHEMA_REFDATA}.gs_database_versions.id", ondelete="CASCADE"), nullable=False)
 18      year_number = db.Column(db.Integer, nullable=False)
 19      scope = db.Column(db.String(20), nullable=False, default="operating",
server_default="operating")
 20      payload = db.Column(db.JSON, nullable=False)
 21      updated_at = db.Column(db.DateTime(timezone=True), server_default=func.now(),
onupdate=func.now(), nullable=False)
```

Маршрут страницы. Источник: app/ecology/routes/operating_tes_routes.py, строки 61–77.

```
 61  @ecology_bp.route("/operating_tes/<territory_slug>/<sheet_slug>/")
 62  @login_required
 63  def operating_tes_table(territory_slug: str, sheet_slug: str):
 64      context = build_operating_tes_table_context(
 65          territory_slug,
 66          sheet_slug,
 67          start_year=request.args.get("start_year"),
 68          end_year=request.args.get("end_year"),
 69          rounding_digits=request.args.get("rounding_digits"),
 70          coefficient_rounding_digits=request.args.get("coefficient_rounding_digits"),
 71          specific_rounding_digits=request.args.get("specific_rounding_digits"),
 72          view_year=request.args.get("view_year"),
 73          affiliation=request.args.get("affiliation", "all"),
 74      )
 75      if context is None:
 76          abort(404)
 77      sheet = context.get("sheet") or {}
```

Сервис и расчетная логика. Источник: app/ecology/services/fuel_exchange_services.py, строки 34–41.

```
  34  def load_accepted_fuel_inputs(years, scope="operating"):
  35      rows = EcologyFuelSnapshot.query.filter_by(database_version_id=_version_id(),
scope=scope).filter(
  36          EcologyFuelSnapshot.year_number.in_(years)
  37      ).all()
  38      result = {year: {} for year in years}
  39      for row in rows:
  40          result[row.year_number] = decode_inputs(row.payload)
  41      return result
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/operating_tes/operating_tes_table.html, строки 787–820.

```
 787            <th class="text-center align-middle ot-label-col">Наименование
 788              {% if sheet.layout in ('fuel', 'coeff') %}
 789              {% include 'ecology/operating_tes/_fuel_filter.html' %}
 790              {% endif %}
 791              {% if territory_filter_storage_key %}
 792              {% include 'ecology/operating_tes/_territory_filter.html' %}
 793              {% endif %}
 794            </th>
 795            {% for col in territory_columns %}
 796            <th class="text-center align-middle ot-terr-col">
 797              {{ col.sheet_name }}{% if sheet.layout == 'coeff' %}
 798              <div class="small">{% if affiliation == 'minen' %}Станции отрасли{% elif affiliation ==
'prompred' %}Промпредприятия{% else %}Все принадлежности{% endif %}, {{ view_year }}</div>
 799              {% endif %}
 800            </th>
 801            {% endfor %}
 802          </tr>
 803        </thead>
 804        <tbody>
 805          {% for row in rows %}
 806          <tr
 807            class="{{ row.css_class }}{% if row.is_empty %} ot-row-empty-hidden{% endif %}"
 808            data-row-key="{{ row.key }}"
 809            data-editable-field="{{ row.editable_field or '' }}"
 810          >
 811            <td
 812              class="text-start ot-row-label{% if row.indent %} ot-indent-{{ row.indent }}{% endif
%}{% if row.kind == 'total' %} fw-bold{% endif %}{% if row.italic %} fst-italic{% endif %}"
 813            >
 814              {{ row.label }}{% if row.key in ('ee_generation', 'heat_output') and row.unit %}, {{
row.unit }}{% endif %} {% if row.formula_tooltip %}
 815              <i
 816                class="bi bi-info-circle text-primary ms-1"
 817                data-bs-toggle="tooltip"
 818                data-bs-placement="top"
 819                {% if row.formula_tooltip_html %}data-bs-html="true" title="{{
row.formula_tooltip_html|forceescape }}"{% else %}title="{{ row.formula_tooltip|e }}"{% endif %}
 820                style="cursor: help; font-size: 0.9em"
```

Формирование выбранного расчетного листа. Источник: app/ecology/services/operating_tes_page_services.py, строки 789–798.

```
 789      if layout == "fuel":
 790          if fuel_source:
 791              from app.ecology.services.fuel_summary_services import build_fuel_summary
 792
 793              summary, errors = build_fuel_summary(years)
 794              inputs = summary[fuel_scope]
 795              fuel_summary_errors = errors[fuel_scope]
 796          else:
 797              inputs = load_accepted_fuel_inputs(years)
 798      elif layout == "coeff":
```

## 10.2. Характеристики топлив и продуктов сгорания

Адрес: /ecology/operating_tes/vsego/khar-topl/

**Функциональные возможности и действия пользователя**

Справочная таблица содержит коэффициент перевода условного топлива в натуральное, низшую теплоту сгорания, зольность, сернистость, доли уноса и связывания, потери от недожога и коэффициенты CO2. Уполномоченный

пользователь меняет значения по виду топлива и году. Характеристики общие для территорий и принадлежностей; локальный территориальный фильтр не создает отдельного набора физических свойств. Общие средства: выбор периода, отображаемого года там, где предусмотрен годовой лист, округление, переход к другим листам и отбор отображаемых строк или территорий. Выгрузка действующих ТЭС собирает расчетные вкладки за выбранные годы.

**Взаимодействие и источники данных**

Расход топлива и энергетические показатели поступают из принятых снимков «Топлива», а не из еще не рассмотренных пакетов. Физические характеристики и коэффициенты берутся из моделей «Экологии» с учетом года и версии БД. Территории согласованы со справочниками ОЭС и группами станций. Полученные результаты входят в сводные расчеты действующих и новых ТЭС.

**Элементы реализации**

Модель OperatingTesFuelCharacteristic. Источник: app/ecology/models/operating_tes_fuel_characteristic_model.py, строки 11–14.

```
11  class OperatingTesFuelCharacteristic(db.Model, AuditMixin):
12      """Общие для всех уровней характеристики топлива по году и версии БД."""
13
14      __tablename__ = "gs_eco_operating_tes_fuel_chars"
```

Маршрут страницы. Источник: app/ecology/routes/operating_tes_routes.py, строки 61–77.

```
61  @ecology_bp.route("/operating_tes/<territory_slug>/<sheet_slug>/")
62  @login_required
63  def operating_tes_table(territory_slug: str, sheet_slug: str):
64      context = build_operating_tes_table_context(
65          territory_slug,
66          sheet_slug,
67          start_year=request.args.get("start_year"),
68          end_year=request.args.get("end_year"),
69          rounding_digits=request.args.get("rounding_digits"),
70          coefficient_rounding_digits=request.args.get("coefficient_rounding_digits"),
71          specific_rounding_digits=request.args.get("specific_rounding_digits"),
72          view_year=request.args.get("view_year"),
73          affiliation=request.args.get("affiliation", "all"),
74      )
75      if context is None:
76          abort(404)
77      sheet = context.get("sheet") or {}
```

Сервис и расчетная логика. Источник: app/ecology/services/operating_tes_page_services.py, строки 383–399.

```
383  def resolve_chars(
384      fuel_key: str,
385      year: int,
386      overrides: dict[tuple[str, str, int], dict[str, Any]],
387      territory_slug: str = DEFAULT_TERRITORY_SLUG,
388  ) -> dict[str, Any]:
389      base = dict(DEFAULT_CHARACTERISTICS.get(fuel_key) or {})
390      if fuel_key in DEFAULT_CHAR_NOTES:
391          base["note"] = DEFAULT_CHAR_NOTES[fuel_key]
392      if fuel_key in DEFAULT_CHAR_SOURCES:
393          base["source_key"] = DEFAULT_CHAR_SOURCES[fuel_key]
394      # Характеристики общие для всех территорий и принадлежностей.
395      extra = overrides.get((DEFAULT_TERRITORY_SLUG, fuel_key, year)) or {}
396      for key, value in extra.items():
397          if value is not None:
398              base[key] = value
399      return base
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/operating_tes/operating_tes_table.html, строки 548–584.

```
548        {% elif sheet.layout == 'chars' %}
549        <colgroup>
550          <col class="ot-col-label" />
551          {% for col in char_columns %}
552          <col />
553          {% endfor %}
554        </colgroup>
555        <thead class="table-success">
556          <tr class="text-center align-middle">
557            <th rowspan="2" scope="col" class="text-center align-middle ot-label-col">
558              Вид топлива
559              {% include 'ecology/operating_tes/_fuel_filter.html' %}
560            </th>
561            {% for group in char_header_groups %}
562            <th colspan="{{ group['keys']|length }}" scope="{{ 'colgroup' if group['keys']|length > 1
else 'col' }}"{% if group.rowspan %} rowspan="{{ group.rowspan }}"{% endif %} class="text-center align-
middle ot-char-title">
563              {{ group.title }}
564              {% if new_tes_capacity_tabs and 'k_co2' in group['keys'] %}
565              <button type="button" class="btn btn-link p-0 text-primary ms-1"
566                      data-bs-toggle="tooltip" data-bs-placement="top"
567                      title="Коэффициент ПГ общий с КСО2 действующих ТЭС."
568                      aria-label="Информация о коэффициенте ПГ">
569                <i class="bi bi-info-circle" aria-hidden="true"></i>
570              </button>
571              {% endif %}
572            </th>
573            {% endfor %}
574          </tr>
575          <tr class="text-center align-middle">
576            {% set char_symbols = {
577              'ash': 'A<sup>p</sup>, %',
578              'ash_share': 'a<sub>ун</sub>',
579              'q_mech': 'q<sub>4</sub>, %',
580              'sulfur': 'S<sup>p</sup>, %',
581              'qh_kj': 'Q<sub>н</sub>, кДж/кг',
582              'qh_kcal': 'Q<sub>н</sub>, ккал/кг',
583              'bj': 'b<sub>j</sub>',
584              'k_co2': 'K<sub>CO<sub>2</sub></sub>',
```

Формирование выбранного расчетного листа. Источник: app/ecology/services/operating_tes_page_services.py, строки 881–906.

```
881      elif layout == "chars":
882          for spec in _characteristic_row_specs():
883              is_header = spec["kind"] == KIND_TOTAL
884              is_group = is_header and not DEFAULT_CHARACTERISTICS.get(spec["key"])
885              values_by_year = {}
886              raw_values = []
887              for year in years:
888                  chars = resolve_chars(spec["key"], year, char_overrides, territory_slug)
889                  values_by_year[year] = {
890                      field: "" if is_group else format_cell(chars.get(field), digits=digits)
891                      for field, _label in CHAR_COLUMNS
892                  }
893                  raw_values.extend(None if is_group else chars.get(field) for field, _label in
CHAR_COLUMNS)
894              rows.append({
895                  "key": spec["key"],
896                  "label": spec["label"],
897                  "kind": spec["kind"],
898                  "indent": spec.get("indent") or 0,
899                  "italic": bool(spec.get("indent")),
900                  "css_class": "table-info fw-bold" if not spec.get("indent") else (
901                      "fw-bold" if spec["indent"] == 1 or is_header else ""
902                  ),
903                  "editable_values": not is_group,
904                  "char_values": values_by_year,
905                  "is_empty": False if is_header else not any(value is not None for value in
raw_values),
906              })
```

### 10.3. Коэффициенты очистки и подавления

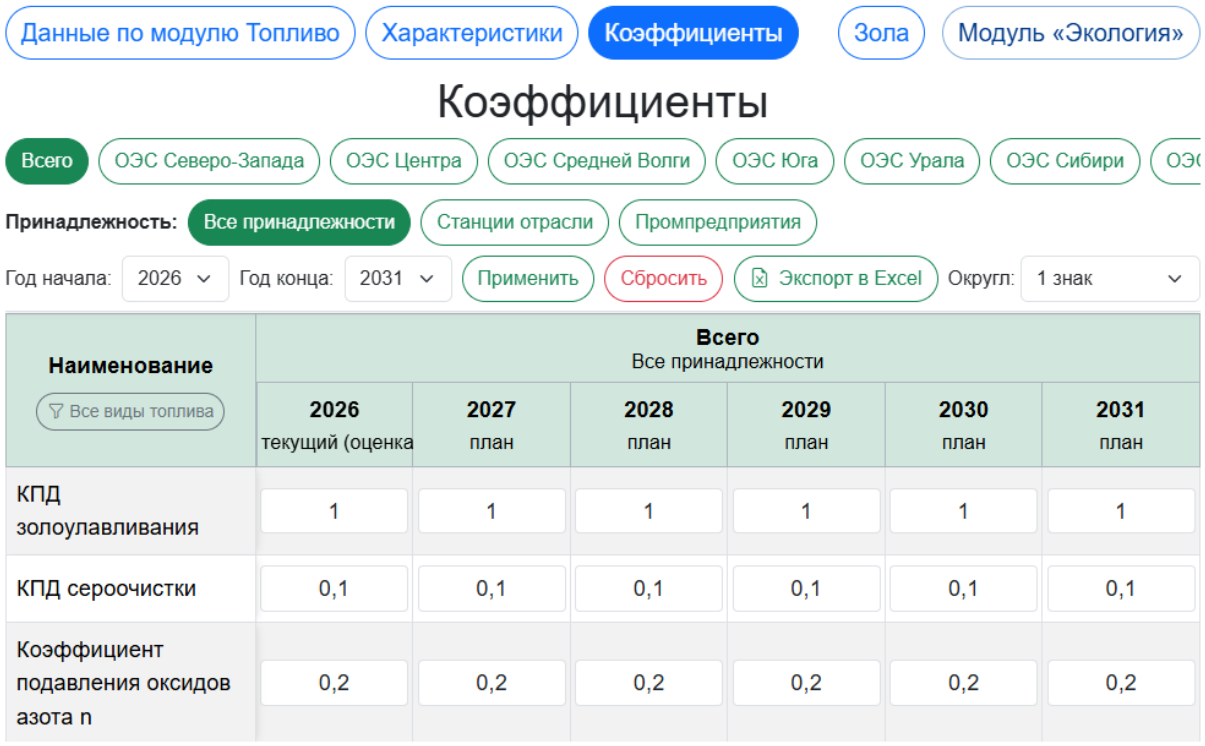
Адрес: /ecology/operating_tes/vsego/koeff/

**Функциональные возможности и действия пользователя**

Матрица коэффициентов содержит КПД золоулавливания и сероочистки, коэффициенты подавления NOx, CO и ПГ. Столбцы соответствуют годам выбранного периода. Территория и принадлежность выбираются независимо; доступны все принадлежности, станции отрасли и промпредприятия. При отсутствии сохраненного значения применяется предусмотренное кодом значение по умолчанию. Правка ячейки сохраняется через PATCH. Общие средства: выбор периода, отображаемого года там, где предусмотрен годовой лист, округление, переход к другим листам и отбор отображаемых строк или территорий. Выгрузка действующих ТЭС собирает расчетные вкладки за выбранные годы.

**Взаимодействие и источники данных**

Расход топлива и энергетические показатели поступают из принятых снимков «Топлива», а не из еще не рассмотренных пакетов. Физические характеристики и коэффициенты берутся из моделей «Экологии» с учетом года и версии БД. Территории согласованы со справочниками ОЭС и группами станций. Полученные результаты входят в сводные расчеты действующих и новых ТЭС.




| Наименование | Всего Все принадлежности | | | | | |
|---|---|---|---|---|---|---|
| Все виды топлива | 2026 текущий (оценка | 2027 план | 2028 план | 2029 план | 2030 план | 2031 план |
| КПД золоулавливания | 1 | 1 | 1 | 1 | 1 | 1 |
| КПД сероочистки | 0,1 | 0,1 | 0,1 | 0,1 | 0,1 | 0,1 |
| Коэффициент подавления оксидов азота n | 0,2 | 0,2 | 0,2 | 0,2 | 0,2 | 0,2 |

Рисунок 13. Коэффициенты очистки и подавления: территория, принадлежность и годы. Действующий сайт, 29.10.2026. Показан фрагмент экранной формы.

**Элементы реализации**

Модель OperatingTesCoefficient. Источник: app/ecology/models/operating_tes_coefficient_model.py, строки 11–14.

```
11  class OperatingTesCoefficient(db.Model, AuditMixin):
12      """КПД / коэффициент подавления по территории, году и виду."""
13
14      __tablename__ = "gs_eco_operating_tes_coefficients"
```

Маршрут страницы. Источник: app/ecology/routes/operating_tes_routes.py, строки 61–77.

```
@ecology_bp.route("/operating_tes/<territory_slug>/<sheet_slug>/")
@login_required
def operating_tes_table(territory_slug: str, sheet_slug: str):
    context = build_operating_tes_table_context(
        territory_slug,
        sheet_slug,
        start_year=request.args.get("start_year"),
        end_year=request.args.get("end_year"),
        rounding_digits=request.args.get("rounding_digits"),
        coefficient_rounding_digits=request.args.get("coefficient_rounding_digits"),
        specific_rounding_digits=request.args.get("specific_rounding_digits"),
        view_year=request.args.get("view_year"),
        affiliation=request.args.get("affiliation", "all"),
    )
    if context is None:
        abort(404)
    sheet = context.get("sheet") or {}
```

Сервис и расчетная логика. Источник: app/ecology/services/operating_tes_page_services.py, строки 1126–1148.

```
def upsert_coefficient(
    territory_slug: str, year: int, kind: str, value: Any, affiliation: str = "all"
) -> OperatingTesCoefficient:
    if get_territory(territory_slug) is None and get_coefficient_territory(territory_slug) is None:
        raise ValueError("Неизвестная территория")
    if affiliation not in {"all", "minen", "prompred"}:
        raise ValueError("Неизвестная принадлежность")
    if kind not in {item["key"] for item in COEFF_KINDS}:
        raise ValueError("Неизвестный коэффициент")
    # Подписи для журнала – до нормализации срезов принадлежности.
    log_context = context_lines(
        ("лист", "Коэффициенты"),
        ("территория", territory_label(territory_slug)),
        ("принадлежность", affiliation_label(affiliation)),
        ("год", int(year)),
    )
    log_action = f"Коэффициенты – {territory_label(territory_slug)}"
    # Сохраняем прежние общие срезы принадлежности, используемые на «Объемах».
    if territory_slug == "vsego" and affiliation != "all":
        territory_slug, affiliation = affiliation, "all"
    elif territory_slug in {"minen", "prompred"} and affiliation != "all":
        raise ValueError("Для принадлежности необходимо выбрать территорию")
    version_id = get_current_db_version_id()
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/operating_tes/operating_tes_table.html, строки 797–820.

```
            {{ col.sheet_name }}{% if sheet.layout == 'coeff' %}
            <div class="small">{% if affiliation == 'minen' %}Станции отрасли{% elif affiliation ==
'prompred' %}Промпредприятия{% else %}Все принадлежности{% endif %}, {{ view_year }}</div>
            {% endif %}
          </th>
          {% endfor %}
        </tr>
      </thead>
      <tbody>
        {% for row in rows %}
        <tr
          class="{{ row.css_class }}{% if row.is_empty %} ot-row-empty-hidden{% endif %}"
          data-row-key="{{ row.key }}"
          data-editable-field="{{ row.editable_field or '' }}"
        >
          <td
            class="text-start ot-row-label{% if row.indent %} ot-indent-{{ row.indent }}{% endif
%}{% if row.kind == 'total' %} fw-bold{% endif %}{% if row.italic %} fst-italic{% endif %}"
          >
            {{ row.label }}{% if row.key in ('ee_generation', 'heat_output') and row.unit %}, {{
row.unit }}{% endif %} {% if row.formula_tooltip %}
            <i
              class="bi bi-info-circle text-primary ms-1"
              data-bs-toggle="tooltip"
              data-bs-placement="top"
              {% if row.formula_tooltip_html %}data-bs-html="true" title="{{
row.formula_tooltip_html|forceescape }}"{% else %}title="{{ row.formula_tooltip|e }}"{% endif %}
              style="cursor: help; font-size: 0.9em"
```

Формирование выбранного расчетного листа. Источник: app/ecology/services/operating_tes_page_services.py, строки 798–811.

```
798     elif layout == "coeff":
799         coeff_overrides = load_coefficient_overrides(years, affiliation)
800         default_slug = (
801             affiliation if territory_slug == "vsego" and affiliation != "all" else territory_slug
802         )
803         for column in value_columns:
804             year = column["year"]
805             bundles_by_terr[column["slug"]] = {"coeffs": {
806                 kind["key"]: coeff_overrides.get(
807                     (territory_slug, year, kind["key"]),
808                     Decimal(str(default_coefficient(year, default_slug, kind["key"]))),
809                 )
810                 for kind in COEFF_KINDS
811             }}
```

## 10.4. Зола: выбросы твердых веществ

Адрес: /ecology/operating_tes/vsego/zola/

**Функциональные возможности и действия пользователя**

Для каждого вида топлива показаны перевод расхода в натуральное выражение по теплоте сгорания и масса отходящих твердых веществ. Формула учитывает долю уноса золы, зольность, механический недожог и теплоту сгорания. Итоговый блок показывает сумму Мотх., КПД улавливания и массу выброса Мвыб. Пользователь просматривает результат, а исходные свойства и КПД меняет на специальных вкладках. Общие средства: выбор периода, отображаемого года там, где предусмотрен годовой лист, округление, переход к другим листам и отбор отображаемых строк или территорий. Выгрузка действующих ТЭС собирает расчетные вкладки за выбранные годы.

**Взаимодействие и источники данных**

Расход топлива и энергетические показатели поступают из принятых снимков «Топлива», а не из еще не рассмотренных пакетов. Физические характеристики и коэффициенты берутся из моделей «Экологии» с учетом года и версии БД. Территории согласованы со справочниками ОЭС и группами станций. Полученные результаты входят в сводные расчеты действующих и новых ТЭС.

**Элементы реализации**

Модель OperatingTesFuelCharacteristic. Источник: app/ecology/models/operating_tes_fuel_characteristic_model.py, строки 11–14.

```
11  class OperatingTesFuelCharacteristic(db.Model, AuditMixin):
12      """Общие для всех уровней характеристики топлива по году и версии БД."""
13
14      __tablename__ = "gs_eco_operating_tes_fuel_chars"
```

Модель OperatingTesCoefficient. Источник: app/ecology/models/operating_tes_coefficient_model.py, строки 11–14.

```
11  class OperatingTesCoefficient(db.Model, AuditMixin):
12      """КПД / коэффициент подавления по территории, году и виду."""
13
14      __tablename__ = "gs_eco_operating_tes_coefficients"
```

Маршрут страницы. Источник: app/ecology/routes/operating_tes_routes.py, строки 61–77.

```
@ecology_bp.route("/operating_tes/<territory_slug>/<sheet_slug>/")
@login_required
def operating_tes_table(territory_slug: str, sheet_slug: str):
    context = build_operating_tes_table_context(
        territory_slug,
        sheet_slug,
        start_year=request.args.get("start_year"),
        end_year=request.args.get("end_year"),
        rounding_digits=request.args.get("rounding_digits"),
        coefficient_rounding_digits=request.args.get("coefficient_rounding_digits"),
        specific_rounding_digits=request.args.get("specific_rounding_digits"),
        view_year=request.args.get("view_year"),
        affiliation=request.args.get("affiliation", "all"),
    )
    if context is None:
        abort(404)
    sheet = context.get("sheet") or {}
```

Сервис и расчетная логика. Источник: app/ecology/services/operating_tes_calc_services.py, строки 44–54.

```
def ash_motkh(bn: Any, chars: Mapping[str, Any] | None) -> Decimal | None:
    """Мотх золы = 0,01·Вн·(аун·Ар + qч·QH/32680)."""
    bn_d = _d(bn)
    if bn_d is None or not chars:
        return None
    aun = _d(chars.get("ash_share")) or Decimal("0")
    ap = _d(chars.get("ash")) or Decimal("0")
    qch = _d(chars.get("q_mech")) or Decimal("0")
    qh = _d(chars.get("qh_kj")) or Decimal("0")
    inner = aun * ap + qch * qh / Q_H_DIVISOR
    return Decimal("0.01") * bn_d * inner
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/operating_tes/operating_tes_table.html, строки 683–711.

```
            {% endif %}
          </th>
          {% endfor %}
        </tr>
      </thead>
      <tbody>
        {% for row in rows %}
        <tr
          class="{{ row.css_class }}{% if row.is_empty %} ot-row-empty-hidden{% endif %}"
          data-row-key="{{ row.key }}"
        >
          <td
            class="text-start ot-row-label{% if row.indent %} ot-indent-{{ row.indent }}{% endif
%}{% if row.kind == 'total' %} fw-bold{% endif %}{% if row.italic %} fst-italic{% endif %}"
          >
            {{ emission_row_label(row) }}
          </td>
          {% for col in value_columns %}
          {% if row.span_pair %}
          <td colspan="2" class="text-center ot-pair-cell fw-bold" data-column="{{ col.slug }}">
            {% include 'ecology/operating_tes/_emission_summary_value.html' %}
          </td>
          {% else %}
          {% set pair =
          row.pair_values[col.slug] if row.pair_values is defined and col.slug
          in row.pair_values else none %}
          <td
            class="text-center ot-pair-cell{% if row.kind == 'total' %} fw-bold{% endif %}"
          >
            {{ pair.left if pair else '' }}
```

Формирование выбранного расчетного листа. Источник: app/ecology/services/operating_tes_page_services.py, строки 929–956.

```
929      elif layout in {"emission", "emission_tj"}:
930          view = EMISSION_SHEET_VIEW[sheet["slug"]]
931          if sheet["slug"] in {"zola", "sera", "co", "azot", "pg", "pg-tj"}:
932              prefix, eta_key, eta_label = {
933                  "zola": ("ash", "ash_eta", "КПД золоулавливания"),
934                  "sera": ("so2", "sulfur_eta", "КПД сероочистки"),
935                  "co": ("co", "co_k", "Коэффициент подавления CO"),
936                  "azot": ("nox", "nox_n", "Коэффициент подавления оксидов азота в топке n"),
937                  "pg": ("pg", "pg_k", "Коэффициент подавления ПГ (оксидов углерода)"),
938                  "pg-tj": ("pg_tj", "pg_k", "Коэффициент подавления ПГ (оксидов углерода)"),
939              }[sheet["slug"]]
940              total_key = f"{prefix}_motkh_total"
941              emitted_key = f"{prefix}_emitted"
942              summary_values = {}
943              for column in value_columns:
944                  slug = column["slug"]
945                  bundle = bundles_by_terr[slug]
946                  motkh = bundle[total_key]
947                  eta = bundle["coeffs"][eta_key]
948                  summary_values[slug] = {
949                      total_key: motkh,
950                      eta_key: eta,
951                      emitted_key: apply_capture(motkh, eta),
952                  }
953              for key, label in (
954                  (total_key, "ΣМотх."),
955                  (eta_key, eta_label),
956                  (emitted_key, "Мвыб."),
```

## 10.5. Сера: выбросы диоксида серы

Адрес: /ecology/operating_tes/vsego/sera/

**Функциональные возможности и действия пользователя**

Рассчитывается масса отходящего SO2 по натуральному расходу топлива, содержанию серы и доле ее связывания в золе. Натуральный расход для этого листа определяется по теплоте сгорания. Итоговая масса выброса учитывает КПД сероочистки. Расчетные ячейки доступны для просмотра, а не прямого ввода. Общие средства: выбор периода, отображаемого года там, где предусмотрен годовой лист, округление, переход к другим листам и отбор отображаемых строк или территорий. Выгрузка действующих ТЭС собирает расчетные вкладки за выбранные годы.

**Взаимодействие и источники данных**

Расход топлива и энергетические показатели поступают из принятых снимков «Топлива», а не из еще не рассмотренных пакетов. Физические характеристики и коэффициенты берутся из моделей «Экологии» с учетом года и версии БД. Территории согласованы со справочниками ОЭС и группами станций. Полученные результаты входят в сводные расчеты действующих и новых ТЭС.

**Элементы реализации**

Модель OperatingTesFuelCharacteristic. Источник: app/ecology/models/operating_tes_fuel_characteristic_model.py, строки 11–14.

```
11  class OperatingTesFuelCharacteristic(db.Model, AuditMixin):
12      """Общие для всех уровней характеристики топлива по году и версии БД."""
13
14      __tablename__ = "gs_eco_operating_tes_fuel_chars"
```

Модель OperatingTesCoefficient. Источник: app/ecology/models/operating_tes_coefficient_model.py, строки 11–14.

```
11  class OperatingTesCoefficient(db.Model, AuditMixin):
12      """КПД / коэффициент подавления по территории, году и виду."""
13
14      __tablename__ = "gs_eco_operating_tes_coefficients"
```

Маршрут страницы. Источник: app/ecology/routes/operating_tes_routes.py, строки 61–77.

```
@ecology_bp.route("/operating_tes/<territory_slug>/<sheet_slug>/")
@login_required
def operating_tes_table(territory_slug: str, sheet_slug: str):
    context = build_operating_tes_table_context(
        territory_slug,
        sheet_slug,
        start_year=request.args.get("start_year"),
        end_year=request.args.get("end_year"),
        rounding_digits=request.args.get("rounding_digits"),
        coefficient_rounding_digits=request.args.get("coefficient_rounding_digits"),
        specific_rounding_digits=request.args.get("specific_rounding_digits"),
        view_year=request.args.get("view_year"),
        affiliation=request.args.get("affiliation", "all"),
    )
    if context is None:
        abort(404)
    sheet = context.get("sheet") or {}
```

Сервис и расчетная логика. Источник: app/ecology/services/operating_tes_calc_services.py, строки 57–64.

```
def so2_motkh(bn: Any, chars: Mapping[str, Any] | None) -> Decimal | None:
    """Мотх SO2 = 0,02·Вн·Sp·(1 − x)."""
    bn_d = _d(bn)
    if bn_d is None or not chars:
        return None
    sp = _d(chars.get("sulfur")) or Decimal("0")
    x = _d(chars.get("sulfur_share")) or Decimal("0")
    return Decimal("0.02") * bn_d * sp * (Decimal("1") - x)
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/operating_tes/operating_tes_table.html, строки 683–711.

```
            {% endif %}
          </th>
          {% endfor %}
        </tr>
      </thead>
      <tbody>
        {% for row in rows %}
        <tr
          class="{{ row.css_class }}{% if row.is_empty %} ot-row-empty-hidden{% endif %}"
          data-row-key="{{ row.key }}"
        >
          <td
            class="text-start ot-row-label{% if row.indent %} ot-indent-{{ row.indent }}{% endif
%}{% if row.kind == 'total' %} fw-bold{% endif %}{% if row.italic %} fst-italic{% endif %}"
          >
            {{ emission_row_label(row) }}
          </td>
          {% for col in value_columns %}
          {% if row.span_pair %}
          <td colspan="2" class="text-center ot-pair-cell fw-bold" data-column="{{ col.slug }}">
            {% include 'ecology/operating_tes/_emission_summary_value.html' %}
          </td>
          {% else %}
          {% set pair =
          row.pair_values[col.slug] if row.pair_values is defined and col.slug
          in row.pair_values else none %}
          <td
            class="text-center ot-pair-cell{% if row.kind == 'total' %} fw-bold{% endif %}"
          >
            {{ pair.left if pair else '' }}
```

Формирование выбранного расчетного листа. Источник: app/ecology/services/operating_tes_page_services.py, строки 929–956.

```
929      elif layout in {"emission", "emission_tj"}:
930          view = EMISSION_SHEET_VIEW[sheet["slug"]]
931          if sheet["slug"] in {"zola", "sera", "co", "azot", "pg", "pg-tj"}:
932              prefix, eta_key, eta_label = {
933                  "zola": ("ash", "ash_eta", "КПД золоулавливания"),
934                  "sera": ("so2", "sulfur_eta", "КПД сероочистки"),
935                  "co": ("co", "co_k", "Коэффициент подавления CO"),
936                  "azot": ("nox", "nox_n", "Коэффициент подавления оксидов азота в топке n"),
937                  "pg": ("pg", "pg_k", "Коэффициент подавления ПГ (оксидов углерода)"),
938                  "pg-tj": ("pg_tj", "pg_k", "Коэффициент подавления ПГ (оксидов углерода)"),
939              }[sheet["slug"]]
940              total_key = f"{prefix}_motkh_total"
941              emitted_key = f"{prefix}_emitted"
942              summary_values = {}
943              for column in value_columns:
944                  slug = column["slug"]
945                  bundle = bundles_by_terr[slug]
946                  motkh = bundle[total_key]
947                  eta = bundle["coeffs"][eta_key]
948                  summary_values[slug] = {
949                      total_key: motkh,
950                      eta_key: eta,
951                      emitted_key: apply_capture(motkh, eta),
952                  }
953              for key, label in (
954                  (total_key, "ΣMотх."),
955                  (eta_key, eta_label),
956                  (emitted_key, "Мвыб."),
```

## 10.6. Азот: выбросы оксидов азота

Адрес: /ecology/operating_tes/vsego/azot/

**Функциональные возможности и действия пользователя**

Расчет Мотх.NOx использует расход условного топлива, механический недожог и коэффициент bj. В основном расчетном столбце показан результат без коэффициента подавления n; в итоговом блоке n применяется к суммарной массе, после чего формируется Мвыб. Такое разделение позволяет видеть исходную массу и эффект подавления. Общие средства: выбор периода, отображаемого года там, где предусмотрен годовой лист, округление, переход к другим листам и отбор отображаемых строк или территорий. Выгрузка действующих ТЭС собирает расчетные вкладки за выбранные годы.

**Взаимодействие и источники данных**

Расход топлива и энергетические показатели поступают из принятых снимков «Топлива», а не из еще не рассмотренных пакетов. Физические характеристики и коэффициенты берутся из моделей «Экологии» с учетом года и версии БД. Территории согласованы со справочниками ОЭС и группами станций. Полученные результаты входят в сводные расчеты действующих и новых ТЭС.

**Элементы реализации**

Модель OperatingTesFuelCharacteristic. Источник: app/ecology/models/operating_tes_fuel_characteristic_model.py, строки 11–14.

```
11  class OperatingTesFuelCharacteristic(db.Model, AuditMixin):
12      """Общие для всех уровней характеристики топлива по году и версии БД."""
13
14      __tablename__ = "gs_eco_operating_tes_fuel_chars"
```

Модель OperatingTesCoefficient. Источник: app/ecology/models/operating_tes_coefficient_model.py, строки 11–14.

```
11  class OperatingTesCoefficient(db.Model, AuditMixin):
12      """КПД / коэффициент подавления по территории, году и виду."""
13
14      __tablename__ = "gs_eco_operating_tes_coefficients"
```

Маршрут страницы. Источник: app/ecology/routes/operating_tes_routes.py, строки 61–77.

```
@ecology_bp.route("/operating_tes/<territory_slug>/<sheet_slug>/")
@login_required
def operating_tes_table(territory_slug: str, sheet_slug: str):
    context = build_operating_tes_table_context(
        territory_slug,
        sheet_slug,
        start_year=request.args.get("start_year"),
        end_year=request.args.get("end_year"),
        rounding_digits=request.args.get("rounding_digits"),
        coefficient_rounding_digits=request.args.get("coefficient_rounding_digits"),
        specific_rounding_digits=request.args.get("specific_rounding_digits"),
        view_year=request.args.get("view_year"),
        affiliation=request.args.get("affiliation", "all"),
    )
    if context is None:
        abort(404)
    sheet = context.get("sheet") or {}
```

Сервис и расчетная логика. Источник: app/ecology/services/operating_tes_calc_services.py, строки 67–74.

```
def nox_motkh(b_tce: Any, chars: Mapping[str, Any] | None) -> Decimal | None:
    """Мотх NOx без n: 0,001·B·(1 − qч/100)·bj."""
    b = _d(b_tce)
    if b is None or not chars:
        return None
    qch = _d(chars.get("q_mech")) or Decimal("0")
    bj = _d(chars.get("bj")) or Decimal("0")
    return Decimal("0.001") * b * (Decimal("1") - qch / Decimal("100")) * bj
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/operating_tes/operating_tes_table.html, строки 727–753.

```
        <col class="ot-col-terr" />
        {% endfor %}
      </colgroup>
      <thead class="table-success">
        <tr class="text-center align-middle">
          <th class="text-center align-middle ot-label-col">
            Вид топлива
            {% include 'ecology/operating_tes/_fuel_filter.html' %}
            {% include 'ecology/operating_tes/_territory_filter.html' %}
          </th>
          {% for col in territory_columns %}
          <th class="text-center align-middle ot-terr-col">
            {{ col.sheet_name }}
            {% if emission_view.metric_formula_html %}
            <div class="fw-normal">M<sub>отх</sub>.
              <button type="button" class="btn btn-link p-0 text-primary ms-1"
                      data-bs-toggle="tooltip" data-bs-placement="top" data-bs-html="true"
                      data-bs-custom-class="ot-summary-tooltip"
                      title="{{ emission_view.metric_formula_html|forceescape }}" aria-
label="Формула Мотх {{ 'азота' if sheet.slug == 'azot' else 'ПГ' }}">
                <i class="bi bi-info-circle" aria-hidden="true"></i>
              </button>
            </div>
            {% endif %}
          </th>
          {% endfor %}
        </tr>
      </thead>
```

Формирование выбранного расчетного листа. Источник: app/ecology/services/operating_tes_page_services.py, строки 929–956.

```
929      elif layout in {"emission", "emission_tj"}:
930          view = EMISSION_SHEET_VIEW[sheet["slug"]]
931          if sheet["slug"] in {"zola", "sera", "co", "azot", "pg", "pg-tj"}:
932              prefix, eta_key, eta_label = {
933                  "zola": ("ash", "ash_eta", "КПД золоулавливания"),
934                  "sera": ("so2", "sulfur_eta", "КПД сероочистки"),
935                  "co": ("co", "co_k", "Коэффициент подавления CO"),
936                  "azot": ("nox", "nox_n", "Коэффициент подавления оксидов азота в топке n"),
937                  "pg": ("pg", "pg_k", "Коэффициент подавления ПГ (оксидов углерода)"),
938                  "pg-tj": ("pg_tj", "pg_k", "Коэффициент подавления ПГ (оксидов углерода)"),
939              }[sheet["slug"]]
940              total_key = f"{prefix}_motkh_total"
941              emitted_key = f"{prefix}_emitted"
942              summary_values = {}
943              for column in value_columns:
944                  slug = column["slug"]
945                  bundle = bundles_by_terr[slug]
946                  motkh = bundle[total_key]
947                  eta = bundle["coeffs"][eta_key]
948                  summary_values[slug] = {
949                      total_key: motkh,
950                      eta_key: eta,
951                      emitted_key: apply_capture(motkh, eta),
952                  }
953              for key, label in (
954                  (total_key, "ΣМотх."),
955                  (eta_key, eta_label),
956                  (emitted_key, "Мвыб."),
```

## 10.7. CO: выбросы монооксида углерода

Адрес: /ecology/operating_tes/vsego/co/

**Функциональные возможности и действия пользователя**

Рассчитывается масса CO по натуральному расходу топлива, химическому недожогу q3, коэффициенту R, теплоте сгорания и механическому недожогу q4. На этом листе масса CO представляется в тоннах, в отличие от ряда листов с тыс. т. Итоговый блок применяет коэффициент подавления CO. Общие средства: выбор периода, отображаемого года там, где предусмотрен годовой лист, округление, переход к другим листам и отбор отображаемых строк или территорий. Выгрузка действующих ТЭС собирает расчетные вкладки за выбранные годы.

**Взаимодействие и источники данных**

Расход топлива и энергетические показатели поступают из принятых снимков «Топлива», а не из еще не рассмотренных пакетов. Физические характеристики и коэффициенты берутся из моделей «Экологии» с учетом года и версии БД. Территории согласованы со справочниками ОЭС и группами станций. Полученные результаты входят в сводные расчеты действующих и новых ТЭС.

**Элементы реализации**

Модель OperatingTesFuelCharacteristic. Источник: app/ecology/models/operating_tes_fuel_characteristic_model.py, строки 11–14.

```
11  class OperatingTesFuelCharacteristic(db.Model, AuditMixin):
12      """Общие для всех уровней характеристики топлива по году и версии БД."""
13
14      __tablename__ = "gs_eco_operating_tes_fuel_chars"
```

Модель OperatingTesCoefficient. Источник: app/ecology/models/operating_tes_coefficient_model.py, строки 11–14.

```
11  class OperatingTesCoefficient(db.Model, AuditMixin):
12      """КПД / коэффициент подавления по территории, году и виду."""
13
14      __tablename__ = "gs_eco_operating_tes_coefficients"
```

Маршрут страницы. Источник: app/ecology/routes/operating_tes_routes.py, строки 61–77.

```
@ecology_bp.route("/operating_tes/<territory_slug>/<sheet_slug>/")
@login_required
def operating_tes_table(territory_slug: str, sheet_slug: str):
    context = build_operating_tes_table_context(
        territory_slug,
        sheet_slug,
        start_year=request.args.get("start_year"),
        end_year=request.args.get("end_year"),
        rounding_digits=request.args.get("rounding_digits"),
        coefficient_rounding_digits=request.args.get("coefficient_rounding_digits"),
        specific_rounding_digits=request.args.get("specific_rounding_digits"),
        view_year=request.args.get("view_year"),
        affiliation=request.args.get("affiliation", "all"),
    )
    if context is None:
        abort(404)
    sheet = context.get("sheet") or {}
```

Сервис и расчетная логика. Источник: app/ecology/services/operating_tes_calc_services.py, строки 77–91.

```
def co_motkh(bn: Any, chars: Mapping[str, Any] | None) -> Decimal | None:
    """Мотх CO, т: 0,001·BH·(q3·R·Qн/1013)·(1 – q4/100)."""
    bn_d = _d(bn)
    if bn_d is None or not chars:
        return None
    q3 = _d(chars.get("q_chem")) or Decimal("0")
    r = _d(chars.get("r_chem")) or Decimal("0")
    qh = _d(chars.get("qh_kj")) or Decimal("0")
    qch = _d(chars.get("q_mech")) or Decimal("0")
    return (
        Decimal("0.001")
        * bn_d
        * (q3 * r * qh / CO_QH_DIVISOR)
        * (Decimal("1") - qch / Decimal("100"))
    )
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/operating_tes/operating_tes_table.html, строки 683–711.

```
            {% endif %}
          </th>
          {% endfor %}
        </tr>
      </thead>
      <tbody>
        {% for row in rows %}
        <tr
          class="{{ row.css_class }}{% if row.is_empty %} ot-row-empty-hidden{% endif %}"
          data-row-key="{{ row.key }}"
        >
          <td
            class="text-start ot-row-label{% if row.indent %} ot-indent-{{ row.indent }}{% endif
%}{% if row.kind == 'total' %} fw-bold{% endif %}{% if row.italic %} fst-italic{% endif %}"
          >
            {{ emission_row_label(row) }}
          </td>
          {% for col in value_columns %}
          {% if row.span_pair %}
          <td colspan="2" class="text-center ot-pair-cell fw-bold" data-column="{{ col.slug }}">
            {% include 'ecology/operating_tes/_emission_summary_value.html' %}
          </td>
          {% else %}
          {% set pair =
          row.pair_values[col.slug] if row.pair_values is defined and col.slug
          in row.pair_values else none %}
          <td
            class="text-center ot-pair-cell{% if row.kind == 'total' %} fw-bold{% endif %}"
          >
            {{ pair.left if pair else '' }}
```

Формирование выбранного расчетного листа. Источник: app/ecology/services/operating_tes_page_services.py, строки 929–956.

```
929      elif layout in {"emission", "emission_tj"}:
930          view = EMISSION_SHEET_VIEW[sheet["slug"]]
931          if sheet["slug"] in {"zola", "sera", "co", "azot", "pg", "pg-tj"}:
932              prefix, eta_key, eta_label = {
933                  "zola": ("ash", "ash_eta", "КПД золоулавливания"),
934                  "sera": ("so2", "sulfur_eta", "КПД сероочистки"),
935                  "co": ("co", "co_k", "Коэффициент подавления CO"),
936                  "azot": ("nox", "nox_n", "Коэффициент подавления оксидов азота в топке n"),
937                  "pg": ("pg", "pg_k", "Коэффициент подавления ПГ (оксидов углерода)"),
938                  "pg-tj": ("pg_tj", "pg_k", "Коэффициент подавления ПГ (оксидов углерода)"),
939              }[sheet["slug"]]
940              total_key = f"{prefix}_motkh_total"
941              emitted_key = f"{prefix}_emitted"
942              summary_values = {}
943              for column in value_columns:
944                  slug = column["slug"]
945                  bundle = bundles_by_terr[slug]
946                  motkh = bundle[total_key]
947                  eta = bundle["coeffs"][eta_key]
948                  summary_values[slug] = {
949                      total_key: motkh,
950                      eta_key: eta,
951                      emitted_key: apply_capture(motkh, eta),
952                  }
953              for key, label in (
954                  (total_key, "ΣМотх."),
955                  (eta_key, eta_label),
956                  (emitted_key, "Мвыб."),
```

## 10.8. ПГ: прямой расчет CO2

Адрес: /ecology/operating_tes/vsego/pg/

**Функциональные возможности и действия пользователя**

Масса CO2 определяется как произведение расхода топлива и KCO2 по каждому виду топлива. Результат представлен в тыс. т; итоговый блок показывает массу до и после применения коэффициента подавления ПГ. Отсутствующий расход или коэффициент не подменяется произвольным расчетным числом. Общие средства: выбор периода, отображаемого года там, где предусмотрен годовой лист, округление, переход к другим листам и отбор отображаемых строк или территорий. Выгрузка действующих ТЭС собирает расчетные вкладки за выбранные годы.

**Взаимодействие и источники данных**

Расход топлива и энергетические показатели поступают из принятых снимков «Топлива», а не из еще не рассмотренных пакетов. Физические характеристики и коэффициенты берутся из моделей «Экологии» с учетом года и версии БД. Территории согласованы со справочниками ОЭС и группами станций. Полученные результаты входят в сводные расчеты действующих и новых ТЭС.

**Элементы реализации**

Модель OperatingTesFuelCharacteristic. Источник: app/ecology/models/operating_tes_fuel_characteristic_model.py, строки 11–14.

```
11  class OperatingTesFuelCharacteristic(db.Model, AuditMixin):
12      """Общие для всех уровней характеристики топлива по году и версии БД."""
13
14      __tablename__ = "gs_eco_operating_tes_fuel_chars"
```

Модель OperatingTesCoefficient. Источник: app/ecology/models/operating_tes_coefficient_model.py, строки 11–14.

```
11  class OperatingTesCoefficient(db.Model, AuditMixin):
12      """КПД / коэффициент подавления по территории, году и виду."""
13
14      __tablename__ = "gs_eco_operating_tes_coefficients"
```

Маршрут страницы. Источник: app/ecology/routes/operating_tes_routes.py, строки 61–77.

```
@ecology_bp.route("/operating_tes/<territory_slug>/<sheet_slug>/")
@login_required
def operating_tes_table(territory_slug: str, sheet_slug: str):
    context = build_operating_tes_table_context(
        territory_slug,
        sheet_slug,
        start_year=request.args.get("start_year"),
        end_year=request.args.get("end_year"),
        rounding_digits=request.args.get("rounding_digits"),
        coefficient_rounding_digits=request.args.get("coefficient_rounding_digits"),
        specific_rounding_digits=request.args.get("specific_rounding_digits"),
        view_year=request.args.get("view_year"),
        affiliation=request.args.get("affiliation", "all"),
    )
    if context is None:
        abort(404)
    sheet = context.get("sheet") or {}
```

Сервис и расчетная логика. Источник: app/ecology/services/operating_tes_calc_services.py, строки 94–102.

```
def pg_motkh(b_tce: Any, chars: Mapping[str, Any] | None) -> Decimal | None:
    """Мотх CO2 = B·KCO2, тыс. т."""
    b = _d(b_tce)
    if b is None or not chars:
        return None
    k = _d(chars.get("k_co2"))
    if k is None:
        return None
    return b * k
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/operating_tes/operating_tes_table.html, строки 727–753.

```
        <col class="ot-col-terr" />
        {% endfor %}
      </colgroup>
      <thead class="table-success">
        <tr class="text-center align-middle">
          <th class="text-center align-middle ot-label-col">
            Вид топлива
            {% include 'ecology/operating_tes/_fuel_filter.html' %}
            {% include 'ecology/operating_tes/_territory_filter.html' %}
          </th>
          {% for col in territory_columns %}
          <th class="text-center align-middle ot-terr-col">
            {{ col.sheet_name }}
            {% if emission_view.metric_formula_html %}
            <div class="fw-normal">М<sub>отх</sub>.
              <button type="button" class="btn btn-link p-0 text-primary ms-1"
                      data-bs-toggle="tooltip" data-bs-placement="top" data-bs-html="true"
                      data-bs-custom-class="ot-summary-tooltip"
                      title="{{ emission_view.metric_formula_html|forceescape }}" aria-
label="Формула Мотх {{ 'азота' if sheet.slug == 'azot' else 'ПГ' }}">
                <i class="bi bi-info-circle" aria-hidden="true"></i>
              </button>
            </div>
            {% endif %}
          </th>
          {% endfor %}
        </tr>
      </thead>
```

Формирование выбранного расчетного листа. Источник: app/ecology/services/operating_tes_page_services.py, строки 929–956.

```
929      elif layout in {"emission", "emission_tj"}:
930          view = EMISSION_SHEET_VIEW[sheet["slug"]]
931          if sheet["slug"] in {"zola", "sera", "co", "azot", "pg", "pg-tj"}:
932              prefix, eta_key, eta_label = {
933                  "zola": ("ash", "ash_eta", "КПД золоулавливания"),
934                  "sera": ("so2", "sulfur_eta", "КПД сероочистки"),
935                  "co": ("co", "co_k", "Коэффициент подавления CO"),
936                  "azot": ("nox", "nox_n", "Коэффициент подавления оксидов азота в топке n"),
937                  "pg": ("pg", "pg_k", "Коэффициент подавления ПГ (оксидов углерода)"),
938                  "pg-tj": ("pg_tj", "pg_k", "Коэффициент подавления ПГ (оксидов углерода)"),
939              }[sheet["slug"]]
940              total_key = f"{prefix}_motkh_total"
941              emitted_key = f"{prefix}_emitted"
942              summary_values = {}
943              for column in value_columns:
944                  slug = column["slug"]
945                  bundle = bundles_by_terr[slug]
946                  motkh = bundle[total_key]
947                  eta = bundle["coeffs"][eta_key]
948                  summary_values[slug] = {
949                      total_key: motkh,
950                      eta_key: eta,
951                      emitted_key: apply_capture(motkh, eta),
952                  }
953              for key, label in (
954                  (total_key, "ΣМотх."),
955                  (eta_key, eta_label),
956                  (emitted_key, "Мвыб."),
```

## 10.9. ПГ через ТДж: расчет CO2 по энергии топлива

Адрес: /ecology/operating_tes/vsego/pg-tj/

**Функциональные возможности и действия пользователя**

Альтернативный расчет переводит расход топлива в энергетическое выражение через коэффициенты т у.т./тн и ТДж/тыс. т, затем применяет коэффициент т CO2/ТДж. Энергия и масса показываются раздельно; масса CO2 выражена в тыс. т. В коде фактическая энергия рассчитывается в ТДж, хотя краткая подпись одного столбца макета содержит «МДж». Описание не приравнивает эти единицы. Общие средства: выбор периода, отображаемого года там, где предусмотрен годовой лист, округление, переход к другим листам и отбор отображаемых строк или территорий. Выгрузка действующих ТЭС собирает расчетные вкладки за выбранные годы.

**Взаимодействие и источники данных**

Расход топлива и энергетические показатели поступают из принятых снимков «Топлива», а не из еще не рассмотренных пакетов. Физические характеристики и коэффициенты берутся из моделей «Экологии» с учетом года и версии БД. Территории согласованы со справочниками ОЭС и группами станций. Полученные результаты входят в сводные расчеты действующих и новых ТЭС.

**Элементы реализации**

Модель OperatingTesFuelCharacteristic. Источник: app/ecology/models/operating_tes_fuel_characteristic_model.py, строки 11–14.

```
11  class OperatingTesFuelCharacteristic(db.Model, AuditMixin):
12      """Общие для всех уровней характеристики топлива по году и версии БД."""
13
14      __tablename__ = "gs_eco_operating_tes_fuel_chars"
```

Модель OperatingTesCoefficient. Источник: app/ecology/models/operating_tes_coefficient_model.py, строки 11–14.

```
11  class OperatingTesCoefficient(db.Model, AuditMixin):
12      """КПД / коэффициент подавления по территории, году и виду."""
13
14      __tablename__ = "gs_eco_operating_tes_coefficients"
```

Маршрут страницы. Источник: app/ecology/routes/operating_tes_routes.py, строки 61–77.

```
@ecology_bp.route("/operating_tes/<territory_slug>/<sheet_slug>/")
@login_required
def operating_tes_table(territory_slug: str, sheet_slug: str):
    context = build_operating_tes_table_context(
        territory_slug,
        sheet_slug,
        start_year=request.args.get("start_year"),
        end_year=request.args.get("end_year"),
        rounding_digits=request.args.get("rounding_digits"),
        coefficient_rounding_digits=request.args.get("coefficient_rounding_digits"),
        specific_rounding_digits=request.args.get("specific_rounding_digits"),
        view_year=request.args.get("view_year"),
        affiliation=request.args.get("affiliation", "all"),
    )
    if context is None:
        abort(404)
    sheet = context.get("sheet") or {}
```

Сервис и расчетная логика. Источник: app/ecology/services/operating_tes_calc_services.py, строки 105–121.

```
def pg_tj_energy_and_mass(
    b_tce: Any, chars: Mapping[str, Any] | None
) -> tuple[Decimal | None, Decimal | None]:
    """Энергия в ТДж (столбец «МДж») и масса CO2 в тыс. т по коэффициентам НД."""
    b = _d(b_tce)
    if b is None or not chars:
        return None, None
    tut_per_t = _d(chars.get("tut_per_t"))
    tj_per_kt = _d(chars.get("tj_per_kt"))
    if tut_per_t is None or tut_per_t <= 0 or tj_per_kt is None:
        return None, None
    energy_tj = b / tut_per_t * tj_per_kt
    t_per_tj = _d(chars.get("t_co2_per_tj"))
    if t_per_tj is None:
        return energy_tj, None
    mass_thousand_t = energy_tj * t_per_tj / Decimal("1000")
    return energy_tj, mass_thousand_t
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/operating_tes/operating_tes_table.html, строки 683–711.

```
            {% endif %}
          </th>
          {% endfor %}
        </tr>
      </thead>
      <tbody>
        {% for row in rows %}
        <tr
          class="{{ row.css_class }}{% if row.is_empty %} ot-row-empty-hidden{% endif %}"
          data-row-key="{{ row.key }}"
        >
          <td
            class="text-start ot-row-label{% if row.indent %} ot-indent-{{ row.indent }}{% endif
%}{% if row.kind == 'total' %} fw-bold{% endif %}{% if row.italic %} fst-italic{% endif %}"
          >
            {{ emission_row_label(row) }}
          </td>
          {% for col in value_columns %}
          {% if row.span_pair %}
          <td colspan="2" class="text-center ot-pair-cell fw-bold" data-column="{{ col.slug }}">
            {% include 'ecology/operating_tes/_emission_summary_value.html' %}
          </td>
          {% else %}
          {% set pair =
          row.pair_values[col.slug] if row.pair_values is defined and col.slug
          in row.pair_values else none %}
          <td
            class="text-center ot-pair-cell{% if row.kind == 'total' %} fw-bold{% endif %}"
          >
            {{ pair.left if pair else '' }}
```

Формирование выбранного расчетного листа. Источник: app/ecology/services/operating_tes_page_services.py, строки 929–956.

```
929      elif layout in {"emission", "emission_tj"}:
930          view = EMISSION_SHEET_VIEW[sheet["slug"]]
931          if sheet["slug"] in {"zola", "sera", "co", "azot", "pg", "pg-tj"}:
932              prefix, eta_key, eta_label = {
933                  "zola": ("ash", "ash_eta", "КПД золоулавливания"),
934                  "sera": ("so2", "sulfur_eta", "КПД сероочистки"),
935                  "co": ("co", "co_k", "Коэффициент подавления CO"),
936                  "azot": ("nox", "nox_n", "Коэффициент подавления оксидов азота в топке n"),
937                  "pg": ("pg", "pg_k", "Коэффициент подавления ПГ (оксидов углерода)"),
938                  "pg-tj": ("pg_tj", "pg_k", "Коэффициент подавления ПГ (оксидов углерода)"),
939              }[sheet["slug"]]
940              total_key = f"{prefix}_motkh_total"
941              emitted_key = f"{prefix}_emitted"
942              summary_values = {}
943              for column in value_columns:
944                  slug = column["slug"]
945                  bundle = bundles_by_terr[slug]
946                  motkh = bundle[total_key]
947                  eta = bundle["coeffs"][eta_key]
948                  summary_values[slug] = {
949                      total_key: motkh,
950                      eta_key: eta,
951                      emitted_key: apply_capture(motkh, eta),
952                  }
953              for key, label in (
954                  (total_key, "ΣМотх."),
955                  (eta_key, eta_label),
956                  (emitted_key, "Мвыб."),
```

## 10.10. Объемы и удельные выбросы действующих ТЭС

Адрес: /ecology/operating_tes/vsego/obemy/

**Функциональные возможности и действия пользователя**

Сводная расчетная форма собирает массы золы, SO2, NOx, CO и CO2 по территориям после учета очистки и подавления. Дополнительные таблицы показывают удельные выбросы на приведенную энергию и на топливо. Используются согласованные единицы массы и энергии; CO отдельно учитывается в тоннах. Удельный показатель не вычисляется при нулевой базе деления. Общие средства: выбор периода, отображаемого года там, где предусмотрен годовой лист, округление, переход к другим листам и отбор отображаемых строк или территорий. Выгрузка действующих ТЭС собирает расчетные вкладки за выбранные годы.

**Взаимодействие и источники данных**

Расход топлива и энергетические показатели поступают из принятых снимков «Топлива», а не из еще не рассмотренных пакетов. Физические характеристики и коэффициенты берутся из моделей «Экологии» с учетом года и версии БД. Территории согласованы со справочниками ОЭС и группами станций. Полученные результаты входят в сводные расчеты действующих и новых ТЭС.

**Элементы реализации**

Модель OperatingTesInputValue. Источник: app/ecology/models/operating_tes_input_model.py, строки 11–14.

```
11  class OperatingTesInputValue(db.Model, AuditMixin):
12      """Переопределение расхода топлива / выработки / тепла по территории и году."""
13
14      __tablename__ = "gs_eco_operating_tes_inputs"
```

Модель OperatingTesCoefficient. Источник: app/ecology/models/operating_tes_coefficient_model.py, строки 11–14.

```
11  class OperatingTesCoefficient(db.Model, AuditMixin):
12      """КПД / коэффициент подавления по территории, году и виду."""
13
14      __tablename__ = "gs_eco_operating_tes_coefficients"
```

Маршрут страницы. Источник: app/ecology/routes/operating_tes_routes.py, строки 61–77.

```
@ecology_bp.route("/operating_tes/<territory_slug>/<sheet_slug>/")
@login_required
def operating_tes_table(territory_slug: str, sheet_slug: str):
    context = build_operating_tes_table_context(
        territory_slug,
        sheet_slug,
        start_year=request.args.get("start_year"),
        end_year=request.args.get("end_year"),
        rounding_digits=request.args.get("rounding_digits"),
        coefficient_rounding_digits=request.args.get("coefficient_rounding_digits"),
        specific_rounding_digits=request.args.get("specific_rounding_digits"),
        view_year=request.args.get("view_year"),
        affiliation=request.args.get("affiliation", "all"),
    )
    if context is None:
        abort(404)
    sheet = context.get("sheet") or {}
```

Сервис и расчетная логика. Источник: app/ecology/services/operating_tes_page_services.py, строки 652–670.

```
def _build_specific_emission_tables(
    value_columns: list[dict[str, Any]],
    bundles: dict[str, Any],
    inputs: dict[int, Any],
    digits: int,
) -> list[dict[str, Any]]:
    energy_title = "Удельные выбросы ЗВ и ПГ на единицу приведенной энергии, г/кВт.ч"
    electricity_title = "Удельные выбросы ЗВ и ПГ на единицу электрической энергии, г/кВт.ч"
    fuel_formula = "М × 1000 / В"
    solid_formula = "М × 100 / (Нефтетопливо + Торф + тв.топливо + Уголь всего)"
    brought_energy_formula = "М / (Выработка электроэнергии + Отпуск тепла / 859,8)"
    electricity_formula = "М / Выработка электроэнергии"
    tables = [
        {"key": "energy", "title": energy_title, "formula": brought_energy_formula},
        {"key": "energy_fuel", "title": electricity_title, "formula": electricity_formula},
        {"key": "fuel", "title": "Удельные выбросы ЗВ и ПГ на единицу топлива, кг/т у.т.",
         "formula": f"{fuel_formula}; для золы и SO2: {solid_formula}"},
    ]
    values = {table["key"]: {} for table in tables}
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/operating_tes/operating_tes_table.html, строки 776–798.

```
      {% elif sheet.layout == 'volumes' %}
      {% include 'ecology/operating_tes/_volume_table_content.html' %}
      {% else %}
      <colgroup>
        <col class="ot-col-label" />
        {% for col in territory_columns %}
        <col class="ot-col-terr" />
        {% endfor %}
      </colgroup>
      <thead class="table-success">
        <tr class="text-center align-middle">
          <th class="text-center align-middle ot-label-col">Наименование
            {% if sheet.layout in ('fuel', 'coeff') %}
            {% include 'ecology/operating_tes/_fuel_filter.html' %}
            {% endif %}
            {% if territory_filter_storage_key %}
            {% include 'ecology/operating_tes/_territory_filter.html' %}
            {% endif %}
          </th>
          {% for col in territory_columns %}
          <th class="text-center align-middle ot-terr-col">
            {{ col.sheet_name }}{% if sheet.layout == 'coeff' %}
            <div class="small">{% if affiliation == 'minen' %}Станции отрасли{% elif affiliation ==
'prompred' %}Промпредприятия{% else %}Все принадлежности{% endif %}, {{ view_year }}</div>
```

Формирование выбранного расчетного листа. Источник: app/ecology/services/operating_tes_page_services.py, строки 493–519.

```
def compute_year_bundle(
    territory_slug: str,
    year: int,
    bucket: dict[str, Decimal] | None,
    char_overrides: dict[tuple[str, str, int], dict[str, Any]],
    coeff_overrides: dict[tuple[str, int, str], Decimal],
) -> dict[str, Any]:
    coeffs = {
        kind["key"]: resolve_coeff(territory_slug, year, kind["key"], coeff_overrides)
        for kind in COEFF_KINDS
    }
    leaf_b: dict[str, Decimal | None] = {}
    by_key: dict[str, dict[str, Decimal | None]] = {}
    # leaves first
    for spec in FUEL_ROWS:
        if spec.get("is_header"):
            continue
        key = spec["key"]
        b_tce = fuel_amount_for_key(bucket, key)
        leaf_b[key] = b_tce
        chars = resolve_chars(key, year, char_overrides, territory_slug)
        by_key[key] = _compute_leaf_emissions(key, b_tce, chars, coeffs)
    header_specs = sorted(
        [spec for spec in FUEL_ROWS if spec.get("is_header")],
        key=lambda item: int(item.get("indent") or 0),
        reverse=True,
    )
```

## 11. Расчет по новым ТЭС

Для новых ТЭС нет самостоятельной формы «Добавить станцию»: состав поступает из топливной подсистемы, а редактируются только разрешенные нормативы. Передаваемые категории определены критериями сводки «Топлива». Внутренние ключи le100/gt100 исторические; актуальная передача new_lt100/new_ge100 разделяет заполненную мощность блока n1 на менее 100 МВт и 100 МВт или более. Старый низкоуровневый помощник _capacity_class_expr использует другой порог по Nуст; эти пути не следует смешивать при описании принятых снимков.

Категории передачи и их сопоставление с capacity_class. Источник: app/ecology/services/fuel_summary_constants.py, строки 3–14.

```
    3  FUEL_SUMMARY_SCOPES = (
    4      ("operating", "Действующие ТЭС"),
    5      ("new_lt100", "до 100 МВт"),
    6      ("new_ge100", "100 МВт и более"),
    7  )
    8  FUEL_SUMMARY_LABELS = dict(FUEL_SUMMARY_SCOPES)
    9  FUEL_SUMMARY_CRITERIA = {
   10      "operating": "Действующие группы оборудования действующих ТЭС с заполненной установленной
мощностью за выбранный год. Ограничений по величине мощности нет.",
   11      "new_lt100": "Новые группы оборудования действующих ТЭС или группы оборудования новых ТЭС.
Мощность блока (вар. 1) (n1) группы оборудования должна быть заполнена и составлять менее 100 МВт.",
   12      "new_ge100": "Новые группы оборудования действующих ТЭС или группы оборудования новых ТЭС.
Мощность блока (вар. 1) (n1) группы оборудования должна быть заполнена и составлять 100 МВт или более.",
   13  }
   14  NEW_TES_SCOPE_CAPACITY = {"new_lt100": "le100", "new_ge100": "gt100"}
```

### 11.1. Принятые топливные показатели новых ТЭС

Адрес: /ecology/new_tes/vsego/toplivo/

**Функциональные возможности и действия пользователя**

Показываются принятые показатели новых ТЭС для выбранного класса мощности: выработка, тепло и расход по видам топлива. Период и год выбираются в той же форме, что и для действующих ТЭС. Наличие нерассмотренного пакета показывается отдельно; отсутствие новых данных не означает отсутствия самой расчетной формы. Прямой ввод расхода на этом листе не разрешен.

**Взаимодействие и источники данных**

Используются два раздельных принятых снимка «Топлива» для новых ТЭС. Ключи capacity_class=le100 и gt100 сохраняются в навигации и сопоставлены с категориями пакетов new_lt100 и new_ge100. По критериям сводки передача разделяется по заполненной мощности блока n1: менее 100 МВт и 100 МВт или более. Обозначения le100/gt100 здесь являются внутренними ключами и не заменяют критерии отбора передаваемого пакета. Результаты обеих категорий включаются в сводные расчеты.

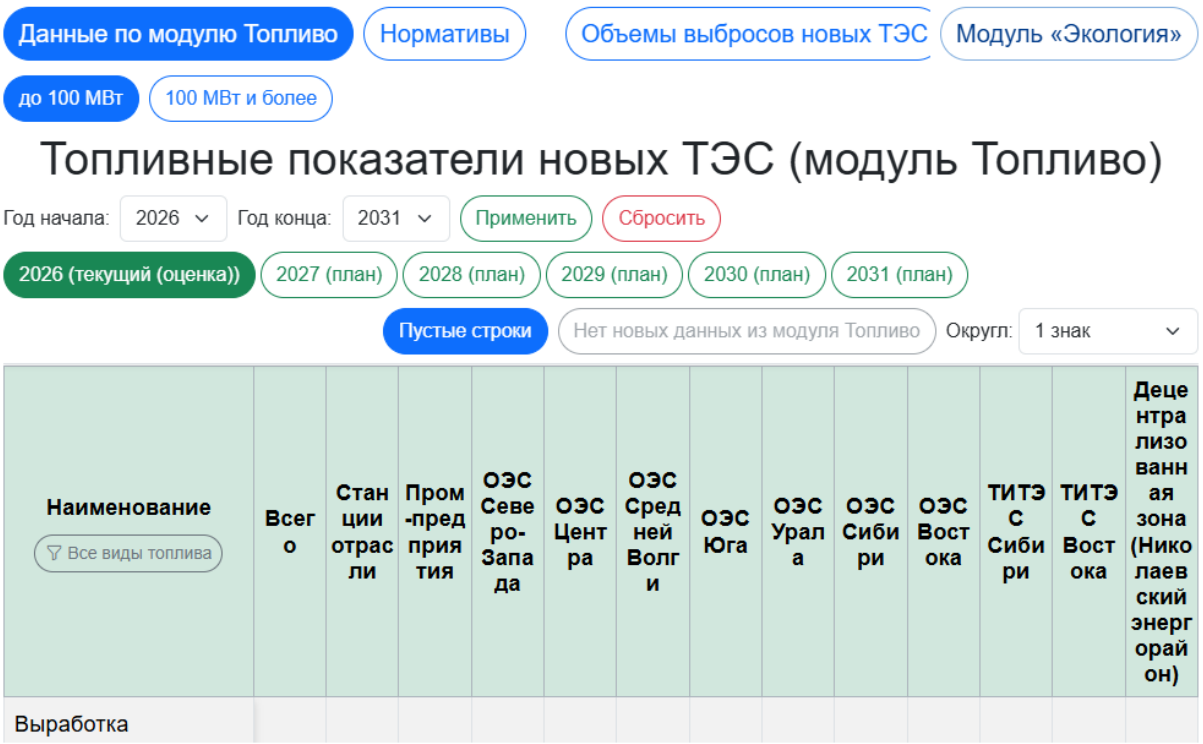



Рисунок 14. Новые ТЭС: принятые топливные показатели и выбор категории мощности. Действующий сайт, 29.10.2026. Показан фрагмент экранной формы.

## Элементы реализации

Модель NewTesNorm. Источник: app/ecology/models/new_tes_norm_model.py, строки 11–14.

```
  11  class NewTesNorm(db.Model, AuditMixin):
  12      """Удельный выброс по виду топлива, классу мощности и году."""
  13
  14      __tablename__ = "gs_eco_new_tes_norms"
```

Модель EcologyFuelSnapshot. Источник: app/ecology/models/fuel_exchange_model.py, строки 9–21.

```
   9  class EcologyFuelSnapshot(db.Model, AuditMixin):
  10      __tablename__ = "gs_eco_fuel_snapshots"
  11      __table_args__ = (
  12          db.UniqueConstraint("database_version_id", "year_number", "scope",
name="uq_eco_fuel_snapshot_version_year_scope"),
  13          {"schema": SCHEMA_ECOLOGY},
  14      )
  15
  16      id = db.Column(db.Integer, primary_key=True)
  17      database_version_id = db.Column(db.Integer,
db.ForeignKey(f"{SCHEMA_REFDATA}.gs_database_versions.id", ondelete="CASCADE"), nullable=False)
  18      year_number = db.Column(db.Integer, nullable=False)
  19      scope = db.Column(db.String(20), nullable=False, default="operating",
server_default="operating")
  20      payload = db.Column(db.JSON, nullable=False)
  21      updated_at = db.Column(db.DateTime(timezone=True), server_default=func.now(),
onupdate=func.now(), nullable=False)
```

Маршрут страницы. Источник: app/ecology/routes/new_tes_routes.py, строки 58–72.

```
@ecology_bp.route("/new_tes/<territory_slug>/<sheet_slug>/")
@login_required
def new_tes_table(territory_slug: str, sheet_slug: str):
    context = build_new_tes_table_context(
        territory_slug,
        sheet_slug,
        start_year=request.args.get("start_year"),
        end_year=request.args.get("end_year"),
        rounding_digits=request.args.get("rounding_digits"),
        view_year=request.args.get("view_year"),
        capacity_class=request.args.get("capacity_class", CAPACITY_LE100_KEY),
    )
    if context is None:
        abort(404)
    sheet = context.get("sheet") or {}
```

Сервис и расчетная логика. Источник: app/ecology/services/fuel_exchange_services.py, строки 44–50.

```
def load_accepted_new_tes_fuel_inputs(years):
    result = {year: {} for year in years}
    for scope, capacity_class in NEW_TES_SCOPE_CAPACITY.items():
        for year, territories in load_accepted_fuel_inputs(years, scope=scope).items():
            for slug, bucket in territories.items():
                result[year].setdefault(slug, {})[capacity_class] = bucket
    return result
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/operating_tes/operating_tes_table.html, строки 66–82.

```
            {% include 'ecology/operating_tes/_sheet_nav.html' %}
            {% if new_tes_capacity_tabs and not new_tes_norm_tables %}
            <nav class="d-flex flex-wrap gap-2" aria-label="Категории новых ТЭС">
              {% for capacity_key, capacity_label in new_tes_capacity_tabs %}
              <a class="btn {{ 'btn-primary' if capacity_class == capacity_key else 'btn-outline-
primary' }}"
                 {% if capacity_class == capacity_key %}aria-current="page"{% endif %}
                 href="{{ url_for(table_endpoint, territory_slug=territory.slug,
sheet_slug=sheet.slug, capacity_class=capacity_key, start_year=start_year, end_year=end_year,
view_year=view_year, rounding_digits=rounding_digits) }}">{{ capacity_label }}</a>
              {% endfor %}
            </nav>
            {% endif %}
            {% endif %}
          </div>
        </div>
      </div>

      {% if fuel_summary_errors %}
      <div class="alert alert-warning" role="alert">
```

Сервис новых ТЭС. Источник: app/ecology/services/new_tes_page_services.py, строки 276–291.

```
    if layout == "fuel":
        raw = load_accepted_new_tes_fuel_inputs(years)
        inputs = {
            year: {
                slug: by_class.get(capacity_class) or empty_fuel_bucket()
                for slug, by_class in (by_terr or {}).items()
            }
            for year, by_terr in raw.items()
        }
    elif layout == "chars":
        ensure_default_norm_rows(years)
        norm_overrides = load_norm_overrides(years)
    else:
        raw = load_accepted_new_tes_fuel_inputs(years)
        ensure_default_norm_rows(years)
        norm_overrides = load_norm_overrides(years)
```

## 11.2. Нормативы удельных выбросов новых ТЭС

Адрес: /ecology/new_tes/vsego/spravochnik/

**Функциональные возможности и действия пользователя**

Справочная форма содержит нормативы золы, SO2 и NOx в кг/т у.т. и CO2 в т/т у.т. Таблицы нормативов формируются для обеих категорий мощности. Разрешенные исходные ячейки сохраняются по виду топлива, классу и году. Эффективный коэффициент CO2 берется из общих характеристик топлива; его изменение на этой странице передается в общий k_co2, а не образует независимую копию норматива ПГ. Норматив CO в текущем расчете новых ТЭС принимается равным нулю.

**Взаимодействие и источники данных**

Используются два раздельных принятых снимка «Топлива» для новых ТЭС. Ключи capacity_class=le100 и gt100 сохраняются в навигации и сопоставлены с категориями пакетов new_lt100 и new_ge100. По критериям сводки передача разделяется по заполненной мощности блока n1: менее 100 МВт и 100 МВт или более. Обозначения le100/gt100 здесь являются внутренними ключами и не заменяют критерии отбора передаваемого пакета. Результаты обеих категорий включаются в сводные расчеты.

**Элементы реализации**

Модель NewTesNorm. Источник: app/ecology/models/new_tes_norm_model.py, строки 11–14.

```
  11  class NewTesNorm(db.Model, AuditMixin):
  12      """Удельный выброс по виду топлива, классу мощности и году."""
  13
  14      __tablename__ = "gs_eco_new_tes_norms"
```

Модель EcologyFuelSnapshot. Источник: app/ecology/models/fuel_exchange_model.py, строки 9–21.

```
   9  class EcologyFuelSnapshot(db.Model, AuditMixin):
  10      __tablename__ = "gs_eco_fuel_snapshots"
  11      __table_args__ = (
  12          db.UniqueConstraint("database_version_id", "year_number", "scope",
name="uq_eco_fuel_snapshot_version_year_scope"),
  13          {"schema": SCHEMA_ECOLOGY},
  14      )
  15
  16      id = db.Column(db.Integer, primary_key=True)
  17      database_version_id = db.Column(db.Integer,
db.ForeignKey(f"{SCHEMA_REFDATA}.gs_database_versions.id", ondelete="CASCADE"), nullable=False)
  18      year_number = db.Column(db.Integer, nullable=False)
  19      scope = db.Column(db.String(20), nullable=False, default="operating",
server_default="operating")
  20      payload = db.Column(db.JSON, nullable=False)
  21      updated_at = db.Column(db.DateTime(timezone=True), server_default=func.now(),
onupdate=func.now(), nullable=False)
```

Маршрут страницы. Источник: app/ecology/routes/new_tes_routes.py, строки 58–72.

```
  58  @ecology_bp.route("/new_tes/<territory_slug>/<sheet_slug>/")
  59  @login_required
  60  def new_tes_table(territory_slug: str, sheet_slug: str):
  61      context = build_new_tes_table_context(
  62          territory_slug,
  63          sheet_slug,
  64          start_year=request.args.get("start_year"),
  65          end_year=request.args.get("end_year"),
  66          rounding_digits=request.args.get("rounding_digits"),
  67          view_year=request.args.get("view_year"),
  68          capacity_class=request.args.get("capacity_class", CAPACITY_LE100_KEY),
  69      )
  70      if context is None:
  71          abort(404)
  72      sheet = context.get("sheet") or {}
```

Сервис и расчетная логика. Источник: app/ecology/services/new_tes_page_services.py, строки 137–155.

```
 137  def resolve_norm(
 138      fuel_key: str,
 139      capacity_class: str,
 140      year: int,
 141      overrides: dict[tuple[str, str, int], dict[str, Any]],
 142      char_overrides: dict | None = None,
 143  ) -> dict[str, Decimal]:
 144      out: dict[str, Decimal] = {}
 145      extra = overrides.get((fuel_key, capacity_class, year)) or {}
 146      for kind in NORM_KINDS:
 147          raw = extra.get(kind)
 148          if raw is None:
 149              raw = default_norm(fuel_key, capacity_class, kind)
 150          amount = _d(raw)
 151          out[kind] = amount if amount is not None else Decimal("0")
 152      out["pg"] = _d(resolve_chars(fuel_key, year, char_overrides or {}).get("k_co2")) or
Decimal("0")
 153      # Норматив CO не задан в новой таблице нормативов.
 154      out["co"] = Decimal("0")
 155      return out
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/operating_tes/operating_tes_table.html, строки 524–548.

```
 524      {% for norm_table in new_tes_norm_tables|default([]) or [{'rows': rows, 'capacity_class':
capacity_class|default(none)}] %}
 525      {% with rows=norm_table.rows, capacity_class=norm_table.capacity_class %}
 526      {% if norm_table.title %}
 527      <section aria-labelledby="ot-norm-{{ capacity_class }}-title">
 528      <h2 class="h3 fw-bold mt-4 mb-3" id="ot-norm-{{ capacity_class }}-title">{{ norm_table.title
}}</h2>
 529      {% endif %}
 530      <table
 531        class="table table-hover table-striped table-bordered align-middle power-demand-summary-table
ot-calculation-table{% if new_tes_volume_columns %} ot-new-tes-volumes-table{% endif %}{% if sheet.layout
not in ('fuel', 'coeff') %} ot-hide-empty{% endif %}{% if sheet.layout in ('fuel', 'coeff', 'volumes') %}
ot-fit-table{% elif sheet.layout == 'chars' %} ot-chars-table{% elif emission_view and emission_view.kind
== 'pair' %} ot-emission-pair-table{% elif emission_view %} ot-emission-single-table{% endif %}"
 532        id="{{ 'operatingTesTable' if loop.first else 'operatingTesTable-' ~ capacity_class }}"
 533        {% if norm_table.title %}aria-labelledby="ot-norm-{{ capacity_class }}-title"{% endif %}
 534        {% if new_tes_volume_columns %}style="min-width: {{ 15 + value_columns|length * 8.5 }}rem;"{%
endif %}
 535        data-update-url="{{ url_for(value_endpoint, territory_slug=territory.slug,
sheet_slug=sheet.slug, capacity_class=capacity_class|default(none)) }}"
 536        data-can-edit="{{ 1 if can_edit else 0 }}"
 537        data-affiliation="{{ affiliation|default('all') }}"
 538        data-autosave="{{ 1 if sheet.layout in ('coeff', 'chars', 'emission', 'emission_tj') or
year_columns_mode else 0 }}"
 539        data-rounding-digits="{{ rounding_digits }}"
 540        data-coefficient-rounding-digits="{{ coefficient_rounding_digits|default(0) }}"
 541      >
 542        {% if new_tes_volume_columns %}
 543        {% include 'ecology/operating_tes/_new_tes_volume_table.html' %}
 544        {% elif dynamics_territories %}
 545        {% include 'ecology/operating_tes/_volume_table_content.html' %}
 546        {% elif year_columns_mode %}
 547        {% include 'ecology/operating_tes/_range_table.html' %}
 548        {% elif sheet.layout == 'chars' %}
```

Сервис новых ТЭС. Источник: app/ecology/services/new_tes_page_services.py, строки 478–498.

```
478  def upsert_norm(fuel_key: str, year: int, field: str, value: Any) -> NewTesNorm:
479      parsed = parse_norm_field(field)
480      if parsed is None or field not in NORM_FIELD_KEYS:
481          raise ValueError("Неизвестное поле норматива")
482      kind, capacity_class = parsed
483      if kind == "pg":
484          return upsert_characteristic(DEFAULT_TERRITORY_SLUG, fuel_key, year, "k_co2", value)
485      if fuel_key not in DEFAULT_NORMS:
486          raise ValueError("Неизвестный вид топлива")
487      version_id = get_current_db_version_id()
488      log_context = context_lines(
489          ("лист", "Нормативы"),
490          ("топливо", fuel_label(fuel_key)),
491          ("год", int(year)),
492      )
493      query = NewTesNorm.query.filter_by(
494          fuel_key=fuel_key,
495          capacity_class=capacity_class,
496          year_number=int(year),
497      )
498      if version_id is not None:
```

## 11.3. Объемы выбросов новых ТЭС

Адрес: /ecology/new_tes/vsego/obemy/

**Функциональные возможности и действия пользователя**

Для выбранной категории мощности рассчитываются ожидаемые массы загрязняющих веществ и CO2 по видам топлива и территориям. Базовая формула М = В × У. При расходе в тыс. т у.т. и нормативе в кг/т у.т. результат ЗВ получается в тоннах; для CO2 норматив в т/т у.т. дает тыс. т. В суммарных формах единицы приводятся явно. Расчетные ячейки не редактируются напрямую.

**Взаимодействие и источники данных**

Используются два раздельных принятых снимка «Топлива» для новых ТЭС. Ключи capacity_class=le100 и gt100 сохраняются в навигации и сопоставлены с категориями пакетов new_lt100 и new_ge100. По критериям сводки передача разделяется по заполненной мощности блока n1: менее 100 МВт и 100 МВт или более. Обозначения le100/gt100 здесь являются внутренними ключами и не заменяют критерии отбора передаваемого пакета. Результаты обеих категорий включаются в сводные расчеты.

**Элементы реализации**

Модель NewTesNorm. Источник: app/ecology/models/new_tes_norm_model.py, строки 11–14.

```
 11  class NewTesNorm(db.Model, AuditMixin):
 12      """Удельный выброс по виду топлива, классу мощности и году."""
 13
 14      __tablename__ = "gs_eco_new_tes_norms"
```

Модель EcologyFuelSnapshot. Источник: app/ecology/models/fuel_exchange_model.py, строки 9–21.

```
  9  class EcologyFuelSnapshot(db.Model, AuditMixin):
 10      __tablename__ = "gs_eco_fuel_snapshots"
 11      __table_args__ = (
 12          db.UniqueConstraint("database_version_id", "year_number", "scope",
name="uq_eco_fuel_snapshot_version_year_scope"),
 13          {"schema": SCHEMA_ECOLOGY},
 14      )
 15
 16      id = db.Column(db.Integer, primary_key=True)
 17      database_version_id = db.Column(db.Integer,
db.ForeignKey(f"{SCHEMA_REFDATA}.gs_database_versions.id", ondelete="CASCADE"), nullable=False)
 18      year_number = db.Column(db.Integer, nullable=False)
 19      scope = db.Column(db.String(20), nullable=False, default="operating",
server_default="operating")
 20      payload = db.Column(db.JSON, nullable=False)
 21      updated_at = db.Column(db.DateTime(timezone=True), server_default=func.now(),
onupdate=func.now(), nullable=False)
```

Маршрут страницы. Источник: app/ecology/routes/new_tes_routes.py, строки 58–72.

```
@ecology_bp.route("/new_tes/<territory_slug>/<sheet_slug>/")
@login_required
def new_tes_table(territory_slug: str, sheet_slug: str):
    context = build_new_tes_table_context(
        territory_slug,
        sheet_slug,
        start_year=request.args.get("start_year"),
        end_year=request.args.get("end_year"),
        rounding_digits=request.args.get("rounding_digits"),
        view_year=request.args.get("view_year"),
        capacity_class=request.args.get("capacity_class", CAPACITY_LE100_KEY),
    )
    if context is None:
        abort(404)
    sheet = context.get("sheet") or {}
```

Сервис и расчетная логика. Источник: app/ecology/services/new_tes_calc_services.py, строки 12–18.

```
def zv_mass_tonnes(b_tce: Any, u_kg_per_tce: Any) -> Decimal | None:
    """М, т = В, тыс. т у.т. × У, кг/т у.т."""
    b = _d(b_tce)
    u = _d(u_kg_per_tce)
    if b is None or u is None:
        return None
    return b * u
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/operating_tes/operating_tes_table.html, строки 539–548.

```
      data-rounding-digits="{{ rounding_digits }}"
      data-coefficient-rounding-digits="{{ coefficient_rounding_digits|default(0) }}"
    >
      {% if new_tes_volume_columns %}
      {% include 'ecology/operating_tes/_new_tes_volume_table.html' %}
      {% elif dynamics_territories %}
      {% include 'ecology/operating_tes/_volume_table_content.html' %}
      {% elif year_columns_mode %}
      {% include 'ecology/operating_tes/_range_table.html' %}
      {% elif sheet.layout == 'chars' %}
```

Сервис новых ТЭС. Источник: app/ecology/services/new_tes_page_services.py, строки 193–218.

```
def compute_new_tes_year_bundle(
    territory_slug: str,
    year: int,
    by_class: dict[str, dict[str, Decimal]] | None,
    norm_overrides: dict[tuple[str, str, int], dict[str, Any]],
    char_overrides: dict | None = None,
) -> dict[str, Any]:
    by_key: dict[str, dict[str, Decimal | None]] = {}
    for spec in FUEL_ROWS:
        if spec.get("is_header"):
            continue
        by_key[spec["key"]] = _leaf_emissions(
            spec["key"], by_class, year, norm_overrides, char_overrides
        )
    header_specs = sorted(
        [spec for spec in FUEL_ROWS if spec.get("is_header")],
        key=lambda item: int(item.get("indent") or 0),
        reverse=True,
    )
    for spec in header_specs:
        children = tuple(spec.get("children") or PARENT_CHILDREN.get(spec["key"]) or ())
        computed = {metric: None for metric in _BUNDLE_METRICS}
        for metric in _BUNDLE_METRICS:
            computed[metric] = _sum_present(
                [(by_key.get(child) or {}).get(metric) for child in children]
            )
```

## 12. Сводные данные расчетов

### 12.1. Общие топливные показатели действующих и новых ТЭС

Адрес: /ecology/calculation_summary/vsego/toplivo/

**Функциональные возможности и действия пользователя**

Суммируются принятые топливные и энергетические показатели действующих ТЭС и двух категорий новых ТЭС. Строки и территориальные столбцы сопоставляются по внутренним ключам, а не по положению в таблице. Форма позволяет выбирать период и год и не разрешает прямую правку суммарных ячеек. Предусмотрена выгрузка сводных расчетных вкладок в Excel с последовательными годовыми блоками.

**Взаимодействие и источники данных**

Объединяются результаты внутренних сервисов действующих и новых ТЭС. Для сравнения сохраняется один и тот же год, территория и версия БД. Модуль «Топливо» поставляет принятую исходную базу; справочники задают доступные годы и состав территорий. Сводная форма не записывает рассчитанные итоги обратно в исходные отчетные таблицы ГИС ТЭК.

**Элементы реализации**

Модель EcologyFuelSnapshot. Источник: app/ecology/models/fuel_exchange_model.py, строки 9–21.

```
  9  class EcologyFuelSnapshot(db.Model, AuditMixin):
 10      __tablename__ = "gs_eco_fuel_snapshots"
 11      __table_args__ = (
 12          db.UniqueConstraint("database_version_id", "year_number", "scope",
name="uq_eco_fuel_snapshot_version_year_scope"),
 13          {"schema": SCHEMA_ECOLOGY},
 14      )
 15
 16      id = db.Column(db.Integer, primary_key=True)
 17      database_version_id = db.Column(db.Integer,
db.ForeignKey(f"{SCHEMA_REFDATA}.gs_database_versions.id", ondelete="CASCADE"), nullable=False)
 18      year_number = db.Column(db.Integer, nullable=False)
 19      scope = db.Column(db.String(20), nullable=False, default="operating",
server_default="operating")
 20      payload = db.Column(db.JSON, nullable=False)
 21      updated_at = db.Column(db.DateTime(timezone=True), server_default=func.now(),
onupdate=func.now(), nullable=False)
```

Модель NewTesNorm. Источник: app/ecology/models/new_tes_norm_model.py, строки 11–14.

```
 11  class NewTesNorm(db.Model, AuditMixin):
 12      """Удельный выброс по виду топлива, классу мощности и году."""
 13
 14      __tablename__ = "gs_eco_new_tes_norms"
```

Маршрут страницы. Источник: app/ecology/routes/calculation_summary_routes.py, строки 34–54.

```
 34  @ecology_bp.route("/calculation_summary/<territory_slug>/<sheet_slug>/")
 35  @login_required
 36  def calculation_summary(territory_slug, sheet_slug):
 37      if get_territory(territory_slug) is None:
 38          abort(404)
 39      context = build_calculation_summary_context(
 40          territory_slug, sheet_slug,
 41          **{key: request.args.get(key) for key in (
 42              "start_year", "end_year", "view_year", "rounding_digits",
"coefficient_rounding_digits",
 43          )},
 44      )
 45      if context is None:
 46          abort(404)
 47      log_page_visit(
 48          "Сводные данные расчетов", tab=context["sheet"]["sheet_name"],
 49          details=context_lines(
 50              ("период", f"{context['start_year']}–{context['end_year']}"),
 51              ("год", context["view_year"]),
 52          ),
 53      )
 54      return render_template("ecology/operating_tes/operating_tes_table.html", **context)
```

Сервис и расчетная логика. Источник: app/ecology/services/calculation_summary_services.py, строки 32–41.

```
  def _sum_rows(context, sources):
      source_rows = [
          {row["key"]: row for row in source["rows"]} for source in sources
      ]
      source_columns = [
          {col.get("territory_slug", col["slug"]): col["slug"] for col in source["value_columns"]}
          for source in sources
      ]
      territories = {col["slug"]: col.get("territory_slug", col["slug"]) for col in
context["value_columns"]}
      rows = []
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/operating_tes/operating_tes_table.html, строки 787–820.

```
            <th class="text-center align-middle ot-label-col">Наименование
              {% if sheet.layout in ('fuel', 'coeff') %}
              {% include 'ecology/operating_tes/_fuel_filter.html' %}
              {% endif %}
              {% if territory_filter_storage_key %}
              {% include 'ecology/operating_tes/_territory_filter.html' %}
              {% endif %}
            </th>
            {% for col in territory_columns %}
            <th class="text-center align-middle ot-terr-col">
              {{ col.sheet_name }}{% if sheet.layout == 'coeff' %}
              <div class="small">{% if affiliation == 'minen' %}Станции отрасли{% elif affiliation ==
'prompred' %}Промпредприятия{% else %}Все принадлежности{% endif %}, {{ view_year }}</div>
              {% endif %}
            </th>
            {% endfor %}
          </tr>
        </thead>
        <tbody>
          {% for row in rows %}
          <tr
            class="{{ row.css_class }}{% if row.is_empty %} ot-row-empty-hidden{% endif %}"
            data-row-key="{{ row.key }}"
            data-editable-field="{{ row.editable_field or '' }}"
          >
            <td
              class="text-start ot-row-label{% if row.indent %} ot-indent-{{ row.indent }}{% endif
%}{% if row.kind == 'total' %} fw-bold{% endif %}{% if row.italic %} fst-italic{% endif %}"
            >
              {{ row.label }}{% if row.key in ('ee_generation', 'heat_output') and row.unit %}, {{
row.unit }}{% endif %} {% if row.formula_tooltip %}
              <i
                class="bi bi-info-circle text-primary ms-1"
                data-bs-toggle="tooltip"
                data-bs-placement="top"
                {% if row.formula_tooltip_html %}data-bs-html="true" title="{{
row.formula_tooltip_html|forceescape }}"{% else %}title="{{ row.formula_tooltip|e }}"{% endif %}
                style="cursor: help; font-size: 0.9em"
```

Сборка сводного контекста. Источник: app/ecology/services/calculation_summary_services.py, строки 185–196.

```
            return None
        if sheet_slug == "toplivo":
            context = build_operating_tes_table_context(territory_slug, source_sheet, **filters)
            context["rows"] = _sum_rows(context, [context, le100, gt100])
            context["page_title"] = "Топливные показатели действующих и новых ТЭС (модуль Топливо)"
            context["page_title_html"] = context["page_title"]
        else:
            new_sources = [_transpose_new_volumes(source) for source in (le100, gt100)]
            context = {**new_sources[0], "rows": _sum_rows(new_sources[0], new_sources)}
            if sheet_slug in {"russia", "specific"}:
                operating = build_operating_tes_table_context(territory_slug, source_sheet,
**filters)
                context = {**operating, "rows": _sum_rows(operating, [operating, context])}
```

## 12.2. Объемы выбросов по России

Адрес: /ecology/calculation_summary/vsego/russia/

**Функциональные возможности и действия пользователя**

Выводятся общий объем и отдельные блоки действующих ТЭС, новых ТЭС le100 и новых ТЭС gt100. Перед сложением массы ЗВ новых ТЭС переводятся из тонн в тыс. т там, где этого требует общая форма; отдельная строка CO остается в тоннах, CO2 в тыс. т. Расчет использует принятые данные, а не сумму видимых округленных строк браузера. Предусмотрена выгрузка сводных расчетных вкладок в Excel с последовательными годовыми блоками.

**Взаимодействие и источники данных**

Объединяются результаты внутренних сервисов действующих и новых ТЭС. Для сравнения сохраняется один и тот же год, территория и версия БД. Модуль «Топливо» поставляет принятую исходную базу; справочники задают доступные годы и состав территорий. Сводная форма не записывает рассчитанные итоги обратно в исходные отчетные таблицы ГИС ТЭК.

**Элементы реализации**

Модель EcologyFuelSnapshot. Источник: app/ecology/models/fuel_exchange_model.py, строки 9–21.

```
  9  class EcologyFuelSnapshot(db.Model, AuditMixin):
 10      __tablename__ = "gs_eco_fuel_snapshots"
 11      __table_args__ = (
 12          db.UniqueConstraint("database_version_id", "year_number", "scope",
name="uq_eco_fuel_snapshot_version_year_scope"),
 13          {"schema": SCHEMA_ECOLOGY},
 14      )
 15
 16      id = db.Column(db.Integer, primary_key=True)
 17      database_version_id = db.Column(db.Integer,
db.ForeignKey(f"{SCHEMA_REFDATA}.gs_database_versions.id", ondelete="CASCADE"), nullable=False)
 18      year_number = db.Column(db.Integer, nullable=False)
 19      scope = db.Column(db.String(20), nullable=False, default="operating",
server_default="operating")
 20      payload = db.Column(db.JSON, nullable=False)
 21      updated_at = db.Column(db.DateTime(timezone=True), server_default=func.now(),
onupdate=func.now(), nullable=False)
```

Модель NewTesNorm. Источник: app/ecology/models/new_tes_norm_model.py, строки 11–14.

```
 11  class NewTesNorm(db.Model, AuditMixin):
 12      """Удельный выброс по виду топлива, классу мощности и году."""
 13
 14      __tablename__ = "gs_eco_new_tes_norms"
```

Маршрут страницы. Источник: app/ecology/routes/calculation_summary_routes.py, строки 34–54.

```
 34  @ecology_bp.route("/calculation_summary/<territory_slug>/<sheet_slug>/")
 35  @login_required
 36  def calculation_summary(territory_slug, sheet_slug):
 37      if get_territory(territory_slug) is None:
 38          abort(404)
 39      context = build_calculation_summary_context(
 40          territory_slug, sheet_slug,
 41          **{key: request.args.get(key) for key in (
 42              "start_year", "end_year", "view_year", "rounding_digits",
"coefficient_rounding_digits",
 43          )},
 44      )
 45      if context is None:
 46          abort(404)
 47      log_page_visit(
 48          "Сводные данные расчетов", tab=context["sheet"]["sheet_name"],
 49          details=context_lines(
 50              ("период", f"{context['start_year']}–{context['end_year']}"),
 51              ("год", context["view_year"]),
 52          ),
 53      )
 54      return render_template("ecology/operating_tes/operating_tes_table.html", **context)
```

Сервис и расчетная логика. Источник: app/ecology/services/calculation_summary_services.py, строки 59–79.

```
 59  def _transpose_new_volumes(context):
 60      total = context["rows"][0]
 61      columns = context["territory_columns"]
 62      rows = []
 63      for spec in VOLUME_ROWS:
 64          metric = spec["key"]
 65
 66          def amount(slug):
 67              value = decimal_or_none(total["col_values_raw"][f"{slug}:{metric}"])
 68              # В исходной таблице новых ТЭС ЗВ – в тоннах. Форма действующих
 69              # ТЭС использует тыс. т, кроме отдельной строки CO (тонны).
 70              if value is not None and metric not in {"co", "co2"}:
 71                  value /= Decimal("1000")
 72              return value
 73
 74          values, raw = _cells_for_territories(columns, amount, context["rounding_digits"])
 75          rows.append(_row_dict(
 76              key=metric, label=spec["label"], unit="т" if metric == "co" else "тыс. т",
 77              kind=spec["kind"], indent=spec["indent"],
 78              col_values=values, col_values_raw=raw,
 79          ))
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/operating_tes/operating_tes_table.html, строки 853–862.

```
 853      {% for volume_table in summary_volume_tables %}
 854      <h2 class="h3 fw-bold mt-4 mb-3" id="ot-summary-{{ volume_table.key }}-title">
 855        {{ volume_table.title }}
 856      </h2>
 857      <table class="table table-hover table-striped table-bordered align-middle power-demand-summary-
table ot-calculation-table ot-fit-table ot-hide-empty"
 858             id="ot-summary-{{ volume_table.key }}" aria-labelledby="ot-summary-{{ volume_table.key
}}-title" data-can-edit="0">
 859        {% with rows = volume_table.rows %}
 860        {% include 'ecology/operating_tes/_volume_table_content.html' %}
 861        {% endwith %}
 862      </table>
```

Сборка сводного контекста. Источник: app/ecology/services/calculation_summary_services.py, строки 191–206.

```
 191          else:
 192              new_sources = [_transpose_new_volumes(source) for source in (le100, gt100)]
 193              context = {**new_sources[0], "rows": _sum_rows(new_sources[0], new_sources)}
 194              if sheet_slug in {"russia", "specific"}:
 195                  operating = build_operating_tes_table_context(territory_slug, source_sheet,
**filters)
 196                  context = {**operating, "rows": _sum_rows(operating, [operating, context])}
 197                  context["specific_emission_tables"] = []
 198                  if sheet_slug == "russia":
 199                      context["main_table_title"] = "Всего"
 200                      context["summary_volume_tables"] = [
 201                          {"key": "operating", "title": "Действующие ТЭС", "rows":
operating["rows"]},
 202                          {"key": "new-gt100", "title": "Новые ТЭС – 100 МВт и более",
 203                           "rows": _sum_rows(operating, [new_sources[1]])},
 204                          {"key": "new-le100", "title": "Новые ТЭС – менее 100 МВт",
 205                           "rows": _sum_rows(operating, [new_sources[0]])},
 206                      ]
```

## 12.3. Удельные выбросы по России

Адрес: /ecology/calculation_summary/vsego/specific/

**Функциональные возможности и действия пользователя**

Из общего объема выбросов и общей топливно-энергетической базы формируются удельные показатели всех ТЭС. Сначала суммируются массы и соответствующие базы, затем вычисляются удельные значения. Простое сложение или среднее арифметическое удельных выбросов отдельных категорий не используется. Предусмотрена выгрузка сводных расчетных вкладок в Excel с последовательными годовыми блоками.

**Взаимодействие и источники данных**

Объединяются результаты внутренних сервисов действующих и новых ТЭС. Для сравнения сохраняется один и тот же год, территория и версия БД. Модуль «Топливо» поставляет принятую исходную базу; справочники задают доступные годы и состав территорий. Сводная форма не записывает рассчитанные итоги обратно в исходные отчетные таблицы ГИС ТЭК.

**Элементы реализации**

Модель EcologyFuelSnapshot. Источник: app/ecology/models/fuel_exchange_model.py, строки 9–21.

```
  9  class EcologyFuelSnapshot(db.Model, AuditMixin):
 10      __tablename__ = "gs_eco_fuel_snapshots"
 11      __table_args__ = (
 12          db.UniqueConstraint("database_version_id", "year_number", "scope",
name="uq_eco_fuel_snapshot_version_year_scope"),
 13          {"schema": SCHEMA_ECOLOGY},
 14      )
 15
 16      id = db.Column(db.Integer, primary_key=True)
 17      database_version_id = db.Column(db.Integer,
db.ForeignKey(f"{SCHEMA_REFDATA}.gs_database_versions.id", ondelete="CASCADE"), nullable=False)
 18      year_number = db.Column(db.Integer, nullable=False)
 19      scope = db.Column(db.String(20), nullable=False, default="operating",
server_default="operating")
 20      payload = db.Column(db.JSON, nullable=False)
 21      updated_at = db.Column(db.DateTime(timezone=True), server_default=func.now(),
onupdate=func.now(), nullable=False)
```

Модель NewTesNorm. Источник: app/ecology/models/new_tes_norm_model.py, строки 11–14.

```
 11  class NewTesNorm(db.Model, AuditMixin):
 12      """Удельный выброс по виду топлива, классу мощности и году."""
 13
 14      __tablename__ = "gs_eco_new_tes_norms"
```

Маршрут страницы. Источник: app/ecology/routes/calculation_summary_routes.py, строки 34–54.

```
 34  @ecology_bp.route("/calculation_summary/<territory_slug>/<sheet_slug>/")
 35  @login_required
 36  def calculation_summary(territory_slug, sheet_slug):
 37      if get_territory(territory_slug) is None:
 38          abort(404)
 39      context = build_calculation_summary_context(
 40          territory_slug, sheet_slug,
 41          **{key: request.args.get(key) for key in (
 42              "start_year", "end_year", "view_year", "rounding_digits",
"coefficient_rounding_digits",
 43          )},
 44      )
 45      if context is None:
 46          abort(404)
 47      log_page_visit(
 48          "Сводные данные расчетов", tab=context["sheet"]["sheet_name"],
 49          details=context_lines(
 50              ("период", f"{context['start_year']}–{context['end_year']}"),
 51              ("год", context["view_year"]),
 52          ),
 53      )
 54      return render_template("ecology/operating_tes/operating_tes_table.html", **context)
```

Сервис и расчетная логика. Источник: app/ecology/services/calculation_summary_services.py, строки 86–105.

```
 86  def _build_russia_specific_emission_tables(context, fuel_context):
 87      inputs = {}
 88      bundles = {}
 89      for column in context["value_columns"]:
 90          slug = column["slug"]
 91          bucket = {}
 92          for row in fuel_context["rows"]:
 93              amount = decimal_or_none(row["col_values_raw"][slug])
 94              if row["key"] in TOP_INPUT_FIELD:
 95                  apply_input_override_to_bucket(bucket, row["key"], amount)
 96              elif amount is not None:
 97                  bucket[row["key"]] = amount
 98          inputs.setdefault(column["year"], {})[column["territory_slug"]] = bucket
 99          bundles[slug] = {"volumes": {
100              row["key"]: decimal_or_none(row["col_values_raw"][slug])
101              for row in context["rows"]
102          }}
103      return _build_specific_emission_tables(
104          context["value_columns"], bundles, inputs, context["rounding_digits"],
105      )
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/operating_tes/operating_tes_table.html, строки 863–879.

```
863      {% endfor %}
864      {% for specific_table in specific_emission_tables %}
865      <div class="d-flex align-items-center justify-content-between gap-2 mt-4 mb-3">
866        <h2 class="h3 fw-bold mb-0 flex-grow-1" id="ot-specific-{{ specific_table.key }}-title">
867          {{ specific_table.title }}
868        </h2>
869        {% if specific_table.key == 'energy' %}
870        {{ rounding_control('specific_rounding_digits', 'Округл (удельные)',
specific_rounding_digits) }}
871        {% endif %}
872      </div>
873      <table class="table table-hover table-striped table-bordered align-middle power-demand-summary-
table ot-calculation-table ot-fit-table ot-specific-table ot-hide-empty"
874             id="ot-specific-{{ specific_table.key }}" aria-labelledby="ot-specific-{{
specific_table.key }}-title" data-can-edit="0">
875        {% with rows = specific_table.rows %}
876        {% include 'ecology/operating_tes/_volume_table_content.html' %}
877        {% endwith %}
878      </table>
879      {% endfor %}
```

Сборка сводного контекста. Источник: app/ecology/services/calculation_summary_services.py, строки 206–212.

```
206                      ]
207                  else:
208                      fuel_context = build_calculation_summary_context(territory_slug, "toplivo",
**filters)
209                      context["specific_emission_tables"] =
_build_russia_specific_emission_tables(context, fuel_context)
210                      context["specific_emissions_only"] = True
211                      context["page_title"] = "Удельные выбросы по России (все ТЭС)"
212                      context["page_title_html"] = context["page_title"]
```

## 12.4. Динамика изменения выбросов

Адрес: /ecology/calculation_summary/vsego/dynamics/

**Функциональные возможности и действия пользователя**

Сравниваются показатели по годам выбранного периода. Доступны территориальные вкладки динамики; строки формируются на основе общих расчетов действующих и новых ТЭС. Форма предназначена для просмотра временного изменения показателей и не создает отдельного набора введенных выбросов. Предусмотрена выгрузка сводных расчетных вкладок в Excel с последовательными годовыми блоками.

**Взаимодействие и источники данных**

Объединяются результаты внутренних сервисов действующих и новых ТЭС. Для сравнения сохраняется один и тот же год, территория и версия БД. Модуль «Топливо» поставляет принятую исходную базу; справочники задают доступные годы и состав территорий. Сводная форма не записывает рассчитанные итоги обратно в исходные отчетные таблицы ГИС ТЭК.

**Элементы реализации**

Модель EcologyFuelSnapshot. Источник: app/ecology/models/fuel_exchange_model.py, строки 9–21.

```
  9  class EcologyFuelSnapshot(db.Model, AuditMixin):
 10      __tablename__ = "gs_eco_fuel_snapshots"
 11      __table_args__ = (
 12          db.UniqueConstraint("database_version_id", "year_number", "scope",
name="uq_eco_fuel_snapshot_version_year_scope"),
 13          {"schema": SCHEMA_ECOLOGY},
 14      )
 15
 16      id = db.Column(db.Integer, primary_key=True)
 17      database_version_id = db.Column(db.Integer,
db.ForeignKey(f"{SCHEMA_REFDATA}.gs_database_versions.id", ondelete="CASCADE"), nullable=False)
 18      year_number = db.Column(db.Integer, nullable=False)
 19      scope = db.Column(db.String(20), nullable=False, default="operating",
server_default="operating")
 20      payload = db.Column(db.JSON, nullable=False)
 21      updated_at = db.Column(db.DateTime(timezone=True), server_default=func.now(),
onupdate=func.now(), nullable=False)
```

Модель NewTesNorm. Источник: app/ecology/models/new_tes_norm_model.py, строки 11–14.

```
 11  class NewTesNorm(db.Model, AuditMixin):
 12      """Удельный выброс по виду топлива, классу мощности и году."""
 13
 14      __tablename__ = "gs_eco_new_tes_norms"
```

Маршрут страницы. Источник: app/ecology/routes/calculation_summary_routes.py, строки 34–54.

```
 34  @ecology_bp.route("/calculation_summary/<territory_slug>/<sheet_slug>/")
 35  @login_required
 36  def calculation_summary(territory_slug, sheet_slug):
 37      if get_territory(territory_slug) is None:
 38          abort(404)
 39      context = build_calculation_summary_context(
 40          territory_slug, sheet_slug,
 41          **{key: request.args.get(key) for key in (
 42              "start_year", "end_year", "view_year", "rounding_digits",
"coefficient_rounding_digits",
 43          )},
 44      )
 45      if context is None:
 46          abort(404)
 47      log_page_visit(
 48          "Сводные данные расчетов", tab=context["sheet"]["sheet_name"],
 49          details=context_lines(
 50              ("период", f"{context['start_year']}–{context['end_year']}"),
 51              ("год", context["view_year"]),
 52          ),
 53      )
 54      return render_template("ecology/operating_tes/operating_tes_table.html", **context)
```

Сервис и расчетная логика. Источник: app/ecology/services/calculation_summary_services.py, строки 108–120.

```
108  def _build_emission_dynamics(territory_slug, filters):
109      context = build_calculation_summary_context(territory_slug, "russia", **filters)
110      annual = {
111          year: context if year == context["view_year"] else build_calculation_summary_context(
112              territory_slug, "russia", **{**filters, "view_year": year},
113          )
114          for year in context["range_years"]
115      }
116      columns = [
117          {"slug": f"{territory_slug}:{year}", "territory_slug": territory_slug,
118           "year": year, "sheet_name": str(year)}
119          for year in annual
120      ]
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/operating_tes/_sheet_nav.html, строки 34–42.

```
  {% if dynamics_territories %}
  <div class="d-flex flex-wrap gap-2" role="navigation" aria-label="Территории динамики выбросов">
    {% for nav in dynamics_territories %}
    <a href="{{ url_for(table_endpoint, territory_slug=nav.slug, sheet_slug=sheet_slug,
start_year=start_year, end_year=end_year, rounding_digits=rounding_digits,
coefficient_rounding_digits=coefficient_rounding_digits|default(none)) }}"
       class="btn {% if nav.slug == territory_slug %}btn-primary{% else %}btn-outline-primary{% endif
%} text-nowrap"{% if nav.slug == territory_slug %} aria-current="page"{% endif %}>
      {{ nav.sheet_name }}
    </a>
    {% endfor %}
  </div>
```

Сборка сводного контекста. Источник: app/ecology/services/calculation_summary_services.py, строки 108–134.

```
def _build_emission_dynamics(territory_slug, filters):
    context = build_calculation_summary_context(territory_slug, "russia", **filters)
    annual = {
        year: context if year == context["view_year"] else build_calculation_summary_context(
            territory_slug, "russia", **{**filters, "view_year": year},
        )
        for year in context["range_years"]
    }
    columns = [
        {"slug": f"{territory_slug}:{year}", "territory_slug": territory_slug,
         "year": year, "sheet_name": str(year)}
        for year in annual
    ]

    def range_rows(tables):
        rows_by_year = {
            year: {row["key"]: row for row in table["rows"]}
            for year, table in tables.items()
        }
        rows = []
        for row in context["rows"]:
            amounts = {
                column["slug"]: decimal_or_none(
                    rows_by_year[column["year"]][row["key"]]["col_values_raw"][column["slug"]]
                )
                for column in columns
            }
```

## 13. Агрегированные суммы отчетно-аналитического контура

После группировки исходные абсолютные показатели суммируются, а производные коэффициенты и удельные показатели вычисляются заново на суммарной базе. Складывать готовые удельные значения станций нельзя. Представления с котельными и без котельных имеют разные исходные выборки.

Суммирование с повторным вычислением производных показателей. Источник: app/ecology/services/gis_tes_fuel_comparison_services.py, строки 3014–3035.

```
3014  def _sum_filled_rows(
3015      filled_list: list[list[Any]],
3016      source_columns: list[dict[str, Any]],
3017      norm_zv_parts: list[str] | tuple[str, ...] = ZV_NORM_DEFAULT_PARTS,
3018  ) -> list[Any] | None:
3019      if not filled_list:
3020          return None
3021      width = max(len(source_columns), max((len(row) for row in filled_list), default=0))
3022      out: list[Any] = [None] * width
3023      for index, column in enumerate(source_columns):
3024          key = str(column.get("key") or "")
3025          if key in SUMMARY_IDENTITY_KEYS or key in SUMMARY_RECOMPUTE_KEYS:
3026              continue
3027          if not column.get("numeric"):
3028              continue
3029          parts = [
3030              row[index] if index < len(row) else None
3031              for row in filled_list
3032          ]
3033          out[index] = _safe_sum(*parts)
3034      # Пересчет d_*/K_*/CO2/UD_* поверх сумм слагаемых (если столбцы есть).
3035      return _compute_derived_columns(out, source_columns, norm_zv_parts)
```

### 13.1. ПГ: агрегированные суммы с котельными

Адрес: /ecology/totals_summary/pg-vse/

**Функциональные возможности и действия пользователя**

Агрегированная форма использует соответствующий лист анализа и показывает годовые итоговые блоки без детальных станционных строк. Суммируются исходные расходы и массы CO2; разности, коэффициенты эмиссии и удельные показатели формируются из суммарной базы. Можно одновременно выбрать ЕЭС, ТИТЭС, Россию, федеральные округа, ОЭС и синхронные зоны. Период, набор уровней и округление передаются в Excel. Итоги доступны только для просмотра.

**Взаимодействие и источники данных**

Источник числовых показателей совпадает с соответствующим листом анализа ПГ или ЗВ и учитывает вариант включения котельных. Состав и имена уровней получаются из справочников энергетических систем, синхронных зон и территорий; общий разбор aggregation_type используется из «Топлива», а разбор периода из «Генерации». Это другой контур, чем сводные расчетные выбросы действующих и новых ТЭС: отчетно-аналитические итоги не следует называть результатом инженерного прогноза.

**Элементы реализации**

Модель PGParamCalc. Источник: app/ecology/models/pg_param_calc_model.py, строки 11–21.

```
 11  class PGParamCalc(db.Model, AuditMixin):
 12      """Строка параметров страниц /ecology/analysis/pg-vse/ и pg-bez-kot/.
 13
 14      Первоначально копируется из PGReportGisTek; далее может
 15      редактироваться отдельно от отчетных данных.
 16      Связь с EquipmentGroup: station_numb (numb1120) = EquipmentGroup.numb
 17      (без FK на numb – поле не уникально).
 18      year_number + database_version_id → Year.number + Year.database_version_id.
 19      """
 20
 21      __tablename__ = "gs_eco_analysis_pg_params"
```

Маршрут страницы. Источник: app/ecology/routes/ecology_totals_summary_routes.py, строки 64–80.

```
@ecology_bp.route("/totals_summary/<sheet_slug>")
@ecology_bp.route("/totals_summary/<sheet_slug>/")
@login_required
def totals_summary(sheet_slug: str):
    context = _ecology_totals_page_context(sheet_slug)
    if context is None:
        abort(404)
    log_page_visit(
        "Итоговые суммы",
        tab=(context.get("sheet") or {}).get("page_title"),
        details=context_lines(
            ("лист", sheet_slug),
            ("период", f"{context.get('start_year')}–{context.get('end_year')}"),
            ("округление", context.get("rounding_digits")),
        ),
    )
    return render_template("ecology/totals_summary/ecology_totals_summary.html", **context)
```

Сервис и расчетная логика. Источник: app/ecology/services/gis_tes_fuel_comparison_services.py, строки 3112–3134.

```
def build_region_summary_block(
    groups: list[dict[str, Any]],
    label: str,
    *,
    source_columns: list[dict[str, Any]],
    visible_columns: list[dict[str, Any]],
    digits: int,
    key_prefix: str,
    emission_digits: int | None = None,
    specific_digits: int | None = None,
    norm_zv_parts: list[str] | tuple[str, ...] = ZV_NORM_DEFAULT_PARTS,
) -> dict[str, Any] | None:
    """Итоговые строки уровня (РЭС/ОЭС/ЕЭС) по годам."""
    by_year: dict[int, list[list[Any]]] = defaultdict(list)
    for group in groups:
        for year, filled in (group.get("filled_by_year") or {}).items():
            if year is None or not filled:
                continue
            by_year[int(year)].append(list(filled))
    if not by_year:
        return None
    summary_label = f"{label}, всего"
    rows: list[dict[str, Any]] = []
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/totals_summary/ecology_totals_summary.html, строки 91–118.

```
  91        <form method="get" action="{{ url_for('ecology_bp.totals_summary', sheet_slug=sheet.slug) }}"
id="totalsFilterForm" class="d-flex align-items-center gap-2 ms-auto flex-wrap">
  92          <input type="hidden" name="start_year" value="{{ start_year }}">
  93          <input type="hidden" name="end_year" value="{{ end_year }}">
  94
  95          <input type="checkbox" class="btn-check" id="aggregationEES" name="aggregation_type"
value="ees" {% if 'ees' in aggregation_types %}checked{% endif %}>
  96          <label for="aggregationEES" class="btn {% if 'ees' in aggregation_types %}btn-primary{%
else %}btn-outline-primary{% endif %} px-3 text-center" style="min-width: 80px;">ЕЭС</label>
  97
  98          <input type="checkbox" class="btn-check" id="aggregationTITES" name="aggregation_type"
value="tites" {% if 'tites' in aggregation_types %}checked{% endif %}>
  99          <label for="aggregationTITES" class="btn {% if 'tites' in aggregation_types %}btn-primary{%
else %}btn-outline-primary{% endif %} px-3 text-center" style="min-width: 80px;">ТИТЭС</label>
 100
 101          <input type="checkbox" class="btn-check" id="aggregationRussia" name="aggregation_type"
value="russia" {% if 'russia' in aggregation_types %}checked{% endif %}>
 102          <label for="aggregationRussia" class="btn {% if 'russia' in aggregation_types %}btn-
primary{% else %}btn-outline-primary{% endif %} px-3 text-center me-3" style="min-width:
80px;">Россия</label>
 103
 104          <input type="checkbox" class="btn-check" id="aggregationFO" name="aggregation_type"
value="fo" {% if 'fo' in aggregation_types %}checked{% endif %}>
 105          <label for="aggregationFO" class="btn {% if 'fo' in aggregation_types %}btn-primary{% else
%}btn-outline-primary{% endif %} px-3 text-center" style="min-width: 80px;">ФО</label>
 106
 107          <input type="checkbox" class="btn-check" id="aggregationOES" name="aggregation_type"
value="oes" {% if 'oes' in aggregation_types %}checked{% endif %}>
 108          <label for="aggregationOES" class="btn {% if 'oes' in aggregation_types %}btn-primary{%
else %}btn-outline-primary{% endif %} px-3 text-center me-3" style="min-width: 80px;">ОЭС</label>
 109
 110          {% if sorted_synchronous_area_ids %}
 111            {% for sa_id in sorted_synchronous_area_ids %}
 112              {% set sa_name = synchronous_area_names[sa_id] %}
 113              {% set sa_checkbox_id = 'aggregationSyncArea' + sa_id|string %}
 114              {% set sa_value = 'sync_area_' + sa_id|string %}
 115              {% set is_checked = sa_value in aggregation_types %}
 116              <input type="checkbox" class="btn-check sync-area-checkbox" id="{{ sa_checkbox_id }}"
name="aggregation_type" value="{{ sa_value }}" {% if is_checked %}checked{% endif %}>
 117              <label for="{{ sa_checkbox_id }}" class="btn {% if is_checked %}btn-primary{% else
%}btn-outline-primary{% endif %} px-3 text-center sync-area-label {% if loop.last %}me-3{% endif %}"
style="min-width: auto; white-space: nowrap;">{{ sa_name }}</label>
 118            {% endfor %}
```

Настройка уровней агрегации. Источник: app/ecology/services/ecology_totals_summary_services.py, строки 305–326.

```
 305      should_show_totals = {
 306          "energy_system_types": {},
 307          "union_energy_systems": False,
 308          "synchronous_areas": {},
 309          "federal_districts": False,
 310          "total": False,
 311      }
 312      if "russia" in agg_types:
 313          should_show_totals["total"] = True
 314      if "fo" in agg_types:
 315          should_show_totals["federal_districts"] = True
 316      if "oes" in agg_types:
 317          should_show_totals["union_energy_systems"] = True
 318      for est_id in sorted_energy_system_type_ids:
 319          name_u = (energy_system_type_names.get(est_id) or "").upper()
 320          if "ees" in agg_types and "ЕЭС" in name_u:
 321              should_show_totals["energy_system_types"][est_id] = True
 322          if "tites" in agg_types and "ТИТЭС" in name_u:
 323              should_show_totals["energy_system_types"][est_id] = True
 324      for sa_id in sorted_synchronous_area_ids:
 325          if f"sync_area_{sa_id}" in agg_types:
 326              should_show_totals["synchronous_areas"][sa_id] = True
```

## 13.2. ПГ: агрегированные суммы без котельных

Адрес: /ecology/totals_summary/pg-bez-kot/

**Функциональные возможности и действия пользователя**

Агрегированная форма использует соответствующий лист анализа и показывает годовые итоговые блоки без детальных станционных строк. Суммируются исходные расходы и массы CO2; разности, коэффициенты эмиссии и удельные показатели формируются из суммарной базы. Можно одновременно выбрать ЕЭС, ТИТЭС, Россию, федеральные округа, ОЭС и синхронные зоны. Период, набор уровней и округление передаются в Excel. Итоги доступны только для просмотра.

**Взаимодействие и источники данных**

Источник числовых показателей совпадает с соответствующим листом анализа ПГ или ЗВ и учитывает вариант включения котельных. Состав и имена уровней получаются из справочников энергетических систем, синхронных зон и территорий; общий разбор aggregation_type используется из «Топлива», а разбор периода из «Генерации». Это другой контур, чем сводные расчетные выбросы действующих и новых ТЭС: отчетно-аналитические итоги не следует называть результатом инженерного прогноза.

**Элементы реализации**

Модель PGParamCalc. Источник: app/ecology/models/pg_param_calc_model.py, строки 11–21.

```
11  class PGParamCalc(db.Model, AuditMixin):
12      """Строка параметров страниц /ecology/analysis/pg-vse/ и pg-bez-kot/.
13
14      Первоначально копируется из PGReportGisTek; далее может
15      редактироваться отдельно от отчетных данных.
16      Связь с EquipmentGroup: station_numb (numb1120) = EquipmentGroup.numb
17      (без FK на numb – поле не уникально).
18      year_number + database_version_id → Year.number + Year.database_version_id.
19      """
20
21      __tablename__ = "gs_eco_analysis_pg_params"
```

Маршрут страницы. Источник: app/ecology/routes/ecology_totals_summary_routes.py, строки 64–80.

```
64  @ecology_bp.route("/totals_summary/<sheet_slug>")
65  @ecology_bp.route("/totals_summary/<sheet_slug>/")
66  @login_required
67  def totals_summary(sheet_slug: str):
68      context = _ecology_totals_page_context(sheet_slug)
69      if context is None:
70          abort(404)
71      log_page_visit(
72          "Итоговые суммы",
73          tab=(context.get("sheet") or {}).get("page_title"),
74          details=context_lines(
75              ("лист", sheet_slug),
76              ("период", f"{context.get('start_year')}–{context.get('end_year')}"),
77              ("округление", context.get("rounding_digits")),
78          ),
79      )
80      return render_template("ecology/totals_summary/ecology_totals_summary.html", **context)
```

Сервис и расчетная логика. Источник: app/ecology/services/gis_tes_fuel_comparison_services.py, строки 3112–3134.

```
def build_region_summary_block(
    groups: list[dict[str, Any]],
    label: str,
    *,
    source_columns: list[dict[str, Any]],
    visible_columns: list[dict[str, Any]],
    digits: int,
    key_prefix: str,
    emission_digits: int | None = None,
    specific_digits: int | None = None,
    norm_zv_parts: list[str] | tuple[str, ...] = ZV_NORM_DEFAULT_PARTS,
) -> dict[str, Any] | None:
    """Итоговые строки уровня (РЭС/ОЭС/ЕЭС) по годам."""
    by_year: dict[int, list[list[Any]]] = defaultdict(list)
    for group in groups:
        for year, filled in (group.get("filled_by_year") or {}).items():
            if year is None or not filled:
                continue
            by_year[int(year)].append(list(filled))
    if not by_year:
        return None
    summary_label = f"{label}, всего"
    rows: list[dict[str, Any]] = []
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/totals_summary/ecology_totals_summary.html, строки 91–118.

```
      <form method="get" action="{{ url_for('ecology_bp.totals_summary', sheet_slug=sheet.slug) }}" id="totalsFilterForm" class="d-flex align-items-center gap-2 ms-auto flex-wrap">
        <input type="hidden" name="start_year" value="{{ start_year }}">
        <input type="hidden" name="end_year" value="{{ end_year }}">

        <input type="checkbox" class="btn-check" id="aggregationEES" name="aggregation_type" value="ees" {% if 'ees' in aggregation_types %}checked{% endif %}>
        <label for="aggregationEES" class="btn {% if 'ees' in aggregation_types %}btn-primary{% else %}btn-outline-primary{% endif %} px-3 text-center" style="min-width: 80px;">ЕЭС</label>

        <input type="checkbox" class="btn-check" id="aggregationTITES" name="aggregation_type" value="tites" {% if 'tites' in aggregation_types %}checked{% endif %}>
        <label for="aggregationTITES" class="btn {% if 'tites' in aggregation_types %}btn-primary{% else %}btn-outline-primary{% endif %} px-3 text-center" style="min-width: 80px;">ТИТЭС</label>

        <input type="checkbox" class="btn-check" id="aggregationRussia" name="aggregation_type" value="russia" {% if 'russia' in aggregation_types %}checked{% endif %}>
        <label for="aggregationRussia" class="btn {% if 'russia' in aggregation_types %}btn-primary{% else %}btn-outline-primary{% endif %} px-3 text-center me-3" style="min-width: 80px;">Россия</label>

        <input type="checkbox" class="btn-check" id="aggregationFO" name="aggregation_type" value="fo" {% if 'fo' in aggregation_types %}checked{% endif %}>
        <label for="aggregationFO" class="btn {% if 'fo' in aggregation_types %}btn-primary{% else %}btn-outline-primary{% endif %} px-3 text-center" style="min-width: 80px;">ФО</label>

        <input type="checkbox" class="btn-check" id="aggregationOES" name="aggregation_type" value="oes" {% if 'oes' in aggregation_types %}checked{% endif %}>
        <label for="aggregationOES" class="btn {% if 'oes' in aggregation_types %}btn-primary{% else %}btn-outline-primary{% endif %} px-3 text-center me-3" style="min-width: 80px;">ОЭС</label>

        {% if sorted_synchronous_area_ids %}
          {% for sa_id in sorted_synchronous_area_ids %}
            {% set sa_name = synchronous_area_names[sa_id] %}
            {% set sa_checkbox_id = 'aggregationSyncArea' + sa_id|string %}
            {% set sa_value = 'sync_area_' + sa_id|string %}
            {% set is_checked = sa_value in aggregation_types %}
            <input type="checkbox" class="btn-check sync-area-checkbox" id="{{ sa_checkbox_id }}" name="aggregation_type" value="{{ sa_value }}" {% if is_checked %}checked{% endif %}>
            <label for="{{ sa_checkbox_id }}" class="btn {% if is_checked %}btn-primary{% else %}btn-outline-primary{% endif %} px-3 text-center sync-area-label {% if loop.last %}me-3{% endif %}" style="min-width: auto; white-space: nowrap;">{{ sa_name }}</label>
          {% endfor %}
```

Настройка уровней агрегации. Источник: app/ecology/services/ecology_totals_summary_services.py, строки 305–326.

```
305      should_show_totals = {
306          "energy_system_types": {},
307          "union_energy_systems": False,
308          "synchronous_areas": {},
309          "federal_districts": False,
310          "total": False,
311      }
312      if "russia" in agg_types:
313          should_show_totals["total"] = True
314      if "fo" in agg_types:
315          should_show_totals["federal_districts"] = True
316      if "oes" in agg_types:
317          should_show_totals["union_energy_systems"] = True
318      for est_id in sorted_energy_system_type_ids:
319          name_u = (energy_system_type_names.get(est_id) or "").upper()
320          if "ees" in agg_types and "ЕЭС" in name_u:
321              should_show_totals["energy_system_types"][est_id] = True
322          if "tites" in agg_types and "ТИТЭС" in name_u:
323              should_show_totals["energy_system_types"][est_id] = True
324      for sa_id in sorted_synchronous_area_ids:
325          if f"sync_area_{sa_id}" in agg_types:
326              should_show_totals["synchronous_areas"][sa_id] = True
```

## 13.3. ЗВ: агрегированные суммы с котельными

Адрес: /ecology/totals_summary/zv-vse/

**Функциональные возможности и действия пользователя**

Агрегированная форма использует соответствующий лист анализа и показывает годовые итоговые блоки без детальных станционных строк. Суммируются массы ЗВ и топливно-энергетическая база; производные и удельные показатели пересчитываются для агрегата. Можно одновременно выбрать ЕЭС, ТИТЭС, Россию, федеральные округа, ОЭС и синхронные зоны. Период, набор уровней и округление передаются в Excel. Итоги доступны только для просмотра.

**Взаимодействие и источники данных**

Источник числовых показателей совпадает с соответствующим листом анализа ПГ или ЗВ и учитывает вариант включения котельных. Состав и имена уровней получаются из справочников энергетических систем, синхронных зон и территорий; общий разбор aggregation_type используется из «Топлива», а разбор периода из «Генерации». Это другой контур, чем сводные расчетные выбросы действующих и новых ТЭС: отчетно-аналитические итоги не следует называть результатом инженерного прогноза.

**Элементы реализации**

Модель ZVParamCalc. Источник: app/ecology/models/zv_param_calc_model.py, строки 11–21.

```
11  class ZVParamCalc(db.Model, AuditMixin):
12      """Строка параметров страниц /ecology/analysis/zv-vse/ и zv-bez-kot/.
13
14      Первоначально копируется из ZVReportGisTek; далее может редактироваться
15      отдельно от отчетных данных.
16      Связь с EquipmentGroup: station_numb (numb1120) = EquipmentGroup.numb
17      (без FK на numb – поле не уникально).
18      year_number + database_version_id → Year.number + Year.database_version_id.
19      """
20
21      __tablename__ = "gs_eco_analysis_zv_params"
```

Маршрут страницы. Источник: app/ecology/routes/ecology_totals_summary_routes.py, строки 64–80.

```
@ecology_bp.route("/totals_summary/<sheet_slug>")
@ecology_bp.route("/totals_summary/<sheet_slug>/")
@login_required
def totals_summary(sheet_slug: str):
    context = _ecology_totals_page_context(sheet_slug)
    if context is None:
        abort(404)
    log_page_visit(
        "Итоговые суммы",
        tab=(context.get("sheet") or {}).get("page_title"),
        details=context_lines(
            ("лист", sheet_slug),
            ("период", f"{context.get('start_year')}–{context.get('end_year')}"),
            ("округление", context.get("rounding_digits")),
        ),
    )
    return render_template("ecology/totals_summary/ecology_totals_summary.html", **context)
```

Сервис и расчетная логика. Источник: app/ecology/services/gis_tes_fuel_comparison_services.py, строки 3112–3134.

```
def build_region_summary_block(
    groups: list[dict[str, Any]],
    label: str,
    *,
    source_columns: list[dict[str, Any]],
    visible_columns: list[dict[str, Any]],
    digits: int,
    key_prefix: str,
    emission_digits: int | None = None,
    specific_digits: int | None = None,
    norm_zv_parts: list[str] | tuple[str, ...] = ZV_NORM_DEFAULT_PARTS,
) -> dict[str, Any] | None:
    """Итоговые строки уровня (РЭС/ОЭС/ЕЭС) по годам."""
    by_year: dict[int, list[list[Any]]] = defaultdict(list)
    for group in groups:
        for year, filled in (group.get("filled_by_year") or {}).items():
            if year is None or not filled:
                continue
            by_year[int(year)].append(list(filled))
    if not by_year:
        return None
    summary_label = f"{label}, всего"
    rows: list[dict[str, Any]] = []
```

HTML-шаблон экранной формы. Источник:
app/templates/ecology/totals_summary/ecology_totals_summary.html, строки 91–118.

```
  91        <form method="get" action="{{ url_for('ecology_bp.totals_summary', sheet_slug=sheet.slug) }}"
id="totalsFilterForm" class="d-flex align-items-center gap-2 ms-auto flex-wrap">
  92          <input type="hidden" name="start_year" value="{{ start_year }}">
  93          <input type="hidden" name="end_year" value="{{ end_year }}">
  94
  95          <input type="checkbox" class="btn-check" id="aggregationEES" name="aggregation_type"
value="ees" {% if 'ees' in aggregation_types %}checked{% endif %}>
  96          <label for="aggregationEES" class="btn {% if 'ees' in aggregation_types %}btn-primary{%
else %}btn-outline-primary{% endif %} px-3 text-center" style="min-width: 80px;">ЕЭС</label>
  97
  98          <input type="checkbox" class="btn-check" id="aggregationTITES" name="aggregation_type"
value="tites" {% if 'tites' in aggregation_types %}checked{% endif %}>
  99          <label for="aggregationTITES" class="btn {% if 'tites' in aggregation_types %}btn-primary{%
else %}btn-outline-primary{% endif %} px-3 text-center" style="min-width: 80px;">ТИТЭС</label>
 100
 101          <input type="checkbox" class="btn-check" id="aggregationRussia" name="aggregation_type"
value="russia" {% if 'russia' in aggregation_types %}checked{% endif %}>
 102          <label for="aggregationRussia" class="btn {% if 'russia' in aggregation_types %}btn-
primary{% else %}btn-outline-primary{% endif %} px-3 text-center me-3" style="min-width:
80px;">Россия</label>
 103
 104          <input type="checkbox" class="btn-check" id="aggregationFO" name="aggregation_type"
value="fo" {% if 'fo' in aggregation_types %}checked{% endif %}>
 105          <label for="aggregationFO" class="btn {% if 'fo' in aggregation_types %}btn-primary{% else
%}btn-outline-primary{% endif %} px-3 text-center" style="min-width: 80px;">ФО</label>
 106
 107          <input type="checkbox" class="btn-check" id="aggregationOES" name="aggregation_type"
value="oes" {% if 'oes' in aggregation_types %}checked{% endif %}>
 108          <label for="aggregationOES" class="btn {% if 'oes' in aggregation_types %}btn-primary{%
else %}btn-outline-primary{% endif %} px-3 text-center me-3" style="min-width: 80px;">ОЭС</label>
 109
 110          {% if sorted_synchronous_area_ids %}
 111            {% for sa_id in sorted_synchronous_area_ids %}
 112              {% set sa_name = synchronous_area_names[sa_id] %}
 113              {% set sa_checkbox_id = 'aggregationSyncArea' + sa_id|string %}
 114              {% set sa_value = 'sync_area_' + sa_id|string %}
 115              {% set is_checked = sa_value in aggregation_types %}
 116              <input type="checkbox" class="btn-check sync-area-checkbox" id="{{ sa_checkbox_id }}"
name="aggregation_type" value="{{ sa_value }}" {% if is_checked %}checked{% endif %}>
 117              <label for="{{ sa_checkbox_id }}" class="btn {% if is_checked %}btn-primary{% else
%}btn-outline-primary{% endif %} px-3 text-center sync-area-label {% if loop.last %}me-3{% endif %}"
style="min-width: auto; white-space: nowrap;">{{ sa_name }}</label>
 118            {% endfor %}
```

Настройка уровней агрегации. Источник: app/ecology/services/ecology_totals_summary_services.py, строки 305–326.

```
 305      should_show_totals = {
 306          "energy_system_types": {},
 307          "union_energy_systems": False,
 308          "synchronous_areas": {},
 309          "federal_districts": False,
 310          "total": False,
 311      }
 312      if "russia" in agg_types:
 313          should_show_totals["total"] = True
 314      if "fo" in agg_types:
 315          should_show_totals["federal_districts"] = True
 316      if "oes" in agg_types:
 317          should_show_totals["union_energy_systems"] = True
 318      for est_id in sorted_energy_system_type_ids:
 319          name_u = (energy_system_type_names.get(est_id) or "").upper()
 320          if "ees" in agg_types and "ЕЭС" in name_u:
 321              should_show_totals["energy_system_types"][est_id] = True
 322          if "tites" in agg_types and "ТИТЭС" in name_u:
 323              should_show_totals["energy_system_types"][est_id] = True
 324      for sa_id in sorted_synchronous_area_ids:
 325          if f"sync_area_{sa_id}" in agg_types:
 326              should_show_totals["synchronous_areas"][sa_id] = True
```

## 13.4. ЗВ: агрегированные суммы без котельных

Адрес: /ecology/totals_summary/zv-bez-kot/

**Функциональные возможности и действия пользователя**

Агрегированная форма использует соответствующий лист анализа и показывает годовые итоговые блоки без детальных станционных строк. Суммируются массы ЗВ и топливно-энергетическая база; производные и удельные показатели пересчитываются для агрегата. Можно одновременно выбрать ЕЭС, ТИТЭС, Россию, федеральные округа, ОЭС и синхронные зоны. Период, набор уровней и округление передаются в Excel. Итоги доступны только для просмотра.

**Взаимодействие и источники данных**

Источник числовых показателей совпадает с соответствующим листом анализа ПГ или ЗВ и учитывает вариант включения котельных. Состав и имена уровней получаются из справочников энергетических систем, синхронных зон и территорий; общий разбор aggregation_type используется из «Топлива», а разбор периода из «Генерации». Это другой контур, чем сводные расчетные выбросы действующих и новых ТЭС: отчетно-аналитические итоги не следует называть результатом инженерного прогноза.

**Элементы реализации**

Модель ZVParamCalc. Источник: app/ecology/models/zv_param_calc_model.py, строки 11–21.

```
11  class ZVParamCalc(db.Model, AuditMixin):
12      """Строка параметров страниц /ecology/analysis/zv-vse/ и zv-bez-kot/.
13
14      Первоначально копируется из ZVReportGisTek; далее может редактироваться
15      отдельно от отчетных данных.
16      Связь с EquipmentGroup: station_numb (numb1120) = EquipmentGroup.numb
17      (без FK на numb – поле не уникально).
18      year_number + database_version_id → Year.number + Year.database_version_id.
19      """
20
21      __tablename__ = "gs_eco_analysis_zv_params"
```

Маршрут страницы. Источник: app/ecology/routes/ecology_totals_summary_routes.py, строки 64–80.

```
64  @ecology_bp.route("/totals_summary/<sheet_slug>")
65  @ecology_bp.route("/totals_summary/<sheet_slug>/")
66  @login_required
67  def totals_summary(sheet_slug: str):
68      context = _ecology_totals_page_context(sheet_slug)
69      if context is None:
70          abort(404)
71      log_page_visit(
72          "Итоговые суммы",
73          tab=(context.get("sheet") or {}).get("page_title"),
74          details=context_lines(
75              ("лист", sheet_slug),
76              ("период", f"{context.get('start_year')}–{context.get('end_year')}"),
77              ("округление", context.get("rounding_digits")),
78          ),
79      )
80      return render_template("ecology/totals_summary/ecology_totals_summary.html", **context)
```

Сервис и расчетная логика. Источник: app/ecology/services/gis_tes_fuel_comparison_services.py, строки 3112–3134.

```
def build_region_summary_block(
    groups: list[dict[str, Any]],
    label: str,
    *,
    source_columns: list[dict[str, Any]],
    visible_columns: list[dict[str, Any]],
    digits: int,
    key_prefix: str,
    emission_digits: int | None = None,
    specific_digits: int | None = None,
    norm_zv_parts: list[str] | tuple[str, ...] = ZV_NORM_DEFAULT_PARTS,
) -> dict[str, Any] | None:
    """Итоговые строки уровня (РЭС/ОЭС/ЕЭС) по годам."""
    by_year: dict[int, list[list[Any]]] = defaultdict(list)
    for group in groups:
        for year, filled in (group.get("filled_by_year") or {}).items():
            if year is None or not filled:
                continue
            by_year[int(year)].append(list(filled))
    if not by_year:
        return None
    summary_label = f"{label}, всего"
    rows: list[dict[str, Any]] = []
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/totals_summary/ecology_totals_summary.html, строки 91–118.

```
      <form method="get" action="{{ url_for('ecology_bp.totals_summary', sheet_slug=sheet.slug) }}" id="totalsFilterForm" class="d-flex align-items-center gap-2 ms-auto flex-wrap">
        <input type="hidden" name="start_year" value="{{ start_year }}">
        <input type="hidden" name="end_year" value="{{ end_year }}">

        <input type="checkbox" class="btn-check" id="aggregationEES" name="aggregation_type" value="ees" {% if 'ees' in aggregation_types %}checked{% endif %}>
        <label for="aggregationEES" class="btn {% if 'ees' in aggregation_types %}btn-primary{% else %}btn-outline-primary{% endif %} px-3 text-center" style="min-width: 80px;">ЕЭС</label>

        <input type="checkbox" class="btn-check" id="aggregationTITES" name="aggregation_type" value="tites" {% if 'tites' in aggregation_types %}checked{% endif %}>
        <label for="aggregationTITES" class="btn {% if 'tites' in aggregation_types %}btn-primary{% else %}btn-outline-primary{% endif %} px-3 text-center" style="min-width: 80px;">ТИТЭС</label>

        <input type="checkbox" class="btn-check" id="aggregationRussia" name="aggregation_type" value="russia" {% if 'russia' in aggregation_types %}checked{% endif %}>
        <label for="aggregationRussia" class="btn {% if 'russia' in aggregation_types %}btn-primary{% else %}btn-outline-primary{% endif %} px-3 text-center me-3" style="min-width: 80px;">Россия</label>

        <input type="checkbox" class="btn-check" id="aggregationFO" name="aggregation_type" value="fo" {% if 'fo' in aggregation_types %}checked{% endif %}>
        <label for="aggregationFO" class="btn {% if 'fo' in aggregation_types %}btn-primary{% else %}btn-outline-primary{% endif %} px-3 text-center" style="min-width: 80px;">ФО</label>

        <input type="checkbox" class="btn-check" id="aggregationOES" name="aggregation_type" value="oes" {% if 'oes' in aggregation_types %}checked{% endif %}>
        <label for="aggregationOES" class="btn {% if 'oes' in aggregation_types %}btn-primary{% else %}btn-outline-primary{% endif %} px-3 text-center me-3" style="min-width: 80px;">ОЭС</label>

        {% if sorted_synchronous_area_ids %}
          {% for sa_id in sorted_synchronous_area_ids %}
            {% set sa_name = synchronous_area_names[sa_id] %}
            {% set sa_checkbox_id = 'aggregationSyncArea' + sa_id|string %}
            {% set sa_value = 'sync_area_' + sa_id|string %}
            {% set is_checked = sa_value in aggregation_types %}
            <input type="checkbox" class="btn-check sync-area-checkbox" id="{{ sa_checkbox_id }}" name="aggregation_type" value="{{ sa_value }}" {% if is_checked %}checked{% endif %}>
            <label for="{{ sa_checkbox_id }}" class="btn {% if is_checked %}btn-primary{% else %}btn-outline-primary{% endif %} px-3 text-center sync-area-label {% if loop.last %}me-3{% endif %}" style="min-width: auto; white-space: nowrap;">{{ sa_name }}</label>
          {% endfor %}
```

Настройка уровней агрегации. Источник: app/ecology/services/ecology_totals_summary_services.py, строки 305–326.

```
    should_show_totals = {
        "energy_system_types": {},
        "union_energy_systems": False,
        "synchronous_areas": {},
        "federal_districts": False,
        "total": False,
    }
    if "russia" in agg_types:
        should_show_totals["total"] = True
    if "fo" in agg_types:
        should_show_totals["federal_districts"] = True
    if "oes" in agg_types:
        should_show_totals["union_energy_systems"] = True
    for est_id in sorted_energy_system_type_ids:
        name_u = (energy_system_type_names.get(est_id) or "").upper()
        if "ees" in agg_types and "ЕЭС" in name_u:
            should_show_totals["energy_system_types"][est_id] = True
        if "tites" in agg_types and "ТИТЭС" in name_u:
            should_show_totals["energy_system_types"][est_id] = True
    for sa_id in sorted_synchronous_area_ids:
        if f"sync_area_{sa_id}" in agg_types:
            should_show_totals["synchronous_areas"][sa_id] = True
```

## 14. Тексты подсказок формул

### 14.1. Редактор текстов подсказок формул

Адрес: /ecology/formulas/

**Функциональные возможности и действия пользователя**

Таблица содержит страницу применения, уровень агрегации, ячейку или показатель и текст формулы с пояснением. Есть предпросмотр, раскрываемая область редактирования, фильтры и поиск. Сохранение отправляет formula_key и formula_text в отдельный POST-маршрут; сброс удаляет переопределение и возвращает исходный текст. Доступ проверяется на сервере и для страницы, и для сохраняющих операций. Изменяется только содержание подсказок «i»: формулы вычисления, алгоритмы импорта и числовые данные не переписываются.

**Взаимодействие и источники данных**

Определения и исходные тексты поступают из operating_tes_formula_registry. Эффективный текст выбирается между сохраненным переопределением и исходным текстом; общий FormulaTextOverrideStore выполняет хранение и кеширование. Подсказки используются в отчетах, анализе и расчетах ТЭС. Журнал фиксирует прежний и новый текст.

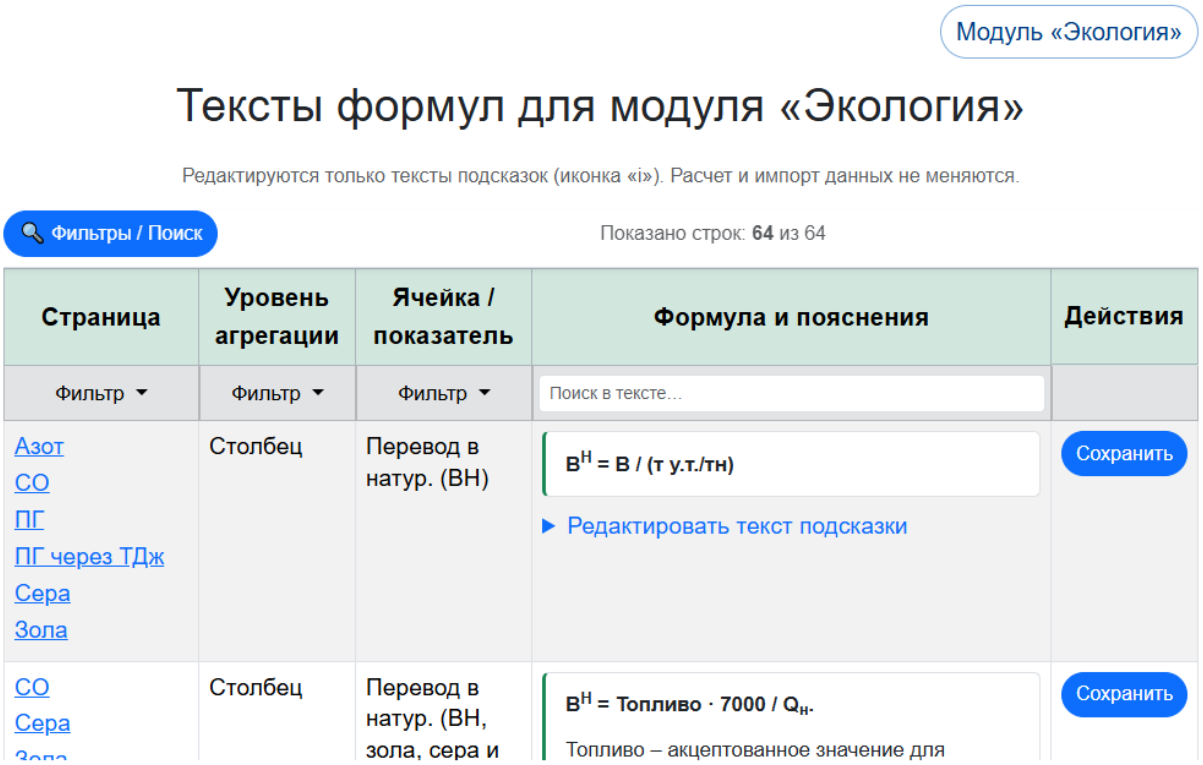



Рисунок 15. Редактор текстов подсказок формул без сохранения изменений. Действующий сайт, 29.10.2026. Показан фрагмент экранной формы.

## Элементы реализации

Модель OperatingTesFormulaText. Источник: app/ecology/models/formula_text/operating_tes_formula_text_model.py, строки 11–25.

```
class OperatingTesFormulaText(db.Model, AuditMixin):
    """Только текст формулы; расчет значений ячеек не меняется."""

    __tablename__ = "gs_eco_operating_tes_formula_texts"
    __table_args__ = (
        db.UniqueConstraint("formula_key", name="uq_gs_eco_operating_tes_formula_texts_key"),
        {"schema": SCHEMA_ECOLOGY},
    )

    # Уникальный идентификатор записи
    id = db.Column(db.Integer, primary_key=True, autoincrement=True)
    # Ключ формулы (иконка «i» на страницах расчета)
    formula_key = db.Column(db.String(128), nullable=False, index=True)
    # Текст формулы (подсказка; на расчет ячеек не влияет)
    formula_text = db.Column(db.Text, nullable=False)
```

Маршрут страницы. Источник: app/ecology/routes/ecology_routes.py, строки 152–163.

```
@ecology_bp.route("/formulas/")
@login_required
def operating_tes_formulas():
    if not can_edit_ecology(current_user):
        flash("Недостаточно прав для редактирования текстов формул.", "danger")
        return redirect(url_for("ecology_bp.hub"))
    log_page_visit("Тексты формул")
    return render_template(
        "ecology/operating_tes/operating_tes_formulas.html",
        page_title="Тексты формул для модуля «Экология»",
        formula_rows=list_formulas_for_admin(),
    )
```

Сервис и расчетная логика. Источник: app/ecology/services/formula_text/operating_tes_formula_text_services.py, строки 35–43.

```
def ot_formula_text(formula_key: str, default: str | None = None) -> str:
    """Эффективный текст формулы: переопределение из БД или значение по умолчанию."""
    definition = get_formula_def(formula_key)
    fallback = default if default is not None else (definition.default_text if definition else "")
    overrides = get_formula_text_overrides()
    override = overrides.get(formula_key)
    if override is not None and str(override).strip() != "":
        return str(override).strip()
    return str(fallback).strip()
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/operating_tes/operating_tes_formulas.html, строки 203–228.

```
                    {% for row in formula_rows %}
                    <tr data-formula-key="{{ row.key }}"
                        data-filter-page-lines="{{ row.page_lines|join('||')|e }}"
                        data-filter-aggregation="{{ row.aggregation_level }}"
                        data-filter-cell="{{ row.cell_name }}">
                        <td class="small ec-formula-page-cell">
                            <ul class="list-unstyled mb-0">
                                {% for page_line in row.page_lines %}
                                <li class="mb-1">
                                    {% if ' (/' in page_line %}
                                    {% set page_parts = page_line.rsplit(' (', 1) %}
                                    <a href="{{ page_parts[1][:-1] }}">{{ page_parts[0] }}</a>
                                    {% else %}{{ page_line }}{% endif %}
                                </li>
                                {% endfor %}
                            </ul>
                        </td>
                        <td class="small">{{ row.aggregation_level }}</td>
                        <td class="small">{{ row.cell_name }}</td>
                        <td>
                            <div class="ecology-formula-preview" role="region" aria-
label="Предпросмотр: {{ row.cell_name }}" aria-live="polite">{{ row.formula_text }}</div>
                            <details class="ecology-formula-editor">
                                <summary>Редактировать текст подсказки</summary>
                                <textarea class="form-control form-control-sm ec-formula-text-input
fuel-param-input" rows="4"
                                          data-default="{{ row.default_text|e }}"
                                          aria-label="Текст формулы {{ row.cell_name }}">{{
row.formula_text }}</textarea>
```

Сохранение подсказки. Источник: app/ecology/routes/ecology_routes.py, строки 166–184.

```
@ecology_bp.route("/formulas/save", methods=["POST"])
@login_required
def operating_tes_formulas_save():
    if not can_edit_ecology(current_user):
        return jsonify(ok=False, error="Недостаточно прав"), 403
    data = request.get_json(silent=True) or {}
    key = str(data.get("formula_key") or "")
    new_text = str(data.get("formula_text") or "")
    log_context = context_lines(("формула", key))
    try:
        previous_text = ot_formula_text(key)
        save_formula_text_override(
            formula_key=key,
            formula_text=new_text,
        )
        db.session.commit()
    except ValueError as exc:
        db.session.rollback()
        return jsonify(ok=False, error=str(exc)), 400
```

Формирование списка определений. Источник: app/ecology/services/formula_text/operating_tes_formula_text_services.py, строки 93–109.

```
def list_formulas_for_admin(*, page: str | None = None) -> list[dict[str, Any]]:
    overrides = get_formula_text_overrides()
    out: list[dict[str, Any]] = []
    for item in iter_formula_defs(page=page):
        out.append(
            {
                "key": item.key,
                "page_labels": item.page_labels(),
                "page_lines": list(item.page_label_lines()),
                "aggregation_level": item.aggregation_level,
                "cell_name": item.cell_name,
                "default_text": item.default_text,
                "formula_text": ot_formula_text(item.key),
                "is_customized": item.key in overrides,
            }
        )
    return out
```

## 15. Рассмотрение данных, переданных из модуля «Топливо»

### 15.1. Рассмотрение пакета данных из модуля «Топливо»

Адрес: /ecology/operating_tes/fuel_submissions/<int:submission_id>/

**Функциональные возможности и действия пользователя**

Страница сравнивает принятые показатели и переданный пакет за конкретный год и категорию. Для каждого отличия выводятся показатель, территория, единица, прежнее значение, новое значение и разница. Фильтры показателя и территории работают в таблице. При статусе pending и наличии прав доступны «Акцептовать» и «Не принимать»; ссылка «Назад без решения» ничего не утверждает. Неизвестный пакет дает 404, недостаточные права на решение дают 403, повторное рассмотрение или замена новой отправкой возвращает конфликт 409.

**Взаимодействие и источники данных**

Пакет передается из «Топлива» как отдельный JSON-снимок. До акцепта он не заменяет принятую топливную базу расчетов. Принятие заменяет только снимок выбранных версии, года и категории, снимает прежние ручные переопределения действующих ТЭС и переводит пакет в accepted. Отклонение сохраняет ранее принятую базу и задает rejected. Блокировка снимка сериализует отправку и рассмотрение. Категории operating, new_lt100 и new_ge100 не объединяются в одну запись.

**Элементы реализации**

Модель EcologyFuelSnapshot. Источник: app/ecology/models/fuel_exchange_model.py, строки 9–21.

```
  9  class EcologyFuelSnapshot(db.Model, AuditMixin):
 10      __tablename__ = "gs_eco_fuel_snapshots"
 11      __table_args__ = (
 12          db.UniqueConstraint("database_version_id", "year_number", "scope",
name="uq_eco_fuel_snapshot_version_year_scope"),
 13          {"schema": SCHEMA_ECOLOGY},
 14      )
 15
 16      id = db.Column(db.Integer, primary_key=True)
 17      database_version_id = db.Column(db.Integer,
db.ForeignKey(f"{SCHEMA_REFDATA}.gs_database_versions.id", ondelete="CASCADE"), nullable=False)
 18      year_number = db.Column(db.Integer, nullable=False)
 19      scope = db.Column(db.String(20), nullable=False, default="operating",
server_default="operating")
 20      payload = db.Column(db.JSON, nullable=False)
 21      updated_at = db.Column(db.DateTime(timezone=True), server_default=func.now(),
onupdate=func.now(), nullable=False)
```

Маршрут страницы. Источник: app/ecology/routes/operating_tes_routes.py, строки 125–143.

```
125  @ecology_bp.route("/operating_tes/fuel_submissions/<int:submission_id>/", methods=["GET", "POST"])
126  @login_required
127  def operating_tes_fuel_review(submission_id):
128      from app.ecology.services.fuel_exchange_services import (
129          get_submission, review_fuel_submission, submission_differences,
130      )
131      from app.ecology.services.operating_tes_page_services import format_cell
132
133      submission = get_submission(submission_id)
134      if submission is None:
135          abort(404)
136      from app.ecology.services.fuel_summary_constants import FUEL_SUMMARY_LABELS
137
138      from app.ecology.services.fuel_summary_constants import NEW_TES_SCOPE_CAPACITY
139
140      scope_label = FUEL_SUMMARY_LABELS[submission.scope]
141      capacity_class = NEW_TES_SCOPE_CAPACITY.get(submission.scope)
142      return_endpoint = "ecology_bp.operating_tes_table" if submission.scope == "operating" else
"ecology_bp.new_tes_table"
143      can_review = can_edit_ecology(current_user)
```

Сервис и расчетная логика. Источник: app/ecology/services/fuel_exchange_services.py, строки 95–115.

```
  95  def review_fuel_submission(submission_id, accept):
  96      submission = get_submission(submission_id)
  97      if submission is None:
  98          raise ValueError("Пакет не найден в текущей версии базы данных")
  99      snapshot = EcologyFuelSnapshot.query.filter_by(
 100          database_version_id=submission.database_version_id, year_number=submission.year_number,
scope=submission.scope,
 101      ).with_for_update().one()
 102      db.session.refresh(submission)
 103      if submission.status != "pending":
 104          raise ValueError("Пакет уже рассмотрен или заменен новой отправкой. Обновите страницу.")
 105      if accept:
 106          snapshot.payload = submission.payload
 107          # Прежние ручные переопределения не должны перекрывать принятый пакет.
 108          if submission.scope == "operating":
 109              OperatingTesInputValue.query.filter_by(
 110                  database_version_id=submission.database_version_id,
year_number=submission.year_number,
 111              ).delete(synchronize_session=False)
 112      submission.status = "accepted" if accept else "rejected"
 113      submission.reviewed_at = func.now()
 114      db.session.commit()
 115      return submission
```

HTML-шаблон экранной формы. Источник: app/templates/ecology/operating_tes/fuel_review.html, строки 17–48.

```
  17    <div class="table-responsive">
  18      <table id="fuel-review-table" class="table table-bordered table-striped">
  19        <thead class="table-light"><tr>
  20          {% for label in ['Показатель', 'Территория'] %}
  21          <th scope="col">{{ label }}
  22            <button type="button" class="btn btn-sm btn-outline-secondary d-block mt-1 fuel-review-
filter"
  23                    data-column="{{ loop.index0 }}" data-label="{{ label }}"
  24                    aria-label="Фильтр: {{ label }}" aria-haspopup="dialog" aria-expanded="false"
hidden>
  25              <i class="bi bi-funnel" aria-hidden="true"></i> <span>Все</span>
  26            </button>
  27          </th>
  28          {% endfor %}
  29          <th scope="col">Ед. изм.</th><th scope="col">Принято сейчас</th><th scope="col">Новые
данные</th><th scope="col">Разница</th>
  30        </tr></thead>
  31        <tbody>{% for row in differences %}
  32          <tr><th>{{ row.label }}</th><td>{{ row.territory }}</td><td>{{ row.unit }}</td>
  33            <td>{{ format_cell(row.before, digits=0) }}</td><td class="table-warning">{{
format_cell(row.after, digits=0) }}</td><td>{{ format_cell(row.delta, digits=0) }}</td></tr>
  34        {% endfor %}
  35          <tr id="fuel-review-empty" hidden><td colspan="6" class="text-center text-muted">Нет строк,
соответствующих фильтрам.</td></tr>
  36        </tbody>
  37      </table>
  38    </div>
  39    <p id="fuel-review-filter-status" class="text-muted" role="status" hidden></p>
  40    <div class="d-flex gap-2 mb-4">
  41      {% if can_review and submission.status == 'pending' %}
  42      <form method="POST" class="d-flex gap-2">
  43        <input type="hidden" name="csrf_token" value="{% if csrf_token is string %}{{ csrf_token }}{%
else %}{{ csrf_token() }}{% endif %}">
  44        <button name="decision" value="accept" class="btn btn-success btn-lg">Акцептовать</button>
  45        <button name="decision" value="reject" class="btn btn-outline-danger btn-lg">Не
принимать</button>
  46      </form>
  47      {% endif %}
  48      <a class="btn btn-secondary" href="{{ url_for(return_endpoint, territory_slug='vsego',
sheet_slug='toplivo', start_year=request.args.get('start_year', submission.year_number),
end_year=request.args.get('end_year', submission.year_number), view_year=submission.year_number,
rounding_digits=request.args.get('rounding_digits', 1), capacity_class=capacity_class|default(none))
}}">Назад без решения</a>
```

## 16. Сквозная логика и взаимодействие с другими модулями

### 16.1. Отчет, выверенные параметры и агрегаты

Последовательность: пользователь выбирает версию БД и год; уполномоченный специалист загружает Excel на странице отчета; импорт разрешает код группы оборудования и год, создает или обновляет исходные строки и возвращает счетчики. Для анализа создаются только отсутствующие параметры, после чего специалист может выверять их отдельно. Лист анализа соединяет эти параметры с топливными и генерационными показателями, вычисляет разности и удельные значения. Агрегированная форма суммирует базовые величины и заново вычисляет производные показатели.

Загрузка ПГ дополнительно инициирует заполнение отсутствующих параметров анализа ПГ. Для ЗВ заполнение предусмотрено при построении аналитических строк. Импорт не является безусловной заменой всех выверенных параметров: существующие строки анализа сохраняются. Пропуск неизвестного года или кода станции и отсутствие найденной группы учитываются отдельными счетчиками; отсутствие группы не следует смешивать с отсутствием кода.

Импорт: обязательные колонки, текущая версия и индекс существующих записей. Источник: app/ecology/services/gis_tek_import_services.py, строки 304–332.

```
304  def import_gis_tek_from_xlsx_bytes(file_bytes: bytes, kind: str, user: str) -> dict:
305      if kind not in REPORT_MODELS:
306          raise ValueError("Неизвестный вид отчетных данных.")
307      logger = _get_logger()
308      t0 = time.perf_counter()
309      df = read_gis_tek_sheet(file_bytes, kind)
310      if "station_numb" not in df.columns or "year_number" not in df.columns:
311          raise ValueError(
312              "В листе нет обязательных столбцов «№_станции» и «Год»."
313          )
314
315      model = REPORT_MODELS[kind]
316      value_attrs = [
317          attr
318          for attr, _label in VALUE_COLUMNS[kind]
319          if kind != REPORT_ZV or attr not in ZV_DERIVED_ATTRS
320      ]
321      version_id = get_current_db_version_id()
322      groups_by_numb = _load_equipment_groups_by_numb(version_id)
323      known_years = _known_year_numbers(version_id)
324
325      existing_query = model.query
326      if version_id is not None:
327          existing_query = existing_query.filter(model.database_version_id == version_id)
328      else:
329          existing_query = existing_query.filter(model.database_version_id.is_(None))
330      existing = {
331          (row.station_numb, row.year_number): row for row in existing_query.all()
332      }
```

Первичное заполнение анализа ПГ после импорта отчета. Источник: app/ecology/services/gis_tek_import_services.py, строки 435–445.

```
435      seeded_params = 0
436      if kind == REPORT_PG:
437          from app.ecology.services.analysis_pg_param_services import (
438              seed_missing_analysis_pg_params_from_reports,
439          )
440          from app.ecology.services.gis_tes_fuel_comparison_constants import (
441              PG_ANALYSIS_SHEET_SLUGS,
442          )
443
444          for sheet_slug in sorted(PG_ANALYSIS_SHEET_SLUGS):
445              seeded_params += seed_missing_analysis_pg_params_from_reports(sheet_slug)
```

Отсутствующие параметры создаются без перезаписи существующих. Источник: app/ecology/services/analysis_param_services.py, строки 64–88.

```
64          existing = {
65              (row.station_numb, row.year_number)
66              for row in param_q.with_entities(
67                  params_model.station_numb, params_model.year_number
68              ).all()
69          }
70          created = 0
71          for report in report_q.all():
72              key = (report.station_numb, report.year_number)
73              if key in existing:
74                  continue
75              db.session.add(
76                  params_model(
77                      sheet_slug=slug,
78                      station_numb=report.station_numb,
79                      year_number=report.year_number,
80                      database_version_id=version_id,
81                      **values_payload(report, fields),
82                  )
83              )
84              existing.add(key)
85              created += 1
86          if created:
87              db.session.commit()
88          return created
```

Отдельное хранилище выверенных параметров. Источник: app/ecology/services/analysis_param_services.py, строки 100–127.

```
100  def upsert_param_values(
101      params_model: Any,
102      reports_model: Any,
103      *,
104      sheet_slug: str,
105      sheet_slugs: frozenset[str],
106      fields: tuple[str, ...],
107      label: str,
108      station_numb: int,
109      year_number: int,
110      values: dict[str, Any],
111  ) -> Any:
112      """Создает или обновляет параметры страницы анализа."""
113      from app.common.services.database_version_filter import get_current_db_version_id
114      from app.extensions import db
115
116      slug = str(sheet_slug or "").strip()
117      if slug not in sheet_slugs:
118          raise ValueError(f"Неизвестный sheet_slug для параметров {label}: {sheet_slug!r}")
119      version_id = get_current_db_version_id()
120      query = params_model.query.filter_by(
121          sheet_slug=slug,
122          station_numb=int(station_numb),
123          year_number=int(year_number),
124      )
125      if version_id is not None:
126          query = query.filter(params_model.database_version_id == version_id)
127      row = query.first()
```

## 16.2. Передача топлива и расчет выбросов ТЭС

Последовательность: в «Топливе» собирается сводка за год и категорию; отправка создает EcologyFuelSubmission со статусом pending, не меняя содержимое уже принятого снимка. Предыдущий ожидающий пакет того же разреза получает superseded. Специалист «Экологии» сравнивает значения и принимает либо отклоняет пакет. Только accepted заменяет payload снимка выбранного разреза. Расчетные сервисы читают принятые EcologyFuelSnapshot, применяют физические характеристики, коэффициенты очистки и нормативы, затем сводят действующие и обе категории новых ТЭС.

Объекты групп оборудования, их топливные параметры и дополнительные виды топлива принадлежат «Топливу». «Генерация» предоставляет связанные станции, мощность, выработку и сведения для проверки соответствия. «Справочники» задают годы, виды топлива, территориальную и энергетическую иерархию. Выбранная версия БД ограничивает предметный разрез. Журнал отражает изменения и решения, но не является источником расчетных значений.

Отправка пакета без замены принятой базы. Источник: app/ecology/services/fuel_exchange_services.py, строки 62–88.

```
  62  def submit_fuel_year(year, scope="operating"):
  63      if scope not in FUEL_SUMMARY_LABELS:
  64          raise ValueError("Неизвестный раздел сводки")
  65      version_id = _version_id()
  66      from app.ecology.services.fuel_summary_services import load_fuel_summary
  67
  68      inputs = load_fuel_summary([year], scope)
  69      # Один замок на раздел и год сериализует отправку и рассмотрение пакета.
  70      snapshot = EcologyFuelSnapshot.query.filter_by(
  71          database_version_id=version_id, year_number=year, scope=scope,
  72      ).with_for_update().first()
  73      if snapshot is None:
  74          db.session.execute(
  75              insert(EcologyFuelSnapshot).values(
  76                  database_version_id=version_id, year_number=year, scope=scope, payload={},
  77              ).on_conflict_do_nothing(index_elements=["database_version_id", "year_number",
"scope"])
  78          )
  79          snapshot = EcologyFuelSnapshot.query.filter_by(
  80              database_version_id=version_id, year_number=year, scope=scope,
  81          ).with_for_update().one()
  82      payload = encode_inputs(inputs.get(year, {}))
  83      for previous in pending_submissions([year], scope=scope):
  84          previous.status = "superseded"
  85      submission = EcologyFuelSubmission(database_version_id=version_id, year_number=year,
scope=scope, payload=payload, status="pending")
  86      db.session.add(submission)
  87      db.session.commit()
  88      return submission
```

Акцепт или отклонение пакета. Источник: app/ecology/services/fuel_exchange_services.py, строки 95–115.

```
  95  def review_fuel_submission(submission_id, accept):
  96      submission = get_submission(submission_id)
  97      if submission is None:
  98          raise ValueError("Пакет не найден в текущей версии базы данных")
  99      snapshot = EcologyFuelSnapshot.query.filter_by(
 100          database_version_id=submission.database_version_id, year_number=submission.year_number,
scope=submission.scope,
 101      ).with_for_update().one()
 102      db.session.refresh(submission)
 103      if submission.status != "pending":
 104          raise ValueError("Пакет уже рассмотрен или заменен новой отправкой. Обновите страницу.")
 105      if accept:
 106          snapshot.payload = submission.payload
 107          # Прежние ручные переопределения не должны перекрывать принятый пакет.
 108          if submission.scope == "operating":
 109              OperatingTesInputValue.query.filter_by(
 110                  database_version_id=submission.database_version_id,
year_number=submission.year_number,
 111              ).delete(synchronize_session=False)
 112      submission.status = "accepted" if accept else "rejected"
 113      submission.reviewed_at = func.now()
 114      db.session.commit()
 115      return submission
```

Вход передачи из модуля «Топливо». Источник: app/fuel/routes/ecology_summary_routes.py, строки 12–35.

```
@fuel_bp.route("/ecology_summary/", methods=["GET", "POST"])
@login_required
def ecology_summary():
    scope = request.args.get("scope", "operating")
    if scope not in FUEL_SUMMARY_LABELS:
        abort(404)
    context = build_operating_tes_table_context(
        "vsego", "toplivo", start_year=request.args.get("start_year"),
        end_year=request.args.get("end_year"), rounding_digits=request.args.get("rounding_digits"),
        view_year=request.args.get("view_year"), fuel_source=True, fuel_scope=scope,
    )
    can_send = bool(current_user.has_admin)
    if request.method == "POST":
        if not can_send:
            return "Недостаточно прав", 403
        year = context["view_year"]
        if year not in context["range_years"]:
            return "Год не входит в выбранный диапазон", 400
        try:
            submit_fuel_year(year, scope=scope)
        except FuelSummaryError as exc:
            return str(exc), 409
        flash(f'{FUEL_SUMMARY_LABELS[scope]}: данные за {year} год переданы в модуль «Экология» и
ожидают акцепта.', "success")
        return redirect(url_for("fuel_bp.ecology_summary", start_year=context["start_year"],
end_year=context["end_year"], rounding_digits=context["rounding_digits"], view_year=year, scope=scope if
scope != "operating" else None))
```

## 16.3. Формулы, единицы измерения и отсутствующие данные

Для действующих ТЭС перевод в натуральное топливо для золы, серы и CO использует B × 7000 / Qн. Прямой расчет CO2 использует B × KCO2; энергетический вариант дополнительно использует ТДж и т CO2/ТДж. Улавливание рассчитывается как Мотх × (1 − коэффициент). В новых ТЭС масса ЗВ = расход в тыс. т у.т. × норматив в кг/т у.т. дает тонны, а масса CO2 = расход × норматив в т/т у.т. дает тыс. т. Перед суммированием выполняется согласование единиц.

Для удельных показателей важны единицы знаменателя. Расчетные ТЭС используют приведенную энергию (E + Q × 1,163) / 1000 в млрд кВт·ч; анализ отчетов использует свою функцию приведения W_en + Teplo / 859,8 × 1000 в млн кВт·ч. Эти выражения описывают текущую реализацию разных форм, а не замену одной формулы другой. Отсутствующее значение и нулевой знаменатель обрабатываются явно; знак отсутствия данных не является автоматически измеренным нулем. Документ не подтверждает научную аттестацию методики или полноту исходных данных.

Натуральный расход топлива по теплоте сгорания. Источник: app/ecology/services/operating_tes_calc_services.py, строки 35–41.

```
def to_natural_by_heat(b_tce: Any, chars: Mapping[str, Any] | None) -> Decimal | None:
    """Перевод для золы, серы и CO: BH = B · 7000 / Qн (ккал/кг)."""
    b = _d(b_tce)
    qh = _d(chars.get("qh_kcal")) if chars else None
    if b is None or qh is None or qh == 0:
        return None
    return b * Decimal("7000") / qh
```

Учет улавливания или подавления. Источник: app/ecology/services/operating_tes_calc_services.py, строки 124–130.

```
def apply_capture(motkh: Any, eta: Any) -> Decimal | None:
    """Мвыб = Мотх·(1 − η)."""
    m = _d(motkh)
    if m is None:
        return None
    e = _d(eta) or Decimal("0")
    return m * (Decimal("1") - e)
```

Приведенная энергия расчетных ТЭС. Источник: app/ecology/services/operating_tes_calc_services.py, строки 133–143.

```
133  def equivalent_energy_billion_kwh(e_mln_kwh: Any, q_ths_gcal: Any) -> Decimal | None:
134      """Приведенная энергия, млрд кВт·ч = (E + Q·1,163) / 1000."""
135      e = _d(e_mln_kwh)
136      q = _d(q_ths_gcal)
137      if e is None and q is None:
138          return None
139      e = e or Decimal("0")
140      q = q or Decimal("0")
141      from app.ecology.services.operating_tes_constants import GCAL_TO_MWH
142
143      return (e + q * GCAL_TO_MWH) / Decimal("1000")
```

Приведенная энергия анализа отчетных данных. Источник: app/ecology/services/gis_tes_fuel_comparison_services.py, строки 1990–2000.

```
1990  def _brought_energy_mln_kwh(w_en: Any, teplo: Any) -> Any:
1991      """Приведенная энергия, млн кВт·ч: W_en + Teplo / 859,8 · 1000."""
1992      energy = _clean_number(w_en)
1993      heat = _clean_number(teplo)
1994      if energy is None and heat is None:
1995          return None
1996      with localcontext() as ctx:
1997          ctx.prec = SIGNIFICANT_DIGITS
1998          ctx.rounding = ROUND_HALF_UP
1999          total = (energy or Decimal(0)) + (heat or Decimal(0)) / Decimal(str(KCAL_PER_KWH)) * 1000
2000          return float(+total)
```

Защита от отсутствующей или нулевой базы деления. Источник: app/ecology/services/gis_tes_fuel_comparison_services.py, строки 1963–1972.

```
1963  def _safe_divide(numerator: Any, denominator: Any) -> Any:
1964      """Безопасное деление: None при пустых значениях или нулевом знаменателе."""
1965      left = _clean_number(numerator)
1966      right = _clean_number(denominator)
1967      if left is None or right is None or right == 0:
1968          return None
1969      with localcontext() as ctx:
1970          ctx.prec = SIGNIFICANT_DIGITS
1971          ctx.rounding = ROUND_HALF_UP
1972          return float(+(left / right))
```

Единицы массы в расчетах новых ТЭС. Источник: app/ecology/services/new_tes_calc_services.py, строки 12–35.

```
 12  def zv_mass_tonnes(b_tce: Any, u_kg_per_tce: Any) -> Decimal | None:
 13      """М, т = В, тыс. т у.т. × У, кг/т у.т."""
 14      b = _d(b_tce)
 15      u = _d(u_kg_per_tce)
 16      if b is None or u is None:
 17          return None
 18      return b * u
 19
 20
 21  def zv_mass_thousand_tonnes(b_tce: Any, u_kg_per_tce: Any) -> Decimal | None:
 22      """М, тыс. т = (В, тыс. т у.т. × У, кг/т у.т.) / 1000."""
 23      tonnes = zv_mass_tonnes(b_tce, u_kg_per_tce)
 24      if tonnes is None:
 25          return None
 26      return tonnes / Decimal("1000")
 27
 28
 29  def pg_mass_thousand_tonnes(b_tce: Any, u_t_per_tce: Any) -> Decimal | None:
 30      """М CO2, тыс. т = В, тыс. т у.т. × У, т/т у.т."""
 31      b = _d(b_tce)
 32      u = _d(u_t_per_tce)
 33      if b is None or u is None:
 34          return None
 35      return b * u
```

## 16.4. Обмен Excel и интерактивные действия

Отчеты ПГ/ЗВ имеют собственные маршруты импорта и выгрузки. Анализ экспортируется без ограничения текущей страницей (per_page=all). Выгрузка

итоговых сумм учитывает выбранные уровни и период. Для действующих ТЭС и сводного расчета Excel формируется по вкладкам и годовым блокам. В текущем new_tes_routes.py отдельного маршрута выгрузки новых ТЭС нет; при описании этой страницы нельзя автоматически приписывать ей экспорт действующих ТЭС.

Ввод в разрешенную числовую ячейку передает PATCH с годом, полем и ключами строки. Сервер проверяет пользователя, допустимый лист, вид поля и корректность числа; листы только для просмотра возвращают отказ. Формы решения по пакету и сохранения подсказок используют POST. Тексты подсказок не исполняются как программные формулы и не меняют расчетный алгоритм.

Excel: расчетные вкладки и годовые блоки. Источник: app/ecology/services/tes_excel_export_services.py, строки 23–31.

```
23  def build_tes_excel_stream(territory_slug, args, *, summary=False):
24      builder = build_calculation_summary_context if summary else build_operating_tes_table_context
25      tabs = CALCULATION_SUMMARY_TABS if summary else CALC_SHEETS
26      years = get_year_columns(args.get('start_year'), args.get('end_year'))
27      filters = {key: args.get(key) for key in ('rounding_digits', 'coefficient_rounding_digits')}
28      if not summary:
29          filters['affiliation'] = args.get('affiliation', 'all')
30      workbook = Workbook()
31      workbook.remove(workbook.active)
```

HTML/JavaScript: PATCH разрешенной числовой ячейки. Источник: app/templates/ecology/operating_tes/operating_tes_table.html, строки 1068–1089.

```
1068            const tr = el.closest('tr');
1069            const cell = el.closest('td');
1070            clearSaveState(el);
1071            state.saving = true;
1072            try {
1073              const response = await fetch(el.dataset.updateUrl || updateUrl, {
1074                method: 'PATCH',
1075                credentials: 'same-origin',
1076                headers: {
1077                  'Content-Type': 'application/json',
1078                  'X-Requested-With': 'XMLHttpRequest',
1079                  'X-CSRFToken': CSRF || ''
1080                },
1081                body: JSON.stringify({
1082                  year: el.dataset.year,
1083                  territory: el.dataset.territory,
1084                  affiliation: el.dataset.affiliation || table.dataset.affiliation,
1085                  row_key: tr?.dataset.rowKey,
1086                  field: el.dataset.field || tr?.dataset.editableField,
1087                  value: value
1088                })
1089              });
```

HTML/JavaScript: сохранение параметра анализа. Источник: app/templates/ecology/analysis/gis_tes_fuel_comparison.html, строки 1098–1118.

```
1098            if (input.value === (input.dataset.original || '')) return;
1099            const tr = input.closest('tr');
1100            const td = input.closest('td');
1101            recalcDerivedRow(tr);
1102            input.disabled = true;
1103            if (td) td.classList.remove('is-save-success', 'is-save-error');
1104            try {
1105              const response = await fetch(updateUrl, {
1106                method: 'PATCH',
1107                credentials: 'same-origin',
1108                headers: {'Content-Type': 'application/json', 'X-CSRFToken': CSRF || ''},
1109                body: JSON.stringify({
1110                  year: input.dataset.year,
1111                  numb: input.dataset.numb,
1112                  field: input.dataset.field,
1113                  value: input.value,
1114                  rounding_digits: roundingDigits,
1115                  emission_rounding_digits: emissionRoundingDigits,
1116                  specific_rounding_digits: specificRoundingDigits,
1117                  norm_zv: {{ norm_zv_selected|default([])|tojson }}
1118                })
```

Проверки сохранения норматива новых ТЭС. Источник: app/ecology/routes/new_tes_routes.py, строки 101–126.

```
@ecology_bp.route(
    "/new_tes/<territory_slug>/<sheet_slug>/value/",
    methods=["PATCH"],
)
@login_required
def new_tes_update_value(territory_slug: str, sheet_slug: str):
    if not can_edit_ecology(current_user):
        return jsonify(ok=False, error="Недостаточно прав"), 403
    if get_territory(territory_slug) is None or get_calc_sheet(sheet_slug) is None:
        abort(404)
    capacity = request.args.get("capacity_class", CAPACITY_LE100_KEY)
    if capacity not in CAPACITY_CLASSES:
        abort(404)
    data = request.get_json(silent=True) or {}
    year = data.get("year")
    field = str(data.get("field") or "").strip()
    row_key = str(data.get("row_key") or "").strip()
    try:
        year_int = int(year)
    except (TypeError, ValueError):
        return jsonify(ok=False, error="Некорректный год"), 400
    raw = data.get("value")
    try:
        amount = parse_decimal_from_display(raw) if raw not in (None, "") else None
    except Exception:
        return jsonify(ok=False, error="Некорректное число"), 400
```

Сброс только текста подсказки. Источник: app/ecology/routes/ecology_routes.py, строки 196–216.

```
@ecology_bp.route("/formulas/reset", methods=["POST"])
@login_required
def operating_tes_formulas_reset():
    if not can_edit_ecology(current_user):
        return jsonify(ok=False, error="Недостаточно прав"), 403
    data = request.get_json(silent=True) or {}
    key = str(data.get("formula_key") or "").strip()
    if not key:
        return jsonify(ok=False, error="Не указан ключ формулы."), 400
    previous_text = ot_formula_text(key)
    reset_formula_text_override(key)
    db.session.commit()
    log_cell_change(
        "Тексты формул",
        f"Текст формулы {key}",
        previous_text,
        ot_formula_text(key),
        context=context_lines(("формула", key), ("действие", "сброс к исходному тексту")),
        number=False,
    )
    return jsonify(ok=True)
```

## 17. Полнота описания модуля «Экология» и источники

### 17.1. Перечень экранных представлений

| Раздел | Представление | Адрес или пример |
|---|---|---|
| 8.1 | Главная страница модуля | /ecology/ |
| 8.2 | Парниковые газы: отчетные данные ГИС ТЭС | /ecology/gis_tek/pg/ |
| 8.3 | Загрязняющие вещества: отчетные данные ГИС ТЭС | /ecology/gis_tek/pollutants/ |
| 9.1 | Парниковые газы: анализ с котельными | /ecology/analysis/pg-vse/ |
| 9.2 | Парниковые газы: анализ без котельных | /ecology/analysis/pg-bez-kot/ |
| 9.3 | Загрязняющие вещества: анализ с котельными | /ecology/analysis/zv-vse/ |
| 9.4 | Загрязняющие вещества: анализ без котельных | /ecology/analysis/zv-bez-kot/ |
| 10.1 | Данные по модулю Топливо | /ecology/operating_tes/vsego/toplivo/ |
| 10.2 | Характеристики топлив и продуктов сгорания | /ecology/operating_tes/vsego/khar-topl/ |
| 10.3 | Коэффициенты очистки и подавления | /ecology/operating_tes/vsego/koeff/ |
| 10.4 | Зола: выбросы твердых веществ | /ecology/operating_tes/vsego/zola/ |
| 10.5 | Сера: выбросы диоксида серы | /ecology/operating_tes/vsego/sera/ |
| 10.6 | Азот: выбросы оксидов азота | /ecology/operating_tes/vsego/azot/ |
| 10.7 | CO: выбросы монооксида углерода | /ecology/operating_tes/vsego/co/ |
| 10.8 | ПГ: прямой расчет CO2 | /ecology/operating_tes/vsego/pg/ |
| 10.9 | ПГ через ТДж: расчет CO2 по энергии топлива | /ecology/operating_tes/vsego/pg-tj/ |
| 10.10 | Объемы и удельные выбросы действующих ТЭС | /ecology/operating_tes/vsego/obemy/ |
| 11.1 | Принятые топливные показатели новых ТЭС | /ecology/new_tes/vsego/toplivo/ |
| 11.2 | Нормативы удельных выбросов новых ТЭС | /ecology/new_tes/vsego/spravochnik/ |
| 11.3 | Объемы выбросов новых ТЭС | /ecology/new_tes/vsego/obemy/ |
| 12.1 | Общие топливные показатели действующих и новых ТЭС | /ecology/calculation_summary/vsego/toplivo/ |
| 12.2 | Объемы выбросов по России | /ecology/calculation_summary/vsego/russia/ |
| 12.3 | Удельные выбросы по России | /ecology/calculation_summary/vsego/specific/ |

| Раздел | Представление | Адрес или пример |
|---|---|---|
| 12.4 | Динамика изменения выбросов | /ecology/calculation_summary/vsego/dynamics/ |
| 13.1 | ПГ: агрегированные суммы с котельными | /ecology/totals_summary/pg-vse/ |
| 13.2 | ПГ: агрегированные суммы без котельных | /ecology/totals_summary/pg-bez-kot/ |
| 13.3 | ЗВ: агрегированные суммы с котельными | /ecology/totals_summary/zv-vse/ |
| 13.4 | ЗВ: агрегированные суммы без котельных | /ecology/totals_summary/zv-bez-kot/ |
| 14.1 | Редактор текстов подсказок формул | /ecology/formulas/ |
| 15.1 | Рассмотрение пакета данных из модуля «Топливо» | /ecology/operating_tes/fuel_submissions/<int:submission_id>/ |

### 17.2. Скрытые совместимые вкладки сводного расчета

17.2.1. Действующие ТЭС

Адрес: /ecology/calculation_summary/vsego/operating/. Показывает расчет действующих ТЭС без добавления новых. Вкладка не выводится в основном списке CALCULATION_SUMMARY_TABS, но доступна по сохраненной ссылке. Общий маршрут calculation_summary, сервис build_calculation_summary_context и HTML operating_tes_table.html совпадают с разделом 12. Используются принятые топливные снимки, NewTesNorm и общие характеристики; отдельная модель скрытой вкладки не создается. Числовые результаты доступны для просмотра.

17.2.2. Новые ТЭС: обе категории

Адрес: /ecology/calculation_summary/vsego/new/. Суммирует объемы новых ТЭС le100 и gt100 без действующих. Вкладка не выводится в основном списке CALCULATION_SUMMARY_TABS, но доступна по сохраненной ссылке. Общий маршрут calculation_summary, сервис build_calculation_summary_context и HTML operating_tes_table.html совпадают с разделом 12. Используются принятые топливные снимки, NewTesNorm и общие характеристики; отдельная модель скрытой вкладки не создается. Числовые результаты доступны для просмотра.

17.2.3. Новые ТЭС le100

Адрес: /ecology/calculation_summary/vsego/new-le100/. Показывает отдельную категорию новых ТЭС, сопоставленную с new_lt100. Вкладка не выводится в основном списке CALCULATION_SUMMARY_TABS, но доступна по сохраненной ссылке. Общий маршрут calculation_summary, сервис build_calculation_summary_context и HTML operating_tes_table.html совпадают с разделом 12. Используются принятые топливные снимки, NewTesNorm и общие характеристики; отдельная модель скрытой вкладки не создается. Числовые результаты доступны для просмотра.

## 17.2.4. Новые ТЭС gt100

Адрес: /ecology/calculation_summary/vsego/new-gt100/. Показывает отдельную категорию новых ТЭС, сопоставленную с new_ge100. Вкладка не выводится в основном списке CALCULATION_SUMMARY_TABS, но доступна по сохраненной ссылке. Общий маршрут calculation_summary, сервис build_calculation_summary_context и HTML operating_tes_table.html совпадают с разделом 12. Используются принятые топливные снимки, NewTesNorm и общие характеристики; отдельная модель скрытой вкладки не создается. Числовые результаты доступны для просмотра.

Основные и скрытые совместимые вкладки. Источник: app/ecology/services/calculation_summary_services.py, строки 16–29.

```
  16  CALCULATION_SUMMARY_TABS = (
  17      {"slug": "toplivo", "sheet_name": "Данные по модулю Топливо"},
  18      {"slug": "russia", "sheet_name": "Объемы выбросов по России"},
  19      {"slug": "specific", "sheet_name": "Удельные выбросы по России"},
  20      {"slug": "dynamics", "sheet_name": "Динамика изменения выбросов"},
  21  )
  22
  23  # Скрытые вкладки остаются доступны по ранее сохраненным ссылкам.
  24  _LEGACY_TABS = (
  25      {"slug": "operating", "sheet_name": "Объемы выбросов для действующих ТЭС"},
  26      {"slug": "new", "sheet_name": "Объемы выбросов для новых ТЭС"},
  27      {"slug": "new-le100", "sheet_name": "Объемы выбросов для новых ТЭС до 100 МВт"},
  28      {"slug": "new-gt100", "sheet_name": "Объемы выбросов для новых ТЭС 100 МВт и более"},
  29  )
```

Выбор источников совместимых сводных представлений. Источник: app/ecology/services/calculation_summary_services.py, строки 162–196.

```
 162  def build_calculation_summary_context(territory_slug, sheet_slug, **filters):
 163      tab = next((tab for tab in (*CALCULATION_SUMMARY_TABS, *_LEGACY_TABS) if tab["slug"] ==
sheet_slug), None)
 164      if tab is None:
 165          return None
 166      source_sheet = "toplivo" if sheet_slug == "toplivo" else "obemy"
 167      new_filters = {key: value for key, value in filters.items() if key !=
"coefficient_rounding_digits"}
 168
 169      def new_context(capacity):
 170          return build_new_tes_table_context(
 171              territory_slug, source_sheet, capacity_class=capacity, **new_filters,
 172          )
 173
 174      if sheet_slug == "dynamics":
 175          context = _build_emission_dynamics(territory_slug, filters)
 176      elif sheet_slug in {"new-le100", "new-gt100"}:
 177          context = new_context(sheet_slug.removeprefix("new-"))
 178          if context is None:
 179              return None
 180      elif sheet_slug == "operating":
 181          context = build_operating_tes_table_context(territory_slug, source_sheet, **filters)
 182      else:
 183          le100, gt100 = new_context("le100"), new_context("gt100")
 184          if le100 is None or gt100 is None:
 185              return None
 186          if sheet_slug == "toplivo":
 187              context = build_operating_tes_table_context(territory_slug, source_sheet, **filters)
 188              context["rows"] = _sum_rows(context, [context, le100, gt100])
 189              context["page_title"] = "Топливные показатели действующих и новых ТЭС (модуль Топливо)"
 190              context["page_title_html"] = context["page_title"]
 191          else:
 192              new_sources = [_transpose_new_volumes(source) for source in (le100, gt100)]
 193              context = {**new_sources[0], "rows": _sum_rows(new_sources[0], new_sources)}
 194              if sheet_slug in {"russia", "specific"}:
 195                  operating = build_operating_tes_table_context(territory_slug, source_sheet,
**filters)
 196                  context = {**operating, "rows": _sum_rows(operating, [operating, context])}
```

### 17.3. Маршруты страниц и служебные операции

| Метод | Адрес | Обработчик |
|---|---|---|
| GET | /ecology/calculation_summary/ | calculation_summary_index |
| GET | /ecology/calculation_summary/<territory_slug>/export.xlsx | calculation_summary_export |
| GET | /ecology/calculation_summary/<territory_slug>/<sheet_slug>/ | calculation_summary |
| GET | /ecology/ | hub |
| GET | /ecology/formulas/ | operating_tes_formulas |
| POST | /ecology/formulas/save | operating_tes_formulas_save |
| POST | /ecology/formulas/reset | operating_tes_formulas_reset |
| GET | /ecology/gis_tek/pg/ | gis_tek_pg |
| POST | /ecology/gis_tek/pg/import | gis_tek_pg_import |
| GET | /ecology/gis_tek/pg/export.xlsx | gis_tek_pg_export |
| GET | /ecology/gis_tek/pollutants/ | gis_tek_pollutants |
| POST | /ecology/gis_tek/pollutants/import | gis_tek_pollutants_import |
| GET | /ecology/gis_tek/pollutants/export.xlsx | gis_tek_pollutants_export |
| GET | /ecology/totals_summary | totals_summary_index |
| GET | /ecology/totals_summary/ | totals_summary_index |
| GET | /ecology/totals_summary/<sheet_slug> | totals_summary |
| GET | /ecology/totals_summary/<sheet_slug>/ | totals_summary |
| GET | /ecology/totals_summary_export | totals_summary_export_index |
| GET | /ecology/totals_summary_export/ | totals_summary_export_index |
| GET | /ecology/totals_summary/<sheet_slug>/export | totals_summary_export |
| GET | /ecology/totals_summary/<sheet_slug>/export/ | totals_summary_export |
| GET | /ecology/analysis/<sheet_slug>/ | gis_tes_fuel_comparison |
| GET | /ecology/analysis/<sheet_slug>/export.xlsx | gis_tes_fuel_comparison_export |
| PATCH | /ecology/analysis/<sheet_slug>/value/ | gis_tes_fuel_comparison_update_value |
| GET | /ecology/new_tes/ | new_tes_hub |
| GET | /ecology/new_tes/<territory_slug>/ | new_tes_territory |
| GET | /ecology/new_tes/<territory_slug>/<sheet_slug>/ | new_tes_table |
| PATCH | /ecology/new_tes/<territory_slug>/<sheet_slug>/value/ | new_tes_update_value |
| GET | /ecology/operating_tes/ | operating_tes_hub |
| GET | /ecology/operating_tes/<territory_slug>/ | operating_tes_territory |
| GET | /ecology/operating_tes/<territory_slug>/<sheet_slug>/ | operating_tes_table |
| GET | /ecology/operating_tes/<territory_slug>/export.xlsx | operating_tes_export |
| GET, POST | /ecology/operating_tes/fuel_submissions/<int:submission_id>/ | operating_tes_fuel_review |

| Метод | Адрес | Обработчик |
|---|---|---|
| PATCH | /ecology/operating_tes/<territory_slug>/<sheet_slug>/value/ | operating_tes_update_value |

Маршруты верхнего уровня operating_tes/, new_tes/, calculation_summary/ и totals_summary/ перенаправляют к исходному листу. Export, value, import, save и reset являются операциями существующих страниц, а не дополнительными экранными формами. Варианты totals_summary со слешем и без него ведут к одному представлению. Технические HEAD и OPTIONS Flask в число отдельных прикладных операций не включены.

### 17.4. Проверка и ограничения

Описание сверено с маршрутами, моделями, сервисами, HTML и включаемыми JavaScript текущей рабочей копии. Снимки подтверждают внешний вид только просмотренных страниц и конкретного состояния БД на дату получения. Изменяющие операции и научная корректность расчетов не испытывались на рабочем сервере. При подаче заявки заявитель по-прежнему подтверждает официальное название программы, авторов, правообладателя, объем и дату завершения. Добавление «Экологии» не заполняет эти сведения автоматически.

Сведения по предоставленному шаблону регистрации сохранены в разделе 1, описание «Справочников» в разделах 2–6. Стили основного текста, заголовков, таблиц и иллюстраций унаследованы из ранее примененного отчета «Отчет_Этап 1_АРМ_ГС_2026-НК.docx». Моноширинный SourceCode сохранен для листингов. Оглавление охватывает оба модуля и обновляется средствами Word.